

SYSTEM DESIGN INTERVIEW SERIES · VOLUME I

The System Design Interview Playbook

A growing compendium of real interview problems, solved end-to-end: rigorous definitions, back-of-the-envelope mathematics, protocol design, and production-grade Rust implementations.

IN THIS VOLUME

- Chapter 01 — Designing a Real-Time Collaborative Text Editor (Google Docs / Notion)
- Chapter 02 — Designing a Maps & Navigation Service (Google Maps)
- Chapter 03 — Designing a Distributed Rate Limiter
- Chapter 04 — Designing a Distributed Logging & Metrics Platform
- Chapter 05 — Designing a Hierarchical Comment System (Reddit)
- Chapter 06 — Designing a Top-K Leaderboard (Gaming)
- Chapter 07 — Designing a Recommendation Engine (YouTube / TikTok)
- Chapter 08 — Designing a Database Index — How B-Trees Work
- Chapter 09 — Designing Conflict-Free Replication — How CRDTs Work
- Chapter 10 — Designing a Distributed Job Scheduler
- Chapter 11 — Designing for Correctness — How ACID Transactions Work
- Chapter 12 — Designing a Ride-Hailing Marketplace (Uber / Lyft)

Providence Salumu

First Edition · September 2026 · A living document — chapters are added incrementally

The System Design Interview Playbook

Volume I · First Edition

Written and typeset by **Providence Salumu**

Composed in Typst · Body set in Noto Serif · Display set in Noto Sans · Code set in Noto Sans Mono

REVISION HISTORY

v0.1	Chapter 1 — Real-Time Collaborative Text Editor	2026-09-05
v0.2	Chapter 2 — Maps & Navigation Service (Google Maps)	2026-09-05
v0.3	Chapter 3 — Distributed Rate Limiter	2026-09-05
v0.4	Chapter 4 — Distributed Logging & Metrics Platform	2026-09-05
v0.5	Chapter 5 — Hierarchical Comment System (Reddit)	2026-09-05
v0.6	Chapter 6 — Top-K Leaderboard (Gaming)	2026-09-05
v0.7	Chapter 7 — Recommendation Engine (YouTube / TikTok)	2026-09-05
v0.8	Chapter 8 — Database Indexes: How B-Trees Work	2026-09-05
v0.9	Chapter 9 — Conflict-Free Replication: How CRDTs Work	2026-09-06
v0.10	Chapter 10 — Distributed Job Scheduler	2026-09-06
v0.11	Chapter 11 — ACID Database Transactions	2026-09-06
v0.12	Chapter 12 — Ride-Hailing Marketplace (Uber / Lyft)	2026-09-06

© 2026 Providence Salumu. This document grows chapter by chapter;
each chapter is a self-contained walkthrough of one interview problem.

Contents

Preface	9
Conventions used in this book	9
How this book grows	10
1 Designing a Real-Time Collaborative Text Editor	11
1.1 The Problem Statement	11
1.2 Scope & Clarifying Questions	12
1.3 Functional Requirements	13
1.4 Non-Functional Requirements	14
1.5 Back-of-the-Envelope Estimation	15
1.6 The Core Challenge: Concurrent Editing	16
1.7 Version Vectors: Tracking What Each Replica Has Seen	21
1.8 API & Protocol Design	22
1.9 Data Model & Storage	25
1.10 High-Level Architecture	26
1.11 Deep Dive: The Edit Pipeline, Step by Step	27
1.12 Deep Dive: Snapshots, Catch-Up & the Two-Phase Lesson	29
1.13 Deep Dive: Rust Reference Implementation	30
1.14 Deep Dive: Presence & Live Cursors	34
1.15 Scaling & Sharding	34
1.16 Failure Modes & Recovery	35
1.17 Trade-offs & Alternatives	36
1.18 Observability & SLOs	36
1.19 Interview Wrap-Up	37
1.20 Summary & Further Reading	38
1.21 Chapter 1 Glossary	38
2 Designing a Maps & Navigation Service	40
2.1 The Problem Statement	40
2.2 Scope & Clarifying Questions	41
2.3 Functional Requirements	42
2.4 Non-Functional Requirements	42
2.5 Back-of-the-Envelope Estimation	43
2.6 Core Challenge I: Serving the Map	44
2.7 Core Challenge II: Modeling the Road Network	46
2.8 Core Challenge III: Shortest Paths at Planetary Scale	47
2.9 API & Protocol Design	50
2.10 Data Model & Storage	52
2.11 High-Level Architecture	53

2.12 Deep Dive: The Live Traffic Loop	55
2.13 Deep Dive: The Turn-by-Turn Navigation Session	57
2.14 Deep Dive: Rust Reference Implementations	57
2.15 Scaling & Geographic Sharding	65
2.16 Failure Modes & Recovery	65
2.17 Trade-offs & Alternatives	66
2.18 Observability & SLOs	67
2.19 Interview Wrap-Up	67
2.20 Summary & Further Reading	68
2.21 Chapter 2 Glossary	69
3 Designing a Distributed Rate Limiter	71
3.1 The Problem Statement	71
3.2 Scope & Clarifying Questions	72
3.3 Functional Requirements	73
3.4 Non-Functional Requirements	74
3.5 Back-of-the-Envelope Estimation	75
3.6 The Core Challenge: Counting Correctly Under Concurrency	76
3.7 The Algorithm Zoo	77
3.8 API & Protocol Design	80
3.9 Data Model & Storage	82
3.10 High-Level Architecture	82
3.11 Deep Dive: The Token Bucket, Precisely	84
3.12 Deep Dive: Distributed Enforcement & the Race	86
3.13 Deep Dive: Rust Reference Implementations	87
3.14 Scaling & Sharding	93
3.15 Failure Modes & Recovery	93
3.16 Trade-offs & Alternatives	95
3.17 Observability & SLOs	95
3.18 Interview Wrap-Up	96
3.19 Summary & Further Reading	96
3.20 Chapter 3 Glossary	97
4 Designing a Logging & Metrics Platform	99
4.1 The Problem Statement	99
4.2 Scope & Clarifying Questions	100
4.3 Functional Requirements	101
4.4 Non-Functional Requirements	102
4.5 Back-of-the-Envelope Estimation	103
4.6 The Core Challenge: One Firehose, Two Shapes of Data	104
4.7 Data Ingestion: Agents, Push vs. Pull, and the Buffer	104
4.8 Log Storage: The Inverted Index	106
4.9 Metrics Storage: The Time-Series Database	107

4.10 API & Query Design	108
4.11 High-Level Architecture	109
4.12 Deep Dive: The Alerting Engine	111
4.13 Rust Reference Implementations	112
4.14 Scaling the Platform	120
4.15 Failure Modes & Degradation	121
4.16 Trade-offs & Alternatives	123
4.17 SLOs & Meta-Observability	124
4.18 Interview Wrap-Up	125
4.19 Summary & Further Reading	125
4.20 Chapter Glossary	126
5 Designing a Hierarchical Comment System	128
5.1 The Problem Statement	128
5.2 Scope & Clarifying Questions	129
5.3 Functional Requirements	130
5.4 Non-Functional Requirements	130
5.5 Back-of-the-Envelope Estimation	132
5.6 The Core Challenge: Trees and Honest Ordering	133
5.7 Data Model: Encoding Comment Trees	133
5.8 Deep Dive: The Ranking Mathematics	135
5.9 API Design	137
5.10 High-Level Architecture	137
5.11 Deep Dive: The Vote Pipeline	139
5.12 Deep Dive: Reading a Thread Window	141
5.13 Rust Reference Implementations	142
5.14 Scaling the Platform	149
5.15 Failure Modes & Degradation	150
5.16 Trade-offs & Alternatives	151
5.17 Observability & SLOs	152
5.18 Interview Wrap-Up	152
5.19 Summary & Further Reading	153
5.20 Chapter Glossary	153
6 Designing a Top-K Leaderboard	155
6.1 The Problem Statement	155
6.2 Scope & Clarifying Questions	156
6.3 Functional Requirements	157
6.4 Non-Functional Requirements	157
6.5 Back-of-the-Envelope Estimation	158
6.6 The Core Challenge: Rank Is Not a Lookup	159
6.7 Deep Dive: The Data Structure — Skip List with Spans	160
6.8 API Design	162

6.9 High-Level Architecture	162
6.10 Deep Dive: Score Ingestion Semantics	164
6.11 Deep Dive: Sharding and the Global Top-K	165
6.12 Deep Dive: Windowed and Friends Boards	166
6.13 Rust Reference Implementations	167
6.14 Scaling the Platform	173
6.15 Failure Modes & Degradation	174
6.16 Trade-offs & Alternatives	175
6.17 Observability & SLOs	176
6.18 Interview Wrap-Up	176
6.19 Summary & Further Reading	177
6.20 Chapter Glossary	177
7 Designing a Recommendation Engine	179
7.1 The Problem Statement	179
7.2 Scope & Clarifying Questions	180
7.3 Functional Requirements	181
7.4 Non-Functional Requirements	182
7.5 Back-of-the-Envelope Estimation	183
7.6 The Core Challenge: The Funnel	184
7.7 Deep Dive: Candidate Generation — Recall at 50 ms	184
7.8 Deep Dive: Ranking — Precision at 150 ms	186
7.9 Deep Dive: Re-Ranking — The Last 50 ms	187
7.10 API Design	187
7.11 High-Level Architecture	188
7.12 Training vs. Serving: The Two-Factory Split	190
7.13 Rust Reference Implementations	190
7.14 Scaling the Platform	197
7.15 Failure Modes & Degradation	198
7.16 Trade-offs & Alternatives	199
7.17 Observability & SLOs	200
7.18 Interview Wrap-Up	201
7.19 Summary & Further Reading	201
7.20 Chapter Glossary	202
8 Designing a Database Index: How B-Trees Work	204
8.1 The Problem Statement	204
8.2 Scope & Clarifying Questions	205
8.3 Functional Requirements	206
8.4 Non-Functional Requirements	206
8.5 Back-of-the-Envelope: The Physics and the Height Math	207
8.6 The Core Challenge: One Structure for Reads and Writes	208
8.7 Candidate Structures	209

8.8 Deep Dive: B-Tree Mechanics	209
8.9 Deep Dive: The B+ Tree Refinements	211
8.10 The Write Path: Buffer Pool and WAL	211
8.11 The Rival: LSM Trees	212
8.12 The Storage Engine's Interface	213
8.13 Rust Reference Implementations	214
8.14 Scaling the Structure	221
8.15 Failure Modes & Degradation	221
8.16 Trade-offs & Alternatives	222
8.17 Observability & SLOs	223
8.18 Interview Wrap-Up	223
8.19 Summary & Further Reading	224
8.20 Chapter Glossary	224
9 Designing Conflict-Free Replication: How CRDTs Work	226
9.1 The Problem Statement	226
9.2 Scope & Clarifying Questions	227
9.3 Functional Requirements	228
9.4 Non-Functional Requirements	229
9.5 Back-of-the-Envelope: Metadata, Tombstones, and Gossip	230
9.6 The Core Challenge: Agreement Without a Coordinator	231
9.7 Deep Dive: The Mathematics of Convergence	232
9.8 Deep Dive: The CRDT Catalog	233
9.9 Deep Dive: Causality — Vector Clocks and Dots	235
9.10 Deep Dive: Tombstones, Deltas, and Anti-Entropy	236
9.11 Deep Dive: Sequence CRDTs — Back to Chapter 1	237
9.12 API Design	239
9.13 High-Level Architecture	240
9.14 Rust Reference Implementations	241
9.15 Scaling the Design	248
9.16 Failure Modes & Degradation	248
9.17 Trade-offs & Alternatives	249
9.18 Observability & SLOs	250
9.19 Interview Wrap-Up	251
9.20 Summary & Further Reading	252
9.21 Chapter Glossary	252
10 Designing a Distributed Job Scheduler	255
10.1 The Problem Statement	255
10.2 Scope & Clarifying Questions	256
10.3 Functional Requirements	257
10.4 Non-Functional Requirements	258
10.5 Back-of-the-Envelope: Timers, Bursts, and Workers	258

10.6	The Core Challenge: “Exactly Once” Is a Lie We Tell Well	259
10.7	Deep Dive: The Trigger Side — Finding What’s Due	260
10.8	Deep Dive: The Firing Side — Leases, Fencing, Idempotency	261
10.9	Deep Dive: Retries, Backoff, Jitter, and the Dead-Letter Queue	263
10.10	Deep Dive: Workflows — DAGs of Jobs	264
10.11	API Design	264
10.12	High-Level Architecture	265
10.13	Rust Reference Implementations	267
10.14	Scaling the Design	274
10.15	Failure Modes & Degradation	274
10.16	Trade-offs & Alternatives	275
10.17	Observability & SLOs	277
10.18	Interview Wrap-Up	277
10.19	Summary & Further Reading	278
10.20	Chapter Glossary	279
11	Designing for Correctness: How ACID Transactions Work	281
11.1	The Problem Statement	281
11.2	Scope & Clarifying Questions	282
11.3	Functional Requirements	283
11.4	Non-Functional Requirements	283
11.5	Back-of-the-Envelope: The Cost of the Four Letters	283
11.6	The Core Challenge: Two Enemies, One Log, One Schedule	284
11.7	Deep Dive: Atomicity & Durability — The Write-Ahead Log	284
11.8	Deep Dive: Isolation Levels and the Anomaly Zoo	285
11.9	Deep Dive: Pessimistic Concurrency Control — Two-Phase Locking	286
11.10	Deep Dive: Optimistic Control and MVCC	287
11.11	Deep Dive: The Distributed Boundary — 2PC and Sagas	288
11.12	API Design	288
11.13	High-Level Architecture	289
11.14	Rust Reference Implementations	289
11.15	Scaling the Design	296
11.16	Failure Modes & Degradation	297
11.17	Trade-offs & Alternatives	298
11.18	Observability & SLOs	298
11.19	Interview Wrap-Up	299
11.20	Summary & Further Reading	299
11.21	Chapter Glossary	300
12	Designing a Ride-Hailing Marketplace (Uber / Lyft)	302
12.1	The Problem Statement	302
12.2	Scope & Clarifying Questions	303
12.3	Functional Requirements	303

12.4 Non-Functional Requirements	304
12.5 Back-of-the-Envelope: A Million Moving Dots	304
12.6 The Core Challenge: A Marketplace That Refuses to Sit Still	304
12.7 Deep Dive: Geospatial Indexing for Moving Points	305
12.8 Deep Dive: The Matching Engine	306
12.9 Deep Dive: The Trip Backbone — State Machine and Money	306
12.10 Deep Dive: Surge Pricing as a Control Loop	307
12.11 API Design	307
12.12 High-Level Design	308
12.13 Rust Reference Implementations	309
12.14 Scaling	314
12.15 Failure Modes	314
12.16 Trade-offs	315
12.17 Observability and SLOs	315
12.18 Interview Wrap-Up	316
12.19 Summary and Further Reading	316
12.20 Chapter Glossary	317

Preface

This book exists because system design interviews reward a very specific skill that ordinary engineering work rarely exercises in full: *taking an ambiguous, enormous problem and shrinking it, in real time and out loud, into a coherent, defensible architecture*. Knowing how Netflix or Google Docs works is not enough; you must be able to **derive** such a design from first principles while an interviewer watches you think.

DEFINITION — System design interview

An interview format, common for senior engineering roles, in which the candidate is asked to design a large-scale software system (for example, “design YouTube” or “design a URL shortener”) within 45–60 minutes. There is no single correct answer. The interviewer evaluates how you clarify ambiguity, structure the problem, justify trade-offs, and reason about scale, failure, and consistency.

Each chapter of this book is a complete, self-contained walkthrough of one such problem. Every chapter follows the same battle-tested structure, which mirrors the natural arc of a strong interview performance:

THE CHAPTER FRAMEWORK

1. **Problem statement** — the prompt exactly as an interviewer would pose it.
2. **Clarifying questions & scope** — shrinking an ambiguous prompt into a concrete target.
3. **Functional requirements** — what the system must *do*.
4. **Non-functional requirements** — how *well* it must do it (scale, latency, availability).
5. **Back-of-the-envelope estimation** — arithmetic that sizes the problem before designing.
6. **The core challenge** — the one or two genuinely hard ideas this problem tests.
7. **API & protocol design** — the contract between clients and servers.
8. **Data model & storage** — what we persist, where, and why.
9. **High-level architecture** — the boxes and the arrows, with every box justified.
10. **Deep dives** — the mechanisms, including complete Rust reference implementations.
11. **Trade-offs & alternatives** — what we gave up, and what we would do differently.
12. **Wrap-up** — likely follow-up questions, common pitfalls, and an interview checklist.

Conventions used in this book

Three conventions deserve explicit mention.

Terms are defined before they are used. System design is full of overloaded vocabulary — “consistency”, “snapshot”, “version vector” — and interviews are lost when a candidate uses a term they cannot define. Whenever a new concept appears, it is introduced in a **Definition** box before the surrounding text relies on it.

All code is written in Rust. Rust's explicit ownership and type system make concurrency logic — the hardest part of most designs — precise and auditable. The listings are not toys: they are faithful, compilable sketches of the real algorithms, with tests.

Numbers are always justified. Every estimate states its assumptions. A number without assumptions is decoration; a number with assumptions is engineering.

How this book grows

This is a living document. New chapters are appended over time, each tackling one new problem. The framework above is fixed, so you always know where to find the requirements, the math, the architecture, and the code — regardless of the problem.

Providence Salumu — September 2026



Designing a Real-Time Collaborative Text Editor

PROBLEM SOURCE

This chapter solves the problem posed in the talk [“12: Design Google Docs / Real Time Text Editor”](#) from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 47 min). The talk walks the problem in mock-interview form — both a high-level design (HLD) and a low-level design (LLD) of the concurrency machinery. This chapter follows the same arc, but slows every step down: full definitions before first use, capacity mathematics with every assumption stated, protocol specifications, and Rust reference implementations you could compile tomorrow.

1.1 The Problem Statement

Picture the scene. The interviewer finishes small talk, leans back, and says:

“Design a real-time collaborative text editor — something like Google Docs. Multiple people should be able to edit the same document at the same time and see each other’s changes as they happen.”

If you have never thought about this problem before, your first instinct might be that it is a CRUD application with a chatty front end: store documents, let people edit them, save often. That instinct survives for about thirty seconds — right up to the moment you ask yourself the question the whole chapter hangs on: *when two people type at the same time, in different places, on different continents, what is the document now?* There is no obviously correct answer, and several plausible answers are wrong in ways that lose user data. Underneath its ordinary surface, this prompt is a distributed-state synchronization problem — one of the richest in the entire interview canon.

Notice what the prompt does **not** tell you. How many users? How many editors per document? Plain text or rich text? What happens when someone loses connectivity? Is there version history? Every one of those omissions is deliberate: the interviewer is watching to see whether you discover the ambiguity and close it, or whether you charge ahead and design whatever product happens to live in your head. Closing those gaps — politely, out loud, and with reasons — is most of your grade.

DEFINITION — High-level design (HLD) / low-level design (LLD)

HLD is the architecture of the whole system: which services exist, which data stores they use, which network paths connect them, and — most importantly — *why* each piece earned its place. *LLD* is the internal mechanics of the one or two components where the real difficulty lives: the data structures, the algorithms, the exact protocols. A strong interview performance does HLD first, in breadth, and then drills into LLD where the problem is genuinely hard. In this problem the hard component is the

concurrency-control core — the machinery that decides what the document is when edits collide — so that is where we will spend our LLD budget.

Here is the roadmap we will follow together. First we negotiate scope, because designing the wrong product elegantly is still designing the wrong product. Then we pin down functional and non-functional requirements, and do the arithmetic that tells us where the scale actually lives. Only then do we open the core challenge — concurrent editing — and give it the several sections it deserves, ending in working Rust. Architecture, protocols, storage, failure handling, and trade-offs follow from that core, not the other way around. That order is itself a lesson: in this problem, correctness considerations drive the architecture, so we let them.

1.2 Scope & Clarifying Questions

There is a saying worth internalizing before any interview: *never design the prompt you were handed; design the prompt you negotiated*. The series this chapter draws on runs its interviews as a dialogue, and that is exactly how a real interview should feel to you — not an interrogation, but two engineers scoping a project together. Watch how a strong opening exchange goes, and pay attention not just to the questions but to what each one **buys**:

Speaker	Dialogue
Candidate	“Are we building plain text editing, or rich text — formatting, embedded images, tables?”
Interviewer	“Plain text is fine. Assume a document is an ordered sequence of characters.”
Candidate	“How many collaborators can edit one document simultaneously?”
Interviewer	“Up to 50 concurrent editors per document.”
Candidate	“And the total scale — how many users and documents are we designing for?”
Interviewer	“10 million daily active users. Documents themselves can be up to a few hundred kilobytes of text.”
Candidate	“Do users need to see each other’s cursors and selections live — presence?”
Interviewer	“Yes, cursors and names, like Google Docs shows.”
Candidate	“Is offline editing in scope, or can we assume clients are connected while editing?”
Interviewer	“Assume connected for the core design; if time remains, sketch how offline would work.”
Candidate	“Do we need version history — the ability to view and restore older revisions?”
Interviewer	“Yes, keep a full history of changes.”

Let us walk through why these particular six questions, because the selection is not arbitrary — each one collapses a different axis of the design space.

The *plain text versus rich text* question decides how we model a document. Plain text lets us say “a document is an ordered sequence of characters,” which means an edit can be described by a position and a character — a tiny, clean model. Rich text would force us to carry formatting ranges, embedded objects, and nested structure through every algorithm we build; the concurrency math stays conceptually identical but the bookkeeping triples. Getting plain text granted is the single biggest simplification available in this interview, so ask for it first.

The *editors-per-document* question is quietly the most important number in the chapter. Fifty concurrent editors means one document's write traffic is trivial — a few hundred operations per second in a typing frenzy — which means a **single server** can own a whole document and serialize its edits. Had the answer been “fifty thousand,” every design decision that follows would invert. Always ask for the per-entity number, not just the global one.

The *total scale* question (10M daily users, documents up to a few hundred kilobytes) feeds the capacity estimation in Section 1.5. The *presence* question adds a real-time broadcast feature with a very different tolerance profile from edits — we will exploit that difference later. The *offline* question probes the deepest axis of all: a system that must merge hours of disconnected work needs a different concurrency mechanism at its heart, so getting “assume connected” is a license for the mainstream design. And the *version history* question commits us to storing not just the document but its entire evolution — which, as you will see, turns out to be nearly free once we choose the right storage model.

From this exchange we can freeze the scope. Everything later in the chapter traces back to these decisions, so state them explicitly — literally summarize them back, as the box below does — and get a nod from the interviewer before moving on.

AGREED SCOPE

Plain-text documents up to 500 KB; up to **50 concurrent editors per document**; **10M daily active users**; live cursors and presence; full version history; connected editing for the core design (offline sketched as an extension).

INTERVIEW TIP — Negotiate scope in both directions

Candidates usually think clarifying questions only **shrink** scope. Used well, they also **expand** it — deliberately. Asking “do we need presence?” signals that you know the feature exists, that it costs something, and that you can build it if wanted. Offer features and let the interviewer decline them; you get credit for knowing the product space, and you protect yourself from discovering a hidden requirement at minute forty.

1.3 Functional Requirements

Before listing what our system must do, let us be precise about what kind of statement a functional requirement even is — interviews are quietly lost by people who mix the two requirement families and end up with a muddled design.

DEFINITION — Functional requirement (FR)

A statement of what the system must **do** — an observable behavior a user can, in principle, verify: “a user can create a document,” “a user sees another user's cursor.” Functional requirements define the feature set. Crucially, they say nothing about how fast, how available, or how consistent the system must be — those qualities are *non-functional requirements*, which we define next, and which are where the architecture actually comes from.

Here are our functional requirements, scoped by the negotiation above. As you read each one, notice that it is testable — you could write an end-to-end test for it without knowing anything about the implementation:

1. **FR-1 — Document management.** Users can create, rename, open, and delete documents. This is ordinary CRUD, and we say so out loud: it tells the interviewer we know which parts of the system are boring and which are not.

2. **FR-2 — Real-time collaborative editing.** Multiple users can edit one document concurrently, and every connected editor sees every other editor's changes within a fraction of a second. This is the requirement the whole chapter serves.
3. **FR-3 — Presence.** Each editor sees who else is in the document, with a live cursor (and selection) per collaborator, labeled with their name. Presence is real-time like FR-2, but unlike FR-2 it tolerates loss and delay — remember that asymmetry; we will design for it explicitly.
4. **FR-4 — Persistence.** A document is never lost because a client disconnects; reopening it later yields its latest committed state. Combined with FR-2, this means every edit must be made durable **while** the real-time dance continues.
5. **FR-5 — Version history.** Users can browse the full history of a document and view or restore any earlier revision. This requirement will end up shaping our storage model more than any other.
6. **FR-6 — Sharing & access control.** A document has an owner; the owner grants view or edit access to others via a shareable link or explicit invite.

And just as important, say what we are **not** building: rich text, comments and suggestions, end-to-end encryption, and offline-first editing as a core flow. Naming exclusions does two things for you: it protects your fifty minutes, and it demonstrates that you know where the boundary of a problem lies — a senior habit.

1.4 Non-Functional Requirements

DEFINITION — Non-functional requirement (NFR)

A statement of how **well** the system must perform its functions: scale, latency, availability, durability, consistency. NFRs are where design decisions actually come from, and this is worth pausing on: two systems with identical functional requirements — say, a chat app and a payment ledger, both of which “store and deliver messages” — can have opposite architectures purely because their NFRs differ. When you state an NFR in an interview, you are really stating which trade-offs you are about to make.

Three qualities dominate this particular problem, and each deserves a precise definition before we use it as a design tool:

DEFINITION — Latency

The time between a cause and its observable effect. Two distinct latencies matter here, and keeping them separate is essential. *Edit latency* is the time from your keystroke to that character appearing in **your own** document — it must feel instant, which in practice means under 16 ms (one display frame), which in turn means it must be applied locally, with no network round trip in the way. *Propagation latency* is the time from your keystroke to its appearance on a **collaborator's** screen — our target is under 150 ms within a region, fast enough that collaboration feels live rather than turn-based.

DEFINITION — Availability

The fraction of time the system is able to serve requests, usually quoted in “nines.” Editing must remain available even when individual servers fail — which, at our scale, they do constantly — so no single machine may be indispensable to the system as a whole. We will aim for the classic “three nines” (99.9%, about 8.8 hours of downtime per year) for the editing path, and we will see exactly which component that constrains.

DEFINITION — Consistency / convergence

In a system where many actors mutate shared state concurrently, *consistency* describes what guarantees observers get about the states they see. For a collaborative editor, the guarantee that matters is **convergence**: once the same set of edits has been delivered to every replica, every replica must hold the **identical document**. If two users could permanently end up looking at different text, the product is broken no matter how fast or how available it is. How convergence is achieved — crucially, **without** locking the document — is the core challenge of this chapter and gets its own section (Section 1.6).

The remaining NFRs, stated as concrete targets we can design against:

Quality	Target
Propagation latency	p50 < 150 ms within a region; p95 < 400 ms cross-region
Edit (local echo) latency	< 16 ms — edits apply locally, synchronously, before any server contact
Editors per document	Up to 50 concurrent, with no degradation
System scale	10M DAU; 1M simultaneous connections at peak
Durability	No committed edit is ever lost — history is the product's memory
Availability	99.9% for editing; reading a document should survive almost anything

KEY INSIGHT — Consistency is the NFR that shapes this design

Most interview problems are dominated by throughput (design Twitter) or storage (design Dropbox). This one is dominated by **correctness under concurrency**. The moment two users type at the same time, you owe the world a mathematical answer to “what is the document now?” — and that answer, not any load number, is what decides the architecture. Section 1.6 is where this interview is won or lost; everything else is infrastructure in service of it.

1.5 Back-of-the-Envelope Estimation

Before drawing a single box, we count. This surprises many candidates the first time — surely architecture comes before arithmetic? — but the arithmetic **decides** the architecture, so it must come first.

DEFINITION — Back-of-the-envelope estimation

Rapid, approximate arithmetic — round numbers, stated assumptions — that sizes a system **before** you design it. Precision is not the goal; nobody cares whether you write 220 GB or 550 GB. The goal is to discover **which constraints bite**: a design for 500 messages per second and a design for 500,000 messages per second are different systems, and you cannot know which one you owe the interviewer until you count.

DEFINITION — DAU / QPS

DAU (daily active users) counts the distinct users active in a day — a business number. *QPS* (queries per second) is the request rate a system actually handles — an engineering number. Interview estimates are mostly the art of converting the first into the second via an activity model: “each user does X, about Y times a day, so the average rate is Z, and peak is some multiple of average.”

Assumptions. Say them out loud, write them on the board, and explicitly invite correction — “does that sound right to you?” is a collaboration signal, not a weakness:

- 10M DAU; each user actively edits on 3 documents per day, in sessions of 20 minutes.
- At peak, 5% of DAU are simultaneously connected: **500k concurrent connections**. We will design for 1M, because capacity you do not need is cheaper than capacity you do not have.
- An active typist produces 3 keystroke-operations per second in bursts, but averaged over a whole session — reading, thinking, chatting, coffee — about 0.5 ops/sec per connected user.
- One operation (a character insert or delete plus its metadata) is about 250 bytes on the wire.
- Average document size is 20 KB of text; 500 KB worst case per our scope.

Derived numbers — each line follows mechanically from the assumptions:

Quantity	Estimate	How
Peak edit operations	$\approx 250\text{k ops/sec}$	$500\text{k conns} \times 0.5\text{ ops/s}$
Edit ingress bandwidth	$\approx 60\text{ MB/s}$	$250\text{k ops/s} \times 250\text{ B}$
Operations stored per day	$\approx 2.2\text{B}$	$25\text{k ops/s avg} \times 86,400\text{ s}$
Operation log growth	$\approx 550\text{ GB/day}$	$2.2\text{B ops} \times 250\text{ B}$
New documents per day	$\approx 3\text{M}$	$10\text{M users} \times 0.3$
Doc-open QPS (REST)	$\approx 600\text{ avg} / 6\text{k peak}$	$10\text{M} \times 5\text{ opens/day} \div 86,400, \times 10\text{ peak factor}$
Connections per WebSocket gateway	50k	commodity servers hold tens of thousands of mostly-idle connections
Gateways needed	20–40	$1\text{M conns} \div 25\text{--}50\text{k each, plus redundancy}$

Now — and this is the part that actually earns the points — read the table back and ask what it is **telling** you.

KEY INSIGHT — What the math tells us

Two facts jump out of these numbers, and between them they dictate the whole architecture. First: the system-wide write rate, hundreds of thousands of operations per second, sounds intimidating — but it is **perfectly sharded by document**. No document ever sees more than its 50 editors, producing maybe 150 ops/sec in a frantic burst. That is trivial load for a single thread, which means the correctness-critical logic can live on one node per document with room to spare. Second: the place where scale genuinely lives is the connection tier — a million mostly-idle WebSocket connections is a pure capacity problem, solved by piling up commodity boxes. So the conclusion writes itself: put the hard, stateful correctness logic **per document** (where throughput is tiny), and make the connection tier **stateless and horizontal** (where throughput is huge). Hold that thought; Section 1.10 turns it into boxes and arrows.

1.6 The Core Challenge: Concurrent Editing

Everything else in this chapter — gateways, storage, protocols — is standard distributed-systems plumbing that you will reuse in a dozen other designs. This section is the intellectual center of the problem, and of the source talk, and we will take it slowly: first we watch the naive approaches fail, then we define precisely what “correct” even means, and only then do we meet the two algorithm families that achieve it.

1.6.1 Why naive approaches fail

To see the difficulty, you need exactly two users and two characters. Suppose Alice and Bob are editing the document "GO" — G at index 0, O at index 1 — and they type **at the same time**, each seeing only the original:

- **Alice** inserts the character A at index 0; she is turning "GO" into "AGO".
- **Bob** deletes the character at index 1; he is turning "GO" into "G".

What should the document be once both edits have been delivered everywhere? Neither user did anything wrong, and their intentions do not conflict in spirit: the only defensible answer is "AG" — Alice's A before the G, and the O gone. Hold that answer in mind, and now watch two strategies you might reach for first both fail to produce it.

Naive strategy 1 — lock the document. Only one editor may type at a time; everyone else waits for the lock. Is it correct? Absolutely — mutual exclusion serializes everything, so no conflict can ever occur. Is it acceptable? Never: you have designed Google Docs with a “please wait your turn” dialog. Real-time collaboration is precisely the absence of global mutual exclusion, so this “correct” answer fails the product.

DEFINITION — Mutual exclusion / locking

A concurrency-control technique in which a resource may be modified by only one actor at a time; everyone else blocks until the lock is released. Locks are the oldest correct answer to shared state, and they work by **serializing** — trading parallelism for safety. That trade is often right (database internals are full of locks, as Chapter 11 will show), but it is fatal when the resource is a shared document with fifty simultaneous editors and the whole point is that nobody ever waits.

Naive strategy 2 — last-writer-wins. Attach a timestamp to each edit, and when edits collide, keep the latest and discard the rest. This is a real, widely used strategy — just not here.

DEFINITION — Last-writer-wins (LWW)

A conflict-resolution rule: when two writes to the same datum race, keep the one with the later timestamp and drop the other. It is simple, fast, and **loses data by design** — one of the two writes silently vanishes. LWW is a fine choice for values that are legitimately overwritten as wholes (your profile picture, a “currently online” flag), and a terrible choice for anything that should be **merged**, like text.

Apply LWW to our example and the failure is concrete: if Bob's delete “wins,” Alice's A disappears from the document she watched herself type. If Alice's insert “wins,” Bob's delete is lost and the O he deleted resurrects. Either way, one user's clear intention is silently destroyed, and the product is untrustworthy.

There is a third failure mode worth seeing, because it is subtler and it is the one that motivates the real solution. Suppose you skip conflict resolution entirely and just **apply** each edit at the position it names. Alice's `insert(0, 'A')` reaches Bob and works fine. But Bob's `delete(1)` reaches Alice's machine **after** her insert, so her document is already "AGO" — and deleting the character at index 1 removes the G instead of the O, producing "AO", a document **neither** user intended. The positions themselves shifted under concurrency. Remember that observation; the fix is built on it.

1.6.2 What “correct” means

If the naive approaches fail, what would a correct approach have to guarantee? Three properties, each of which we can state precisely — and once stated precisely, they become testable.

DEFINITION — Causality

Event A *causally precedes* event B if A happened first **and could have influenced** B — for instance, B is an edit made by a user who had already seen A applied to their screen. Two events with no causal order in either direction are **concurrent**. Alice's and Bob's edits above are concurrent: neither had seen the other's. The deep point is that concurrency — not time — is what creates conflicts, and wall-clock timestamps cannot detect concurrency (two machines' clocks never agree well enough). That is why Section 1.7 introduces version vectors, which track causality directly.

DEFINITION — Convergence

A set of replicas converges if, once the **same set** of edits has been delivered to all of them, every replica holds the **identical** document — regardless of the order in which the edits arrived. Note that convergence is a property of the **algorithm**, not of luck or timing: it must hold for every possible interleaving, including the ugly ones that only happen at 3 a.m. during a deploy.

DEFINITION — Intention preservation

The effect of an edit, as observed by its author at the moment they made it, must survive concurrency with other edits. Alice meant “ A before G ”; Bob meant “ O is gone ”; the converged document “AG” honors both. This property is what rules out degenerate “solutions”: an algorithm could converge trivially by always producing the empty string — every replica agrees! — and be worthless. So we want convergence **with** intention preservation, and any algorithm we accept must demonstrate both.

With the target defined, we can go shopping for algorithms. Two families are in wide production use and deliver these properties: **Operational Transformation** and **Conflict-free Replicated Data Types**. A candidate who can define both, contrast them honestly, and implement one owns this interview.

1.6.3 Approach A — Operational Transformation (OT)

DEFINITION — Operational Transformation (OT)

A concurrency-control technique in which every edit is expressed as an **operation** — for plain text, `insert(position, char)` or `delete(position)` — and, whenever two operations are concurrent, one is **transformed** against the other: its position is adjusted so that it still does what its author meant on a document the other operation has already modified. Remember the “A0” failure above, where positions shifted under concurrency? Transformation is exactly the repair for that shift. OT is the machinery inside Google Docs, with a lineage running from the Jupiter system (1995) through Google Wave.

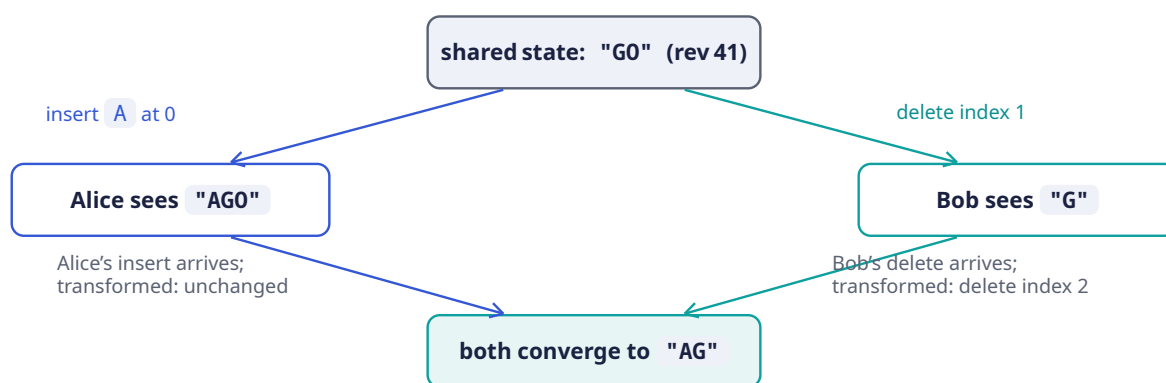
The transformation rules fit in a small matrix, and the matrix is the entire algorithm. To apply operation `a` on a replica where the concurrent operation `b` has already been applied, you transform `a` against `b` like this:

Pair	Rule for transforming <code>a</code> against already-applied <code>b</code>
<code>insert(p_a)</code> vs <code>insert(p_b)</code>	If $p_b < p_a$, shift <code>a</code> right by 1. If $p_b == p_a$, break the tie deterministically (e.g., by client ID) so every replica shifts the same way.
<code>insert(p_a)</code> vs <code>delete(p_b)</code>	If $p_b < p_a$, shift <code>a</code> left by 1. Otherwise unchanged.
<code>delete(p_a)</code> vs <code>insert(p_b)</code>	If $p_b \leq p_a$, shift <code>a</code> right by 1. Otherwise unchanged.

Pair	Rule for transforming a against already-applied b
<code>delete(p_a)</code> vs <code>delete(p_b)</code>	If $p_b < p_a$, shift <code>a</code> left by 1. If $p_b == p_a$, both deletes target the same character: <code>a</code> becomes a no-op.

Read one row slowly to be sure the idea lands. Take “`insert(p_a)` vs `delete(p_b)` : if $p_b < p_a$, shift `a` left by 1.” Why left? Because a character **before** your insert position has been removed, so every position after it moved one slot toward the start of the document. Your insert still means “after the same characters as before,” so its index must follow the shift. Every other row is the same idea in a different case: track what the already-applied operation did to the text, and adjust the incoming operation so its **meaning** survives.

The whole of OT is this matrix plus one discipline: **every replica applies the same operations in the same total order, transforming as needed**. Here is our running example drawn as the classic convergence diamond — both paths away from the concurrent state must land on the same document:



This little diamond repays a slow reading, because it **is** the problem and its solution in one picture. Start at the top: both users see the same document “G0” at revision 41. Now follow the left edge downward. Alice types first from her own point of view, so her machine applies her insert immediately (remember the 16 ms local-echo budget) and she sees “AG0”. Bob’s machine, on the right, does the symmetric thing: applies his delete locally and shows him “G”. At this instant the two screens genuinely disagree — that is unavoidable, and fine, because nobody has communicated yet.

Now the edits cross the network, and the bottom half of the diamond is where OT earns its keep. On the left, Alice’s machine receives Bob’s `delete(1)` — but her document is now “AG0”, where index 1 is the G, not the 0. So her client transforms Bob’s operation against her already-applied `insert(0, 'A')`: the insert happened at or before the delete position, shifting everything right, so the delete becomes `delete(2)`. Applied to “AG0”, that removes the 0 and yields “AG”. On the right, Bob’s machine receives Alice’s `insert(0, 'A')` and transforms it against his already-applied `delete(1)`: the insert position is before the delete, so the delete did not shift it, and the operation applies unchanged to “G”, yielding “AG”. Both paths land on the bottom box — same operations, same final document, both intentions preserved. When you can narrate this diamond fluently at a whiteboard, the interviewer knows the hardest concept in the chapter is safely yours.

NOTE — Where the total order comes from

OT has a hidden precondition: every replica must transform against the **same** concurrent history, or the matrices produce different results on different machines. The standard way to arrange that is a **single sequencer per document**: the server assigns every operation the next integer **revision** (42, 43, 44, ...), and that assignment is the total order. Clients send operations tagged with the revision they were based on; the server transforms each operation across any operations committed in between,

and only then broadcasts it. We make this precise in the deep dives (Sections 1.11 and 1.13), where you will see that the sequencer turns “distributed consensus” into a counter and a loop.

1.6.4 Approach B — Conflict-free Replicated Data Types (CRDTs)

Before the second approach makes sense, you need the consistency vocabulary it improves on.

DEFINITION — Eventual consistency

A consistency model in which replicas that have stopped receiving new writes will **eventually** reach the same state. It promises convergence, but says nothing about **when** convergence happens, and nothing about what intermediate states look like — a replica might serve nonsense for an unbounded window before settling. Weak, but often enough.

DEFINITION — Conflict-free Replicated Data Type (CRDT)

A data structure whose concurrent updates are merged by rules that are commutative (order does not matter), associative (grouping does not matter), and idempotent (duplicates do not matter). Those three properties together mean that replicas applying the same updates in **any** order, even with duplicates, provably converge. The upgraded guarantee is called **strong eventual consistency**: convergence as soon as updates are delivered, with no central ordering authority and no transformation step at all.

For text, the canonical CRDT trick is elegant: stop addressing characters by integer positions — positions are exactly what shifts under concurrency, the problem OT spends its matrix repairing — and instead give every character a **unique, immutable, orderable identifier** at birth. In the RGA family of algorithms, an insert is recorded as “insert this new character, with a fresh globally-unique ID, immediately after character ID X.” Because identifiers never move, nothing ever needs transforming: concurrent inserts after the same character interleave deterministically by ID order, and deletes merely mark a character as a **tombstone** — present in the structure, invisible in the text. Merging two replicas is set union, which is why any delivery order works. If this sounds appealing, it is — Chapter 9 is a full-length treatment of CRDTs, including an RGA text implementation in Rust, and it reads this chapter as “the road not taken.”

1.6.5 Choosing between them

Lay the two families side by side and the trade-off structure becomes visible — neither dominates; each buys something the other cannot cheaply provide:

Criterion	OT	CRDT
Message size	Tiny: position + char	Larger: globally-unique IDs per character
Memory / metadata	None beyond the text	ID + tombstone per character; tombstone growth needs garbage collection
Needs central sequencer	Yes (classic form)	No — tolerates any delivery order
Algorithmic subtlety	Transformation matrix + history; easy to get subtly wrong	Merge rules; conceptually cleaner, harder to keep compact
Offline / peer-to-peer	Awkward (needs the ordering authority)	Natural fit

Criterion	OT	CRDT
Used in production by	Google Docs, older Etherpad	Figma (variants), many local-first tools

Our decision: classic OT behind a per-document sequencer — the architecture the source talk describes, and the one Google Docs runs on. Walk through the reasoning the way you would out loud. The sequencer’s obvious cost is that it is a single point of **ordering** per document — but is it a bottleneck? Section 1.5 already answered that: a document peaks at 150 ops/sec, which a single thread laughs at. And what the sequencer buys is enormous: it collapses the hard problem into “transform against a linear log,” gives us version history almost for free (the log **is** the history), and is simple enough to implement exactly right. We accept the failover complexity of single-ownership, and we keep version vectors (next section) for the edges of the design — reconnect, catch-up, and the offline extension — where causal bookkeeping still matters.

INTERVIEW TIP — Name the trade-off out loud

Compare two candidates. The first says: “I’ll use OT because Google Docs does.” That earns nothing — it is an appeal to authority. The second says: “I’ll use OT with a per-document sequencer. That gives me small messages and a linear history that makes version history trivial; in exchange I accept a single ordering point per document, which I mitigate with lease-based failover.” Same design — but the second candidate has demonstrated the skill the interview actually measures. Every choice in this chapter is presented as a benefit purchased at a cost, and you should narrate yours the same way.

1.7 Version Vectors: Tracking What Each Replica Has Seen

The OT diamond resolves the moment of concurrency, but a running system faces a quieter book-keeping question every few seconds: **which operations has this particular client already received?** Think about when the question comes up. A client’s Wi-Fi drops for thirty seconds and reconnects — what should the server send it? A user opens a document after a week away — where do you start? An offline-editing extension needs to merge a laptop’s airplane edits with the server’s state — which operations does each side lack? All three are the same question in different clothes: compare “what you have seen” with “what exists.”

Your first instinct might be timestamps — “give me everything after 14:32:07.” We already know why that fails: clock skew. Two machines’ wall clocks routinely disagree by seconds, sometimes minutes, and correctness cannot rest on that. What we need is a **logical** notion of time — one that tracks “what caused what” instead of “when the clock said.”

DEFINITION — Logical clock / Lamport timestamp

A counter, maintained by each participant, that ticks on every local event and rides along on every message; a receiver merges it by taking the maximum and ticking on. Logical clocks order events by **causality** rather than wall-clock time — if A’s clock value is smaller than B’s and causality flowed between them, A happened first. This is exactly the notion of “before” that distributed state synchronization needs, and it costs nothing but a few bytes per message.

DEFINITION — Version vector

A Lamport clock generalized to many writers: a map from participant ID to a counter, where {Alice: 3, Bob: 1} reads as “I have seen 3 events from Alice and 1 from Bob.” Two version vectors compare pointwise, and the comparison has exactly four outcomes. V_1 **dominates** V_2 — equivalently, V_2 happened-before V_1 — if every entry of V_1 is greater than or equal to V_2 ’s and at least one is strictly

greater: V_1 has seen everything V_2 has, and more. If neither dominates, the states are **concurrent**: each has absorbed something the other has not. A version vector, in other words, is a compact summary of **precisely which causal history a replica has seen** — and that is exactly what catch-up logic needs to know.

Now, an honest simplification. In our design the server already assigns a single integer revision per document through the sequencer, so in the **steady state** the sync marker between server and client is just a scalar — “I am at rev 47.” You do not need a full vector when a total order exists; the integer is a complete causal summary. Version vectors earn their keep at the **edges** of the design, where total orders break down: when a reconnecting client’s state was captured against a snapshot rather than the live log, when replicas in different regions compare notes, and in the offline extension where a client accumulated edits no server saw. The comparison logic is identical in all three: if the server’s knowledge dominates the client’s, send exactly the difference; if concurrent, compute a two-sided diff.

COMMON PITFALL — The snapshot / version-vector trap

The source talk’s wry observation — a client “never received those writes on your local copy since your version vector was more up to date than the document snapshot” — describes a real and depressingly common production bug. Here is the scenario in slow motion. A snapshot of the document is written to one store, while the version metadata recording **which revisions that snapshot covers** is written to another, and the two writes are not atomic. A crash intervenes between them. Now a reconnecting client compares its version vector against **stale** snapshot metadata, concludes — correctly, given what it was told — that it is up to date, and is never sent the missing operations. The user sees a document silently behind everyone else’s, and no error ever fires. The cure is atomicity between a snapshot and its version metadata; Section 1.12 shows the cheap way to get it, and defines the two-phase commit protocol the talk name-drops for exactly this seam.

1.8 API & Protocol Design

With the correctness machinery understood, we can design the surface the system shows the world. The first decision is that there are **two** surfaces, and saying that out loud is itself worth credit:

- The **control plane** — create, rename, share, and open documents; fetch history. These are classic request/response operations: infrequent, latency-tolerant, and naturally stateless. Plain HTTPS REST is not merely adequate here, it is **correct** — anything fancier would be engineering theater.
- The **data plane** — the live editing session itself: join, send operations, receive operations, stream presence. This plane has opposite physics: a persistent, bidirectional, low-latency channel per client, held open for the whole session.

DEFINITION — REST

An API style layered on HTTP in which resources — nouns like `/documents/{id}` — are manipulated with a fixed vocabulary of methods — verbs: `GET`, `POST`, `PATCH`, `DELETE`. Its defining discipline is statelessness: each request carries everything the server needs to answer it, so any server can handle any request. That is precisely what makes REST ideal for our control plane, where we want ordinary horizontal scaling and no session affinity.

DEFINITION — WebSocket

A protocol (RFC 6455) that upgrades an ordinary HTTP connection into a long-lived, full-duplex channel: after an initial handshake, **either** side may send a message at **any** time, with only a few bytes of framing overhead per message. Full-duplex persistence is exactly what makes sub-150 ms propagation

possible — the server pushes an edit the instant it commits, with no client polling loop and no per-message connection setup.

Why not the older real-time mechanisms? Each is worth a sentence, because the interviewer may well ask:

Mechanism	Behavior	Verdict
Short polling	Client re-asks “anything new?” every few seconds	Latency measured in seconds; thousands of wasted requests — rejected
Long polling	Server holds each request open until data exists	Near-real-time, but a fresh HTTP request per message burst — heavy at our scale
Server-Sent Events	Server → client push over HTTP; client sends via separate requests	Workable, but asymmetric; WebSocket is cleaner for a two-way edit stream
WebSocket	One persistent, bidirectional connection	Chosen for the data plane

1.8.1 Control-plane endpoints

The control plane is deliberately unremarkable — you have seen a hundred APIs like it, and that is the point. A resource per noun, a verb per operation:

Method	Path	Purpose
POST	/documents	Create a document; returns its <code>doc_id</code>
GET	/documents/{id}	Fetch metadata + the latest committed revision number
PATCH	/documents/{id}	Rename, change settings
DELETE	/documents/{id}	Delete (owner only)
POST	/documents/{id}/share	Grant view/edit access to a user or link
GET	/documents/{id}/history?from=&to=	List revisions for the version-history UI
GET	/documents/{id}/snapshot?rev=	Fetch the document as of any revision (view/restore)

Two of these deserve a second look. `GET /documents/{id}` returns not just the document but its **current revision number** — the client needs that number to say `last_seen_rev` when it joins the live session, so the open-then-join sequence is how catch-up begins. And the `history` / `snapshot` pair is FR-5 made concrete: list revisions to render the timeline UI, then fetch the document **as of** any revision the user clicks.

1.8.2 Data-plane protocol (WebSocket messages)

After the REST `GET`, the client upgrades to a WebSocket and speaks a small JSON protocol. Every message carries a `type`; the important ones:

Message	Direction	Payload & semantics
join	client → server	{doc_id, auth_token, last_seen_rev} — enter the session; last_seen_rev is the catch-up handle
joined	server → client	{doc_id, rev, snapshot, collaborators[]} — current state, after any needed catch-up
op	client → server	{doc_id, base_rev, op, client_seq} — one edit, based on base_rev, deduplicated by client_seq
ack	server → client	{client_seq, rev} — “your edit is committed as revision rev”
op (broadcast)	server → client	{rev, author, op} — a committed edit to apply (transform locally if you have pending edits)
presence	both	{cursor, selection, name} — ephemeral, throttled, never persisted
sync	server → client	{rev} / snapshot — sent when the client is too far behind to patch with ops

A committed operation, on the wire, looks like this:

```
{
  "type": "op",
  "doc_id": "d_8f31c2",
  "rev": 47,
  "author": "alice@example.com",
  "op": { "kind": "insert", "pos": 0, "ch": "A" }
}
```

Two fields in this little protocol carry the entire correctness story, and interviewers probe both. The first is `base_rev` — the revision the client was looking at when it made the edit. That single number is how the server **detects** concurrency: if the document is at revision 52 and your operation says `base_rev: 49`, then revisions 50 through 52 were invisible to you, your edit is concurrent with them, and the server must transform it across those three revisions before committing (Section 1.11 walks the pipeline; Section 1.13 implements the loop). The second is `client_seq`, which makes the whole protocol survivable on real networks:

DEFINITION — Idempotency / idempotency key

An operation is *idempotent* if performing it twice has exactly the same effect as performing it once. Why you should care: real networks retry. Connections drop **after** the server processed your message but **before** you heard the acknowledgment, and the client’s only sane move is to send the operation again. A unique client-supplied key per operation — here, the pair (`client_id`, `client_seq`) — lets the server recognize the duplicate delivery and answer with the original result instead of applying the edit twice. This is the difference between at-least-once **delivery**, which networks give you whether you want it or not, and effectively exactly-once **effect**, which you must engineer. Chapter 10 builds an entire system on this idea.

1.9 Data Model & Storage

What do we actually persist? Four entities, each with one clear job — and notice, as you read the table, that each entity has a **different access pattern**, which is the fact that will choose our storage engines for us:

Entity	Contents	Store
Document	doc_id , owner, title, ACL, current rev	Metadata DB (relational or document store; sharded by doc_id)
Operation	doc_id , rev , author, transformed op payload, client_seq , timestamp	Operation log — append-only store, ordered by rev per document
Snapshot	doc_id , base_rev , full document text	Blob/object store; pointer + base_rev in metadata DB
Session	who is connected, cursors	Ephemeral presence cache — not persisted

Two of these storage ideas are the ones interviewers lean forward for, so let us give each a proper definition and — more importantly — the reasoning behind it.

DEFINITION — Operation log (event log)

The discipline of storing every change as an immutable, ordered, append-only record instead of overwriting state in place. The current state is always **derivable** — replay the log from the beginning and you rebuild the present. For us this single choice cascades beautifully: version history (FR-5) is free, because history is what we store; auditing is free, for the same reason; and crash recovery is just replay. There is even a performance dividend: appending to the end of a file is the fastest write a storage device can do — sequential I/O, no random updates, no in-place locking of hot rows.

DEFINITION — Snapshot

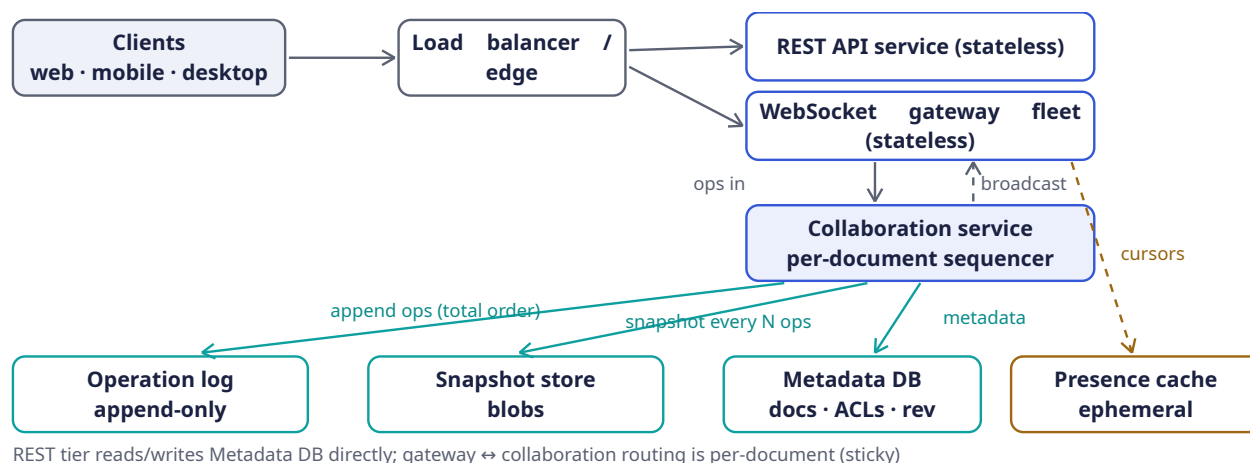
A materialized copy of the full document as of a specific revision. Pure logs have an embarrassing read problem: replaying two billion operations to open one document is absurd. So every N operations (say 500), we freeze the current text into a snapshot, and reconstruction becomes latest snapshot + the handful of operations since . Keep the hierarchy straight in your head and say it out loud in the interview: snapshots are a **read optimization** over the log, never the source of truth — the log is. You can delete every snapshot and lose nothing but time.

NOTE — Why three stores and not one database

Look back at the table and you will see the access patterns pulling in three directions. The metadata is small, hot, and transactional — it wants indexes, conditional updates, and joins, the home turf of a PostgreSQL-class store. The snapshots are large, immutable, and cold — they want cheap object storage, an S3-class blob store, where durability is high and nobody ever asks for a partial update. The operation log is append-only and strictly ordered per document — it wants a log-structured store, Kafka-class or Cassandra-class. Choosing one engine per access pattern, and being able to **say why each property matches**, is exactly the answer this section exists to elicit. Name the properties first and the brands second; the brands are replaceable, the reasoning is the answer.

1.10 High-Level Architecture

Now we can assemble the machine. Section 1.5 handed us the organizing principle — **a thin, stateless, horizontally scaled connection tier in front of a small, stateful, correctness-critical core** — and every box you are about to see exists to serve that principle. Here is the whole system at a glance; we will walk through it slowly right after.



Do not let the nine boxes blur together — each has one job, and the **shape** of the diagram is the argument. Read it the way traffic flows.

Start at the top left with the clients. Every client holds two logical connections: an ordinary HTTPS channel for control-plane work (creating a document, fetching history), and one long-lived WebSocket for the editing session. Both pass through the load balancer, whose only job is to spread connections across the stateless tiers behind it — it makes no per-document decisions and holds no per-document state, which is why it can be boring, redundant, and invisible.

The next row splits the two planes we designed in Section 1.8. The **REST API service** is a stateless request/response tier: any request can land on any instance, so scaling it means adding instances, the dullest kind of scaling there is. The **WebSocket gateway fleet** is where the million-connection problem from Section 1.5 lives: each gateway is a commodity box holding tens of thousands of mostly-idle connections, terminating TLS, and forwarding messages. It is stateless in the only sense that matters — it knows nothing about documents — so a client whose gateway dies simply reconnects to any other. We will never have to think hard about this tier again, and that is exactly why it exists: it absorbs the huge, dumb, connection-count load so the next tier does not have to.

That next tier — the **collaboration service**, drawn in blue because it is the heart of the design — is where every operation on a document converges on the single node that owns that document. Follow the solid arrow labeled “ops in”: gateways forward client operations here, routing by `doc_id`. The owning node sequences, transforms, and commits each operation (that is Section 1.11’s whole story), and then — the dashed “broadcast” arrow — pushes the committed result back to the gateways, which fan it out to every connected editor. One owner per document is the load-bearing decision of the entire architecture, so the insight box below is devoted to it.

The bottom row is storage, and it mirrors Section 1.9’s access patterns. Three teal arrows leave the collaboration node, one per store. The longest arrow, sweeping left to the **operation log**, is the write that matters most: every committed operation is appended here, in revision order, and this log is the source of truth — everything else can be rebuilt from it. The middle arrow, “snapshot every N ops,” is the read optimization: periodically the node freezes the document text into the blob store so that future opens and reconnects are $O(1)$ instead of $O(\text{history})$. The short arrow to the **metadata DB** keeps the small, hot, transactional records — documents, ACLs, and the current revision counter. And off to the right, drawn in amber and **dashed** on purpose, is the **presence cache**: cursors flow gateway-

to-cache-to-gateway without ever touching the sequencer or the log. The dashed styling is a design statement — presence is ephemeral, lossy, and throttled, and drawing it on the same solid lines as edits would claim a guarantee we deliberately do not provide.

Finally, the caption under the diagram records the one routing rule everything else depends on. Here it is again, with its full weight:

KEY INSIGHT — The one routing rule that makes it all work

All traffic for a given document flows through the single collaboration node that owns it. When a gateway receives an operation, it looks up `doc_id → owner` in a routing table — a cheap read — and forwards. Because every operation on a document passes through one owner, the owner's revision counter **is** a total order, and a total order is exactly what OT needs (Section 1.6). Ownership is not permanent: if a node dies, its lease expires and ownership migrates (Section 1.16). The invariant is only that there is **exactly one owner at a time**. Notice what this gives us: we never run a consensus protocol on the edit path at all. Consensus is what you need when many nodes must agree on an order; we sidestepped the need by construction.

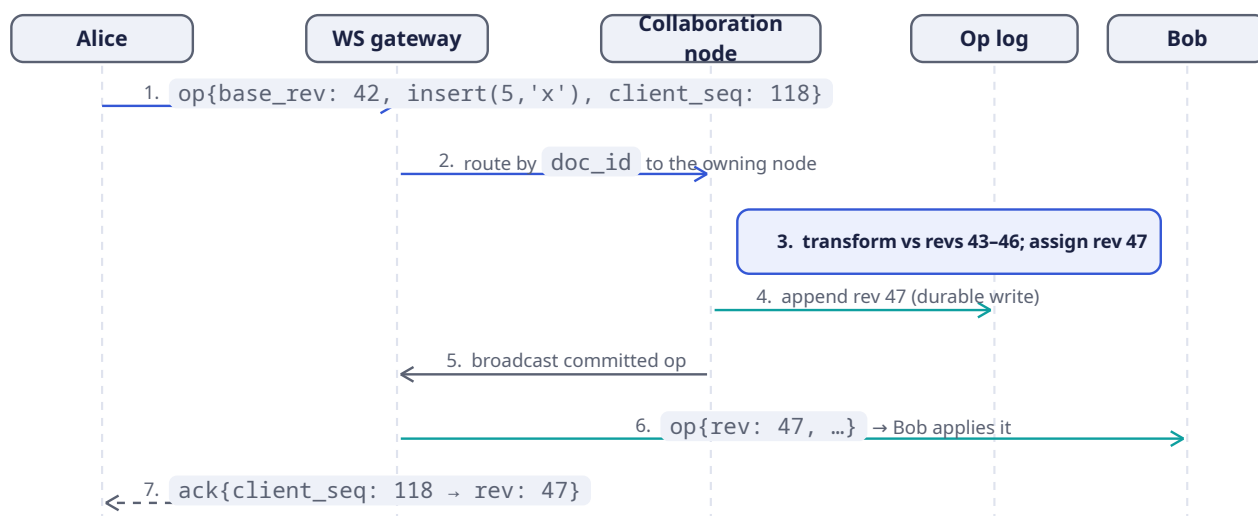
For completeness, the same responsibilities as a checklist — use it in the interview to make sure you have justified every box you drew:

Component	Responsibility	Why it is shaped this way
WebSocket gateway	Hold 50k client connections each; terminate TLS; forward messages	Stateless: any client can reconnect to any gateway. Scale by adding boxes — this tier absorbs the 1M-connection problem
Collaboration service	Owns each active document: sequences ops, transforms, commits, broadcasts	Stateful but sharded by document : a document's whole session lives on one node, so its sequencer is just an in-memory counter. Throughput per doc is tiny (Section 1.5), so one node hosts thousands of docs
Operation log	Durable, ordered, append-only record per document	Source of truth; powers history (FR-5), replay, recovery
Snapshot store	Immutable document snapshots every N ops	Makes open/reconnect $O(1)$ instead of $O(\text{history})$
Metadata DB	Documents, ACLs, current revision counters	Small, hot, transactional reads/writes
Presence cache	Live cursors and collaborator lists	Ephemeral by nature — losing it costs nothing; it rebuilds in seconds
REST API service	Control plane (Section 1.8)	Stateless request/response work; ordinary horizontal scaling

1.11 Deep Dive: The Edit Pipeline, Step by Step

Architecture diagrams tell you where things live; sequence diagrams tell you what **happens**. Let us follow one keystroke the whole way from Alice's keyboard to Bob's screen, because every design

decision from the last six sections fires somewhere along this path. The setup is chosen to make it interesting: document `d_8f31c2` is at revision 46, but Alice believes it is at revision 42 — four operations from other editors are committed on the server and have not yet reached her. Her edit is therefore **concurrent** with four committed operations, and the system has to cope.



Read the diagram top to bottom — time flows downward, each column is one of the participants, and every arrow is one message crossing the network. Then let us narrate the seven numbered steps, because each one exists for a reason you should be able to defend:

1. **Send (Alice → gateway).** The instant Alice types, her client applies the keystroke to her **local** document — this is the local echo that keeps edit latency under 16 ms; her screen never waits for the network. In parallel, the edit goes out as `op{base_rev: 42, ...}` carrying her next `client_seq`. Both fields are load-bearing: `base_rev` honestly reports what she had seen when she typed, and `client_seq` makes the message safe to retry.
2. **Route (gateway → collaboration node).** The gateway does **no** correctness work — it does not even understand the operation. It reads `doc_id`, consults the routing table, and forwards to the node that owns this document. Keeping the gateway dumb is what keeps it stateless, and statelessness is what lets the fleet scale to a million connections.
3. **Sequence & transform (the collaboration node, highlighted).** This is the step where the chapter's core challenge is actually solved, once per operation. The owner compares `base_rev: 42` with the current revision 46 and thereby **knows** Alice's edit is concurrent with revisions 43–46. It transforms her operation against each of those four committed operations, in order, using the matrix from Section 1.6 — and only then stamps the result with the next revision, 47. From this moment, her edit has a fixed place in the document's history.
4. **Commit (collaboration node → op log).** The transformed operation is appended to the operation log, and the append must become durable before anything else happens. Once it is durable, the edit is **committed**: it can never be lost, never be reordered, never be un-seen. The pitfall box below explains why this step must precede the next one.
5. **Broadcast (collaboration node → gateways).** Only after durability does the committed operation flow back to the gateway tier, which fans it out to every connected editor of the document — including Alice's own gateway, though her client will mostly treat the broadcast as a formality.
6. **Apply (gateway → Bob).** Bob's client checks the incoming `rev` against its own state. Bob is fully caught up, so the operation applies verbatim. Had Bob been nursing a **pending** local edit — typed, echoed, not yet acked — his client would first transform that pending edit against the incoming operation, using the same matrix the server runs. The transform code is symmetric on purpose:

one implementation, shared by client and server, keeps local echo and committed state from drifting apart.

7. **Acknowledge (gateway → Alice).** Finally, the `ack` tells Alice her operation is committed as revision 47, and her pending buffer clears. Notice what her eyes experienced: the character appeared instantly at step 1, and nothing visible happened at step 7. The local echo and the committed result are identical **by construction** — that is the user-visible payoff of all this machinery.

COMMON PITFALL — Order matters: commit before broadcast

It is tempting to reorder steps 4 and 5 — broadcast first, persist “in parallel,” save a few milliseconds. Resist it. If a node crashes after the broadcast but before the durable append, you have shown users edits that no longer exist; on recovery, the official history contradicts what people watched happen, and there is no clean way to retract a character from a collaborator’s screen. The rule is absolute: durability first, **then** visibility. It is the write-path version of “do not promise what you cannot guarantee,” and Chapter 11’s write-ahead log is the same rule wearing different clothes.

1.12 Deep Dive: Snapshots, Catch-Up & the Two-Phase Lesson

Two housekeeping jobs remain before the design is operationally complete, and they live at the same seam — the boundary between the operation log and the snapshot store. The first is making **open and reconnect fast**; the second is keeping a snapshot and its metadata **atomic**. Both are easy to describe and easy to get subtly wrong.

1.12.1 Catch-up: what to send a rejoining client

When a client joins with `join{last_seen_rev}`, the owning node compares that number against the current revision `R` and picks one of three strategies. Think of it as a cost comparison: what is the cheapest way to make this client current?

- **Close behind** (`R - last_seen_rev` is small, say ≤ 500): send the missing operations straight from the log. The client transforms any pending local edits across them and is current in milliseconds. This is the common case — network blips are short.
- **Far behind** (the laptop that was closed for a week): replaying tens of thousands of operations is strictly slower than sending their **result**. Send `latest snapshot + operations since the snapshot's base_rev`. This is exactly what the snapshot store exists for; without it, every long-absent open would be a full log replay.
- **Offline edits** (the extension): now the client is not merely behind — it has edits of its own that the server never saw. A scalar revision cannot express that, so the client presents its **version vector** (Section 1.7) instead. The server computes the set difference — the operations the client missed — and the client rebases its offline operations on top, transforming each across the diff. Notice why vectors and not timestamps, one more time: they compare **causal** histories, and causality is the only “when” both sides can agree on.

1.12.2 The atomicity requirement

A snapshot without correct metadata is worse than no snapshot at all. The metadata — above all `base_rev`, the revision through which the snapshot covers the log — is what catch-up logic **trusts**. Stale metadata is precisely the failure from Section 1.7’s pitfall: the client compares its version against a lie, concludes it needs nothing, and silently misses writes forever. So two writes must succeed or fail **together**: the snapshot blob itself, and the `{snapshot_pointer, base_rev}` record in the metadata DB. Two stores, one atomic outcome — the classic distributed-commit problem.

DEFINITION — Two-phase commit (2PC)

The textbook protocol for committing one atomic operation across **multiple** stores. In phase 1 (*prepare*), a coordinator asks every participant “can you commit?” and each participant does everything short of committing, then votes yes or no. In phase 2 (*commit*), the coordinator broadcasts the verdict: commit if all voted yes, abort otherwise. 2PC delivers genuine cross-store atomicity, but the price is steep: it is **blocking** — a coordinator that crashes at the wrong moment leaves participants locked, holding resources, unable to proceed — and it costs two network round trips. So the engineering question is never “how do I run 2PC?” but “how do I get the same guarantee more cheaply?”

For our seam, three constructions are on the table, in increasing order of machinery:

1. **One store, one transaction.** If the snapshot blob and its metadata happen to live in the same database, a single local transaction gives atomicity for free and 2PC never enters the conversation. Always check this option first; sometimes a schema decision makes it available.
2. **Immutable blob + conditional pointer update** — our choice, and worth understanding line by line. First, write the snapshot to the blob store under a content-addressed name, so the blob is immutable and self-identifying. **Only after** that write is durable, update the metadata row with a conditional compare-and-swap: `UPDATE ... WHERE base_rev = <old>`. Now consider every crash point. Crash before the blob write: nothing happened. Crash between the steps: an **orphan blob** sits in the store — wasted bytes, which a garbage collector reclaims, but **never incorrect**, because nobody’s metadata points at it. Crash during the pointer update: the conditional write fails or completes atomically, by the store’s own row-level guarantees. No coordinator, no blocking, no second round trip — atomicity assembled from idempotence and ordering.
3. **Full 2PC** remains for the case where two genuinely independent systems must commit together and neither trick above applies. Name it in the interview — and then argue you rarely need it. Knowing the expensive tool and declining it is stronger than not knowing it.

1.13 Deep Dive: Rust Reference Implementation

The concurrency core of this entire system is small enough to hold in your head — small enough, in fact, to show in full. Three pieces: the wire protocol types, the OT transform matrix with its convergence test, and the version vector. As you read, notice how little code the architecture’s discipline has bought us.

1.13.1 Protocol types

The data-plane messages from Section 1.8, as Rust types, with `serde` deriving the JSON representation:

```
use serde::{Deserialize, Serialize};

/// One edit to a plain-text document.
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum Op {
    Insert { pos: usize, ch: char },
    Delete { pos: usize },
}

/// Data-plane messages (client ⇄ server), serialized as JSON.
#[derive(Clone, Debug, Serialize, Deserialize)]
#[serde(tag = "type", rename_all = "snake_case")]
pub enum Msg {
    Join { doc_id: String, auth: String, last_seen_rev: u64 },
    Joined { doc_id: String, rev: u64, snapshot: String },
    Op { doc_id: String, base_rev: u64, client_seq: u64, op: Op },
    Ack { client_seq: u64, rev: u64 },
}

/// Server → clients: a committed operation.
```

```
Bcast { doc_id: String, rev: u64, author: String, op: Op },
}
```

The `#[serde(tag = "type")]` attribute is what makes the JSON look exactly like the protocol table — a `type` field distinguishes the variants, so the wire format and the type system say the same thing in two languages.

1.13.2 The OT transform matrix

This is Section 1.6's table made executable. `transform(a, b, a_wins_ties)` returns operation `a` adjusted to apply **after** the concurrent operation `b` has been applied — or `None` when `a` has been transformed into a no-op:

```
use crate::Op;

/// Transform `a` so it applies correctly *after* concurrent op `b`.
/// `a_wins_ties` breaks same-position insert/insert ties identically
/// on every replica (e.g., by client id) — determinism is what matters.
pub fn transform(a: &Op, b: &Op, a_wins_ties: bool) -> Option<Op> {
    match (a, b) {
        (Op::Insert { pos: pa, ch: ca }, Op::Insert { pos: pb, .. }) => {
            let shifted = if *pa > *pb || (*pa == *pb && !a_wins_ties) { pa + 1 } else
{ *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
        }
        (Op::Insert { pos: pa, ch: ca }, Op::Delete { pos: pb }) => {
            let shifted = if *pa > *pb { pa - 1 } else { *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
        }
        (Op::Delete { pos: pa }, Op::Insert { pos: pb, .. }) => {
            let shifted = if *pa >= *pb { pa + 1 } else { *pa };
            Some(Op::Delete { pos: shifted })
        }
        (Op::Delete { pos: pa }, Op::Delete { pos: pb }) => {
            if pa == pb { None } // both deleted the same char
            else { Some(Op::Delete { pos: if *pa > *pb { pa - 1 } else { *pa } }) }
        }
    }
}

/// Apply an operation to a document buffer.
pub fn apply(doc: &mut String, op: &Op) {
    match op {
        Op::Insert { pos, ch } => {
            let byte = doc.char_indices().nth(*pos).map_or(doc.len(), |(i, _)| i);
            doc.insert(byte, *ch);
        }
        Op::Delete { pos } => {
            if let Some((byte, c)) = doc.char_indices().nth(*pos) {
                doc.drain(byte..byte + c.len_utf8());
            }
        }
    }
}
```

Two disciplines in this code deserve a sentence each, because both are classic production bugs when ignored. First, `apply` works in **character** positions, not byte offsets — Rust strings are UTF-8, and an edit protocol that confuses the two will corrupt any document containing an emoji the day it ships. Second, the delete/delete collision returns `None`: when both users deleted the same character, the

second delete is transformed into a no-op, which is exactly the “both intentions honored” outcome the definition of intention preservation demands.

And here is the convergence diamond from Section 1.6 — the picture we walked through — expressed as an executable test, so the claim “both paths converge to `"AG"`” is checked, not asserted:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn go_diamond_converges() {
        let alice_op = Op::Insert { pos: 0, ch: 'A' }; // "G0" -> "AGO"
        let bob_op = Op::Delete { pos: 1 }; // "G0" -> "G"

        // Alice's side: apply hers, transform Bob's against it, apply.
        let mut a_doc = String::from("G0");
        apply(&mut a_doc, &alice_op);
        let bob_at_alice = transform(&bob_op, &alice_op, false).unwrap();
        apply(&mut a_doc, &bob_at_alice);

        // Bob's side: apply his, transform Alice's against it, apply.
        let mut b_doc = String::from("G0");
        apply(&mut b_doc, &bob_op);
        let alice_at_bob = transform(&alice_op, &bob_op, true).unwrap();
        apply(&mut b_doc, &alice_at_bob);

        assert_eq!(a_doc, b_doc); // convergence
        assert_eq!(a_doc, "AG"); // intention preservation
    }
}
```

1.13.3 The server-side session: transforming against history

Step 3 of the pipeline — “transform against revs 43–46” — is where the server’s copy of the algorithm lives. The owning node keeps one `DocSession` per active document, and when an operation arrives based on an old revision, the session walks it forward across everything committed since:

```
pub struct DocSession {
    pub rev: u64,
    pub doc: String,
    pub log: Vec<(u64, Op)>, // (revision, committed op); the op log in memory
    pub since_snapshot: usize, // ops committed since last snapshot
}

impl DocSession {
    pub fn commit(&mut self, base_rev: u64, mut op: Op) -> Result<u64, &'static str> {
        if base_rev > self.rev { return Err("base_rev from the future"); }
        // Transform across every revision the client had not seen.
        for (_, past) in self.log.iter().skip(base_rev as usize) {
            op = transform(&op, past, false).ok_or("op became a no-op")?;
        }
        self.rev += 1;
        apply(&mut self.doc, &op);
        self.log.push((self.rev, op.clone()));
        self.since_snapshot += 1;
        if self.since_snapshot >= 500 { self.snapshot(); } // Section 1.9 policy
        Ok(self.rev) // -> ack + broadcast
    }
}
```

```
fn snapshot(&mut self) { /* write blob, then conditional pointer update */
    self.since_snapshot = 0;
}
}
```

Stand back and look at how unglamorous this is: a counter, a vector, a `for` loop. There is no consensus protocol, no lock service, no distributed anything — because the architecture already did that work. Since **all** operations for a document funnel through one owner, the “distributed consensus” the problem seemed to demand has been reduced to no consensus at all. When you present this in an interview, that sentence is the punchline of the whole chapter.

1.13.4 Version vector

Section 1.7’s definition, implemented with its three-way comparison:

```
use std::collections::HashMap;

#[derive(Clone, Debug, Default, PartialEq, Eq)]
pub struct VersionVector(pub HashMap<String, u64>);

#[derive(Debug, PartialEq, Eq)]
pub enum Ordering { Before, After, Equal, Concurrent }

impl VersionVector {
    pub fn tick(&mut self, who: &str) { *self.0.entry(who.into()).or_insert(0) += 1; }

    /// Pointwise comparison: Before = self is causally dominated by other.
    pub fn compare(&self, other: &Self) -> Ordering {
        let mut less = false;
        let mut greater = false;
        for id in self.0.keys().chain(other.0.keys()) {
            let a = self.0.get(id).copied().unwrap_or(0);
            let b = other.0.get(id).copied().unwrap_or(0);
            less |= a < b;
            greater |= a > b;
        }
        match (less, greater) {
            (false, false) => Ordering::Equal,
            (true, false) => Ordering::Before,
            (false, true) => Ordering::After,
            (true, true) => Ordering::Concurrent,
        }
    }

    /// Element-wise maximum: the merged causal history of two replicas.
    pub fn merge(&mut self, other: &Self) {
        for (id, &n) in &other.0 {
            let e = self.0.entry(id.clone()).or_insert(0);
            *e = (*e).max(n);
        }
    }
}
```

Two details in the comparison deserve your attention, because both are easy to get wrong on a whiteboard. Iterating `self.0.keys().chain(other.0.keys())` — rather than just one side’s keys — is what makes the comparison correct when one vector knows about a participant the other has never heard of: the missing entry counts as zero on both sides. And the two-boolean accumulation (`less` , `greater`) is what makes **concurrency** detectable: if some entry is less and some other entry is greater, neither vector dominates, and the honest answer is `Concurrent` . With this type in hand, Section 1.12’s

catch-up decision collapses to three lines: if the client's vector is `Before` the server's, send the operations it lacks; if `Equal`, send nothing — and, per Section 1.7's pitfall, verify the **snapshot metadata** told the same story; if `Concurrent`, the client carries offline edits, so send the diff and rebase.

1.14 Deep Dive: Presence & Live Cursors

Presence — “who is here, and where exactly is their cursor” — looks like a trivial side feature, and it hides one elegant detail that interviewers love to probe: **cursor positions are text positions**, so they are subject to exactly the same shifting problem as edits. Suppose Alice's cursor sits at position 10, and Bob inserts three characters at position 4. If we render Alice's cursor at position 10 unchanged, we draw it three characters early — on the wrong letter, silently. The fix is already built: clients transform remote cursors against every incoming operation using the **same** OT matrix as edits, so a cursor is just an operation-free position that rides the transform rules. The server never computes cursors at all; it relays them, and each client keeps every remote cursor correct locally.

Beyond that detail, three design rules keep presence cheap, and each is really an application of one principle — **presence is disposable**, so nothing expensive should ever be spent on it:

- **Never persist it.** Presence flows through the ephemeral cache and the broadcast path, refreshed by a client heartbeat every few seconds. If the whole presence tier vanished this instant, the only consequence would be that cursors reappear within one heartbeat — which is why the architecture diagram draws it as a dashed, separate path off to the side.
- **Don't sequence it.** Presence messages carry no `rev` and bypass the operation log entirely. Ask yourself why: a stale cursor for 100 ms is invisible to users, but a stale **edit** is a correctness bug. Paying sequencer and durability costs for data nobody will ever verify is the classic over-engineering trap, and declining it out loud is worth credit.
- **Throttle it.** Cursor movement fires dozens of events per second per editor; left unshaped, presence would dwarf edit traffic. Cap updates at, say, 10 per second per client — fifty editors then produce 500 tiny messages a second, which the per-document owner fans out without breaking stride.

1.15 Scaling & Sharding

DEFINITION — Sharding (horizontal partitioning)

Splitting data or work across many machines by a key, so that each machine owns a disjoint slice. Sharding is the fundamental answer to “one machine is not enough”: pick the right key and the system grows by **adding** machines rather than by growing them — horizontal rather than vertical scale. The art is entirely in the choice of key: it decides whether load spreads evenly, and whether the data that must be together stays together.

Every tier of our system shards by its natural key, and walking the tiers is a good way to see why the keys differ:

- **Gateways** hold no document state at all, so any consistent routing works — connections balance across the fleet, and a reconnect can land anywhere. When there is no state, the sharding question answers itself.
- **Collaboration nodes** are sharded **by document**, with a routing service that maps `doc_id` → `owner`. Ownership is granted as a **lease** — a time-limited claim the node must keep renewing. If a node crashes, its leases expire on their own, the routing service assigns a new owner, and the invariant “exactly one sequencer per document” survives the failure with no human involved.

- The **operation log, snapshots, and metadata** are sharded by `doc_id` as well, so all of a document's data lives together on one shard. Co-location is what keeps the commit path — append the op, bump the revision — a single-shard, single-round-trip affair.

KEY INSIGHT — The 50-editor cap is a load-shedding decision

Why can a document never become a “hot shard” that melts its owner? Look back at Section 1.2: we capped concurrency at fifty editors. That product decision has an enormous architectural dividend — it bounds per-document load so tightly that the single-owner design can never be overwhelmed by traffic alone. A live-blog with a million **writers** would need something else entirely (a tree of sequencers, or CRDTs with no central order), and the honest thing to say in the interview is exactly that: “my single-owner design is safe **because** of this cap; lift the cap and I redesign.” Naming which NFR protects you from which failure mode is precisely the reasoning interviewers listen for.

1.16 Failure Modes & Recovery

A senior answer enumerates what breaks **before** being asked — not because interviewers enjoy gloom, but because a design that only works when nothing fails is a design that does not work. Walk the table slowly; each row is a small argument about why the system survives:

Failure	Handling
Client disconnects	Its edits are already durable in the log (that is what commit-before-broadcast bought us). On reconnect: <code>join with last_seen_rev</code> , catch up per Section 1.12. Its presence entry simply expires on its own.
WebSocket gateway dies	Clients reconnect to any other gateway — they are stateless, so any will do. In-flight unacknowledged operations are resent, and <code>client_seq</code> idempotency makes the retry harmless.
Collaboration node dies	The document's lease expires; the routing service assigns a new owner, which rebuilds session state as latest snapshot + log tail and resumes sequencing. Clients experience a reconnect, not data loss.
Duplicate message delivery	Impossible to prevent on real networks — so we make it harmless instead: <code>client_seq</code> idempotency on the write path, revision numbers on the read path (already have rev 47? ignore the re-broadcast).
Op-log replica loss	The log is replicated (3× is standard); a lost replica is replaced from its peers, and the source of truth survives any single loss.
Network partition	A client cut off from the server keeps editing locally and rebases on reconnect via the version-vector diff (Section 1.12) — or shows “offline, changes will sync,” if we choose availability over freshness. A partitioned server-side minority, by contrast, refuses ownership transfers: on the ordering authority, correctness beats liveness.

Step back and notice the pattern across the rows, because it generalizes far beyond this chapter: **stateless things are replaced, stateful things are rebuilt from the log, and duplicates are made harmless rather than prevented.** If you classify your components along those three lines, failure handling stops being a list to memorize and becomes a reflex.

1.17 Trade-offs & Alternatives

The design is now complete, and it is time for the most honest section of the chapter. Every architecture is a bundle of purchased benefits and accepted costs; a candidate who recites only the benefits has not finished the answer. Here is the whole ledger:

Decision	Benefit purchased	Cost accepted
Per-document sequencer + OT	Small messages, a total order, free version history, simple reasoning	A single point of ordering per doc (mitigated by lease failover — seconds of unavailability on crash); the OT matrix must be exactly right
CRDT instead	No sequencer; offline and peer-to-peer for free	Per-character IDs and tombstones; heavier storage and garbage collection; harder to bolt on a clean linear history
WebSocket data plane	Sub-150 ms push; bidirectional	A million long-lived connections to hold; load balancers must tolerate them
Op log as source of truth	History, audit, replay, recovery — all free	Storage growth (550 GB/day at our scale) and the snapshot machinery that keeps reads fast
Single-region write ownership per doc	No cross-region consensus anywhere on the hot path	Editors far from the owner's region see higher propagation latency

Read the first row one more time, because it is the whole interview in miniature: we bought simplicity and a free feature (history) by accepting a single point of ordering — and then we **engineered the cost down** with lease failover, rather than pretending the cost did not exist. That rhythm — benefit, cost, mitigation — is the voice of every strong system design answer.

NOTE — PACELC, briefly

A refinement of the CAP theorem worth having at your fingertips: **if** there is a Partition, a system must choose between Availability and Consistency; **else**, in normal operation, it chooses between Latency and Consistency. Our design makes its choices per-subsystem rather than globally: the edit path picks **consistency** on both branches (single owner, total order, commit before broadcast), while presence picks **availability and latency** (ephemeral, best-effort, never persisted). Different subsystems may legitimately make different PACELC choices — and saying which subsystem chose what, and why, is a far stronger answer than declaring the whole system “CP” or “AP.”

1.18 Observability & SLOs

DEFINITION — SLI / SLO

A *Service Level Indicator* is a measurable quantity — for example, propagation latency at the 95th percentile. A *Service Level Objective* is its target — “p95 < 400 ms.” SLOs are how “the system feels slow” becomes an engineering signal: each SLO carries an **error budget** (how much violation you can afford per month), and the budget, not anyone’s feelings, decides when to stop shipping features and start fixing reliability.

What do we instrument, at minimum? Per-document ops/sec and transform rates (the health of the core loop); end-to-end propagation latency, measured **on the client** and tagged by region (the user-visible promise); ack latency (the writer’s promise); WebSocket connection counts and message rates per gateway (the million-connection tier); the catch-up size distribution — where reconnect storms

reveal themselves before they hurt; snapshot lag (`current_rev - snapshot.base_rev`), which tells you whether opens are drifting toward O(history); and the lease-failover count, which should be near zero and interesting when it is not.

One operating principle ties the list together: **alerts fire on SLO burn, not on individual machine failures**. The failure table in Section 1.16 exists precisely so that single failures are non-events; paging a human because one gateway died would mean the redundancy was decorative.

1.19 Interview Wrap-Up

The interviewer still has ten minutes. Here are the follow-ups that usually arrive, with the shape of a good answer for each:

- “*Rich text?*” — Operations gain attributes (bold ranges, styles, embeds); the transform matrix grows cases, but the architecture — sequencer, log, snapshots — does not change. A reassuring answer, because it is true.
- “*Comments and suggestions?*” — Anchored to character ranges, and those ranges transform exactly like cursors (Section 1.14). They live in the metadata DB, off the hot path, because they are read-side annotations, not document content.
- “*End-to-end encryption?*” — Now the server cannot read content, so server-side OT is impossible — you cannot transform what you cannot parse. This single requirement pushes you all the way to client-side CRDTs. A beautiful example of one requirement that **re-architects** the core; say so explicitly.
- “*5,000 simultaneous editors?*” — Our 50-editor cap is precisely what licenses the single sequencer; at a hundred times that, shard the document itself or move to CRDTs. (Notice this is Section 1.15’s insight, arriving as a question.)
- “*Undo?*” — Per-user inverse operations, transformed through the log like everything else. Easy to describe, subtly hard to make **intuitive** — whose edits should your undo cross? Mention it, scope it, do not build it live.

If you remember five things from this chapter:

1. The product is a distributed-state synchronizer wearing a text editor’s clothes; design the synchronizer and the editor takes care of itself.
2. Convergence plus intention preservation is the correctness bar; locks and last-writer-wins both fail it, each in an instructive way.
3. A per-document total order — the sequencer — reduces “distributed consensus” to a counter and a transform loop.
4. Commit durably, then broadcast: never show a user an edit you cannot guarantee.
5. Snapshots and their version metadata commit atomically, or reconnecting clients silently miss writes (Section 1.12).

1.20 Summary & Further Reading

Let us close the loop. We designed a real-time collaborative plain-text editor for 10M daily users: a stateless WebSocket gateway tier holding a million connections; a collaboration service that owns each document and imposes a total order on its operations; operational transformation to merge concurrent edits while preserving their authors' intentions; an append-only operation log as the source of truth, with periodic snapshots to keep reads fast; version vectors for catch-up and offline diffing; and idempotent protocols that survive everything real networks do. If the design felt inevitable by the end, that is the estimation and the core challenge doing their work — the architecture was **derived**, not invented.

Primary source for this chapter:

- [“12: Design Google Docs/Real Time Text Editor” — Systems Design Interview Questions With Ex-Google SWE \(Jordan has no life\)](#) — the mock-interview walkthrough this chapter expands.

Foundations worth reading:

- Ellis & Gibbs, *Concurrency Control in Groupware Systems* (1989) — the original OT paper.
- Nichols et al., *Jupiter: high-latency collaboration* (1995) — OT with a central server, the direct ancestor of our design.
- Shapiro et al., *Conflict-free Replicated Data Types* (2011) — the CRDT formulation; our Chapter 9 builds directly on it.
- Google Wave's OT whitepapers, and Figma's engineering blog on multiplayer — production war stories from both families.

1.21 Chapter 1 Glossary

A one-glance index of every term this chapter defined. Later chapters assume it.

Term	Meaning in one line
System design interview	Open-ended architecture design under a 45–60 minute clock
HLD / LLD	Whole-system architecture / internal mechanics of the hard components
Functional requirement	What the system must do
Non-functional requirement	How well it must do it: scale, latency, availability, consistency
DAU / QPS	Daily active users / queries per second
Back-of-the-envelope estimation	Approximate arithmetic that sizes the problem before designing
Latency	Cause → observable effect; here: local echo < 16 ms, propagation < 150 ms
Availability	Fraction of time the system serves requests
Mutual exclusion (lock)	One writer at a time — correct, but kills real-time collaboration
Last-writer-wins	Latest timestamp overwrites — simple, but silently loses edits
Causality / concurrency	“Could A have influenced B?” If neither way: concurrent
Convergence	Same delivered edits = identical document on every replica
Intention preservation	Each author's observed edit effect survives concurrency

Term	Meaning in one line
Operational Transformation	Adjust concurrent operations' positions so all replicas converge
CRDT	Data type whose merge rules converge without a central order
(Strong) eventual consistency	Replicas converge (immediately upon delivery, for SEC)
Logical clock / Lamport	Causality-tracking counters instead of wall-clock time
Version vector	Per-participant counters recording exactly what a replica has seen
REST	Stateless resource-oriented HTTPS API — our control plane
WebSocket	Persistent full-duplex client↔server channel — our data plane
Idempotency key	Client-supplied unique key making retried operations safe
Operation log	Immutable append-only record of all edits; the source of truth
Snapshot	Materialized document at a revision; read optimization over the log
Two-phase commit	Prepare-then-commit protocol for atomic writes across stores
Sharding	Partitioning work/data by key across machines — here, by document
Presence	Ephemeral who's-here-and-where state; never persisted
SLI / SLO	A measured indicator / its target, defining “healthy”

— End of Chapter 1 · Next: Chapter 2 —

Designing a Maps & Navigation Service

PROBLEM SOURCE

This chapter solves the problem posed in the talk “13: Google Maps” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 35 min). The talk designs a mapping and navigation product: how the map itself is served, how the road network becomes a graph, how routes and ETAs are computed, and how live traffic feeds back into both. We follow the same arc here, slowing every step down: full definitions before first use, capacity mathematics with every assumption stated, protocol specifications, and Rust reference implementations of the pieces an interviewer is most likely to ask you to sketch.

2.1 The Problem Statement

The interviewer finishes the small talk and says:

“Design Google Maps. Users should be able to look at a map, search for places, and get directions from A to B — with an estimated time of arrival that accounts for current traffic.”

Where Chapter 1’s problem hid its difficulty behind a familiar CRUD interface, this one hides its difficulty behind a familiar *picture* — and that makes it dangerous in a different way. A map **feels** like content: something you serve, like an image on a website. Directions **feel** like a lookup: something you query, like a row in a database. If you follow those feelings, you will design a file server and a database, and both designs will be wrong. The map is a planet-sized rendering and caching problem. Directions are a graph algorithm running at planetary scale. And “current traffic” means the system’s answers change **under you, every few minutes**, driven by a firehose of GPS signals from millions of moving phones. Three genuinely hard subsystems — geospatial serving, large-scale graph search, and real-time stream processing — share one interview prompt, and part of what the interviewer is measuring is whether you notice all three.

One term will appear in every section of this chapter, so let us define it before anything else:

DEFINITION — ETA (estimated time of arrival)

The system’s prediction of how long a journey from A to B will take, usually expressed either as a duration (“47 minutes”) or as an arrival clock time (“arrive 6:15 PM”). The word **prediction** is doing the work in that sentence: an ETA must be computed **before** the journey happens, from a model of the road network plus everything currently known about conditions on it — and it will be compared, by every user, against the reality that follows. ETA accuracy is the quality bar this whole chapter is judged against; Section 2.4 makes it a formal requirement and Section 2.18 shows how to measure it honestly.

2.2 Scope & Clarifying Questions

Chapter 1 gave you the rule — never design the prompt you were handed; design the prompt you **negotiated** — and this prompt needs it more than most, because Google Maps is a dozen products wearing one icon. Map rendering, place search, directions, turn-by-turn navigation, live traffic, Street View, satellite imagery, transit schedules, offline maps, business listings, reviews... If you try to design all of that, you will design none of it well. So the first job is to shrink it, out loud, with the interviewer's help:

Speaker	Dialogue
Candidate	"The full product is enormous — map rendering, place search, directions, turn-by-turn navigation, live traffic, Street View, satellite imagery, transit schedules, offline maps. Which parts are we designing?"
Interviewer	"Focus on five: render the map, search for places, compute directions with an ETA, show live traffic, and run a turn-by-turn navigation session. Skip Street View, satellite, transit, and offline."
Candidate	"What scale — how many users, and how many actively navigating at once?"
Interviewer	"One billion monthly users, a few hundred million daily. Assume up to 15 million people in an active navigation session at peak."
Candidate	"Latency expectations?"
Interviewer	"The map must feel instant while panning. A route request should come back within a second or two."
Candidate	"How accurate does the ETA need to be?"
Interviewer	"Within about ten percent of actual travel time on typical trips."
Candidate	"Do drivers' phones report GPS positions back to us during navigation? That would be our live traffic signal."
Interviewer	"Yes — anonymized location updates, with user consent, every few seconds while navigating."
Candidate	"Availability target? People use this while driving; a dead navigation app mid-highway is a safety problem."
Interviewer	"Four nines for the navigation path."

Before we freeze the scope, look at what two of these questions quietly bought. The **feature-list** question did not just trim scope — it established that the interviewer will let you treat a giant product as a menu, which is the correct posture for every "design X" prompt where X is a mature product. And the question about **GPS reporting** is the single most valuable line in the exchange: without phone-reported positions there is no live traffic, and without live traffic the prompt collapses into "serve static files and run Dijkstra." One question about **inputs** unlocked the most interesting subsystem of the entire design. (The tip box below generalizes this.)

AGREED SCOPE

Five features: **map rendering, place search, directions with ETA, live traffic, turn-by-turn navigation**. Scale: **1B MAU**, 200M DAU, **15M concurrent navigation sessions** at peak. Latency: map feels instant, routes within 1–2 s. ETA accurate to $\pm 10\%$. Navigation availability **99.99%**. Phones send anonymized GPS updates every few seconds while navigating.

INTERVIEW TIP — Ask where the data comes from

Most candidates ask about users and features; very few ask “**what data do we get to see?**” That single question unlocks this entire problem: the answer — GPS updates from phones, every few seconds, with consent — is what makes live traffic possible at all. In a real interview, the questions that reveal **inputs** are worth as much as the questions that reveal **scale**, and they are rarer, which makes them more memorable.

2.3 Functional Requirements

Chapter 1 defined functional and non-functional requirements; recall that FRs are what the system must **do** — observable behaviors, testable without knowing the implementation. Here are ours, with a sentence each about where its difficulty hides:

1. **FR-1 — Map rendering.** The user can view an interactive map of the world and pan and zoom it smoothly, from a whole-planet view down to individual streets. Sounds like serving images; is actually a planetary caching problem (Section 2.6).
2. **FR-2 — Place search.** The user can find places by name, address, or category (“coffee near me”) and see them on the map. Sounds like text search; is actually text search **intersected with geography**, which is the interesting part (Section 2.9).
3. **FR-3 — Directions.** The user can request a route between two points and receives a good route: distance, step-by-step instructions, and a path drawn on the map. This is the graph algorithm (Section 2.8).
4. **FR-4 — ETA.** Every route comes with a travel-time estimate that reflects **current** conditions, not just speed limits. This requirement is what forces routing and traffic to be one mechanism rather than two (Section 2.7).
5. **FR-5 — Turn-by-turn navigation.** The user can start a navigation session and receives timely spoken and visual instructions, a live ETA that updates en route, and an automatic new route if they stray. This is the only stateful, session-oriented requirement — and therefore the one with the strict availability target (Section 2.13).
6. **FR-6 — Traffic overlay.** The map shows current congestion (green, yellow, red) on roads, fresh to within a few minutes. The visible tip of the streaming pipeline (Section 2.12).

Out of scope, stated explicitly so nobody can ambush us with them at minute fifty: Street View and satellite imagery, public transit timetables, offline maps, business-owner tooling, and social features.

2.4 Non-Functional Requirements

This problem has one NFR that Chapter 1 never needed, and it is the one that turns the design into a streaming system — so let us define it before the target table.

DEFINITION — Freshness

The age of the information behind an answer, measured from when reality changed to when the system’s output reflects it. A traffic overlay that shows speeds from an hour ago is **stale**; our target is that any road’s displayed condition reflects measurements from the last few minutes. Freshness is different from latency: latency asks “how fast do you answer?”, freshness asks “how new is the answer’s evidence?” A system can be fast and stale, or slow and fresh. It is the freshness requirement — not any throughput number — that makes this problem a **streaming** system rather than a static one.

The full set, stated as targets we can design and measure against:

Quality	Target
Map latency	Tiles visible within 100 ms while panning (p95); panning must never stutter
Route latency	Directions within 1 s for typical trips; 2 s for cross-country
ETA accuracy	Within $\pm 10\%$ of realized travel time on typical trips
Traffic freshness	Displayed speeds reflect the last 2–5 minutes of reality
Availability	99.99% on the navigation path (4.4 s of allowed downtime per day); 99.9% elsewhere
Scale	1B MAU; 200M DAU; 15M concurrent navigation sessions at peak

KEY INSIGHT — Different subsystems, different NFRs

Run your eye down that table and notice that “the system” has no single latency or availability number — and cannot have one, because its parts want opposite things. Tile serving is a massive, cacheable **read** workload where 100 ms matters and a stale tile is harmless. Routing is a **compute** workload where one second is generous but the algorithm must be exact. The traffic pipeline is a **write** workload where a few minutes of lag degrade quality, not correctness. And navigation is a **session** workload where availability is a safety property. If an interviewer presses you for “the availability target,” the senior answer is: “it depends which plane you mean” — followed by the planes. Chapter 1 drew the same lesson with its control plane / data plane split; this chapter splits further.

2.5 Back-of-the-Envelope Estimation

Chapter 1 taught the discipline — state assumptions, write them down, invite correction — and it is identical here. What changes is **where** the numbers will surprise you: in this problem the read path, the compute path, and the write path each have their own scale story, and they point at three different architectures.

Assumptions:

- 1B MAU, 200M DAU. Each daily user views 60 map tiles’ worth of panning, searches 2 places, and requests 3 routes per day.
- 15M concurrent navigation sessions at peak; each phone uploads one GPS update every 5 seconds while navigating; one update is 150 bytes on the wire.
- The world’s routable road network is on the order of **300 million directed edges** (two directions per piece of road, worldwide).
- A raster map tile averages 20 KB; the world is mapped at zoom levels 0–20.

Derived numbers:

Quantity	Estimate	How
Map tile requests	$\approx 140\text{k QPS avg, } 700\text{k peak}$	$200\text{M users} \times 60 \text{ tiles/day} \div 86,400 \text{ s, } \times 5 \text{ peak factor}$
Place search QPS	$\approx 5\text{k avg, } 25\text{k peak}$	$200\text{M} \times 2/\text{day} \div 86,400, \times 5$
Route request QPS	$\approx 7\text{k avg, } 20\text{k peak}$	$200\text{M} \times 3/\text{day} \div 86,400, \times 3$
GPS update ingress	$\approx 3\text{M updates/s}$	$15\text{M navigators} \div 5 \text{ s per update}$
Ingress bandwidth	$\approx 450 \text{ MB/s}$	$3\text{M updates/s} \times 150 \text{ B}$

Quantity	Estimate	How
Road graph size	≈ 20 GB	300M edges $\times$ 64 B per adjacency record
Naive tile storage	≈ 29 PB	$\sum_{z=0}^{20} 4^z \approx 1.47$ trillion tiles $\times$ 20 KB

KEY INSIGHT — What the math tells us

Three conclusions drive everything that follows, and each one pre-answers a later section. First, tile traffic is enormous — 700k requests per second at peak — but **the content barely changes**, which makes it the textbook caching workload; Section 2.6 exists to make 95%+ of those requests vanish into a CDN before they ever touch us. Second, the road graph is only 20 GB: it fits in the RAM of a handful of machines, so routing is an **in-memory** problem. But — and this is the arithmetic that justifies the chapter's most famous section — 20k route QPS multiplied by 3 CPU-seconds for a naive shortest-path search would demand **60,000 CPU cores**, which no fleet absorbs. Section 2.8's whole purpose is to shave those 3 seconds down to 1 millisecond, at which point 20 cores suffice. Third, 3M GPS updates per second is far too much for any request/response design; it demands a streaming pipeline, which Section 2.12 builds. Read-heavy caching, in-memory compute, streaming writes: three subsystems, three shapes, one product.

2.6 Core Challenge I: Serving the Map

Let us start with the thing the user actually sees. Someone opens the app and looks at their city. They drag the map; new streets slide in. They pinch; the view dives from the whole country to one neighborhood. Now think about what each of those gestures **demand**s: the right piece of the planet, rendered, on a glass screen, in tens of milliseconds — for 200 million people a day. Rendering the world on the fly for every gesture is out of the question; no fleet of renderers survives 700k requests per second at 100 ms each. The entire solution rests on one idea, and everything in this section is an unfolding of it: **precompute the map as tiny, independently addressable squares, and cache those squares everywhere.**

DEFINITION — Map tile

A small, fixed-size square of the map — conventionally 256 $\times$ 256 pixels as an image, or the equivalent bundle of geometry as data — covering a specific rectangular patch of the Earth's surface. A screen full of map is assembled from a grid of tiles, like mosaic pieces laid edge to edge. Tiles are the unit of storage, of caching, and of transfer for every mainstream map service: the server thinks in tiles, the cache thinks in tiles, and the client assembles tiles.

DEFINITION — Zoom level

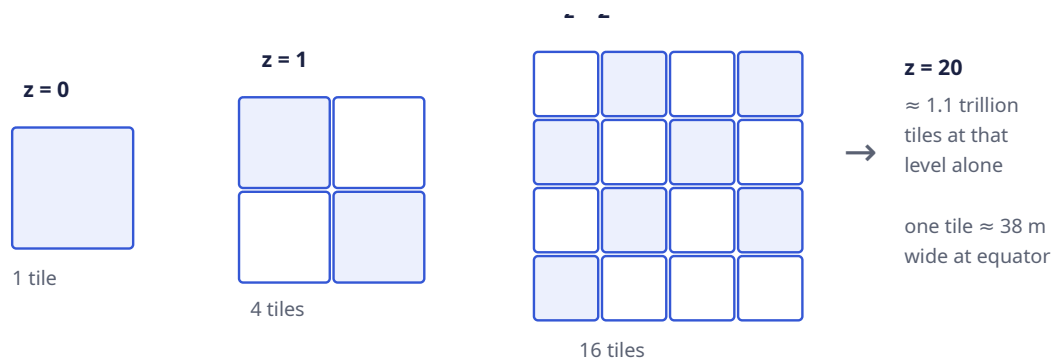
The map's scale, expressed as an integer z from 0 upward. At zoom 0 the **entire world** is one tile. Each step down splits every tile into four children, so zoom z has 2^z tiles per axis and 4^z tiles in total. Zoom 5 is roughly a country; zoom 10 a city; zoom 15 a neighborhood; zoom 20 shows a patch about 38 meters wide at the equator — individual buildings. The powers of four matter: they are why the total tile count explodes, and the explosion is what the storage section below must tame.

DEFINITION — Slippy-map tiling scheme

The near-universal convention for **addressing** tiles: every tile is identified by three integers (**z , x , y**) — its zoom level and its grid coordinates, counted from the top-left of the world. To build the grid, the Earth's surface is first flattened with the Web Mercator projection (which maps the sphere to a square

and caps latitude near $\pm 85.05^\circ$, which is why polar regions never quite appear), then cut into the $2^z \times 2^z$ grid. The payoff of this convention is determinism: given a latitude, a longitude, and a zoom, **anyone can compute which tile contains that point** with a closed-form formula — Section 2.14 implements it in eleven lines of Rust. So the client never asks the server “which tiles do I need?”; it already knows, and requests them by name.

The pyramid shape that falls out of the zoom rule is the whole storage story, so let us look at it before taming it:



Read the diagram left to right and watch the explosion happen. On the far left, zoom 0: the entire planet in one square — one tile, mostly blue. At zoom 1, that tile has split into its four children; the shaded squares track one patch of the Earth as it subdivides. At zoom 2, sixteen tiles cover the world, and the same patch is now a quarter of a quarter. The arrow then jumps the sequence forward fifteen more levels to zoom 20, where the count is no longer drawable: about 1.1 trillion tiles at that level alone, each covering a patch 38 meters wide. One tile became four became sixteen became a trillion; that geometric progression, multiplied by 20 KB a tile, is exactly where the 29 PB estimate in Section 2.5 came from.

So how do we afford a 29-petabyte map? We do not — two observations collapse it:

1. **Most tiles are uniform.** Roughly 70% of the planet is ocean; enormous areas beyond that are desert, ice, or forest. A solid-blue 256×256 square compresses to almost nothing — and, more powerfully, **it is the same square everywhere on Earth**. If we store tiles content-addressed (named by the hash of their contents; Section 2.10), every identical tile in the world is stored exactly **once**. The trillion collapse to the few percent of tiles that actually contain roads and labels, and 29 PB becomes merely large.
2. **Nobody looks at most of the world, most of the time.** Demand is violently skewed toward populated areas and common zooms: midtown Manhattan at zoom 16 is served constantly; the mid-Pacific at zoom 20 essentially never. Skew like that is what caching feeds on — a cache holding the popular few percent serves the overwhelming majority of requests.

Which brings us to the machinery that exploits both observations:

DEFINITION — Content Delivery Network (CDN)

A geographically distributed fleet of **edge caches**: servers in hundreds of cities, each holding copies of popular content physically close to users. A request is routed to the nearest edge location; on a **cache hit** the content is served locally — single-digit milliseconds, and zero load on our infrastructure; on a **miss** the edge fetches from our origin once and caches the copy for the next requester. A CDN turns a read-heavy, mostly-static workload into someone else’s bandwidth, and the tile pyramid is the most read-heavy, most static workload imaginable.

DEFINITION — TTL (time to live) / cache hit ratio

A cached entry's *TTL* is how long an edge may serve it before revalidating with the origin — it is the dial between freshness and load, and you turn it per content type. The *hit ratio* is the fraction of requests served from cache, and it is the number that decides whether the CDN strategy is working. For base map tiles we choose long TTLs (days; roads move slowly) and expect hit ratios above 95%. The **traffic overlay** tiles of Section 2.12 get TTLs of a minute or two, because stale traffic is worse than no traffic. Same cache, two TTLs — freshness is per-layer, not per-system.

COMMON PITFALL — Serving tiles from your own fleet

The single most common junior mistake on this problem is to draw a “tile service” that renders or reads tiles on demand, per request. Run the numbers on that design: at 700k peak QPS with even a cheap 5 ms read, you need thousands of machines whose only job is to re-serve the same static squares forever — slow for distant users, vast, and pointless. The correct shape is: **pre-render tiles offline, push them to object storage, put a CDN in front, and let the origin see a single-digit percentage of traffic**. Rendering is a batch job that runs when the map changes, not a request path that runs when a user pans. Say that sentence in the interview and the map-serving discussion is over.

One design fork is worth naming now and deciding later (Section 2.17): tiles can be **raster** (pre-rendered images) or **vector** (compact geometry the client renders on its own GPU). We will serve vector tiles to modern clients — roughly half the bytes, restyleable without re-rendering, and one tile set serves every zoom-adjacent gesture smoothly — while keeping raster fallbacks for old clients. Either way, everything you just learned — the pyramid, the addressing, the CDN — is identical; only the payload changes.

Place search (FR-2) rides on the same geospatial ideas as tiles: a **spatial index** lets us find “all places inside this patch of Earth” without scanning 200M records. We will define that structure once, in Section 2.7's storage discussion and Section 2.14's code, because routing needs it too.

2.7 Core Challenge II: Modeling the Road Network

Directions require a mathematical object we can search. Maps as humans know them — pretty pictures of roads — are useless to an algorithm; what we need is a structure that encodes **what connects to what, and at what cost**. That structure is a graph, and building it from the road network is the second core challenge.

DEFINITION — Graph / weighted directed graph

A *graph* is a set of **nodes** connected by **edges**. It is *directed* when edges have a direction of travel — a one-way street is traversable only along its arrow — and *weighted* when every edge carries a number measuring the **cost** of crossing it. We model intersections as nodes and stretches of road between them as edges, and we call one such directed road piece a **segment**. If this feels abstract, hold a city in your head: every intersection is a dot, every block between two intersections is an arrow or two, and the whole city's road network is exactly those dots and arrows.

DEFINITION — Segment

The atomic unit of the road network: one directed piece of road between two nodes, carrying metadata — its geometry (the polyline of its shape), length, road class (residential, arterial, highway), speed limit, and any turn restrictions at its far end. Segments are also the unit of **traffic**: when we say “this road is slow,” we mean “this segment's current crossing time is high,” and every GPS update we receive is

eventually attributed to exactly one segment (Section 2.12). One concept, two jobs: structure for the router, evidence for the traffic loop.

The crucial modeling choice is the edge **weight**, and it is worth thinking through rather than accepting. Distance is the obvious candidate — it is stable, measurable, and simple. But drivers do not minimize distance; they minimize **time**, and the shortest route in miles is often the slowest route in minutes. So

an edge's weight is its **expected crossing time**: $\frac{\text{length}}{\text{current expected speed}}$. Now watch what this one choice does for the whole chapter. On an empty road, “current expected speed” is the speed limit. As Section 2.12's pipeline observes real vehicles slowing down, it lowers that speed — and the edge weight rises, automatically. Routing and traffic collapse into one mechanism: **traffic is just edge weights that move**. FR-4, the traffic-aware ETA, needed no new algorithm at all.

DEFINITION — Adjacency list

The standard memory layout for sparse graphs: for every node, a list of its outgoing edges. Routing graphs are extremely sparse — an intersection has roughly 3–6 outgoing segments — so an adjacency list stores 300M small records, the 20 GB from Section 2.5, where a dense matrix representation would need space quadratic in the node count (petabytes of mostly zeros). Our routing engine holds adjacency lists **in RAM**: there is no database on the per-request path, which is how route latency survives its one-second budget.

One subtlety interviewers enjoy raising: a plain graph edge cannot express “you may enter this intersection from Main Street but may **not** turn left onto 1st Avenue.” Turn restrictions break the fiction that cost lives on edges alone. Two standard repairs: split nodes (one node per incoming direction, with edges only for legal turns), or attach per-edge metadata that the search consults. Either way, the core model — nodes, directed edges, time weights — survives intact, and knowing the repair is enough at this level.

2.8 Core Challenge III: Shortest Paths at Planetary Scale

With the road network modeled as a time-weighted graph, computing directions becomes a precise mathematical problem:

DEFINITION — Shortest path problem

Given a weighted graph and two nodes, find the path between them whose total edge weight is minimal. With our time-weighted segments, the shortest path is the **fastest route**, and its total weight is the route's ETA — the answer to FR-3 and FR-4 falls out of the same computation. That tidy reduction is why the modeling work in Section 2.7 mattered: one good model turns two product features into one well-studied algorithmic problem.

Now, which algorithm? The interviewer expects you to know the canonical answer **and** to know why you cannot ship it. Let us build the ladder one rung at a time — that ordering, from “correct but too slow” to “fast enough to serve,” is itself the story of this section.

DEFINITION — Dijkstra's algorithm

The classic exact solution, and still the right mental model. The idea: explore the graph outward from the source in order of increasing distance from it, like a stain spreading. Maintain for each node the best known distance (initially ∞ everywhere, 0 at the source); repeatedly take the not-yet-finalized node with the smallest best-known distance, declare it **final** — its distance can never improve, because any other route to it would have to pass through a node with an even larger distance — and **relax**

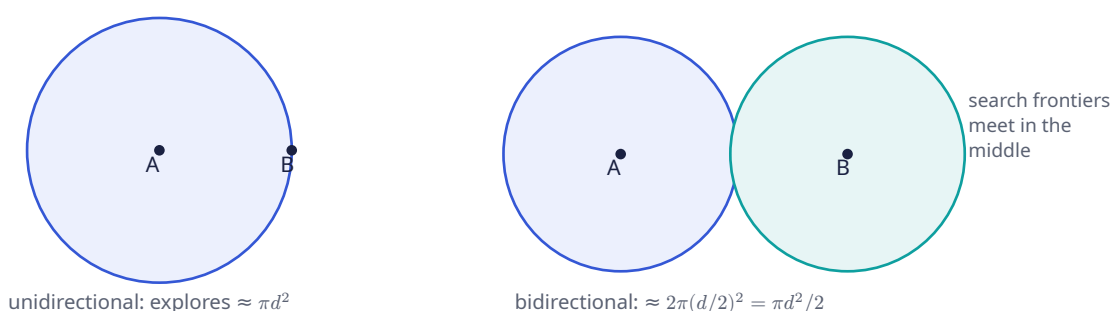
its edges: for each outgoing edge, check whether reaching its neighbor through this node beats the neighbor's best-known distance, and if so, update it. With a min-priority queue picking the next node, the running time is $O((V + E) \log V)$, and the answer is provably exact whenever weights are non-negative — which crossing times always are.

DEFINITION — Priority queue (min-heap)

A data structure supporting “insert with a priority” and “remove the minimum-priority element,” both in $O(\log n)$; a binary heap inside an array is the standard implementation. Dijkstra's algorithm spends its entire life asking one question — “which unfinished node is currently closest to the source?” — and a priority queue is precisely that question, as a data structure.

Here is the uncomfortable arithmetic that frames the rest of the section. Dijkstra is correct — and unusable at our scale. A cross-country query over 100M nodes takes seconds of CPU; Section 2.5 showed that seconds $\times$ 20k QPS is 60,000 cores of pure search. We cannot buy our way out; we must **search less of the graph**. Each of the next three refinements is a different way of doing exactly that.

Refinement 1 — bidirectional Dijkstra. Run two searches simultaneously: one forward from the origin, one **backward** from the destination over reversed edges. Stop when the frontiers meet; the route is the two half-paths joined. Why does that help? The picture makes it obvious:



The left panel shows the unidirectional search. Dijkstra knows nothing about where B is, so it explores **everything** within reach of A: a disk whose radius grows until it touches B. If A and B are distance d apart, that disk has radius d and — in a roughly planar road network, where node count grows with area — contains on the order of πd^2 nodes. The right panel shows the bidirectional search: a blue disk spreading from A and a teal one spreading backward from B, and the algorithm stops the moment the two frontiers touch. Each disk has radius only $d/2$, so together they explore about $2 \cdot \pi(d/2)^2 = \pi d^2/2$ — **half** the nodes. The saving comes from the quadratic: halving the radius quarters each disk, and two quarter-disks beat one whole disk. Same provably-exact answer, half the CPU, and the two searches are trivially run on two threads. A real improvement — and nowhere near enough.

Refinement 2 — A* search. Look at the left disk again and something should offend you: Dijkstra explores equally in **all** directions, including straight away from the destination. Every node it finalizes on the far side of A from B is provably wasted work. A* fixes this by reordering the priority queue so the frontier grows **toward** B instead of uniformly.

DEFINITION — Heuristic / admissible heuristic

In A*, each node's queue priority becomes $\text{known distance from source} + h(\text{node})$, where the *heuristic* h estimates the **remaining** distance to the destination. The heuristic is *admissible* if it never **overestimates** the true remaining cost — and admissibility is the whole ballgame: with it, A* keeps Dijkstra's

exactness guarantee while pulling the search frontier toward the goal, because nodes in promising directions get smaller priorities and are popped first. Overestimate even once and you can finalize a node through the wrong path; the guarantee dies quietly.

For a road network, nature hands us a perfect admissible heuristic: the **straight-line distance to the destination, divided by the fastest speed anywhere in the network**. No legal route can beat the crow flying at top speed, so this h never overestimates — it is admissible by construction. Computing it requires distances between points on a sphere, which needs one more definition:

DEFINITION — Haversine distance

The great-circle distance between two latitude/longitude points on a sphere — the length of the shortest path over the Earth’s surface, “as the crow flies.” A closed-form trigonometric formula computes it in microseconds; Section 2.14 implements it. Haversine is our heuristic’s yardstick and, more generally, our fallback whenever the question “how far apart are these two coordinates?” appears anywhere in the system.

Refinement 3 — hierarchy: mega-segments, then contraction hierarchies. Even A* has a blind spot: it still explores every side street near the origin and the destination, because near the endpoints the heuristic cannot yet distinguish promising from unpromising detours. Now think about how **you** actually drive cross-country: local streets for a minute, then a highway for three hundred miles, then local streets again. The middle of a long route almost never touches small roads — and that human observation can be turned into a mechanism: precompute the hierarchy and let queries climb it.

DEFINITION — Mega-segment

A precomputed synthetic edge that summarizes a chain of ordinary segments — for example, one edge meaning “highway, exit 12 to exit 47, 52 km, 31 minutes at current speeds.” A long-distance search can then hop between on-ramps and off-ramps on mega-segments instead of expanding thousands of small ones. The source talk builds its long-range routing this way: segments roll up into mega-segments, which roll up further, so a query quickly climbs from street level to highway level, crosses the country on a handful of edges, and descends again. And because a mega-segment’s weight is the **sum** of its members’ weights, when traffic updates a segment (Section 2.12), the change simply **bubbles up** to every mega-segment containing it — the hierarchy stays live without being rebuilt.

The literature’s formal, optimal version of the same idea:

DEFINITION — Contraction hierarchy (CH)

A preprocessed form of the road graph, built offline in two steps. First, order all nodes by importance — highway junctions outrank cul-de-sacs. Second, **contract** the nodes in that order: remove a low-importance node, and whenever the shortest path between two of its neighbors ran through it, insert a **shortcut edge** with the summed weight, so every shortest-path distance in the graph is preserved exactly. At query time, run bidirectional Dijkstra with one extra rule: only follow edges that go **up** in importance. The two upward searches meet at the most important node on the route — and because unimportant nodes were contracted away, the search spaces are tiny. Preprocessing takes hours and adds 30–100% more edges; in exchange, queries drop from seconds to **microseconds-to-milliseconds** — the roughly 1000× speedup that Section 2.5’s arithmetic demands. Production engines (OSRM, Valhalla, and the systems the source talk gestures at with mega-segments) all run variants of this playbook.

The full ladder, with the numbers that justify each rung:

Algorithm	Preprocessing	Query time	Verdict
Dijkstra	none	seconds (continental)	Teaches the principle; too slow to serve
Bidirectional Dijkstra	none	2× faster, still seconds	Free win; still too slow
A* + haversine	none	often 5–10× faster	Great for short trips; long trips still expand too much
Mega-segments / CH	hours, offline	≈ 0.1–1 ms	Production answer: the 1000× that makes 20k QPS fit on 20 cores

KEY INSIGHT — Precomputation is the theme of the whole chapter

Step back and notice what the last three sections have in common. Tiles are pre-rendered so reads become cache hits (Section 2.6). The graph is pre-contracted so queries touch only highways-in-the-middle (this section). Traffic speeds are pre-aggregated per segment so ETAs become table lookups (Section 2.12). A maps system is a machine for moving work **off** the request path and into batch and stream pipelines that ran earlier. So when an interviewer asks “how does it answer so fast?”, the honest one-word answer is: **earlier**.

2.9 API & Protocol Design

Chapter 1 split its API into a control plane and a data plane, and the same split applies here — with one twist worth savoring: our heaviest “API” is barely an API at all.

Tiles are plain GETs — and mostly not ours. Because the slippy-map scheme is deterministic, the client computes the (z, x, y) address of every tile in view by itself (Section 2.14 has the eleven lines) and issues ordinary `GET /v1/tiles/{z}/{x}/{y}` requests against a tile hostname. Nearly all of those requests terminate at a CDN edge and never reach our infrastructure. Sit with that for a second, because it inverts the usual API-design story: the tile endpoint has **no interesting request parameters**, no session, no state — all the intelligence lives in the addressing scheme, which the client and the cache both understand. The best-designed endpoint in this system is the one we barely operate.

DEFINITION — Polyline encoding

A compact ASCII encoding of a sequence of coordinates. Google’s variant encodes latitude/longitude **deltas** as variable-length, base-64-ish text, so a 500-point route becomes a few hundred bytes instead of a JSON array of floats ten times larger. Routes are transmitted as encoded polylines and decoded client-side. The format exists because a route is geometrically huge — it crosses a route-length’s worth of geography — but must fit inside one small, cacheable response.

The remaining request/response endpoints are ordinary REST (as defined in Chapter 1):

Method	Path	Purpose
GET	<code>/v1/tiles/{z}/{x}/{y}</code>	Map tiles; served by CDN edge, origin on miss
GET	<code>/v1/search?q=&near=lat,lng</code>	Place search: text + spatial filter; ranked results
GET	<code>/v1/search/autocomplete?q=</code>	Prefix suggestions as the user types

Method	Path	Purpose
POST	/v1/routes	Directions: origin, destination, mode, departure time → candidate routes with distance, ETA, ETA-in-traffic, steps, encoded polyline
POST	/v1/geocode	Address → coordinates (and reverse: coordinates → address)

A route request and its response, so you can see exactly what the routing engine's output becomes on the wire:

```
POST /v1/routes
{
  "origin": { "lat": 37.7749, "lng": -122.4194 },
  "destination": { "lat": 37.3861, "lng": -122.0839 },
  "mode": "driving",
  "depart_at": "now"
}

200 OK
{
  "routes": [{
    "route_id": "rt_9c31",
    "distance_m": 56300,
    "eta_seconds": 2820,
    "eta_in_traffic_seconds": 3450,
    "polyline": "a~l~Fjk~u0wIb@_...",
    "steps": [
      { "instruction": "Head north on Market St",
        "segment_ids": ["seg_7712", "seg_7713"],
        "distance_m": 400, "eta_seconds": 61 }
    ]
  }]
}
```

Two fields in that response deserve a pause. `eta_seconds` versus `eta_in_traffic_seconds` is Section 2.7's design made visible: the first is the route at free-flow weights, the second at **current** weights, and showing both is what lets the UI say “10 minutes slower than usual.” And each step carries `segment_ids` — the route is not just a picture, it is a list of the exact road segments the user will traverse, which is precisely what the navigation session needs to track progress and what the traffic pipeline needs to match this user's GPS updates back to the roads they are actually on.

Navigation (FR-5) is a **session**, not a request: the phone streams its position up, and the server streams guidance down, for as long as the trip lasts. That is a bidirectional, long-lived, low-latency channel — in other words, exactly the WebSocket data plane Chapter 1 built, reused verbatim:

Message	Direction	Payload & semantics
nav_start	client → server	{route_id, auth} — begin the session on the chosen route
location_update	client → server	{lat, lng, speed_mps, heading_deg, t} — one GPS fix, every 5 s; drives guidance and the traffic pipeline
instruction	server → client	{text, voice, distance_to_maneuver_m} — the next turn, delivered early enough to speak

Message	Direction	Payload & semantics
eta_update	server → client	{eta_seconds, remaining_m} — refreshed as segment weights move
reroute	server → client	{reason, new_route} — deviation, or a newly faster alternative
nav_end	either	Arrived, cancelled, or the connection dropped

Two protocol notes before we move on. First, look at `location_update` carefully: it is **one stream feeding two masters**. The navigation service uses it to guide this user, right now; the traffic pipeline uses it — anonymized and aggregated — to update everyone’s edge weights. Section 2.12 draws that fork as a picture. Second, navigation is the one place in the whole system where **we push down** to a moving phone, and that raises a plumbing question the diagram in Section 2.11 must answer: with millions of WebSocket gateways, which one holds this user’s connection? The connection manager, introduced there, exists to answer exactly that.

2.10 Data Model & Storage

Six entity families, six different access patterns — which is this chapter’s excuse to define a term you should reach for whenever one database is asked to be four things:

DEFINITION — Polyglot persistence

Using different storage engines for different access patterns within one system, instead of forcing every entity into a single database. The cost is real — operational complexity, more moving parts, more consistency questions — but the benefit is that each workload gets the structure it actually needs. A maps system is the canonical case for it, as the table below shows: our six entities want six different shapes, and pretending otherwise would punish every one of them.

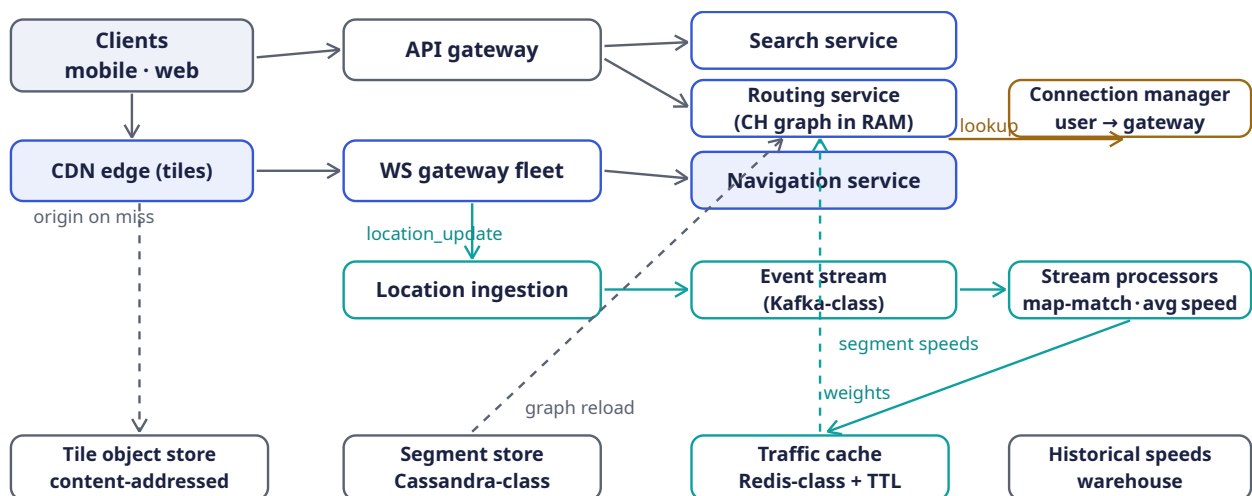
Entity	Contents	Store
Tile	(z,x,y) → rendered image or vector geometry	Object store behind the CDN; content-addressed so identical tiles are stored once
Place	name, category, address, location, ratings, hours	Text-inverted index × spatial index; 200M records
Segment	geometry, length, road class, speed limit, turn restrictions	Wide-column store (Cassandra-class), sharded by geography; read-heavy, batch-written
Road graph	adjacency lists + current weights + shortcuts	In RAM on routing nodes; rebuilt/swapped from the segment store + traffic cache
Traffic	segment_id → current average speed, sample count, updated-at	In-memory key-value cache (Redis-class) with a TTL — absence means “no fresh data,” triggering the historical fallback

Entity	Contents	Store
Historical speeds	$\text{segment_id} \times (\text{day-of-week, time bucket}) \rightarrow \text{average speed}$	Analytics warehouse, batch-computed nightly from the GPS archive
Nav session	route, progress along it, last fix, ETA	In-memory on the navigation node; ephemeral like Chapter 1's presence data

Three ideas in this table are the ones interviewers reward, so let us pull them out of the rows. First, tiles are **content-addressed**: Section 2.5's 29 PB nightmare collapses because duplicate oceans are stored once, under one hash. Second, the routing graph lives **in memory** — the stores are only its source of truth during reloads, never on the request path; that is how route latency survives its budget. Third, and most elegant: the traffic cache's TTL is the freshness guarantee. A segment whose entry has expired simply stops claiming live data, and the system degrades to historical patterns without anyone handling an error. Absence as a signal is a pattern worth stealing.

2.11 High-Level Architecture

Section 2.4 promised that this system has separate planes with separate NFRs; here they are, in one picture. The **read plane** (tiles, search, routes) is stateless and cache-fronted. The **streaming plane** (GPS in, traffic out) is a pipeline. Navigation sits astride both. Then we will walk the picture slowly.



Read plane: stateless, cache-fronted. Streaming plane: GPS in → traffic out. Dashed: control/reload paths.

This diagram has a lot of moving parts, so let us walk it in the order data actually flows, and make sure every arrow earns its place.

The read plane, top half. Begin at the top left. When a client pans the map, its tile requests drop almost immediately into the **CDN edge** box directly below — the vertical arrow — and, for 95%+ of them, the journey ends right there. Only on a cache miss does the long dashed arrow down the left margin carry the request to our **tile object store**, the “origin on miss.” The base map, remember, changes weekly at worst, so even a stale edge copy is safe to serve while revalidating.

Everything else the client asks for — search, routes, navigation — flows right through the two gateway boxes. Plain REST calls (search, routes) go through the **API gateway**, which does the ordinary edge work: authentication, rate limiting, routing. Long-lived navigation connections go through the **WebSocket gateway fleet**, Chapter 1's stateless connection-holders reused without changes. From

the API gateway, three short arrows fan out to the read-plane services: the **search service** answers place queries over its text × spatial index; the **routing service** answers directions on its in-memory CH graph; and the **navigation service** — highlighted, because it is the session-ful one — runs the per-user guidance state machine of Section 2.13.

The streaming plane, middle row. Now watch a GPS fix take the other road. While navigating, the phone sends a `location_update` every five seconds; the teal arrow carries it down from the WS gateway to **location ingestion**, whose job is to absorb 3M updates per second, validate them, strip identity, and absorb bursts so processing never sees them. From there the fix enters the **event stream** — a durable, partitioned log (defined properly in Section 2.12) — and then the **stream processors**, which do the two hard jobs: map-matching the noisy fix onto a real segment, and folding its speed into that segment’s running average.

Where the two planes meet. The long teal arrow from the stream processors down to the **traffic cache** is the handshake between the planes: per-segment speeds land in a small, hot, TTL-governed cache. And then the arrow labeled “weights” — dashed teal, from the traffic cache up to the routing service — is the single most important arrow in the whole design: it is how **reality changes the answers**. Every few minutes, the routing engine’s edge weights are re-read from the cache, so the very next route request routes around a jam that formed ninety seconds ago. Meanwhile a second dashed arrow, “graph reload,” runs from the segment store up to the routing service: that is the slow path by which new roads and edited geometry enter the in-memory graph on a periodic rebuild-and-swap cycle.

The amber plumbing, top right. One small box is easy to overlook: the **connection manager**. When the navigation service decides a user needs a `reroute` push, it knows **what** to send but not **where the user’s socket lives** among thousands of gateways. The connection manager is the registry that answers — the amber “lookup” arrow — and Section 2.13 shows how it is kept honest.

Finally, the bottom row of stores should look familiar from Section 2.10 — they are drawn in the same left-to-right order as the workloads above them: tiles at the far left behind the CDN, segments under ingestion, the traffic cache under the stream processors, and the **historical speeds** warehouse at the far right, quietly batch-computed each night from the day’s GPS archive and standing ready as the fallback whenever live data runs dry.

Component	Responsibility	Why it is shaped this way
CDN edge	Serve 95%+ of tile requests	Tiles are static and skewed — the textbook cache workload (Section 2.6)
API gateway	Auth, rate limiting, routing for REST endpoints	Stateless; also the paid-API front door, where per-customer quotas are enforced
Search service	Text × spatial place lookup	Read-heavy over a slowly-changing index; replicas scale it
Routing service	Compute routes + ETAs on the CH graph	Holds the graph in RAM ; scales by region shards and replicas (Section 2.15)
WS gateway fleet	Hold millions of navigation connections	Stateless connection holders, exactly as in Chapter 1
Navigation service	Per-session guidance: progress, instructions, ETA refresh, reroutes	Stateful per session — but sessions are small and independent

Component	Responsibility	Why it is shaped this way
Connection manager	Registry: user → which gateway	So the navigation service can push reroute to the right socket (Section 2.13)
Location ingestion	Absorb 3M updates/s, validate, anonymize	A buffer that decouples phone bursts from processing (Section 2.12)
Event stream	Durable, replayable pipe of GPS updates	Defined in Section 2.12; lets many consumers read the same firehose
Stream processors	Map-match pings to segments; average speeds; write traffic cache; bubble weights up mega-segments	The batch-quality work of traffic, done continuously

2.12 Deep Dive: The Live Traffic Loop

This is the subsystem that turns “a map” into “**current** conditions,” and it is the heart of the source talk. It is also a beautiful example of a feedback loop in systems form: the product’s own users generate the data that improves the product’s answers for everyone. Let us define the four moving parts first, then follow one GPS update until it changes a stranger’s ETA.

DEFINITION — Event streaming platform (Kafka-class)

A distributed, durable, append-only log organized into **topics**. Producers append events; any number of independent **consumers** read them at their own pace, and the log remembers, so a slow or crashed consumer resumes where it left off. Topics are **partitioned** — we partition by geographic key, a geohash prefix — so hundreds of consumer instances process the stream in parallel, each owning a disjoint slice of the world. For us it absorbs the 3M updates/second, decouples ingestion from processing, and — because it is replayable — lets us rebuild derived state after failures. Chapter 4 studies this kind of platform as a design problem in its own right.

DEFINITION — Stream processing

Computing over data **as it arrives** — event by event, or in small windows — rather than in scheduled batch runs over stored data. The distinction matters because freshness demands it: a nightly batch over today’s GPS data would give you yesterday’s traffic. Here, every GPS update is map-matched and folded into a per-segment running average within seconds of leaving the phone.

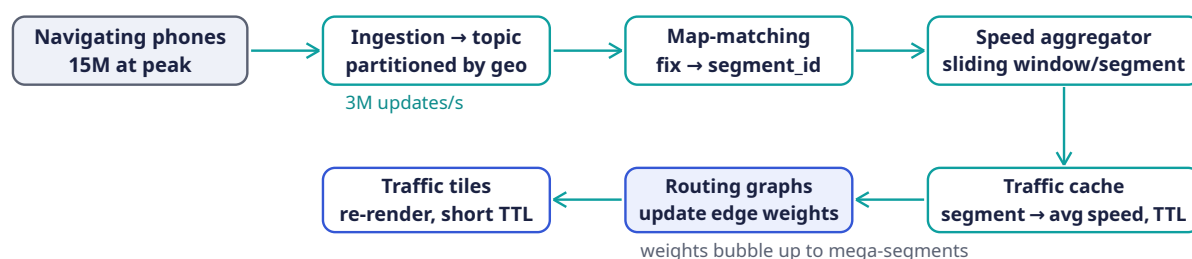
DEFINITION — Map-matching

Snapping a noisy GPS fix to the road segment the vehicle is most likely **actually** on. Raw GPS is wrong by 5–50 meters — enough to place a car on the wrong street, on a parallel frontage road, or inside a building. The production-standard approach models the trace as a **hidden Markov model**: hidden states are the candidate segments near each fix, emission scores measure how close the fix is to each candidate, and transition scores measure how plausible the implied movement between consecutive candidates is; the Viterbi algorithm then finds the most likely segment sequence. Map-matching is why the traffic pipeline’s first stage is a **processor**, not a database write — the raw fix is evidence, not fact.

DEFINITION — Sliding window

A moving time interval over a stream — “the last 15 minutes,” recomputed as time advances. A per-segment sliding-window average of observed speeds is our operational definition of “current speed”: old enough data falls out of the window and stops influencing the present, so the average tracks reality with a bounded lag and a bounded memory. Section 2.14 implements the window in about twenty lines.

Now the loop itself, end to end:



Every few minutes, reality → graph. ETAs and reroutes on new routes then use fresh weights automatically.

Trace one fix through the picture. Top row, left to right: your phone, mid- navigation, emits a GPS fix every five seconds; ingestion absorbs it (with 3M of its siblings every second) and drops it onto the geo-partitioned topic; the map-matcher consumes it and decides which segment you are really on; the aggregator folds your speed into that segment’s sliding window. Now the loop turns downward: the fresh per-segment average lands in the **traffic cache**, TTL ticking. From there the picture forks left along the bottom row, and the fork is the two places “current speed” becomes user-visible. One arrow feeds the **routing graphs**: edge weights update, mega-segment weights bubble up as their members change, and every **subsequent** route request quietly routes around the jam — no code path changed, because traffic was always just weights. The other arrow feeds the **traffic tiles**: segment speeds become green/yellow/red classes rendered into the overlay layer with its one-minute TTL, so the next pan shows the jam in red. Reality to pixels to routing decisions, in a few minutes, forever.

Three design details carry the interview discussion of this loop:

1. **Speed, not position, is the aggregate.** Per segment and window, we average the **speeds** of map-matched vehicles. One slow vehicle is noise — a delivery van at the curb with its hazards on; the average over dozens of vehicles in fifteen minutes is signal. And notice the happy alignment: because only segment statistics survive, nobody’s individual trace is stored in the traffic cache at all. Privacy and accuracy, for once, pull in the same direction.
2. **Sparse segments fall back to history.** At 3 a.m. on a rural road, the window holds too few samples to trust. Below a minimum count, we blend toward the **historical** speed for that segment, day-of-week, and time-of-day — and traffic is blessedly periodic, so Tuesday-8 a.m. looks remarkably like last Tuesday-8 a.m. The rule of thumb: live data where it exists, patterns where it does not, the speed limit as the floor of last resort.
3. **Third-party feeds are scored, not trusted.** We also ingest incident and flow feeds from external providers — and the talk is explicit about the discipline: validate each feed against our own observed reality. If a provider claims congestion on a corridor where our measured speeds never dropped, the claim is wrong; a provider wrong too often is down-weighted or ignored at the organization level. Every external signal is guilty until statistically innocent.

The same machinery, one more time, paints the traffic overlay of FR-6 — segment speeds become color classes, rendered into a **separate tile layer** with a TTL of a minute or two: short-lived tiles stacked on the long-lived base map, each layer cached at its own freshness, exactly as Section 2.6’s TTL discussion promised.

2.13 Deep Dive: The Turn-by-Turn Navigation Session

A navigation session is a small state machine per user, held in the navigation service: `ON_ROUTE → OFF_ROUTE_SUSPECTED → REROUTING → ON_ROUTE → ARRIVED`. Each incoming `location_update` advances it through four steps, and each step has a design decision embedded in it:

1. **Project.** Map-match the fix onto the planned route's polyline: how far along the route is the user, and how far **off** it? Projection turns raw geography into progress.
2. **Guide.** From progress, emit the next `instruction` early enough to be spoken — “in 300 meters, turn right” must arrive with time for the voice to say it and the driver to change lanes. Guidance is scheduled by distance to the maneuver, not by wall-clock.
3. **Refresh.** Recompute the ETA over the remaining segments using **current** weights from the traffic cache; push an `eta_update` only when the value moves by more than a small threshold. Users notice a jumping ETA and read it as the product being confused; hysteresis here is a **feature**, not sloppiness.
4. **Reroute.** If the fix sits beyond a distance threshold from the polyline **and** heading disagrees with the route — **sustained** across a few consecutive updates, because one bad fix must never trigger a reroute (GPS noise is constant) — recompute from the current position and push `reroute` with the new route. Section 2.14 implements this exact decision as the `RerouteFilter`.

The push path needs one piece of plumbing we deferred: the navigation service knows **what** to send but not **where the user's socket lives**. The **connection manager** — a replicated key-value registry mapping `user_id → gateway_id` — answers that question: gateways register each connection on connect and remove it on drop; the navigation service looks the user up and hands the message to the right gateway. If the registry entry is stale because a gateway died, the client's reconnect registers a fresh one within seconds, and at worst one push is retried. This is Chapter 1's stateless-gateway story, completed with its directory.

COMMON PITFALL — Rerouting on a single GPS fix

GPS in a downtown canyon can teleport a user two blocks sideways for one update and then snap back. A reroute fired on that ghost fix is a terrible user experience — the voice confidently announces a new route to a driver who never left the old one — and it has a subtler cost: it **pollutes the traffic pipeline** with a phantom slow-down on a road the user never touched. Both loops therefore consume map-matched, sustained signals, never raw single fixes. The same discipline, applied twice, in two subsystems that never meet.

INTERVIEW TIP — Navigation availability is a degradation story, not an uptime story

Four nines on the navigation path does not mean “the server never dies” — at our scale the server dies daily. It means **the phone copes when it does**. The client caches its route, its upcoming tiles, and its step list; if the session drops, guidance continues locally from the last known state while a new session is established in the background. Saying “the client is the final fallback layer of the server” out loud is exactly the kind of answer the 99.99% requirement is fishing for.

2.14 Deep Dive: Rust Reference Implementations

Four pieces of this chapter are small enough — and interview-critical enough — to show in full: tile addressing, the routing core, the traffic window, and the reroute decision. As you read them, keep mapping each one back to the section it implements; that mapping is what turns “I read about it” into “I can build it.”

2.14.1 Tile addressing: `lat/lng → tile → quadkey`

The slippy-map formulas of Section 2.6, plus the quadkey encoding that makes tiles prefix-searchable:

```

/// Web-Mercator slippy-map addressing (Section 2.6).

/// Which tile (x, y) at `zoom` contains this lat/lng?
pub fn lat_lng_to_tile(lat: f64, lng: f64, zoom: u8) -> (u32, u32) {
    let n = 2f64.powi(zoom as i32); // tiles per axis: 2^z
    let x = ((lng + 180.0) / 360.0 * n) as u32;
    let lat = lat.to_radians();
    let y = ((1.0 - lat.tan().asinh() / std::f64::consts::PI) / 2.0 * n) as u32;
    let max = n as u32 - 1;
    (x.min(max), y.min(max)) // clamp the antimeridian edge
}

/// The same address as a quadkey: one digit per zoom level, '0'..'3',
/// interleaving y/x bits from the most significant down. Neighboring tiles
/// share prefixes – a quadkey IS a quadtree path – so a plain sorted
/// key-value store can answer "all tiles in this area" as a range scan.
pub fn quadkey(x: u32, y: u32, zoom: u8) -> String {
    let mut key = String::with_capacity(zoom as usize);
    for level in (0..zoom).rev() {
        let mask = 1u32 << level;
        let mut digit = 0u8;
        if x & mask != 0 { digit += 1; }
        if y & mask != 0 { digit += 2; }
        key.push((b'0' + digit) as char);
    }
    key
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn whole_world_is_tile_zero() {
        assert_eq!(lat_lng_to_tile(0.0, 0.0, 0), (0, 0));
    }

    #[test]
    fn san_francisco_zoom_16() {
        // Verified against the slippy-map formulas.
        assert_eq!(lat_lng_to_tile(37.7749, -122.4194, 16), (10482, 25331));
        assert_eq!(quadkey(10482, 25331, 16), "023010203330032");
    }

    #[test]
    fn bing_canonical_quadkey() {
        // The documentation example: tile (3, 5) at level 3 -> "213".
        assert_eq!(quadkey(3, 5, 3), "213");
    }
}

```

Pause on the quadkey, because it is the bridge from map-serving to place search. Since a quadkey is a quadtree path, all tiles inside a region share the region’s key as a prefix — so if you index every place under the quadkey of its location, then “places near me” becomes “places whose key shares my tile’s prefix,” widened one ring of neighbors at a time. A two-dimensional spatial query collapses into a string-prefix range scan, and any sorted key-value store can serve it. Chapter 12’s geohash index is the same trick with a different alphabet.

2.14.2 The routing core: haversine, Dijkstra, A*

Section 2.8's algorithm ladder, executable. Weights are integer **milliseconds** of expected crossing time — exact arithmetic, and exactly what the traffic loop updates:

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

pub type NodeId = u32;

/// One directed segment leaving a node (Section 2.7).
#[derive(Clone, Copy, Debug)]
pub struct Edge {
    pub to: NodeId,
    pub millis: u32, // expected crossing time – the weight traffic moves
}

/// In-memory adjacency-list road graph (Section 2.7).
pub struct RoadGraph {
    pub adj: Vec<Vec<Edge>>,
    pub lat: Vec<f64>, // node coordinates, for haversine and A*
    pub lng: Vec<f64>,
}

/// Great-circle distance in meters (Section 2.8).
pub fn haversine_m(a: (f64, f64), b: (f64, f64)) -> f64 {
    const R: f64 = 6_371_000.0; // Earth radius, meters
    let (la1, la2) = (a.0.to_radians(), b.0.to_radians());
    let dla = (b.0 - a.0).to_radians();
    let dlo = (b.1 - a.1).to_radians();
    let h = (dla / 2.0).sin().powi(2) + la1.cos() * la2.cos() * (dlo / 2.0).sin().powi(2);
    2.0 * R * h.sqrt().asin()
}

/// Dijkstra: exact shortest path by total crossing time.
pub fn dijkstra(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>>) {
    {
        let mut dist = vec![u64::MAX; g.adj.len()];
        let mut prev = vec![u32::MAX; g.adj.len()];
        let mut heap = BinaryHeap::new();
        dist[source as usize] = 0;
        heap.push(Reverse((0u64, source)));
        while let Some(Reverse((d, u))) = heap.pop() {
            if u == target { break; } // popped => finalized => optimal
            if d > dist[u as usize] { continue; } // stale heap entry
            for e in &g.adj[u as usize] {
                let nd = d + e.millis as u64; // relax the edge
                if nd < dist[e.to as usize] {
                    dist[e.to as usize] = nd;
                    prev[e.to as usize] = u;
                    heap.push(Reverse((nd, e.to)));
                }
            }
        }
        if dist[target as usize] == u64::MAX { return None; }
        let mut path = vec![target]; // walk predecessors back
        while *path.last().unwrap() != source {
            path.push(prev[*path.last().unwrap() as usize]);
        }
        path.reverse();
        Some((dist[target as usize], path))
    }
}
```



```

}

/// A*: the same search, re-prioritized by an admissible heuristic —
/// remaining straight-line distance at the fastest plausible speed, so it
/// never overestimates (and, on a road graph, is *consistent*, which is what
/// licenses the early exit when the target is popped).
pub fn astar(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>)> {
    const MAX_SPEED_MPS: f64 = 70.0; // ~250 km/h: faster than any legal road
    let t = target as usize;
    let h = |n: NodeId| -> u64 {
        let i = n as usize;
        (haversine_m((g.lat[i], g.lng[i]), (g.lat[t], g.lng[t]))) / MAX_SPEED_MPS * 1000.0)
    } as u64;
    let mut dist = vec![u64::MAX; g.adj.len()];
    let mut prev = vec![u32::MAX; g.adj.len()];
    let mut heap = BinaryHeap::new();
    dist[source as usize] = 0;
    heap.push(Reverse((h(source), 0u64, source))); // (priority, dist, node)
    while let Some(Reverse((_, d, u))) = heap.pop() {
        if u == target { break; }
        if d > dist[u as usize] { continue; }
        for e in &g.adj[u as usize] {
            let nd = d + e.millis as u64;
            if nd < dist[e.to as usize] {
                dist[e.to as usize] = nd;
                prev[e.to as usize] = u;
                heap.push(Reverse((nd + h(e.to), nd, e.to)));
            }
        }
    }
    if dist[t] == u64::MAX { return None; }
    let mut path = vec![target];
    while *path.last().unwrap() != source {
        path.push(prev[*path.last().unwrap() as usize]);
    }
    path.reverse();
    Some((dist[t], path))
}

```

Read the two functions side by side and you will see that A* is Dijkstra — same loop, same relaxation, same predecessor walk — with a single difference: what the heap orders by. That is the honest way to present A* in an interview: not a new algorithm, but Dijkstra with its priorities aimed at the goal. And here is the test that demonstrates the chapter’s slogan — **traffic is just edge weights that move**:

```

#[cfg(test)]
mod tests {
    use super::*;

    /// A tiny city: two ways from node 0 to node 4, edges ~1.1-1.6 km.
    /// 0 -> 1 -> 2 -> 4 : 60 s + 60 s + 60 s = 180 s
    /// 0 -> 3 -> 4 : 30 s + 120 s = 150 s (normally fastest)
    /// (Times are consistent with the A* speed cap of 70 m/s, so the
    /// haversine heuristic stays admissible on this graph.)
    fn tiny_city() -> RoadGraph {
        let mut adj = vec![Vec::new(); 5];
        let e = |adj: &mut Vec<Vec<Edge>>, from: u32, to: u32, millis: u32| {
            adj[from as usize].push(Edge { to, millis });
            adj[to as usize].push(Edge { to: from, millis }); // two-way streets
        };
        e(&mut adj, 0, 1, 60);
        e(&mut adj, 1, 2, 60);
        e(&mut adj, 2, 4, 60);
        e(&mut adj, 0, 3, 30);
        e(&mut adj, 3, 4, 120);
    }
}

```

```

    };
    e(&mut adj, 0, 1, 60_000);
    e(&mut adj, 1, 2, 60_000);
    e(&mut adj, 2, 4, 60_000);
    e(&mut adj, 0, 3, 30_000);
    e(&mut adj, 3, 4, 120_000);
    RoadGraph {
        adj,
        lat: vec![0.0, 0.0, 0.01, -0.01, 0.0], // rough grid around origin
        lng: vec![0.0, 0.01, 0.01, 0.01, 0.02],
    }
}

#[test]
fn dijkstra_and_astar_agree() {
    let g = tiny_city();
    assert_eq!(dijkstra(&g, 0, 4).unwrap().0, 150_000);
    let (millis, path) = astar(&g, 0, 4).unwrap();
    assert_eq!(millis, 150_000);
    assert_eq!(path, vec![0, 3, 4]);
}

#[test]
fn traffic_reroutes() {
    let mut g = tiny_city();
    // Congestion report: the 3 -> 4 segment now takes 600 s.
    g.adj[3].iter_mut().find(|e| e.to == 4).unwrap().millis = 600_000;
    g.adj[4].iter_mut().find(|e| e.to == 3).unwrap().millis = 600_000;
    // No code path changes – the "traffic-aware" router is the same
    // algorithm reading different weights.
    assert_eq!(dijkstra(&g, 0, 4).unwrap().1, vec![0, 1, 2, 4]);
}
}

```

The `traffic_reroutes` test is the one to internalize. We mutate two weights — a jam forms on the 3→4 segment — and the **same unmodified Dijkstra** now returns the long way around. No “traffic mode,” no special case: the router does not know traffic exists. That is what Section 2.7 bought by making weight mean expected crossing time.

2.14.3 The traffic window: from GPS fixes to edge weights

The aggregator at the heart of Section 2.12: a per-segment sliding window of observed speeds, the live/historical blend, and the weight the graph stores.

```

use std::collections::VecDeque;

/// Per-segment sliding-window speed average (Section 2.12).
pub struct SpeedWindow {
    samples: VecDeque<(u64, f64)>, // (timestamp_secs, observed speed m/s)
    window_secs: u64,             // e.g. 15 * 60
}

impl SpeedWindow {
    pub fn new(window_secs: u64) -> Self {
        Self { samples: VecDeque::new(), window_secs }
    }

    pub fn add(&mut self, now: u64, speed_mps: f64) {
        self.samples.push_back((now, speed_mps));
        self.evict(now);
    }
}

```

```

    }

    fn evict(&mut self, now: u64) {
        while let Some(&(t, _)) = self.samples.front() {
            if now.saturating_sub(t) > self.window_secs { self.samples.pop_front(); }
            else { break; }
        }
    }

    /// Current average, or None when there is too little live data –
    /// the caller then blends in the historical pattern (Section 2.12).
    pub fn average(&mut self, now: u64, min_samples: usize) -> Option<f64> {
        self.evict(now);
        if self.samples.len() < min_samples { return None; }
        let sum: f64 = self.samples.iter().map(|&(_, s)| s).sum();
        Some(sum / self.samples.len() as f64)
    }
}

/// Effective speed for a segment: live average where we trust it,
/// the historical pattern for this day/time otherwise, and never above
/// the posted limit (we do not route people as if speeding were planned).
pub fn effective_speed(live: Option<f64>, historical: f64, limit: f64) -> f64 {
    live.unwrap_or(historical).min(limit)
}

/// The edge weight the routing graph stores (Section 2.7): expected
/// crossing time. Clamped away from zero so weights stay finite and
/// Dijkstra's non-negativity requirement is never endangered.
pub fn weight_millis(segment_length_m: f64, speed_mps: f64) -> u32 {
    (segment_length_m / speed_mps.max(0.5) * 1000.0) as u32
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn window_evicts_old_samples() {
        let mut w = SpeedWindow::new(900); // 15-minute window
        w.add(1_000, 30.0); // highway at full speed
        w.add(1_700, 8.0); // jam begins
        w.add(1_800, 9.0);
        // The t=1000 sample is now 800 s old: inside the window.
        assert_eq!(w.samples.len(), 3);
        w.add(2_000, 10.0);
        // t=1000 is 1000 s old: evicted. Average over the jam only.
        let avg = w.average(2_000, 3).unwrap();
        assert!((avg - 9.0).abs() < 0.01);
    }

    #[test]
    fn sparse_segments_fall_back_to_history() {
        let mut w = SpeedWindow::new(900);
        w.add(100, 22.0); // one car at 3 a.m.
        assert_eq!(w.average(200, 10), None); // not enough samples
        // Historical pattern says 20 m/s here at this day/time; limit 25.
        assert_eq!(effective_speed(None, 20.0, 25.0), 20.0);
        // Live data says 30 m/s, but we never plan on speeding.
        assert_eq!(effective_speed(Some(30.0), 20.0, 25.0), 25.0);
    }
}

```

```

    }

    #[test]
    fn weight_is_crossing_time() {
        assert_eq!(weight_millis(1_000.0, 10.0), 100_000); // 1 km at 10 m/s
        assert!(weight_millis(1_000.0, 0.0) > 0);          // clamped, finite
    }
}

```

Notice how `average` returning `Option<f64>` makes the sparse-data fallback **type-safe**: “we don’t have enough live data” is not a sentinel value or an error — it is `None`, and the compiler forces the caller to handle it. And `weight_millis` clamping speed away from zero is a quiet act of care: a segment gridlocked to a standstill must produce a huge weight, not a division by zero or an infinite crossing time that breaks Dijkstra’s assumptions.

2.14.4 The reroute decision

Section 2.13’s state machine, reduced to its durable core: distance from the planned polyline, sustained across fixes. (Distance uses a local flat-earth approximation — within a few hundred meters of the route, the Earth’s curvature is below GPS noise; production systems run the full map-matching of Section 2.12 here.)

```

/// Approximate meters-per-degree at a given latitude: 111.32 km per degree
/// of latitude everywhere; longitude degrees shrink with cos(latitude).
fn meters_per_deg(lat: f64) -> (f64, f64) {
    (111_320.0, 111_320.0 * lat.to_radians().cos())
}

/// Distance from point p to segment a-b, in meters (local approximation).
fn point_segment_m(p: (f64, f64), a: (f64, f64), b: (f64, f64)) -> f64 {
    let (m_lat, m_lng) = meters_per_deg(p.0);
    let (px, py) = (p.1 * m_lng, p.0 * m_lat);
    let (ax, ay) = (a.1 * m_lng, a.0 * m_lat);
    let (bx, by) = (b.1 * m_lng, b.0 * m_lat);
    let (dx, dy) = (bx - ax, by - ay);
    let len2 = dx * dx + dy * dy;
    let t = if len2 == 0.0 { 0.0 } else {
        (((px - ax) * dx + (py - ay) * dy) / len2).clamp(0.0, 1.0)
    };
    let (cx, cy) = (ax + t * dx, ay + t * dy); // closest point on a-b
    ((px - cx).powi(2) + (py - cy).powi(2)).sqrt()
}

/// Distance from a fix to the nearest point anywhere on the route polyline.
pub fn distance_to_route_m(p: (f64, f64), route: &[(f64, f64)]) -> f64 {
    route.windows(2)
        .map(|w| point_segment_m(p, w[0], w[1]))
        .fold(f64::MAX, f64::min)
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Guidance { OnRoute, OffRouteSuspected, Reroute }

/// The hysteresis filter: ONE noisy fix must never trigger a reroute
/// (Section 2.13's pitfall). `strikes` counts consecutive off-route fixes.
pub struct RerouteFilter {
    pub distance_threshold_m: f64, // e.g. 40 m
    pub required_consecutive: u8,  // e.g. 3 fixes
    strikes: u8,
}

```

```

}

impl RerouteFilter {
    pub fn new(distance_threshold_m: f64, required_consecutive: u8) -> Self {
        Self { distance_threshold_m, required_consecutive, strikes: 0 }
    }

    pub fn update(&mut self, fix: (f64, f64), route: &[(f64, f64)]) -> Guidance {
        if distance_to_route_m(fix, route) > self.distance_threshold_m {
            self.strikes += 1;
        } else {
            self.strikes = 0;
        }
        if self.strikes >= self.required_consecutive { Guidance::Reroute }
        else if self.strikes > 0 { Guidance::OffRouteSuspected }
        else { Guidance::OnRoute }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    // A straight eastbound route along lat 37.0.
    fn route() -> Vec<(f64, f64)> { vec![(37.0, -122.0), (37.0, -121.9)] }

    #[test]
    fn on_route_fix_stays_quiet() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0001, -121.95), &route()), Guidance::OnRoute);
    }

    #[test]
    fn sustained_deviation_reroutes() {
        let mut f = RerouteFilter::new(40.0, 3);
        // ~56 m north of the route: beyond threshold.
        let lost = (37.0005, -121.95);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::Reroute);
    }

    #[test]
    fn one_ghost_fix_recovers() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0005, -121.95), &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update((37.0, -121.95), &route()), Guidance::OnRoute);
    }
}

```

The three tests are the three user stories: the normal drive (stay quiet), the genuine wrong turn (three strikes, then reroute), and the downtown ghost fix (suspect once, recover, never bother the driver). Forty lines of Rust, and the entire “what should happen when the user misses a turn?” discussion has a falsifiable answer.

2.15 Scaling & Geographic Sharding

Chapter 1 defined sharding and warned that the art is in the key. Here the key chooses itself: **geography**. Every workload in this system is naturally located — tiles are places, segments are places, GPS fixes are places — and that makes the sharding table almost suspiciously clean:

Tier	Sharding strategy	Why
Tiles	No sharding logic at all — the (z, x, y) address space partitions naturally; CDN edges cache by URL	Addressing is the partition scheme; popularity skew does the rest
Road graph	Partitioned by region (continent → subregion); each shard is a full in-memory copy of its region's CH graph, replicated	A route is overwhelmingly inside one region; replicas add both capacity and failover
Cross-region routes	Solved hierarchically: mega-segments cross region borders at a handful of highway gateways	The hierarchy of Section 2.8 doubles as the distributed-systems answer
GPS stream	Topic partitioned by geohash prefix; each stream-processor instance owns a disjoint set of cells	Map-matching and aggregation are embarrassingly parallel over geography
Traffic cache	Sharded by <code>segment_id</code> hash; total size 500 MB	Tiny, hot, and latency-critical — keep it in memory, close to routing
Place index	Replicated per region; sharded by quadkey prefix within a region	Search is read-heavy and geographically scoped (Section 2.14)

Linger on the third row, because it is the one that looks like a problem and is not. “What about a route from Lisbon to Warsaw — two graph shards?” feels like it should force distributed queries. It does not, and the reason is beautiful: Section 2.8's hierarchy, which we built to make search fast, **also** makes distribution unnecessary. A Lisbon–Warsaw query climbs to the highway level inside the origin region's shard, crosses on mega-segments that span the border at a handful of highway gateways, and descends inside the destination shard. The algorithmic optimization and the scaling strategy are the same object.

KEY INSIGHT — Geography is a forgiving shard key

Compare geography to Chapter 1's key — the document — and you will see why this chapter's scaling story is calm. Documents grow legs: a shared document can go viral, concentrate fifty editors, and migrate between owners. Geography does none of that. A region's road graph changes on the scale of weeks; its traffic weights change on the scale of minutes but are **regenerated** in place, never migrated. The one genuinely hot spot — a stadium emptying at 11 p.m. — is absorbed **inside** one partition by the aggregation pipeline, not spread across the system. When your shard key is this well-behaved, most of distributed systems' horror stories simply do not apply to you.

2.16 Failure Modes & Recovery

The enumeration, before the interviewer asks for it — with the pattern from Chapter 1 still in force: stateless things are replaced, stateful things are rebuilt from the log, and degradation is designed rather than improvised.

Failure	Handling
WS gateway dies	Clients reconnect to any gateway; the connection manager re-registers them; the navigation service resumes pushes to the new socket. Unacknowledged client updates are retried and deduplicated — Chapter 1’s idempotency discipline, unchanged.
Navigation node dies	Sessions are small: a route plus progress along it. The client’s reconnect carries its last known position; a fresh node rebuilds the session from the route store and continues. The user sees a pause, not a crash.
Stream backpressure / Kafka lag	Freshness degrades first and visibly : per-segment entries age past their TTL, and the system slides to historical speeds automatically — Section 2.12’s fallback is the degradation mode, by design, not by apology. Recovery replays the log.
Traffic cache loss	Rebuilt from the stream within one window (15 min); historical speeds serve in the meantime and nobody pages anyone.
Routing node dies	Each region shard is replicated; traffic shifts to a replica. A lost node reloads its graph from the segment store plus the traffic cache (Section 2.10).
Tile origin down	The CDN keeps serving — 95%+ of requests never knew the origin existed. <code>stale-while-revalidate</code> semantics make even expired tiles safe, since the base map changes weekly at worst.
Bad or spoofed GPS input	Per-segment sample thresholds ignore singletons; per-source scoring quarantines systematically wrong feeds (Section 2.12).
Full region outage	The phone is the last fallback: cached route, tiles, and step list keep guiding offline (Section 2.13) while sessions re-home to another region.

Read the third row twice. In most systems, “the pipeline is behind” is a page-worthy emergency. Here it is a **designed, user-visible, self-healing degradation**: colors fade to historical patterns, ETAs lean on Tuesday-8 a.m. instead of right now, and when the pipeline catches up, the live colors come back. The freshness NFR is what makes this softness possible — a system whose promise is “recent” can always fall back to “typical.”

2.17 Trade-offs & Alternatives

The ledger, benefit against cost, as always:

Decision	Benefit purchased	Cost accepted
Vector tiles over raster	50% fewer bytes; restyle without re-render; smooth zoom	The client’s GPU does the work; a raster fallback must exist for old clients
CH / mega-segments over plain A*	1000× query speedup; the only way 20k QPS fits a small fleet	Hours of preprocessing; shortcut weights must be rebuilt or bubbled up

Decision	Benefit purchased	Cost accepted
		when traffic moves — extra machinery (Section 2.8)
WebSocket push for navigation	Reroutes and ETA updates arrive instantly; one connection also carries GPS upstream	Millions of long-lived connections to hold, and a registry (the connection manager) to find them
Hybrid live + historical traffic	Coverage everywhere; graceful under sparse data and pipeline lag	Two data paths to keep consistent, and blending logic to get right
Quadkey/geohash indexing	Proximity becomes a string-prefix range scan in ordinary key-value stores	Cell-boundary artifacts — a place just across a boundary is missed unless you always query the neighboring too
Precompute over on-demand render	Tiles and hierarchy make reads $O(\text{cache hit})$	A planet of batch jobs to run, version, and roll back

The last row is the chapter's thesis wearing a cost column. Moving work off the request path bought us every latency number we promised — and the bill arrives as operational surface area: batch renderers, graph rebuilds, warehouse jobs, each with versions and rollbacks and monitors of their own. There is no free millisecond; there are only milliseconds you paid for in advance.

2.18 Observability & SLOs

The accuracy requirement ($\pm 10\%$ ETA) cannot be an aspiration; it must be a measured, per-city, per-hour number. That needs a metric:

DEFINITION — MAPE (mean absolute percentage error)

The average of $|\text{predicted} - \text{actual}| / \text{actual}$ over many predictions — the standard accuracy metric for forecasts, and the natural SLI for ETA quality. Here we are unusually lucky: per **completed** trip, we know both the ETA we gave and the travel time that reality delivered, so ETA MAPE is measurable exactly, per city, per hour, per route class. The source talk is emphatic on this point and it deserves the emphasis: you cannot improve what you do not score, and ETA accuracy is the metric this product lives by.

Beyond MAPE, the dashboard that keeps this system honest: **route acceptance rate** — how often users actually drive the recommended route, because a recommendation nobody takes is a signal something is wrong with the routes (the talk's analytics discussion makes exactly this point); reroute rate per session, which is route quality measured by contradiction; tile cache hit ratio and origin QPS, guarding the 95% promise of Section 2.6; route latency p50/p99; pipeline lag from fix to weight update, the freshness SLI in seconds; median age of live segment speeds; and navigation session drop-and-recovery times, the 99.99% promise measured where the user feels it.

2.19 Interview Wrap-Up

The likely follow-ups, with the shape of a strong answer for each:

- “*Offline maps?*” — Ship regional bundles (tiles plus the contracted graph) to the device; routing runs locally; traffic and reroute quality silently degrade until reconnect. Notice that everything we precomputed server-side is **exactly** what the bundle contains — the theme of precomputation, one final time.
- “*Public transit?*” — A **time-dependent** graph, where edges exist only when a vehicle does. Dijkstra generalizes, but production transit uses schedule-based algorithms (RAPTOR and friends). Name it, admire it, do not build it in the interview.
- “*GPS noise in urban canyons?*” — The hidden-Markov map-matching of Section 2.12, plus sensor fusion: accelerometer, gyroscope, and wheel ticks where the vehicle offers them.
- “*Better ETAs?*” — Features into a learned model: current and historical speeds, weather, events, turn and signal penalties; graph neural networks can model congestion **spreading** along the road graph. Google reported up to 50% ETA-error reductions in some cities from exactly this move. Say the number as motivation, then defend the simple hybrid of Section 2.12 as the baseline any model must beat.
- “*Walking and cycling?*” — Same graph, same algorithms, different edge eligibility and weights: stairs, bike lanes, no highways. The design’s generality is the answer.
- “*Location privacy?*” — Aggregate, don’t store: segment statistics instead of traces, retention limits on raw updates, and minimum sample counts that double as k-anonymity for sparse segments. Section 2.12’s design made the private choice the convenient one — point that out.

If you remember five things:

1. The map is a precomputed pyramid of addressable squares; the CDN, not your fleet, serves it.
2. Roads are a time-weighted directed graph, and **traffic is just edge weights that move**.
3. Dijkstra is exact and unusable; bidirectional and A* are free wins; hierarchy (mega-segments, contraction hierarchies) is the 1000× that turns an algorithm into a product.
4. Live traffic is a streaming pipeline: GPS fix → map-match → per-segment windowed average → weights and overlay tiles, with history as the fallback.
5. Navigation correctness lives on the phone: the client caches the route and survives the server.

2.20 Summary & Further Reading

We designed a maps and navigation service for 1B monthly users, and — if the chapter did its job — every piece felt **derived** rather than invented: a pre-rendered, CDN-served tile pyramid addressed by `(z, x, y)` and quadkeys; a time-weighted road graph held in memory and searched with an algorithm ladder that ends at contraction hierarchies; a live traffic loop that turns 3M GPS updates per second into per-segment speeds, edge weights, and overlay tiles, with historical patterns as the fallback; and a turn-by-turn navigation service that guides, refreshes ETAs, and reroutes with hysteresis — degrading gracefully to the phone when the network, or we, fail.

Primary source for this chapter:

- “**13: Google Maps**” — [Systems Design Interview Questions With Ex-Google SWE \(Jordan has no life\)](#) — the walkthrough this chapter expands.

Foundations worth reading:

- The OSRM and Valhalla open-source routing engines — contraction hierarchies in production code, not just papers.
- Geisberger et al., *Exact Routing in Large Road Networks Using Contraction Hierarchies* (2008) — the CH formulation.
- Newson & Krumm, *Hidden Markov Map Matching Through Noise and Sparseness* (2009) — the map-matching standard.
- Google’s S2 geometry library documentation, and Uber’s H3 — the real spatial indexes behind quadkey-style addressing (Chapter 12 uses H3’s geohash cousin directly).

- The Bing Maps tile system documentation — the canonical quadkey reference.
- Google DeepMind’s write-up on learning ETAs with graph neural networks (2020) — the ML direction for Section 2.19’s follow-up.

2.21 Chapter 2 Glossary

A one-glance index of every term this chapter defined. Chapter 1’s glossary is assumed; later chapters assume both.

Term	Meaning in one line
ETA	Predicted travel time / arrival time; the quality bar of this chapter
Freshness	Age of the data behind an answer; traffic targets minutes
Map tile	Fixed 256×256 square of map; the unit of storage, cache, transfer
Zoom level	Scale integer z ; each step splits every tile into four
Slippy-map scheme	Deterministic (z, x, y) tile addressing over Web Mercator
Quadkey	Tile address as a digit string; a quadtree path, prefix-searchable
CDN	Planet-wide edge caches that absorb static reads near the user
TTL / hit ratio	How long cache entries may live / fraction of requests they absorb
Content addressing	Identical tiles (oceans) stored once, by content hash
Polyline encoding	Compact ASCII encoding of a route’s coordinate sequence
Graph	Nodes + edges; directed and weighted for roads
Segment	One directed road piece between nodes; the unit of traffic
Adjacency list	Per-node outgoing-edge lists; the in-memory graph layout
Shortest path	Minimum-total-weight route; with time weights, the fastest path
Dijkstra’s algorithm	Exact breadth-by-distance search with a priority queue
Priority queue	$O(\log n)$ “give me the current minimum” structure
Bidirectional search	Two half-searches meeting mid-route; half the work
A* / heuristic	Dijkstra steered by a remaining-distance estimate
Admissible heuristic	Never overestimates; preserves exactness
Haversine distance	Great-circle distance between coordinates
Mega-segment	Precomputed synthetic edge summarizing a segment chain
Contraction hierarchy	Node-importance ordering + shortcuts; μ s–ms queries
Kafka-class stream	Durable partitioned event log; decouples and replays
Stream processing	Computing over data as it arrives, not in batch
Map-matching	Snapping noisy GPS fixes to the true road segment
Sliding window	Moving time interval defining “current” speed per segment
Polyglot persistence	Different engines for different access patterns
Connection manager	Registry of which gateway holds which user’s socket

Term	Meaning in one line
MAPE	Mean absolute percentage error; the ETA accuracy SLI

— End of Chapter 2 · Next: Chapter 3 —

Designing a Distributed Rate Limiter

PROBLEM SOURCE

This chapter solves the problem posed in the talk “7: Design a Rate Limiter” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The talk designs a rate-limiting service for a public API: why limiting is a business feature and not just a shield, the classic limiting algorithms, and how to enforce limits correctly across a fleet of stateless servers. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol details, and Rust reference implementations.

3.1 The Problem Statement

The interview for this one starts deceptively quietly. The interviewer looks up and says:

“Design a rate limiter. Our public API is used by a million developers, and we need to make sure no single caller can overwhelm the service — or use more than their plan allows.”

If Chapters 1 and 2 felt like designing **products**, this one will feel different, and it is worth noticing why. You are not designing something a user ever sees; you are designing a piece of **infrastructure** — a component that sits in front of every single request your company receives and decides, in well under a millisecond, whether that request is allowed to proceed. Nobody visits your rate limiter. Everybody passes through it. That changes what “good” means: the best limiter is one nobody knows exists until the day it saves the service.

And here is the trap the prompt lays for you. The **concept** is trivially simple — “count requests, reject past a threshold” is a five-line program on one machine, and if you say so out loud you will watch the interviewer’s interest drain away in real time. The difficulty is not the concept; it is the **setting**. The counting has to happen correctly and cheaply on the hot path of a distributed system: fifty gateway servers must enforce **one shared limit** per caller, under heavy concurrency, without adding meaningful latency, and — the subtlest requirement of all — without the limiter itself ever becoming the outage it exists to prevent. Every interesting decision in this chapter is a consequence of those four words: correct, cheap, shared, and safe to fail.

Before we touch any design, let us define the vocabulary precisely, because half of all weak interview answers on this topic come from confusing two words that sound interchangeable and are not.

DEFINITION — Rate limiting / throttling

Controlling the **rate** at which a caller may invoke a system: at most n requests per time window per identity (API key, user, IP address, tenant). Calls beyond the limit are rejected (or queued, or slowed — hence *throttling*). Rate limiting serves three distinct masters: **protection** (abuse, brute force, accidental stampede), **capacity** (one noisy caller must not starve the rest), and **monetization** (usage tiers are limits with a price tag). Keep all three in mind: they produce different requirements.

It is worth pausing on those three masters, because they will quietly steer every later trade-off. If your limiter exists for **protection**, approximate enforcement is fine — a brute-forcer who gets 4% extra attempts is still thwarted. If it exists for **capacity**, you care mostly about worst-case fairness, and small per-caller errors average out. But if it exists for **monetization** — if a paying customer's contract says "1,000 requests per second" — then the limiter is adjacent to billing, and suddenly every rejection had better be correct and every admission had better be defensible, because customers will audit you. Same component, three different accuracy budgets. When the interviewer asks "how exact does this need to be?", this is the reasoning they are hoping you will produce unprompted.

DEFINITION — Quota vs. rate limit

A *quota* is a budget over a long period ("100,000 calls per month") used for billing and plan enforcement; it tolerates approximate, batched accounting. A *rate limit* governs short windows ("100 calls per second") to shape live traffic; it must be enforced **now**, on the request's critical path. This chapter is about rate limits; Section 3.18 discusses how quotas differ.

Why does the distinction matter so much? Because the two systems live in different worlds. A quota can be computed an hour late from a log file, and nobody suffers — the invoice is still right. A rate limit computed an hour late is not a rate limit at all; the damage it was meant to prevent has already happened. This single fact — **the decision must be made before the request runs** — is what forces everything you are about to design onto the critical path, with all the latency and availability pain that implies. Hold onto it.

3.2 Scope & Clarifying Questions

Look again at the prompt and count what it leaves unspecified. Are we limiting HTTP calls, messages, bytes, login attempts? One limit for everyone, or tiers? How exact must enforcement be? What happens when the limiter's own machinery breaks? The prompt spans everything from login-page brute-force protection to planetary DDoS absorption, and a design that tries to cover all of that will cover none of it well. So you narrow it — out loud, with the interviewer, the way you would scope a real project with a colleague. Watch how the exchange buys information, question by question:

Speaker	Dialogue
Candidate	"What are we limiting — HTTP API calls, or something else like messages or bytes?"
Interviewer	"HTTP requests to our public REST API. Limit per API key."
Candidate	"Are limits the same for everyone, or tiered — free keys vs. paid plans?"
Interviewer	"Tiered. Rules must be configurable per key and per route, and changeable without redeploying anything."
Candidate	"Scale of the API being protected?"
Interviewer	"Peak 200,000 requests per second, spread across a fleet of about 50 stateless gateway servers. One million registered API keys."
Candidate	"How exact must enforcement be? If a key is limited to 100 requests per second, is 101 a scandal?"
Interviewer	"Never reject a caller who is under their limit. Small overshoot during failures is acceptable; systematic overshoot is not."
Candidate	"How much latency may the check add to each request?"

Speaker	Dialogue
Interviewer	“A millisecond or two at most — this runs on every single request.”
Candidate	“And if the limiter’s own state store is down — block everything, or let traffic through?”
Interviewer	“Great question. Decide, and defend the decision.”

Read that dialogue again and notice what each answer **unlocks**. The first answer tells you the identity you count by: the API key, which is convenient because it arrives with every request and shards beautifully. The second tells you limits are **configuration**, not code — which means a rules subsystem with runtime reload is a requirement, not a luxury. The third gives you the numbers that decide whether one machine can hold all the state (it can, almost — wait for Section 3.5’s arithmetic). The fourth is the most valuable sentence in the whole interview: it tells you the accuracy requirement is **asymmetric**, and we will build the entire error policy on that asymmetry. The fifth hands you a latency budget so small it eliminates whole categories of designs before you draw a single box. And the last one — well, the last one is the interview, as the tip below explains.

AGREED SCOPE

Per-API-key rate limits on a public REST API; tiered limits, configurable per key and per route, hot-reloadable; enforced **globally** across 50 stateless gateway nodes at **200k RPS** peak; $\leq 1\text{--}2$ ms added latency; **no false rejections**, bounded overshoot; and an explicit, defended policy for limiter failure (fail-open vs. fail-closed — Section 3.15).

INTERVIEW TIP — The last question is the interview

“What do we do when the limiter breaks?” separates candidates who build components from candidates who build **systems**. Ask it yourself if the interviewer doesn’t — every subsequent design decision (centralized store, local fallbacks, timeouts) is downstream of the answer. A limiter that takes the API down with it has inverted its purpose: it was hired to absorb failures, not to manufacture them. Everything in Sections 3.10 through 3.15 is shaped by keeping that inversion impossible.

3.3 Functional Requirements

Chapter 1 defined functional requirements as the promises the system makes about what it **does**. Six promises cover this problem — and notice, as you read them, that each one traces back to a specific line of the dialogue you just conducted. That traceability is not decoration; in the interview, it is how you prove the requirements came from the conversation rather than from a template you memorized.

1. **FR-1 — Enforcement.** A caller exceeding its limit receives HTTP **429 Too Many Requests** with a `Retry-After` hint; everyone else passes through.
2. **FR-2 — Configurable rules.** Limits are defined per API key, optionally per route or method, with named tiers (free, pro, enterprise); an admin API changes them at runtime.
3. **FR-3 — Transparency.** Responses carry the caller’s limit state (limit, remaining, reset time) in standard headers, so well-behaved clients can self-throttle **before** being rejected.
4. **FR-4 — Global enforcement.** The limit holds across the entire gateway fleet, not per server — a caller with 100 req/s cannot get 5,000 req/s by being load-balanced across 50 nodes.
5. **FR-5 — Burst policy.** Short bursts above the sustained rate are allowed up to a defined allowance (real traffic is bursty; rejecting every microburst punishes normal clients).
6. **FR-6 — No false rejections.** A caller under its limit is never rejected. (Overshoot is a soft failure; false rejection is a hard one — it breaks innocent customers deterministically.)

Three of these deserve a second look because they are easy to state and hard to honor. FR-3 sounds like politeness — and it is — but it is also load shedding: every header you hand a well-behaved client is a rejected request that never has to be sent, retried, or counted. Transparency is cheaper than enforcement. FR-4 is the requirement that kills the naive “count on each server” design before it starts; Section 3.6 is entirely about why. And FR-5 encodes a fact about real traffic that your algorithm choice must respect: legitimate callers do not arrive in a smooth stream, they arrive in clumps — a mobile app syncing after the subway, a CI pipeline fanning out, a cron job at the top of the minute. An algorithm that treats any clump as abuse will generate false rejections against perfectly innocent customers, which FR-6 forbids. Keep FR-5 and FR-6 side by side in your head; the tension between them is exactly what the token bucket in Section 3.7 resolves.

3.4 Non-Functional Requirements

Functional requirements tell you what to build; non-functional requirements tell you what will get you fired if you build it naively. Three qualities dominate this problem, and here is the part most candidates miss: **they pull against each other**. Naming that tension explicitly — before drawing anything — is half the answer, because it converts three vague adjectives into a triangle you can navigate deliberately instead of discovering by accident.

Quality	Target
Added latency	≤ 1 ms in-region, p99 — the check is on every request’s critical path
Accuracy	Zero false rejections; steady-state overshoot within 1% of the limit; degraded-mode overshoot explicitly bounded
Availability of the API	The limiter must never be a new single point of failure: if the limiter fails, the API stays up (fail-open), with a local safety cap
Scale	200k RPS enforcement across 50 nodes; 1M keys; 100M+ active keys per day tolerable
Config freshness	Rule changes propagate to all gateways within seconds

Walk the rows and feel the constraints bite. The latency row says the check is on **every** request’s critical path — so whatever machinery you invent, its cost is multiplied by 200,000 per second and paid by every caller, including the ones nowhere near their limits. A design that is “only a little slow” still makes the entire API a little slow, forever. The accuracy row encodes the asymmetry the interviewer handed you: one direction of error is forbidden, the other merely rationed — say that sentence in the interview and watch the room warm to you. The availability row is FR-6’s older sibling: the limiter is a bodyguard, and a bodyguard who shoots the client during a faint is worse than no bodyguard. Scale and config freshness are quieter, but both eliminate options — the first rules out single-node solutions, the second rules out “redploy to change a limit.”

DEFINITION — False rejection / overshoot

The two ways a limiter can be wrong. A *false rejection* denies a caller who is under the limit — an availability bug, visible and unforgivable. *Overshoot* allows more than the limit — a protection degradation, tolerable in small doses. Accuracy requirements should always be stated in this asymmetric vocabulary, because the two errors have opposite costs and different causes.

This asymmetry deserves one more paragraph, because it will quietly govern Sections 3.12 and 3.15. A false rejection lands on a customer who did everything right; it is deterministic, reproducible, and it will be reported, screenshotted, and escalated. Overshoot, by contrast, is statistical — a few extra requests slip through a window, the backend absorbs them (your capacity planning already

assumed spikes far larger), and nobody notices anything but a metric. So when you are forced to choose which error your failure mode commits, you already know the answer: err toward letting traffic through, bound how much, and make the bound observable. You will see this principle return wearing different clothes in the failure-mode table of Section 3.15.

KEY INSIGHT — Latency, accuracy, availability: pick the trade-off consciously

Exact global counting wants a synchronous round trip to a strongly consistent store (latency + availability cost). **Zero added latency** wants purely local counting (accuracy cost: 50 nodes × local limit). **Maximum availability** wants fail-open everywhere (protection cost). Every real design is a point in this triangle; the rest of the chapter locates ours and prices it. The skill on display is not finding a design with no costs — there is none — it is demonstrating that you know which vertex you are nearest, what you paid to stand there, and what it would take to move.

3.5 Back-of-the-Envelope Estimation

Before choosing anything, run the numbers — not because the interviewer loves arithmetic, but because five minutes of estimation here will tell you **what kind of problem this actually is**, and it is probably not the kind you expected. Chapter 1 taught the discipline: state every assumption, derive only what follows.

Assumptions (stated, per Chapter 1’s discipline):

- Peak gateway traffic: **200k requests/sec**; every request needs exactly one limit check.
- 1M registered keys; up to **100M keys active in a day** (many keys are used once and idle for weeks).
- Typical limit: 100 req/s per key; token-bucket state per active key is two numbers plus the key itself — 50–60 bytes.
- A modern in-memory store serves 100k simple ops/sec per shard.

Derived numbers:

Quantity	Estimate	How
Check operations	200k ops/s peak	one per request; reads+writes combined
Store shards needed	4–6 (+ replicas)	200k ops/s ÷ 100k ops/s per shard, with headroom
State memory (counters)	≈ 6 GB worst case	100M active keys × 60 B
State memory (sliding log)	≈ 800 GB worst case	100M keys × up to 1,000 time-stamps × 8 B — infeasible
Added latency budget	≤ 1 ms	one in-region round trip, or zero with local state
Config size	a few MB	1M keys × rule record; trivially cacheable everywhere

Two rows in that table do the real work. The **counters** row says the entire planet of state fits in six gigabytes — a rounding error on modern hardware. The **sliding log** row, two lines below, says that one particular algorithm for the same problem would need eight hundred gigabytes. Same requirement, same scale, a hundred-fold difference produced purely by **what you choose to remember per key**. That is your first concrete lesson in algorithm selection, and it arrives before you have drawn a single box: memory per key is a design axis, and the sliding log loses on it so badly that it is eliminated on

paper. (This is why Section 3.7 calls the log “the algorithm you describe to show you know the trade-off, not the one you ship.”)

The config row earns a mention too: a few megabytes of rules, read by every request, means the only sane placement is **inside each gateway’s process memory** — you will see that decision resurface as the rule cache in Section 3.10’s architecture.

KEY INSIGHT — The scarce resource is round trips, not hardware

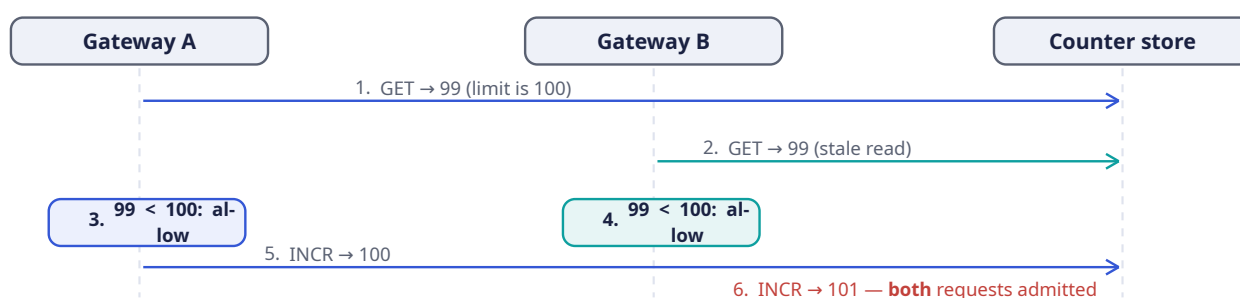
Six gigabytes of state and a few hundred thousand ops per second are, by Chapter 2’s standards, nothing. The entire design problem is: **where does the check happen relative to the request?** A centralized exact check costs one network round trip (1 ms in-region — the whole budget). A local check costs zero round trips but is approximate. Estimation tells you this is a **latency topology** problem, not a capacity problem — design accordingly. Candidates who skip the estimation step routinely reach for complex distributed machinery to solve a capacity problem this system does not have, while ignoring the placement problem it does have. Do the arithmetic; let it aim your attention.

3.6 The Core Challenge: Counting Correctly Under Concurrency

Now you can state the real problem in one sentence: the limit check is a read-modify-write. **Read** the caller’s count, **decide** whether another request fits, **write** the incremented count. Three steps, and the correctness of the whole system lives in the seams between them. Two naive placements of this operation fail, and both failures are instructive — walk through them slowly, because the production design in Section 3.10 is nothing more than “the naive design with both failure modes engineered out.”

Naive strategy 1 — count locally on each gateway. This is the placement your latency budget loves: no shared state, no network hop, zero added latency, and nothing to fail. It dies on a single observation: the limit is **per key**, and a key’s requests land on all 50 gateways, because that is what a load balancer is **for**. If each node enforces 100 req/s locally, a caller spraying requests across the fleet effectively holds $100 \times 50 = 5,000 \text{ req/s}$ — fifty times the limit, exactly proportional to your fleet size, and FR-4 lies in ruins. Your first instinct might be to divide: give each node $100/50 = 2 \text{ req/s}$ locally. But now watch what happens to an innocent caller whose requests happen to hash onto one node for a while — they are strangled at 2 req/s against a 100 req/s contract, which is a false rejection, which FR-6 forbids. Per-instance limits cannot express a global limit. There is no divisor that fixes this, because the flaw is not the arithmetic — it is that the **state** is in the wrong place. The count is a property of the caller, so the count must live somewhere all the caller’s requests can reach.

Naive strategy 2 — a shared counter store with plain reads and writes. Fine, you say: put the counters in one shared, in-memory store, and have each gateway do `GET`, compare, `INCR`. State placement fixed. But you have just walked into the oldest trap in concurrent programming, and the diagram below shows it happening in six numbered steps. Two gateways, one shared store, one unlucky interleaving:



Take this diagram apart slowly, because understanding exactly **why** it breaks is the whole interview in miniature. Time flows downward along the three vertical lifelines — Gateway A on the left, Gateway

B in the middle, the shared counter store on the right — and each horizontal arrow is a network message. At step 1, Gateway A asks the store for the current count of some API key and gets 99, against a limit of 100. So far, so good: one slot remains, and A is entitled to take it. But before A acts on that information, step 2 happens: Gateway B — serving a **different** request from the **same** key, which is exactly what a load-balanced fleet produces — issues the same `GET` and receives the same 99. The label calls this a **stale read**, and the word “stale” is precise: B’s 99 was true when it was sent, but B is about to make a decision whose validity depends on it **remaining** true until B’s write lands, and nothing guarantees that.

Steps 3 and 4 are the quiet catastrophe: both gateways, working from individually correct information, conclude “99 < 100, allow” — and both are right **at the moment they decide**. Step 5 lands A’s increment, taking the counter to 100. Step 6 lands B’s, taking it to 101 — and now **both** requests have been admitted into a window with room for one of them. Count the villains in this story. There are none. Every component did its local job perfectly; the failure lives entirely in the **gap between check and use**, a gap that exists only because you split one logical operation across a network. Neither gateway retried anything, timed anything out, or misread a value — which is why no amount of retrying, faster hardware, or “being more careful” closes this hole.

DEFINITION — Race condition / check-then-act (TOCTOU)

A *race condition* is a bug whose outcome depends on the uncontrolled interleaving of concurrent operations. The specific species above is **time-of-check to time-of-use**: the decision (“count is 99, under the limit”) is based on state that another actor changes before our write lands. Both gateways checked honestly; both were correct **at check time**; the limit was still exceeded. No amount of retries fixes a TOCTOU hole — the check and the update must become **one indivisible operation**.

DEFINITION — Atomicity / compare-and-swap (CAS)

An operation is *atomic* if it executes entirely or not at all, with no observable intermediate state. *Compare-and-swap* is the primitive form: “set this value to V_2 only if it currently equals V_1 ,” performed as one hardware/server-side step. Our options for making the check atomic: a server-side script executed atomically by the store (Section 3.12), a CAS loop, or a single atomic increment whose **returned value** is the decision input. The last two need no scripting at all — `INCR` already returns the post-increment count, which **is** the atomic observation we need.

Notice the shape of the fix the definitions are steering you toward: do not try to make the **gap** safe — eliminate the gap. If the store hands you the post-increment count as the return value of a single atomic operation, then there is no window in which another gateway’s write can invalidate your information, because the reading **is** the writing. That one idea reappears as Fix 1 in Section 3.12 and as the Lua script in Section 3.13.

So the core challenge decomposes into two questions that the rest of the chapter answers, in order: **what exactly is the count** — which algorithm defines “at most n per window” precisely enough to implement (Section 3.7) — and **how is the check made atomic across the fleet** (Section 3.12). Keep them separate in your head. Candidates who conflate them end up comparing algorithms when asked about races, and vice versa.

3.7 The Algorithm Zoo

Five algorithms cover the entire design space, and it is worth seeing all five even though you will only ship one — because each is a different answer to a question the prompt never spelled out: “what does ‘at most n per window’ **mean**, exactly?” As you read them, watch how the definition itself is the

design decision. Two algorithms can enforce “100 per minute” and behave completely differently at the edges.

DEFINITION — Fixed window counter

Divide time into fixed windows (e.g., each clock minute) keyed `rl:{key}:{window_id}`; each request increments the current window’s counter; reject when it exceeds the limit. $O(1)$ memory per key, one atomic increment per request — and a famous flaw: **the boundary burst**. A caller can fire 100 requests at 0:59.9 and 100 more at 1:00.0 — 200 requests inside one second, all legal, because the two bursts sit in different windows.

Sit with that flaw for a moment, because it is the single most-asked follow-up in rate-limiter interviews. The fixed window never lies — every individual window honored its limit. The problem is that “windows” were your invention, not the caller’s constraint; the caller experienced one second containing 200 admitted requests, and if your backend’s capacity plan assumed 100 per minute, that one second can hurt. The algorithm is **locally** correct and **globally** surprising, which is exactly the species of bug that passes code review and fails in production. Section 3.13 includes a test that demonstrates this burst executing, because a flaw you can reproduce is a flaw you understand.

DEFINITION — Sliding window log

Store the **timestamp of every request** in a per-key log; on each request, drop timestamps older than the window and reject if the remaining count reaches the limit. Perfectly accurate, no boundary problem — but memory is $O(\text{limit})$ per key. Section 3.5 priced this at 800 GB at our scale: the log is the algorithm you describe to show you know the trade-off, not the one you ship.

DEFINITION — Sliding window counter

The fixed-window/log hybrid. Keep the current and previous window counters only, and estimate the sliding window as:

$$\text{estimate} = \text{current_count} + \text{previous_count} \times (1 - \text{elapsed} / \text{window})$$

If 15 seconds of a 60-second window have elapsed, the previous window’s 84 requests count as $84 \times 0.75 = 63$. $O(1)$ memory, one increment per request, no boundary burst — at the price of being an **estimate** (it assumes the previous window’s traffic was spread evenly, which is fair on aggregate; published analyses put the misclassification rate well under 1%).

Read that formula until the intuition clicks, because it is a beautiful piece of thrift. You cannot afford to remember **when** last window’s requests happened — that was the log’s 800 GB mistake — so instead you make one defensible assumption: however they were spread across the previous window, only the fraction that overlaps **this** moment’s sliding window should still count against the caller. Fifteen seconds into a minute-long window, the trailing 45 seconds of the previous window are still “inside” a true sliding window, so the previous window’s count is weighted by 0.75. You are buying $O(1)$ memory with a single statistical assumption, and the price — a sub-1% misclassification rate — lands comfortably inside the accuracy budget you negotiated in Section 3.2. When an interviewer asks “but is it exact?”, the senior answer is: no, and here is the measured size of the error, and here is why the error is acceptable for this use case.

DEFINITION — Token bucket

Each key owns a bucket holding up to b tokens, refilled continuously at r tokens per second. A request takes one token; an empty bucket rejects. The parameters map directly onto product language: r is the sustained rate, b is the burst allowance (FR-5). $O(1)$ memory (two numbers: tokens, last-refill

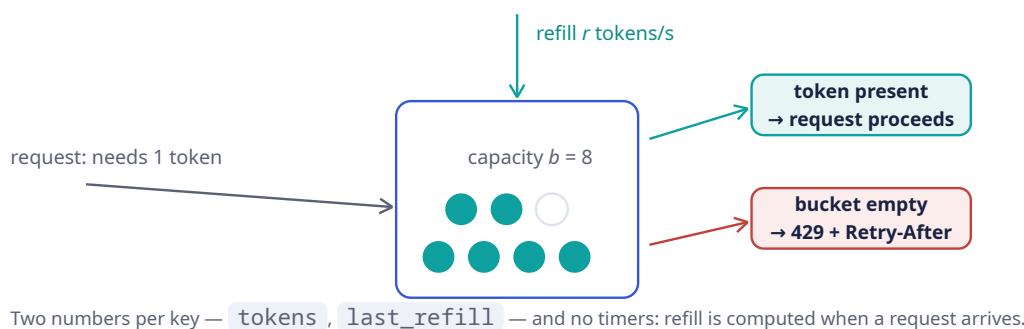
timestamp), and refill is computed **lazily** on each request — no timers, no background jobs. This is the industry’s default for API limiting, and our choice; Section 3.11 deep-dives it and Section 3.13 implements it.

DEFINITION — Leaky bucket / GCRA

Requests enter a conceptual queue and “leak” out at a fixed rate; a request is admitted only if the queue has room. Where the token bucket **permits** bursts, the leaky bucket **forbids** them — output is perfectly smooth. Its stateless formulation is the **Generic Cell Rate Algorithm**: store one theoretical-arrival-time per key; admit if now is not earlier than that time minus tolerance. The right tool when you meter **into** a fragile downstream (payment gateways, SMS providers), overkill for request admission.

The contrast in that last definition is the exam answer: token bucket and leaky bucket are mirror images, and choosing between them is choosing **who feels the burst**. Token bucket absorbs bursts at the limiter and releases them at the downstream — good when the downstream is your own robust API and bursts are a product feature (FR-5 says they are). Leaky bucket smooths bursts away entirely — good when the downstream is a third party who will fail or fine you for clumpy traffic. Same $O(1)$ memory, opposite temperament. If the interviewer ever asks you to “rate limit calls to Stripe’s API,” reach for leaky/GCRA without hesitation and say why.

Here is the token bucket drawn as a machine, because it is genuinely easier to picture than to parse:



Walk around the diagram once, slowly, because every element carries a design decision. The large outlined box in the center is the bucket itself, labeled with capacity $b = 8$ — the burst allowance from FR-5 drawn as physical volume. Inside it you can count seven teal discs: full tokens, each one permission for exactly one request. The eighth disc is drawn hollow, and that is not decoration: it marks the headroom between **current balance** and **capacity**, the visual form of the `min(b, ...)` cap you will meet in the pseudocode of Section 3.11. Tokens cannot accumulate past the rim; a caller who goes quiet for an hour banks eight requests, not eight hundred. That cap is what stops “saving up” from becoming an unlimited liability, and if an interviewer asks “can a user hoard quota by staying idle?”, this hollow disc is your answer.

Now the two arrows feeding the machine. From the top, a teal arrow drips downward into the bucket: the refill, r tokens per second — the sustained rate from your requirements, drawn as a faucet. From the left, a slate arrow carries an incoming request into the bucket’s wall, labeled with its cost: every request must withdraw exactly one token to proceed. The bucket is where those two flows negotiate.

On the right, the two outcomes split. If the withdrawal succeeds — at least one teal disc present — the upper arrow carries the request onward and the API serves it. If the bucket is empty, the lower, crimson arrow ends in the rejection box: HTTP 429 plus a `Retry-After` hint. And here is the detail to say out loud, because it is the caption under the whole figure: **there are no timers anywhere in this machine**. The refill arrow is not a background job topping the bucket up on a schedule; the

bucket's contents are recomputed lazily, from two stored numbers — `tokens` and `last_refill` — at the instant each request arrives. Time never has to be **advanced**; it only has to be **read**. Section 3.11 turns this picture into five lines of pseudocode, and Section 3.13 turns the pseudocode into Rust with a test proving eight threads cannot collectively extract more than the capacity.

The comparison that ends the discussion — every candidate should be able to reproduce this table from a standing start:

Algorithm	Memory/key	Exact?	Bursts?	Notes
Fixed window	$O(1)$	window-granular	$2\times$ at boundary	Simplest; the boundary burst is the interview trap
Sliding log	$O(\text{limit})$	exact	no	Memory kills it at scale (Section 3.5)
Sliding counter	$O(1)$	1% error	smoothed	Best accuracy-per-byte when bursts are unwanted
Token bucket	$O(1)$	exact accounting	allowed up to b	Our choice: burst policy is a product feature here
Leaky / GCRA	$O(1)$	exact pacing	forbidden	Choose when downstream needs smooth flow

Read the table as an elimination bracket, not a catalog. Sliding log: dead on memory in Section 3.5. Leaky bucket: wrong temperament — your FR-5 wants bursts **allowed**, not smoothed away. Fixed window: alive and $O(1)$ -cheap, but carrying the boundary-burst flaw you just dissected. Sliding counter: fixes the burst, pays with a 1% estimation error, and — subtle but real — still **thinks in windows**, so its burst tolerance is incidental rather than expressible. Token bucket: $O(1)$, exact accounting of what it tracks, and the only row where “bursts allowed up to b ” appears as a **parameter you can hand to product managers** instead of an accident of the algorithm. That last property is the decider: when your limit is a product feature (tiered plans, FR-2), the algorithm's knobs should speak the product's language. Rate and burst do; window-weights do not.

3.8 API & Protocol Design

Here is a small mental shift that separates experienced candidates from checklist readers: a rate limiter's “API” is mostly **other people's responses**. Nobody calls your limiter; your limiter annotates — or terminates — everybody else's calls. So the contract that matters is not an endpoint you expose but a behavior you impose on every response the API emits, whether the answer is “here is your data” or “slow down.” Design that behavior with the same care you would give a real endpoint, because a million client applications will be written against it.

DEFINITION — HTTP 429 / Retry-After

Status **429 Too Many Requests** (RFC 6585) means “you, specifically, are calling too fast.” The **Retry-After** header tells the caller how many seconds to wait before retrying. Together they convert a rejection into a negotiation: a well-built client backs off exactly as instructed instead of hammering harder. A limiter that rejects without `Retry-After` trains clients to retry blindly — worse for everyone.

Underline the word **negotiation**. A bare 429 is an insult: it spends the client's request, tells it nothing useful, and — here is the operational consequence — invites an immediate retry, because most HTTP clients are written to treat unknown failures as transient. Multiply that by a fleet of clients and your limiter has **created** a retry storm in the act of preventing one. `Retry-After` defuses this by giving the client a cheaper action than retrying: waiting a known number of seconds. You are not just rejecting load; you are **scheduling** it into the future, at a moment you know you can absorb. The same logic, writ large, is why Section 3.15's failure table worries about "retry storms after a 429 wave" — rejections reshape traffic, and a good contract shapes it **away** from the wall.

The transparency headers (FR-3) complete the contract:

Header	Meaning
<code>X-RateLimit-Limit</code>	The caller's limit for this route, e.g. 100
<code>X-RateLimit-Remaining</code>	Requests left in the current window / tokens left
<code>X-RateLimit-Reset</code>	When the budget refills (epoch seconds or seconds-from-now)
<code>Retry-After</code>	On 429 only: minimum wait before retry, seconds

Notice the division of labor across the four rows. `Limit` and `Reset` are **static context** — they let a client plan. `Remaining` is the load-bearing one: a client that watches it can throttle **itself** before ever seeing a 429, which converts your enforcement problem into their scheduling problem — exactly the outsourcing FR-3 promised. And `Retry-After` appears only on the 429 itself, because only then does the client need a **command**, not context. Every header earns its place; resist the urge to add more.

(An IETF draft standardizes these as `RateLimit-*` fields; the `X-` forms remain the de-facto convention. Either is defensible — knowing both exist is the point. If the interviewer asks which you would ship, say: `X-` forms today for compatibility, the standard fields alongside them as they finalize, and both documented. Ten seconds, and you have signaled that you read RFCs **and** ship product.)

A rejection response, end to end — read it the way a client developer would, top to bottom, asking at each line "what can I do with this?":

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
Retry-After: 1
Content-Type: application/json

{ "error": "rate_limit_exceeded", "limit": 100, "window": "1s", "plan": "free" }
```

The status line tells the client **what happened**; the headers tell it **when to come back**; and the JSON body tells it **why** — including, deliberately, the word `"free"`. That last field is monetization doing quiet work: the error message itself names the caller's plan, which is both a debugging aid and a gentle, perfectly-timed upsell delivered at the exact moment the caller feels the limit's cost. Section 3.17 will note that the list of most-rejected keys doubles as the sales team's lead list; this body is where that observation starts.

The **admin** surface (FR-2, FR-6) is ordinary REST by comparison: rules as records — `{ api_key or tier, route pattern, algorithm, limit/rate, burst, window }` — created and updated through an admin API, versioned, and pushed to every gateway within seconds (Section 3.10). Two properties matter more than the endpoints themselves. **Versioned**, because a bad rule push must be reversible in seconds — you will see "bad rule pushed" in the failure table of Section 3.15, and "roll back" is only an option if history exists. And **pushed**, not polled, because "seconds" of propagation lag is hard to

guarantee when fifty gateways each poll on their own schedule; a push invalidates everyone at once. Keep this in mind when Section 3.10 draws the rule fan-out as a first-class arrow.

3.9 Data Model & Storage

Only three kinds of data exist in this entire system, and the table below is short enough that you should be suspicious of any design that needs a fourth. Read the **Store** column as a series of placement decisions, each one forced by the access pattern in the **Contents** column:

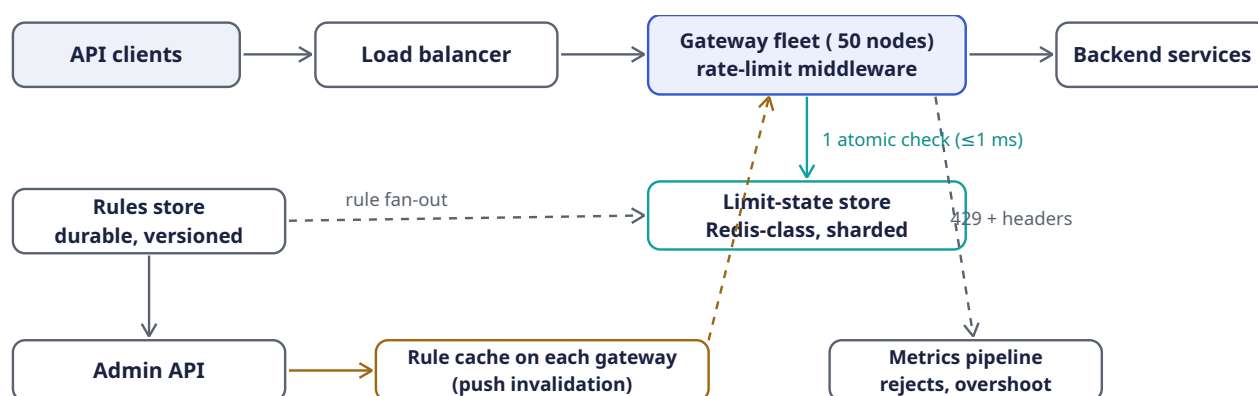
Entity	Contents	Store
Limit state	rl:{api_key}:{route} → {tokens, last_refill_ms} (bucket) or window counters; TTL = window so idle keys self-clean	In-memory store cluster (Redis-class), sharded by key hash
Rule	tier or key → {algorithm, rate, burst, window, route pattern}; versioned	Durable KV/relational store, fanned out to gateway-local caches
Quota ledger	monthly usage per key for billing	Append-only log → batch aggregation; not on the request path (Section 3.2)

Row by row. **Limit state** is tiny (two numbers per key, from Section 3.7), mutated on every request, and worthless once its window passes — so it lives in an in-memory cluster where a read-modify-write costs microseconds, sharded by key hash because every check touches exactly one key (no cross-shard transactions on the hot path — ever). **Rules** are the opposite temperament: read constantly, written rarely, and disastrous to lose — so they live in a durable store behind the admin API, with copies pushed into gateway memory. And the **quota ledger** — monthly billing usage — is listed here mostly to be exiled: it flows to an append-only log and batch jobs, explicitly **not** on the request path, honoring the quota-versus-rate-limit wall you built in Section 3.1. If you find yourself incrementing billing counters inside the limit check, stop: you have put an hour-tolerant system inside a millisecond-budget one, and the millisecond budget will lose.

Two decisions to name, because interviewers probe both. First, state entries carry a **TTL** (Chapter 2) equal to the window, so the 100M-active-keys problem from Section 3.5 is self-cleaning: a key used once at 3 a.m. and never again simply evaporates when its window passes. No garbage collector, no compaction job, no “state cleanup” cron quietly becoming a second system to operate — the store’s expiry machinery, which you are already paying for, does it for free. Second, rules are cached **on each gateway** with push-based invalidation — because a per-request rule lookup must never add a second network hop to a budget that only contains one.

3.10 High-Level Architecture

Everything you have decided so far — one shared atomic store for state, rules in gateway memory, decisions in middleware — assembles into a remarkably small picture. Draw it in the interview exactly like this, left to right, and narrate as you draw:



The middleware is a read-modify-write on the hot path: one atomic store op per request, rules read locally.

Now walk the picture the way a request experiences it, because the diagram is really two stories layered on one canvas — a **request story** running left to right along the top, and a **control story** running along the bottom — and keeping them separate in your narration is what makes the design sound obvious instead of improvised.

The request story starts at the top-left, where API clients send traffic into a load balancer. The balancer’s only job here is spray: it spreads each key’s requests across the 50-node gateway fleet, which is precisely why FR-4 forced the count to live off the nodes — remember, this spraying is what killed naive strategy 1 back in Section 3.6. The request lands on a gateway, and the **first code it touches** is the rate-limit middleware inside the big blue box. That middleware does exactly two lookups. First, a local one: which rule applies to this key and route? That answer is already in process memory — zero network hops — for reasons the control story will explain in a moment. Second, the single remote operation the design allows itself: the teal arrow dropping straight down from the gateway box into the **limit-state store**, labeled “1 atomic check (≤ 1 ms)”. This is the atomic increment or server-side script of Section 3.12, and it is the only synchronous dependency the hot path tolerates. The store answers allow-or-deny. On allow, the request continues rightward into the backend services, and the response flows back carrying the transparency headers of Section 3.8. On deny, the request never reaches the backend at all; the dashed slate arrow angling down from the gateway to the bottom-right labeled “429 + headers” is the rejection path — the API’s data plane is protected because the rejection happens **before** any backend work is spent.

The control story runs along the bottom row and explains how the middleware always knows the current rules without ever asking at request time. An operator (or a pricing change, or an abuse response) updates rules through the **Admin API** at bottom-left, which writes versioned records into the durable **rules store** above it. From there, a dashed “rule fan-out” arrow pushes the new rules toward every gateway, and the amber arrows complete the circuit: rules land in the **rule cache on each gateway** — the box at bottom-center — and a dashed amber arrow rises from that cache back up into the gateway fleet, which is the picture’s way of saying “the middleware reads its rules from local memory.” Trace any request and you will find it never touches the bottom row: config changes propagate in seconds, but they travel on a completely separate path from the traffic they govern. That separation is not tidiness; it is what lets you push a bad rule, roll it back, and never add a microsecond to a single request while doing it.

Finally, the quiet dashed arrow into the **metrics pipeline** at bottom-right: every decision — allow or deny — is emitted as an event off the hot path. It looks optional. It is not. Section 3.17 will argue that for a limiter, observability **is** the correctness evidence: a system whose job is to reject traffic cannot distinguish “working” from “broken” without these counters, because both states look like rejections from the outside.

Component	Responsibility	Why it is shaped this way
Rate-limit middleware	First code every request touches on the gateway: load rule → atomic check → pass or 429	Middleware placement means no backend code changes and one consistent policy for every route
Limit-state store	Atomic counters/buckets for all active keys	One shared store is what makes the limit global (FR-4); sharded by key hash, replicated for availability
Rules store + admin API	Versioned rule records; runtime updates	Config is tiny and read-hot: push it to gateway memory, never fetch per request
Gateway rule cache	Rules in process memory, invalidated by push	Zero added hops for config (Section 3.9)
Metrics pipeline	Every decision emitted as an event	Section 3.17: you cannot tune what you cannot see

Read the middle column as the minimal job description of each box, and the right column as the **argument** for each box's existence — because in the interview, every box you draw will be challenged with “why is this here?”, and the right column is your rehearsed answer. The middleware row's argument is worth memorizing as a sentence: putting the check in middleware means every route gets one consistent policy **and no backend team changes a line of code**. Adoption costs decide whether infrastructure actually gets used; a limiter that requires every service to integrate it will be integrated by none of them.

KEY INSIGHT — The one-op rule

Per request, the design allows itself **exactly one** atomic store operation and zero synchronous config reads. Everything — the algorithm choice ($O(1)$ state), the TTL self-cleaning, the gateway rule caches — exists to preserve that invariant. When an interviewer pushes on any component, return to it: “that would cost a second op on the hot path, and the one-op rule is what buys the millisecond budget.” A single crisp invariant you defend consistently is worth more than five features you cannot.

3.11 Deep Dive: The Token Bucket, Precisely

You have the picture (Section 3.7's bucket) and the placement (Section 3.10's one-op rule). Now you need the machine itself, stated precisely enough that two engineers implementing it on different continents produce byte-identical behavior. The token bucket's deepest elegance is that **time is never advanced by a timer** — nothing ticks, nothing fires, nothing drifts. State is two numbers, updated lazily when a request arrives:

1. `tokens` — current balance, capped at capacity `b`.
2. `last_refill_ms` — when the balance was last recomputed.

On each request at time `now`:

```
elapsed_s = (now - last_refill_ms) / 1000
tokens = min(b, tokens + elapsed_s * r) // lazy refill, capped
last_refill = now
if tokens >= 1: tokens -= 1; ALLOW (remaining = floor(tokens))
else: DENY (retry_after_ms = (1 - tokens) / r * 1000)
```

Read the five lines as three acts. The first two lines **catch the balance up to the present**: however many seconds have passed since the last request, that many refill-rate's worth of tokens have dripped in — computed arithmetically, not simulated, which is why no timer exists. The `min(b, ...)` cap is the hollow disc from the diagram doing its work: idleness banks at most one full burst. The third act is the decision, and notice how much falls out of it for free: on denial, `retry_after_ms` is exactly the time until one token will exist — the `Retry-After` header of Section 3.8 computed from the same arithmetic that made the decision, so the hint you give clients can never disagree with the policy you enforce.

Reading the parameters as product language: r = sustained rate (“100 req/s”), b = burst allowance (“...with bursts up to 150”). When product asks for a new plan tier, they speak in exactly these two numbers — the algorithm has no third knob, which means there is nothing for a well-meaning operator to misconfigure at 2 a.m.

A worked trace (limit 5 req/s, burst 8), to have in hand at the whiteboard — run it yourself once, right now, because being able to **simulate your own algorithm out loud** is what separates “I read about token buckets” from “I can operate one”:

Time	Event	Balance after	Why
<code>t = 0.0 s</code>	8 requests arrive together	$8 \rightarrow 0$	Full bucket: all 8 pass — the burst allowance
<code>t = 0.0 s</code>	9th request	0	Empty: 429, <code>Retry-After</code> : 1 (1/5 s until one token)
<code>t = 1.0 s</code>	1 request	$5 \rightarrow 4$	One second refilled 5 tokens
<code>t = 3.0 s</code>	1 request	$8 \rightarrow 7$	Two more seconds refill 10, capped at capacity 8

The fourth row is the one interviewers probe. Two idle seconds earned 10 tokens, but the balance shows 8 — the cap fired. Then ask yourself the follow-up they will ask you: **is the cap right?** For this design, yes — uncapped banking would let a dormant key return after a week with a million-token balance and become a denial-of-service against your own backend. The cap is the moment the token bucket stops being an accounting device and starts being a safety device.

COMMON PITFALL — Refill by background timer

Implementing refill as a cron or timer per key means a million timers, clock drift between timer and request paths, and state that never cleans itself up. Lazy refill has none of these: no timers, no drift (one clock, read once per request), and a TTL on the key reclaims idle state for free. If you find yourself scheduling work per key, you have re-invented the sliding log's costs inside the token bucket. This is the single most common implementation mistake in take-home versions of this problem — the algorithm looks like it “ticks,” so people build a ticker. It does not tick; it **settles**, on demand, when asked.

NOTE — One clock, and make it monotonic

Distributed machines disagree about wall-clock time (clock skew), and even on one machine the wall clock can jump backwards (NTP corrections) — a backward jump would **refund** tokens. Two defenses: (1) perform the bucket update **inside the store** with a server-side script, so one clock governs every gateway (Section 3.12); (2) where local time is unavoidable, read a **monotonic** clock (one that never moves backwards), as Section 3.13's Rust does. The refund scenario is worth picturing once: NTP notices the gateway is 400 ms fast and steps the clock back; every bucket on the host suddenly computes a **negative** elapsed time; if your code adds elapsed tokens without guarding the sign, you have just

minted free requests fleet-wide. One subtraction, done carelessly, becomes a money-losing bug — because for tiered plans, tokens are revenue.

3.12 Deep Dive: Distributed Enforcement & the Race

Section 3.6's race came from splitting **check** and **update** across a network — two round trips with a decision made in the no-man's-land between them. You now know the cure in principle (“make it one indivisible operation”); this section gives you the three production-grade forms that cure actually takes, in increasing order of sophistication, and — more importantly — teaches you when each is **allowed**.

Fix 1 — atomic increment-as-decision. For window algorithms, one atomic `INCR` already returns the post-increment value: the store's returned count **is** the check. Reject when the returned value exceeds the limit. One round trip, no client-side race, and the first increment of a window sets the key's TTL in the same atomic step (self-cleaning, from Section 3.9). Admire the economy: the observation and the mutation are the same network message, so the TOCTOU gap has literally nowhere to exist. Fixed window and sliding counter ship this way.

Fix 2 — server-side script for read-modify-write. The token bucket needs more than an increment — it needs read-compute-write (refill from the stored timestamp, compare against one, decrement). Executing that sequence **on the store**, as one atomic script, removes the race identically: the store serializes script execution, so no two requests ever observe the same balance. Think of it as moving the `Mutex` from Section 3.13's single-process code into the one place all fifty gateways share. Section 3.13 shows the client side, with the script embedded as data — and the embedding matters: the gateway ships **code to the data**, once, instead of shipping **data to the code** three times per request.

Fix 3 — approximate local counting. The radical option, and the one that separates interviewees who have operated systems from those who have only diagrammed them: skip the store on the hot path entirely. Each gateway keeps local counters and periodically **publishes** them; a background aggregator broadcasts fleet-wide totals, and each node denies when `local_estimate + fleet_total ≥ limit`. Zero added latency, and the store is off the critical path (fail-open for free) — but between syncs, overshoot is bounded by roughly `limit × (sync_lag / window)`. Choose it when protection matters more than precision (abuse mitigation at enormous scale); **never** for billing-adjacent limits, because an approximate count next to an invoice is a lawsuit with extra steps. Notice, though, that Fix 3 is not a rejection of the design — it is the design's **degraded mode promoted to a feature**. You will meet it again in Section 3.15 as the fallback when the store fails, and recognizing that your emergency plan and your extreme-scale plan are the same plan is a genuinely senior observation.

Approach	Latency	Accuracy	Choose when
Centralized, atomic op	+1 RTT (1 ms)	exact	Default. Paid tiers, contractual limits
Centralized, non-atomic	+1 RTT	races under concurrency	Never — Section 3.6
Local + async sync	0	bounded overshoot	Abuse protection at extreme scale, or store-failure fallback
Sticky routing + local	0	near-exact	Only if your load balancer can pin keys to nodes — fragile on failover

The fourth row deserves a word, because it tempts people: if the load balancer could pin each API key to one gateway, local counting would be exact and free. It works — until a gateway dies, the pin breaks, the traffic re-hashes, and your “exact” limit scatters across whatever node caught it, with all accumulated state stranded on the corpse. You have rebuilt naive strategy 1 with extra machinery and a failover bug. Renting exactness from your load balancer’s stability is renting, not owning; say so if it comes up.

INTERVIEW TIP — Say the quiet part: exactness is a product decision

“How exact does this need to be?” has no technical answer — it depends on whether the limit protects revenue (exact: paid tiers), infrastructure (approximate fine: abuse), or other users’ experience (approximate fine: fairness). Volunteering this framing turns an algorithm comparison into a senior-level requirements discussion. The interviewer cannot teach you anything about token buckets you have not already read; what they are testing is whether you know that the **choice among** them is a business input wearing a technical costume.

3.13 Deep Dive: Rust Reference Implementations

Time to make all of it executable. Three pieces follow, and each exists to **prove** a claim the prose has been making — read them that way, as arguments with compilers. The token bucket comes with deterministic-time tests, because a time-dependent algorithm tested against the real clock is a superstition, not a test. The fixed-window/sliding-counter pair demonstrates the boundary burst and its cure side by side, so the flaw from Section 3.7 stops being a story and becomes a failing-then-passing assertion. And the fleet-wide check embeds the server-side script as data, showing exactly where the atomicity of Section 3.12 physically lives.

3.13.1 Token bucket with a testable clock

Time is injected through a `Clock` trait so tests control it exactly — the difference between a test that **suggests** correctness and one that **proves** it. Watch for the `Mutex` around the state tuple: within one process, that is what makes the read-modify-write of Section 3.6 indivisible. (Across fifty processes it cannot help you — that is what the third listing is for.)

```
use std::sync::Mutex;

/// Time source, injected so tests can control it (Section 3.11).
pub trait Clock: Send + Sync {
    fn now_ms(&self) -> u64;
}

/// Production clock: monotonic — it can never jump backwards and refund
/// tokens (Section 3.11's note).
pub struct MonotonicClock(std::time::Instant);
impl MonotonicClock {
    pub fn new() -> Self { Self(std::time::Instant::now()) }
}
impl Clock for MonotonicClock {
    fn now_ms(&self) -> u64 { self.0.elapsed().as_millis() as u64 }
}

/// The decision every algorithm returns: pass, or reject with a retry hint
/// that becomes the Retry-After header (Section 3.8).
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Decision {
    Allow { remaining: u64 },
    Deny { retry_after_ms: u64 },
}
```



```

/// Token bucket (Section 3.7): `capacity` tokens, refilled lazily at
/// `refill_per_sec`. The Mutex makes the read-modify-write atomic *within*
/// one process; Section 3.12 handles the fleet-wide version.
pub struct TokenBucket<C: Clock> {
    capacity: f64,
    refill_per_sec: f64,
    state: Mutex<(f64, u64)>, // (tokens, last_refill_ms)
    clock: C,
}

impl<C: Clock> TokenBucket<C> {
    pub fn new(capacity: f64, refill_per_sec: f64, clock: C) -> Self {
        let now = clock.now_ms();
        Self { capacity, refill_per_sec,
            state: Mutex::new((capacity, now)), clock }
    }

    pub fn try_acquire(&self) -> Decision {
        let now = self.clock.now_ms();
        let mut s = self.state.lock().unwrap();
        let elapsed_s = (now - s.1) as f64 / 1000.0;
        let mut tokens = (s.0 + elapsed_s * self.refill_per_sec).min(self.capacity);
        s.1 = now;
        if tokens >= 1.0 {
            tokens -= 1.0;
            s.0 = tokens;
            Decision::Allow { remaining: tokens as u64 }
        } else {
            s.0 = tokens;
            let wait_ms = ((1.0 - tokens) / self.refill_per_sec * 1000.0).ceil() as u64;
            Decision::Deny { retry_after_ms: wait_ms.max(1) }
        }
    }
}

```

Map the code back onto Section 3.11’s five pseudocode lines and you will find a one-to-one correspondence — plus two Rust-specific choices worth being able to defend. The `Decision` enum forces every caller to **handle** both outcomes: there is no way to “forget” the deny path, because the compiler will make you match on it. And the balance is stored even on denial (`s.0 = tokens` in the `else` arm) — a denied request still **consumed time**, and the fractional token it earned during its wait must be banked, or a caller hammering an empty bucket would find their retries perpetually starved by rounding. Small line, real behavior.

Tests, including the concurrency proof that `Mutex` buys us exactly-once accounting across threads:

```

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::{Arc, atomic::{AtomicU64, Ordering}};

    /// A clock the test controls explicitly.
    struct TestClock(AtomicU64);
    impl Clock for TestClock {
        fn now_ms(&self) -> u64 { self.0.load(Ordering::SeqCst) }
    }

    #[test]
    fn burst_then_reject_then_refill() {

```

```

let clock = Arc::new(TestClock(AtomicU64::new(0)));
let b = TokenBucket::new(8.0, 5.0, clock.clone()); // 8 burst, 5/s
for i in 0..8 {
    assert!(matches!(b.try_acquire(), Decision::Allow { .. }), "burst #{i}");
}
// Bucket empty: denied, and told to wait 1/5s for one token.
assert_eq!(b.try_acquire(), Decision::Deny { retry_after_ms: 200 });
// One second passes: 5 tokens return (capped at capacity).
clock.0.store(1000, Ordering::SeqCst);
assert_eq!(b.try_acquire(), Decision::Allow { remaining: 4 });
}

#[test]
fn exactly_capacity_across_threads() {
    let clock = Arc::new(TestClock(AtomicU64::new(0)));
    let b = Arc::new(TokenBucket::new(100.0, 1.0, clock));
    let allowed = Arc::new(AtomicU64::new(0));
    std::thread::scope(|s| {
        for _ in 0..8 {
            let (b, allowed) = (b.clone(), allowed.clone());
            s.spawn(move || {
                for _ in 0..25 { // 8 threads x 25 = 200 attempts
                    if matches!(b.try_acquire(), Decision::Allow { .. }) {
                        allowed.fetch_add(1, Ordering::SeqCst);
                    }
                }
            });
        }
    });
    assert_eq!(allowed.load(Ordering::SeqCst), 100); // never one more
}

```

The second test is the one to talk about in the room. Eight threads, 200 attempts, capacity 100, frozen clock—and the assertion is not “roughly 100” or “at most 105” but **exactly** 100, every run, no flakiness. That is the entire promise of Section 3.6’s fix, demonstrated in miniature: when the read-modify-write is indivisible, the interleaving of threads stops mattering. If you ever doubt whether a mutex is “worth it” on a hot path, remember that this assertion is what you are buying.

3.13.2 Fixed window, its boundary burst, and the sliding counter’s cure

```

/// Fixed window counter (Section 3.7): one counter per window per key.
pub struct FixedWindow {
    window_ms: u64,
    count: u64,
    window_start_ms: u64,
}

impl FixedWindow {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, count: 0, window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms; // window start
        if w != self.window_start_ms { // new window
            self.window_start_ms = w;
            self.count = 0;
        }
    }
}

```

```

        self.count += 1;
        self.count <= limit
    }
}

/// Sliding window counter: previous window, weighted by overlap (Section 3.7).
pub struct SlidingWindowCounter {
    window_ms: u64,
    prev_count: u64,
    curr_count: u64,
    curr_window_start_ms: u64,
}

impl SlidingWindowCounter {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, prev_count: 0, curr_count: 0, curr_window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms;
        if w != self.curr_window_start_ms {
            self.prev_count = if w == self.curr_window_start_ms + self.window_ms {
                self.curr_count // adjacent window: carry over
            } else { 0 }; // gap longer than a window
            self.curr_count = 0;
            self.curr_window_start_ms = w;
        }
        let elapsed = (now_ms - w) as f64 / self.window_ms as f64;
        let estimate = self.curr_count as f64
            + self.prev_count as f64 * (1.0 - elapsed);
        if estimate + 1.0 <= limit as f64 {
            self.curr_count += 1;
            true
        } else {
            false
        }
    }
}

```

One subtlety in the sliding counter deserves attention, because it is easy to misread: when the window rolls, the code asks **how far** it rolled. If the new window is exactly the next one, the current count is **carried** into `prev_count` — the estimate needs it. But if the key went quiet for longer than a whole window, `prev_count` resets to zero instead, because traffic from two windows ago has legitimately fallen outside any sliding window anchored now. That one conditional is the difference between an estimate and a superstition about the distant past.

And the test that **demonstrates the flaw** — fixed window admits a 2× burst at the boundary, sliding counter rejects it:

```

#[cfg(test)]
mod window_tests {
    use super::*;

    #[test]
    fn fixed_window_admits_boundary_burst() {
        let mut fw = FixedWindow::new(60_000); // 100 req/min
        // 100 requests at t = 59.5 s .. 59.9 s: all pass (window 0).
        for i in 0..100 {
            assert!(fw.try_acquire(59_500 + i, 100));
        }
    }
}

```

```

    }
    // 100 MORE at t = 60.0 s .. 60.4 s: all pass (window 1).
    // 200 requests in under a second – legal. That is the bug.
    for i in 0..100 {
        assert!(fw.try_acquire(60_000 + i * 4, 100));
    }
}

#[test]
fn sliding_counter_rejects_the_same_burst() {
    let mut sw = SlidingWindowCounter::new(60_000);
    for i in 0..100 {
        assert!(sw.try_acquire(59_500 + i, 100));
    }
    // Just past the boundary the estimate is ~100 x overlap: the
    // previous window still counts almost fully. Burst denied.
    assert!(!sw.try_acquire(60_200, 100));
    assert!(!sw.try_acquire(61_000, 100));
    // Half a window later, weight has decayed: traffic flows again.
    let mut ok = 0;
    for i in 0..60 {
        if sw.try_acquire(90_000 + i * 500, 100) { ok += 1; }
    }
    assert!(ok > 40, "expected roughly half the limit to be available");
}
}

```

Run the first test in your head and feel the insult of it: two hundred requests, inside one second, every single one **legal** — the assertion suite passes precisely because the algorithm is doing exactly what it was specified to do, and the specification was wrong. Then the second test shows the cure working through time: denied just past the boundary (the previous window still weighs almost fully), and flowing again half a window later, with roughly half the budget returned — which is exactly the overlap math of Section 3.7 made concrete. Two tests, one story: **same traffic, different definition of “window,” opposite outcomes**. That is why the definition is the design.

3.13.3 The fleet-wide atomic check

Within one process, a `Mutex` makes read-modify-write atomic. Across fifty gateways, the atomicity must live **in the store**: the whole check ships as one server-side script that the store executes without interleaving. In production Rust this is a string constant handed to the client library — the script is data; the engineering is Rust:

```

/// Fleet-wide fixed-window check (Section 3.12, Fix 1). Executed atomically
/// by the store, so the interleaving of Section 3.6 is impossible.
/// KEYS[1] = "rl:{api_key}:{route}:{window_id}", ARGV[1] = window seconds.
const FIXED_WINDOW_LUA: &str = r#"
    local n = redis.call("INCR", KEYS[1])
    if n == 1 then redis.call("EXPIRE", KEYS[1], ARGV[1]) end
    return n
"#;

/// One gateway node's decision: exactly one round trip, and the *returned*
/// count is the check – no client-side read-modify-write at all.
pub async fn allowed(
    con: &mut redis::aio::MultiplexedConnection,
    api_key: &str,
    route: &str,
    limit: u64,
    window_secs: u64,

```

```

    now_secs: u64,
) -> redis::RedisResult<Decision> {
    let window_id = now_secs / window_secs;
    let key = format!("rl:{api_key}:{route}:{window_id}");
    let n: u64 = redis::Script::new(FIXED_WINDOW_LUA)
        .key(key)
        .arg(window_secs)
        .invoke_async(con)
        .await?;
    Ok(if n <= limit {
        Decision::Allow { remaining: limit - n }
    } else {
        let reset = (window_id + 1) * window_secs;
        Decision::Deny { retry_after_ms: (reset - now_secs) * 1000 }
    })
}

```

Note what the code makes structural: the key **embeds the window id**, so window rollover is just a new key (old windows expire by TTL — self-cleaning, Section 3.9); and the retry hint falls out of the window arithmetic for free. Look also at what is **absent**: no `GET` before the script, no client-side comparison against a fetched value, no retry loop. There is exactly one round trip and one atomic observation, and the interleaving from Section 3.6's diagram has no gap to live in. If you trace gateways A and B from that diagram through **this** code, one of them receives `n = 100` and passes while the other receives `n = 101` and is denied — the race is not managed, it is **eliminated**.

3.13.4 Where the check sits

The middleware contract, sketched against a generic handler: rules come from the gateway-local cache (zero hops), the decision from the fleet store (one hop), and `Deny` maps to the 429 contract of Section 3.8.

```

/// Gateway middleware shape (Section 3.10). `handler` is the real API.
pub async fn handle(
    req: Request,
    rules: &RuleCache,           // gateway-local, push-refreshed
    store: &mut Store,           // fleet-wide limit state
) -> Response {
    let rule = rules.for_key_and_route(req.api_key(), req.route());
    match store.check(req.api_key(), req.route(), &rule).await {
        Ok(Decision::Allow { remaining }) => {
            let mut resp = handler(req).await;
            resp.headers_mut().insert("X-RateLimit-Limit", rule.limit.into());
            resp.headers_mut().insert("X-RateLimit-Remaining", remaining.into());
            resp
        }
        Ok(Decision::Deny { retry_after_ms }) => Response::too_many_requests(
            retry_after_ms,        // -> Retry-After + X-RateLimit-Reset
            &rule,
        ),
        Err(_) => fail_open(req).await, // Section 3.15: never take the API down
    }
}

```

This listing is the architecture diagram compiled. Compare its three match arms against Section 3.10's picture: the happy path decorates the real response with transparency headers (FR-3) and costs the one allowed hop; the deny path never calls `handler` at all — the backend spends nothing on rejected traffic, which is the entire point of placing the check in middleware; and the error arm — the store itself failing — routes to `fail_open`, the decision Section 3.15 defends at length. Notice that the error

arm exists **in the type system**: `store.check` returns a `Result`, so “what if the limiter breaks?” is not a question you remember to ask in a design review, it is a branch the compiler required you to write. That is the last question from the scope dialogue, answered in code.

3.14 Scaling & Sharding

Sharding (Chapter 1) is by API key — the natural key, since every limit check is scoped to exactly one. That sentence sounds obvious, but stop and check it against the alternatives, because the **choice** of shard key is where scaling designs are won or lost. Shard by route and one hot endpoint concentrates a fleet’s worth of checks onto one shard. Shard by time window and every shard rotation becomes a fleet-wide migration. Shard by API key and each check touches exactly one shard, keys are uniformly random enough to spread load, and no shard ever needs to talk to another on the hot path. The requirement you established in Section 3.6 — the count is a property of the caller — turns out to dictate the sharding for free.

- **Limit-state store**: keys hash-sharded across 4-6 shards with replicas (Section 3.5’s arithmetic). Each check touches exactly one key, so there is no cross-shard coordination on the hot path.
- **Gateways**: stateless; any request can be checked anywhere, because the state lives in the shared store.
- **Rules**: fully replicated to every gateway — config is megabytes, and the read pattern (every request) demands local memory.

Notice the three different scaling answers for the three data kinds from Section 3.9: **shard** what is large and hot (limit state), **replicate** what is small and read-hot (rules), and **leave alone** what must stay cheap and replaceable (gateways). Uniform answers (“shard everything!” / “replicate everything!”) are a tell of template thinking; the variance across these three rows is the actual design.

DEFINITION — Hot key

A single key whose traffic concentrates on one shard hard enough to matter — here, one viral API key doing tens of thousands of checks per second. A single atomic `INCR` or script call is $O(1)$ and microseconds; a hot key is far more likely to be **rejected** fast than to melt a shard. If a key ever outgrows one shard, split its counter across sub-shards (`r1:key#0..15`) and sum at check time — the same trick Chapter 2’s stream used per segment.

Multi-region note. A truly global limit across regions forces synchronous cross-region round trips on the hot path — hundreds of milliseconds, violating the latency NFR. Production answer: enforce per-region limits sized to divide the global one, and reconcile asynchronously (Chapter 1’s PACELC framing: consistency would cost latency, so availability and latency win, and the **bounded overshoot** vocabulary of Section 3.4 is how you state the price). When the interviewer asks “and if we go multi-region?”, the trap to avoid is reaching for a globally consistent counter; the answer they want is the sentence you just read — divide the budget by region, accept bounded overshoot at the seams, and say the trade-off out loud before they can ask.

3.15 Failure Modes & Recovery

Here is where the interview’s last scoping question gets its full answer. You already decided **that** the limiter must not take the API down; this section decides **how**, failure by failure. The definitions first, because the two words are load-bearing for the rest of your career, not just this chapter:

DEFINITION — Fail-open / fail-closed

What a dependent system does when its dependency fails. *Fail-open* serves traffic anyway (availability over enforcement); *fail-closed* refuses (enforcement over availability). The choice is per-route: abuse

protection on a public API fails open with a local cap; a login endpoint defending against brute force fails **closed-ish** — refusing logins during a store outage is cheaper than admitting a credential-stuffing wave.

That last contrast is the sentence to say in the room, because it dissolves the false binary. “Fail open or closed?” has no global answer; it has a **per-route** answer driven by which error is cheaper **for that route**. A rate-limited product API during a store outage: every minute closed costs revenue and trust, and the risk is a few minutes of uncounted traffic — open wins. A login endpoint during the same outage: every minute open is an unthrottled credential-stuffing window against your users’ accounts — closed wins. Same limiter, same outage, opposite policies, both correct. Candidates who answer “fail open, obviously” have answered a different, easier question.

Failure	Handling
Limit-state store shard down	Fail over to the replica. If all replicas are gone: fail open with a per-gateway local token bucket sized $\text{limit} / \text{fleet_size}$ — traffic flows, protection degrades to a bounded multiple of the limit, metrics scream (Section 3.17).
Store unreachable from some gateways (partition)	Those gateways degrade to local caps; the rest enforce exactly. Converges on heal, no manual action.
Clock skew between gateways	Structurally impossible to matter: bucket/window math runs on the store’s clock via the server-side script (Section 3.12); local code uses monotonic time (Section 3.13).
Bad rule pushed	Rules are versioned; roll back to the previous version. Rate of change is small, so a human-in-the-loop push is fine.
Rule fan-out stalls	Gateways enforce last-known rules and alert; a stale rule for 60 s beats a missing check for 60 s.
Retry storm after a 429 wave	<code>Retry-After</code> spreads retries; add jitter client-side guidance in docs. A limiter that herds retries into the same second recreates the burst it just rejected.
Hot key	Sub-shard the counter and sum (Section 3.14). Rare in practice — hot keys are usually already over their limit.

Read the first row until the **degraded-mode** logic is second nature, because it is the design’s load-bearing failure story. Store down → local token bucket per gateway, sized $\text{limit} / \text{fleet_size}$. Wait — didn’t Section 3.6 prove per-instance limits can’t express a global limit? It did, and look at why it is acceptable **here**: the failure is temporary, the local cap bounds total overshoot at roughly $\text{limit} \times (\text{fleet_reached} / \text{fleet_size})$ in the worst case, and the alternative — refusing all traffic because a **protection component** is ill — violates the availability NFR that outranks accuracy. You are not contradicting Section 3.6; you are **spending** its lesson deliberately, in an emergency, with a bound

and a pager attached. Degraded mode is a design, not an apology. The clock-skew row, by contrast, should satisfy you: a failure mode made **structurally impossible** by an earlier decision (the store-side script owns the clock) is worth ten failure modes handled by procedures.

3.16 Trade-offs & Alternatives

Every design is a purchase: this table is the receipt. Read each row as “benefit bought, cost accepted” — and notice that none of the costs are hidden, which is the entire point of writing them down:

Decision	Benefit purchased	Cost accepted
Token bucket	Bursts as a product feature; O(1) state; lazy refill needs no timers	Two-number accounting must be atomic fleet-wide (script in the store)
Sliding window counter	No boundary burst, still O(1)	1% estimate error; slightly more math at check time
Fixed window alone	One atomic INCR; trivially explainable	2× boundary burst (Section 3.13’s test proves it)
Centralized exact check	FR-4 truly holds; no false rejections	+1 RTT on every request; store on the critical path
Local + async sync	Zero added latency; fail-open for free	Bounded overshoot; unusable for billing-adjacent limits
Fail-open default	The API never dies because the limiter did	During store outages, protection is a bounded approximation

If the interviewer pushes on any row, the strongest move is to **agree with the cost and re-state the budget that justifies it**. “Your centralized check adds a millisecond to every request.” — Yes; the negotiated budget was 1–2 ms (Section 3.2), and what it buys is FR-4 holding exactly, which the tiered pricing model depends on. Costs you have already priced are not weaknesses; they are evidence the design was chosen rather than defaulted into.

3.17 Observability & SLOs

SLIs and SLOs (Chapter 1) for a limiter are unusual in one way that you should name up front, because it inverts the usual dashboard instinct: for this system, **working correctly looks like rejections**. A graph full of 429s might be the limiter failing — or the limiter heroically holding the line against an abuse wave. Without instrumentation that separates those two stories, you cannot tell success from outage. Instrument accordingly:

- **Decision rates** per rule: allows vs. 429s, over time. A 429 spike after a rule change is a config bug until proven otherwise.
- **Overshoot audit**: sampled per-key realized rates vs. limits — the direct measurement of Section 3.4’s accuracy NFR.
- **Check latency** p50/p99, split by path (local rules lookup vs. store call); SLO: the ≤ 1 ms budget of Section 3.4.
- **Store health**: shard ops/sec, script latency, replica lag — the one dependency on the hot path.
- **Degraded-mode counters**: how many gateways are currently failing open, and with what local cap. This number should be zero; when it is not, paging is legitimate.
- **Top rejected keys**: the abuse list and the sales list are the same list — chronically limited keys are upgrade candidates (limits as monetization, Section 3.1).

Two of these bullets are doing more than monitoring. The **overshoot audit** is what converts your accuracy NFR from a promise into a measurement — sampled per-key realized rates against limits is the only way to **prove** steady-state overshoot stays near 1%, and being able to point at the proof is what makes “bounded overshoot” an engineering statement instead of a hope. And the **degraded-mode counter** closes the loop with Section 3.15: fail-open is only a defensible policy if you can see it happen, size it, and page on it. A silent fail-open is not resilience; it is an outage wearing a costume.

3.18 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- “*Per-key AND per-IP AND per-route at once?*” — Composite rules: evaluate each applicable limit; deny if any denies. Order checks cheapest-first.
- “*Monthly quotas for billing?*” — Different system: append-only usage log, batch aggregation, delayed enforcement. Rate limits shape traffic; quotas shape invoices (Section 3.1).
- “*Limiters as a shared service vs. a library?*” — A library removes a hop but re-introduces per-language drift and per-instance state; a sidecar/service centralizes policy at the price of the hop. State which you picked and why.
- “*Exactly-distributed without a central store?*” — Gossiped counters or CRDTs (Chapter 1) give you **approximate** global counts with zero coordination; exactness without coordination is not a thing, and saying so is the answer.
- “*L3/L4 DDoS?*” — Out of scope by layer: SYN floods and volumetric attacks are absorbed at the CDN/edge (Chapter 2) with connection-level defenses. This design is L7, per-authenticated-caller.
- “*Adaptive limits under load?*” — Load-shedding’s cousin: when the backend browns out, tighten limits dynamically by a global multiplier. Nice extension; say it, don’t build it live.

Notice what each of these answers has in common: none of them is a feature list. Each one **locates** the follow-up relative to the design you already built — inside it (composite rules), beside it (quotas), below it (L3/L4), or beyond it (adaptive limits). That reflex — classifying a new requirement against an existing architecture instead of bolting it on — is the interviewer’s final test, and it is a habit you can practice on every chapter of this book.

If you remember five things:

1. A rate limiter is a read-modify-write on the hot path; the design is the art of making that operation atomic and ≤ 1 ms.
2. Local counters cannot express a global limit; a non-atomic shared counter races. Atomicity lives **in the store**.
3. Token bucket = sustained rate + burst allowance, with lazy refill and self-cleaning TTL state.
4. Accuracy is asymmetric: never false-reject, bound the overshoot — and know that exactness is a product decision, not a technical one.
5. The limiter must never be the outage: fail open with a local cap, and page on degraded mode.

3.19 Summary & Further Reading

We designed a distributed rate limiter for a 200k-RPS public API: a gateway middleware making exactly one atomic check per request against a sharded in-memory store; the algorithm zoo (fixed window, sliding log, sliding counter, token bucket, leaky/GCRA) with token bucket chosen for its burst semantics; the TOCTOU race and its three fixes (atomic increment-as-decision, server-side scripts, approximate local counting); the 429 + `Retry-After` contract that turns rejections into negotiations; runtime-reloadable tiered rules pushed to gateway memory; and a defended fail-open policy with bounded local caps.

Primary source for this chapter:

- “7: Design a Rate Limiter” — [Systems Design Interview Questions With Ex-Google SWE \(Jordan has no life\)](#) — the walkthrough this chapter expands.

Foundations worth reading:

- Stripe’s engineering blog on rate limiters — production limiter design at API-company scale.
- Figma’s *An alternative approach to rate limiting* — the leaky-bucket-in-Redis variant, honestly costed.
- The IETF draft *RateLimit header fields for HTTP* — the emerging standard response contract.
- Brandur Leach’s writing on GCRA and the Redis-cell module — the stateless leaky bucket, implemented.
- NGINX’s rate-limiting documentation — the leaky bucket hiding inside a commodity reverse proxy.

3.20 Chapter 3 Glossary

A one-glance index of every term this chapter defined. Chapters 1–2 are assumed; later chapters assume all three. If any row feels unfamiliar, the section that defines it is worth a re-read before you move on — later chapters will use these words without re-defining them.

Term	Meaning in one line
Rate limiting / throttling	Capping requests per identity per window; protection, capacity, monetization
Quota	Long-period budget for billing; batched, approximate accounting
False rejection	Denying an under-limit caller — the unforgivable error
Overshoot	Admitting over-limit traffic — the bounded, tolerable error
Race condition / TOCTOU	Check and use separated in time; state changes in between
Atomicity / CAS	All-or-nothing operation / conditional swap as one step
Fixed window counter	One counter per window; $O(1)$, but $2\times$ boundary bursts
Sliding window log	Every timestamp kept; exact; memory $O(\text{limit})$ — infeasible at scale
Sliding window counter	Two windows weighted by overlap; $O(1)$, 1% error, no burst
Token bucket	Capacity b refilled at r/s ; bursts allowed; lazy refill
Leaky bucket / GCRA	Constant-rate drip; bursts forbidden; smooth output
HTTP 429 / Retry-After	“Too fast” status + how long to wait — rejection as negotiation
X-RateLimit-* headers	Limit, remaining, reset — client-side self-throttling data
API gateway middleware	First code every request meets; policy without backend changes
Server-side script (Lua)	Read-modify-write executed atomically inside the store
Local + async sync	Zero-latency approximate enforcement with bounded overshoot
Monotonic clock	Time source that never moves backwards; no refunded tokens
Fail-open / fail-closed	Dependency down: serve anyway / refuse — chosen per route
Hot key	One key concentrating load on one shard; sub-shard and sum
Tiered limits	Different limits per plan; limits as pricing
Rule cache	Gateway-local copy of config; zero hops per request

Term	Meaning in one line
Key TTL self-cleaning	Idle keys expire with their window; no garbage collector
Boundary burst	Two legal window-fulls fired at a window edge — 2× in seconds
Degraded-mode cap	Local emergency limit while the fleet store is down

— End of Chapter 3 · Next: Chapter 4, Logging & Metrics at Scale —

Designing a Logging & Metrics Platform

PROBLEM SOURCE

This chapter solves the problem posed in the talk “[14: Distributed Logging & Metrics Framework](#)” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The talk designs the observability backbone for a microservices fleet: how telemetry leaves thousands of hosts, where it is buffered, how logs and metrics are stored and queried, and how alerts fire. This chapter follows the same arc, deepened with full definitions, capacity mathematics, query and retention design, and Rust reference implementations.

4.1 The Problem Statement

The interviewer looks up and says:

“We run thousands of services across tens of thousands of machines. When something breaks at 3 a.m., engineers need to see what happened — search every log, graph every metric, and get paged before users notice. Design the logging and metrics platform that makes that possible.”

Feel how different this prompt is from everything before it. Chapters 1 through 3 designed systems that **serve users** — editors, maps, API infrastructure. This chapter designs the system that **watches all of them** — including, recursively, the very systems you built in those chapters. Your “users” now are other engineers, and they show up at the worst possible moment: during an outage, stressed, impatient, asking very expensive questions of your data. And here is the cruel twist that should shape your whole design intuition: at that same moment, the fleet you are watching is also screaming — error storms, stack traces, retry floods. The system’s busiest day and its most important day are **the same day**. Strip away the operations vocabulary and what remains is a data pipeline problem: an enormous, never-ending write stream on one side, and on the other a handful of humans issuing rare, costly, latency-sensitive queries. Almost every decision in this chapter flows from that asymmetry, so hold onto it.

DEFINITION — Observability / telemetry

The ability to answer questions about a system’s internal state from its **outputs**: the data it emits as it runs. *Telemetry* is that emitted data. The canonical “three pillars” are **logs** (timestamped event records), **metrics** (numeric measurements over time), and **traces** (the path of one request through many services). This chapter designs the platform for the first two; traces are a follow-up (Section 4.18).

Notice the definition’s one hard constraint: you may only use **outputs**. You cannot pause a production service at 3 a.m. and inspect its variables; the telemetry it already emitted is the only evidence that will ever exist. That is why durability shows up later as a first-class requirement rather than a nicety — a

log line dropped at emission is not delayed evidence, it is **destroyed** evidence, and the investigation it would have closed stays open.

DEFINITION — Log vs. metric

A *log record* is a discrete, semi-structured event: “at 03:14:22.017, service checkout on host i-9f2 logged ERROR payment timeout user=...” — high volume, low per-record value, queried by **searching text**. A *metric* is a named numeric measurement repeated over time: `http_requests_total{service="checkout", status="500"} = 87341` — small, structured, queried by **range scans and aggregations**. They look like cousins and demand opposite storage engines. Holding that distinction is the core of this chapter (Section 4.6).

One more intuition before we scope: read those two examples again and ask yourself which one you would want **during** an incident and which you would want **before** it. Metrics tell you something is wrong — a red line bending the wrong way, often minutes before any user complains. Logs tell you **why** — the precise exception, on the precise host, for the precise request. Metrics are the smoke detector; logs are the fire investigation. A platform that stored only one would leave you either unaware or helpless, and the engineering consequence is that you are really designing two storage systems that happen to share a firehose. Section 4.6 makes that fork explicit.

4.2 Scope & Clarifying Questions

The prompt is enormous — “log and measure everything, forever, for everyone” — so your first job is to shrink it to something designable. Watch the dialogue and, as always, pay attention to what each answer **buys**:

Speaker	Dialogue
Candidate	“What’s in scope — logs, metrics, distributed tracing, all three?”
Interviewer	“Logs and metrics. Tracing is a future extension; don’t design it, but don’t preclude it.”
Candidate	“Scale of the fleet being observed?”
Interviewer	“About 50,000 service instances across a few thousand hosts, in several regions.”
Candidate	“How quickly must emitted data be queryable?”
Interviewer	“Logs within about a minute; metrics within about thirty seconds. Alerts within a minute of the condition.”
Candidate	“How long do we keep data?”
Interviewer	“Logs: two weeks fast-searchable, three months retrievable. Metrics: thirteen months, and nobody needs per-15-second resolution on data from last year.”
Candidate	“Is some data loss acceptable under extreme overload?”
Interviewer	“Metrics — yes, they’re statistical; drop samples rather than fall over. Logs — avoid it; engineers notice gaps. State how you’d degrade.”
Candidate	“Who queries, and how often?”
Interviewer	“A few thousand engineers. Dashboards refresh continuously; searches spike during incidents.”

Six answers, six load-bearing decisions. The first carves tracing out of scope but plants a constraint you should honor cheaply — “don’t preclude it” is why trace ids will reappear in Section 4.18 as a

bolt-on rather than a rebuild. The fleet size (50k instances) is the number that makes manual, per-service anything impossible and forces the **agent** model of Section 4.7. The freshness targets (60 s logs, 30 s metrics) are interesting precisely because they are **not** milliseconds: unlike Chapter 3's limiter, nothing here sits on a user-facing critical path, which means you are allowed — even encouraged — to trade seconds of delay for enormous throughput. That permission is what makes the buffer-first architecture possible. The retention answer hands you a cost model disguised as a policy: “nobody needs 15-second resolution from last year” is an engraved invitation to **downsample**, and Section 4.9 accepts it. The loss answer gives you a degradation contract you must quote back later: metrics may shed, logs must never vanish silently. And the query answer contains the seed of the whole availability philosophy — dashboards refresh **continuously**, but searches **spike during incidents**, which is the correlated-load problem from the problem statement, now confirmed by the person grading you.

AGREED SCOPE

Logs + metrics for a **50k-instance** fleet. Logs: full-text search, queryable ≤ 1 min after emission, 2 weeks hot + 3 months cold. Metrics: label-filtered queries and dashboards, fresh ≤ 30 s, 13 months retained with downsampling. Alerting on both, notification ≤ 1 min from condition. Metrics may drop under extreme load; logs degrade by **sampling**, never silent gaps.

4.3 Functional Requirements

Six promises this time, and as you read them, notice how they partition into three pairs: two about getting data **in** (FR-1, FR-5), two about getting answers **out** (FR-2, FR-3), and two about the platform being a **product** for engineers rather than a pile of infrastructure (FR-4, FR-6). A design that nails the pipeline but forgets the humans paging through it at 3 a.m. has failed the prompt.

1. **FR-1 — Collection.** Every instance's logs and metrics flow to the platform automatically, within seconds of emission, with no per-service plumbing by engineers.
2. **FR-2 — Log search.** Engineers query logs by full text, structured fields, and time range; results over hot data return in seconds.
3. **FR-3 — Metric queries & dashboards.** Engineers query series by name and labels over time ranges, with aggregations (rate, sum, percentile), rendered as auto-refreshing dashboards.
4. **FR-4 — Alerting.** Rules over logs and metrics evaluate continuously; when a condition holds for its configured duration, notifications fire — once, not repeatedly (Section 4.12).
5. **FR-5 — Retention & tiering.** Data ages automatically through hot $\rightarrow$ warm $\rightarrow$ cold $\rightarrow$ deleted, per configured policy, with no manual intervention.
6. **FR-6 — Self-service config.** Teams register services, set retention, and define alerts through an API/UI — the platform team is not in the loop.

Two of these deserve a beat. FR-1's phrase “no per-service plumbing” is doing enormous work: with 50,000 instances, any collection scheme that requires engineers to modify their services will be adopted by a fraction of them, and unobserved services are exactly the ones whose failures surprise you. Collection must be something the **environment** provides, which is why Section 4.7 puts a daemon on every host rather than a library in every service. And FR-4's “once, not repeatedly” sounds like politeness; it is actually the hardest engineering constraint in the chapter, because a paging system that cries wolf trains its humans to ignore it, and an ignored pager is strictly worse than no pager — you pay the operational cost and get none of the safety. Section 4.12 exists almost entirely to honor those four words.

4.4 Non-Functional Requirements

Before the table, one term needs its own definition, because the word “freshness” already meant something different in Chapter 2 and you should not let the two meanings blur:

DEFINITION — Ingestion latency (freshness, telemetry sense)

The time from an event being emitted on a host to it being **queryable**. Chapter 2’s freshness measured data age; this one measures pipeline speed. Our targets: logs ≤ 60 s, metrics ≤ 30 s — fast enough that an engineer watching a deploy sees its effect while watching.

That last clause is the real requirement hiding inside the numbers. “Fast enough to watch a deploy” is a **human-loop** criterion: an engineer rolls out a change, stares at a dashboard, and needs the graph to answer “better or worse?” within one attention span. Nobody negotiated 60 seconds because the machines need it; they negotiated it because human patience during an incident is about a minute. When you derive requirements from the humans they serve, you get numbers you can defend.

Quality	Target
Ingestion throughput	10M log events/s average, 3× that in bursts; 0.7M metric samples/s (Section 4.5)
Ingestion latency	Logs queryable ≤ 60 s; metrics ≤ 30 s
Query latency	Hot log search p95 ≤ 5 s; dashboard refresh ≤ 2 s
Durability	Logs: at-least-once, gaps are visible and rare. Metrics: loss-tolerant by nature (sampling tolerates gaps)
Availability	Ingestion path $\geq 99.9\%$ — it is the fleet’s black-box recorder; query path may degrade first
Cost discipline	Storage dominates; compression and tiering are first-class requirements, not optimizations

Walk the rows and watch them disagree with each other — the disagreements are the design. Throughput says millions of writes per second; query latency says interactive reads; durability says lose nothing; cost says store it for cheap. You cannot have all four maximally, and the rows themselves tell you how to split the difference: durability attaches to **logs** while metrics are explicitly loss-tolerant, and the availability row makes a ranking you should say out loud in the interview — **the ingestion path outranks the query path**. If forced to choose, a platform that keeps recording but answers slowly is a black-box flight recorder doing its job; a platform that answers instantly but stopped recording an hour ago is a museum. Queries can catch up later; unrecorded history cannot be re-created. That one ranking decision will show up again as the buffer-first architecture of Section 4.11.

KEY INSIGHT — The system’s worst day is everyone else’s

When the fleet has an incident, two things happen at once: services emit **more** telemetry (error storms, stack traces) and engineers query **harder** (everyone opens dashboards). The platform must be sized for the correlated spike — the day it is needed most is the day it is loaded most. This observation drives the buffer-first architecture of Section 4.11. Most systems get to choose between capacity planning for average load or peak load; this system’s peak load arrives correlated with its peak **importance**, so the choice is made for you. A telemetry platform sized for the average is a platform that dies exactly when it is needed.

4.5 Back-of-the-Envelope Estimation

Now quantify the firehose, because the numbers are going to make two architecture decisions for you before Section 4.6 even starts.

Assumptions:

- 50k service instances; each emits 200 log lines/s average (quiet services less, chatty ones more), 200 B per line.
- Each instance exposes 200 metric series (name + label combinations); sampled every 15 s.
- Dashboards/searches: 2k engineers, spiking to 5k queries/s during incidents.

Derived numbers:

Quantity	Estimate	How
Log event rate	$\approx 10\text{M events/s avg, } 30\text{M burst}$	50k instances $\times$ 200 lines/s, $\times 3$ incident factor
Log bandwidth	$\approx 2\text{ GB/s avg}$	10M $\times$ 200 B
Raw logs per day	$\approx 170\text{ TB}$	2 GB/s $\times$ 86,400 s
Hot log tier (14 days)	$\approx 0.5\text{--}1\text{ PB}$	sampling/filtering trims to 30–50 TB/day indexed; $\times 14$ days, plus index overhead
Metric sample rate	$\approx 0.7\text{M samples/s}$	50k $\times$ 200 series $\div 15\text{ s}$
Metric storage per day	$\approx 120\text{ GB compressed}$	0.7M/s $\times$ 86,400 s $\times$ 2 B per compressed sample (Section 4.9)
Metrics, 13 months	tens of TB	after rollups; an order of magnitude less than logs' daily raw volume
Buffer capacity	$\approx 2\text{--}5\text{ GB/s sustained}$	a Kafka-class cluster of 20–30 brokers, 200 partitions

Do not skim past the third and sixth rows — hold them side by side and let the absurdity sink in. Logs: a hundred and seventy **terabytes per day**. Metrics, thirteen months, **after** rollups: tens of terabytes **total**. One of your two data classes produces more raw data **every day** than the other produces in its **entire retention window**. Same pipeline, same fleet, same chapter — three orders of magnitude apart. If someone proposes storing these two things the same way, the estimation table has already refuted them; you just have to read it out loud.

KEY INSIGHT — What the math tells us

Two facts shape everything. First, **logs are the cost center**: 170 TB/day raw versus metrics' 120 GB/day — three orders of magnitude apart. Storage design for logs is really **cost** design (sampling, tiering, compression); storage design for metrics is **query-shape** design (fast range scans over tiny values). Second, the write rate utterly dwarfs the query rate, so the system is organized as a pipeline: absorb first, index second, query third. Notice that neither fact came from cleverness — they fell out of multiplication. This is why you always estimate before you design: the envelope tells you which problems are big, and big problems are the only ones worth architecture.

4.6 The Core Challenge: One Firehose, Two Shapes of Data

Here is the chapter’s central tension, stated as plainly as possible: logs and metrics arrive on the same wire, from the same hosts, carried by the same agents — and they must part ways almost immediately, because everything about them is different. Not just their formats: their **economics**, their **query patterns**, even their **relationship to loss**. The table below is the whole argument; every row justifies the fork you will see in the architecture:

Property	Logs	Metrics
Record	Semi-structured text event	Name + labels + (timestamp, number)
Volume	170 TB/day raw	120 GB/day compressed
Per-record value	Low individually; priceless during one incident	Low individually; aggregates are the product
Query pattern	Full-text search, field filters, recent time ranges	Range scan by name/labels; aggregations over time
Index needed	Inverted index over tokens (Section 4.8)	Compressed time-ordered blocks (Section 4.9)
Loss tolerance	Low — gaps break investigations	High — statistics tolerate dropped samples

Read the last row twice, because it is the one candidates fumble. A missing log line during an incident is a hole in the evidence — and investigators **notice holes**, because the gap always seems to be exactly where the root cause was hiding. A missing metric **sample**, by contrast, is one absent point in a series of thousands; every aggregate you compute barely moves, and the dashboard’s line barely flickers. This asymmetry is a gift: it tells you exactly which data class to sacrifice first when the pipeline is drowning (Section 4.15’s degradation order), and it came straight from the interviewer’s own answer in Section 4.2.

COMMON PITFALL — One store for both

The classic junior move: “just put it all in one database.” Indexing metrics as log lines wastes the log pipeline’s expensive text machinery on tiny numbers; storing logs in a time-series engine forfeits text search entirely. The chapter’s architecture forks **right after the buffer** into two purpose-built stores — Section 4.8 and 4.9 exist because this fork exists. If an interviewer challenges the fork (“isn’t two stores twice the operational burden?”), the answer is yes — and worth it, because the alternative is one store that is bad at both jobs: the table above shows the two workloads disagree on volume, query shape, **and** loss tolerance, so any single engine must betray at least one of them.

4.7 Data Ingestion: Agents, Push vs. Pull, and the Buffer

How does telemetry physically leave 50,000 instances? Your first instinct might be “services send it” — a library in every application, an SDK, a handful of import statements. Kill that instinct quickly, because FR-1 killed it: with 50k services written by hundreds of teams in a dozen languages, any scheme requiring per-service work guarantees patchy adoption, and patchy observability is a trap worse than none — you stop trusting your own dashboards. The answer is to move collection out of the services entirely:

DEFINITION — Telemetry agent

A small daemon running on every host (or beside every workload as a sidecar), owned by the platform team: it tails log files, collects or receives metrics, batches, compresses, and ships both upstream — buffering locally when upstream is unavailable. Agents are how collection (FR-1) happens “automatically”: a service writes to stdout and exposes a metrics endpoint; the agent does the rest.

Two deployment facts make this work, and both are worth saying aloud. `stdout` and a `/metrics` endpoint are **conventions**, not integrations — a service gets observed the moment it is scheduled onto a host, with zero code changes, which is the only adoption curve that reaches 100%. And the agent is owned by **your** team, the platform team — so batching policy, compression, buffering, and upgrades roll out centrally instead of waiting on fifty thousand dependency bumps. You have converted an adoption problem into an operations problem, and operations problems are the kind platform teams can actually solve.

The one genuine design fork in collection is **who initiates the metrics transfer** — and this is a question with a real answer in the industry, so learn both sides well enough to argue either:

DEFINITION — Push vs. pull (scrape)

Push: the source (or its agent) sends metrics upstream when it has them. *Pull*: the collector periodically **requests** metrics from each target’s endpoint (“scrapes” `/metrics` every 15 s). Pull gives the collector control of sample timing, automatic **up/down detection** (a target that doesn’t answer is itself a signal), and easy correctness (one place enforces the schedule). Push fits short-lived jobs that vanish before they can be scraped, and crossing trust boundaries outward. Production fleets commonly run both: pull for services, push gateways for batch jobs.

Savor the up/down detection point, because it is the deepest argument in the exchange. Under pull, a target that stops answering **generates information**: the collector’s missed scrape is itself a health signal, for free, on a schedule you control. Under push, silence is ambiguous — is the service down, or merely quiet? Distinguishing the two requires extra machinery (heartbeats, timeouts on staleness) that pull gets as a side effect of asking. That single asymmetry is why the industry’s default for services is pull. But then run the reverse case: a batch job that lives for forty seconds can die between scrapes and take its metrics to the grave — for those, push is the only honest option. “Pull for services, push for batch jobs” is not a compromise; it is each model deployed where its weakness cannot bite.

DEFINITION — Backpressure

What a pipeline does when a downstream stage is slower than upstream: the pressure must propagate backward and be **absorbed somewhere** — a queue grows, a producer is slowed, or data is deliberately shed. A pipeline without a backpressure plan converts every slow consumer into a cascading outage. Our answer: a durable buffer between agents and processors, plus explicit drop policies per data class (metrics shed first, logs sampled — Section 4.2’s degradation agreement).

Here is the mental model that makes backpressure intuitive: a pipeline with no shock absorber is **rigid**, and rigid systems transmit force. If the log indexer slows down and nothing absorbs the difference, the slowness travels upstream at the speed of TCP until it reaches the agents — and then, if the agents have nowhere to put data, it reaches **the application hosts themselves**. Your observability platform has just become a source of production outages, which is the one failure mode you can never be forgiven for. Every stage of the design below is an answer to the question “where does the pressure go?”

KEY INSIGHT — The buffer is the shock absorber

A Kafka-class log (Chapter 2) between agents and processors buys four things at once: spikes are absorbed (the incident-storm insight of Section 4.4); indexing can lag and replay after outages; a bad deploy of the indexing fleet costs **lag**, not data; and new consumers (the future tracing pipeline, a billing tap) attach without touching producers. Decoupling ingestion from indexing is the single highest-leverage decision in the chapter. When an interviewer asks what one component you would fight to keep, it is this one — and the reason is that the buffer converts **failure domains** into **lag domains**: downstream outages stop being data loss and become a number (consumer lag) you can watch recover.

And a deliberate callback, because platforms compound: the buffer's tenants are protected from each other with per-team **ingestion quotas** — Chapter 3's rate limiter, applied to telemetry. One team's debug-level flood must not evict everyone else's logs. When you say this in the interview, name the connection explicitly; showing that last chapter's design is a **component** of this one is exactly the kind of cross-system fluency that separates staff engineers from diagram reciters.

4.8 Log Storage: The Inverted Index

Now follow the log fork of the firehose into its store. Log search must answer “find the records containing these tokens, in this time range, with these field values, among billions of records” in seconds. Sit with that requirement for a moment and let it eliminate options. A scan is impossible — 170 TB/day laughs at scans. A hash index is useless — queries are about **containment** of tokens, not equality of keys. What remains is the structure every text engine since the 1960s has converged on, and you should be able to derive it on the spot: if queries ask “which records contain token X?”, then at write time you must build the answer to exactly that question — a map from tokens to records.

DEFINITION — Inverted index / posting list

The data structure behind every text search engine: a map from **token** to the sorted list of records containing it (the *posting list*). “payment” → [rec 17, rec 203, rec 8811, ...]. A query like payment AND timeout intersects two posting lists — set intersection over sorted lists, not a scan of the data. Building the index is work done **at write time** so that reads stay fast: logs are a write-once, read-rarely, read-expensively workload, and the inverted index prices exactly that.

Internalize the workload characterization, because it is the defense of the whole design. You pay indexing cost on **every one** of ten million writes per second so that a query asked perhaps a few thousand times a day answers in seconds. Is that insane? Only if reads and writes were symmetric — they are not. The alternative (“schema-on-read”: store raw, grep at query time) moves the cost to the read side, which sounds thrifty until the read is a paged engineer waiting for a scan of a petabyte at 3 a.m. Writes are cheap because they are amortizable, parallelizable, and never urgent; reads are expensive because a human is staring at them. The inverted index is what “spend write capacity to buy read latency” looks like as a data structure.

DEFINITION — Tokenization / structured logging

Tokenization splits raw text into searchable terms (lower-cased, split on punctuation). *Structured logging* emits logs as JSON records — {“level”:“ERROR”, “service”:“checkout”, “msg”:“payment timeout”, ...} — so fields are indexed **as fields** (service:checkout) rather than dug out of text at query time. The platform ingests both; structure makes queries faster and sampling smarter, and FR-6's self-service story nudges teams toward it.

Three storage decisions carry the load — and as you read them, notice that each one converts a **requirement** into a **mechanism**:

1. **Immutable segments, merged in the background.** Incoming records are buffered into small index **segments** written to disk immutably; a background process merges small segments into larger ones. Appends and merges are sequential I/O — the cheapest disk work there is (same lesson as Chapter 1's operation log). Immutability is the quiet superpower here: a segment never changes once written, so it can be cached, replicated, checksummed, and merged without a single lock.
2. **Sharding by time.** The index is partitioned into time buckets (e.g., one index per day per tenant class). Queries prune whole days by timestamp before touching a posting list — and **retention becomes deletion of an old index file**, not a billion per-record deletes. Time-sharding turns FR-5 into `rm`. Compare this with the naive alternative — issuing per-record deletions against a live index, fragmenting posting lists and burning I/O to destroy data nobody wanted — and you will never design retention any other way.
3. **Hot / warm / cold tiers.** Today's indices live on fast SSD nodes with replicas (hot); last week's on cheaper nodes (warm); months-old data as compressed archives in object storage (cold), rehydrated on the rare historical query. Cost per GB falls by an order of magnitude per tier — Section 4.5 made clear this **is** the design. The tiers work because of a statistical mercy: engineers overwhelmingly query **recent** data (the incident is always now), so the expensive hardware only ever has to hold the small, hot slice of history.

4.9 Metrics Storage: The Time-Series Database

Across the fork, the metrics stream needs the opposite kind of store. Where logs demanded text machinery, metrics demand **numerical** machinery — and the first concept you need is the atom of the whole pillar:

DEFINITION — Time series / sample / label set

A *time series* is a named, labeled sequence of **(timestamp, value)** pairs: `http_requests_total{service="checkout", status="500"}` sampled every 15 s. A *sample* is one such pair. The *labels* (tags) are the dimensions queries filter and group by. The number of **distinct series** — the product of all label-value combinations — is the *cardinality*, and it is the metric system's one true nemesis.

COMMON PITFALL — Cardinality explosion

Labels must be **dimensions**, never **identifiers**. `status="500"` is a dimension (a handful of values); `user_id="u_91827"` is an identifier — millions of series, each seen once, index structures bloated beyond RAM, queries scanning garbage. Rule of thumb: a label whose value set grows with **traffic or users** does not belong on a metric; it belongs in a log or a trace. Every metrics interview eventually arrives here — arrive first. And if the interviewer asks why one innocent-looking label can hurt the **whole cluster**, give the real answer: series indices are shared infrastructure, so one team's `user_id` label is a denial-of-service attack on everyone's dashboards — which is why Section 4.15 enforces per-metric cardinality budgets at write time.

Why a purpose-built store at all? Because samples of one series arrive in time order and change slowly, and data with that shape compresses almost to nothing — **if** your storage engine is built to notice. A general-purpose database stores each sample as a row and pays row overhead for it; a time-series engine exploits the regularity:

DEFINITION — Delta encoding / delta-of-delta

Store differences, not values. Timestamps sampled every 15 s delta-encode to a constant 15000 ms; the **delta-of-delta** (difference of consecutive deltas) is 0 almost always — compressible to a single bit per timestamp in the production scheme (Facebook’s Gorilla, which also XOR-encodes values so only meaningful bits are stored). 16-byte samples become 1–2 bytes. Section 4.13 implements the delta+zigzag+varint core of the idea.

Take the compression seriously as a **design argument**, not a trivia fact. Remember Section 4.5’s estimate: $0.7\text{M samples/s} \times 86,400\text{ s} = 60\text{ billion samples a day}$. At 16 uncompressed bytes that is a petabyte a year; at 1.4 compressed bytes it is a rack’s worth of disks. The compression ratio is not an optimization you apply later — it is the difference between a feasible design and an unfundable one. This is the same pattern you saw in Chapter 3’s estimation table, wearing new clothes: **what you choose to store** is the cost model.

DEFINITION — Downsampling / rollup

Replacing old high-resolution samples with precomputed aggregates — keep 15-second resolution for a week, 1-minute for a quarter, 1-hour beyond: avg/min/max/sum/count per bucket, computable incrementally. Nobody asks for 15-second data from last year (Section 4.2), and rollups turn 13 months of retention from a petabyte problem into a terabyte one.

Rollups deserve one skeptical question, because the interviewer will ask it: “if you throw away the raw samples, what did you lose?” The honest answer: the ability to ask **new questions at old resolution** — you can never recompute last quarter’s 99th-percentile at 15-second granularity from hourly averages. You accept this because the interviewer already told you (in Section 4.2’s retention answer) that nobody needs it, and because the aggregates you **do** keep were chosen to be the ones capacity reviews and trend analyses actually consume. Trade-offs stated this precisely are unassailable.

Queries are then range scans over compressed blocks, sharded by metric-name hash: fetch the blocks for matching series in the range, decompress, aggregate. Dashboard panels are exactly this, cached and refreshed — which means the entire dashboarding product, with all its Grafana glamour, reduces to “a cache in front of a range scan.” Saying that out loud in an interview is a delightful way to demystify a billion-dollar product category.

4.10 API & Query Design

Three surfaces — ingestion, query, config — each deliberately boring. That word choice is a compliment you should learn to pay: after chapters of exotic protocols, the correct answer here is plain HTTP with obvious endpoints, because an observability platform’s API is operated at 3 a.m. by humans under stress, and boring APIs fail in boring, debuggable ways.

Method	Path	Purpose
POST	/v1/logs:batch	Agent push: compressed batch of log records with source metadata
GET	/metrics (on targets)	The scrape endpoint pull collectors call every 15 s
POST	/v1/metrics:write	Push path for short-lived jobs
GET	/v1/logs/search?q=&from=&to=	Log search: query string, time range, cursor-paginated
POST	/v1/metrics/query	Instant and range queries over series

Method	Path	Purpose
POST	/v1/alerts/rules	Create/update alert rules (FR-4, FR-6); versioned like Chapter 3's rules

Two rows repay attention. The logs:batch endpoint takes **compressed batches**, not individual records — at 10M events/s, per-record HTTP would spend more bytes on headers than on data, and batching is where the agent earns its keep (amortizing connection costs across thousands of records). And the alerts endpoint is **versioned like Chapter 3's rules** — the same pattern you just learned for limiter config reappears here unchanged, because “config that pages humans” has the same lifecycle needs as “config that rejects traffic”: reviewable, reversible, pushable.

Query languages, sketched to show the shape (not to design in full):

```
-- logs: field filters + free text + time range
service:checkout AND level:ERROR AND "payment timeout" @ last 1h

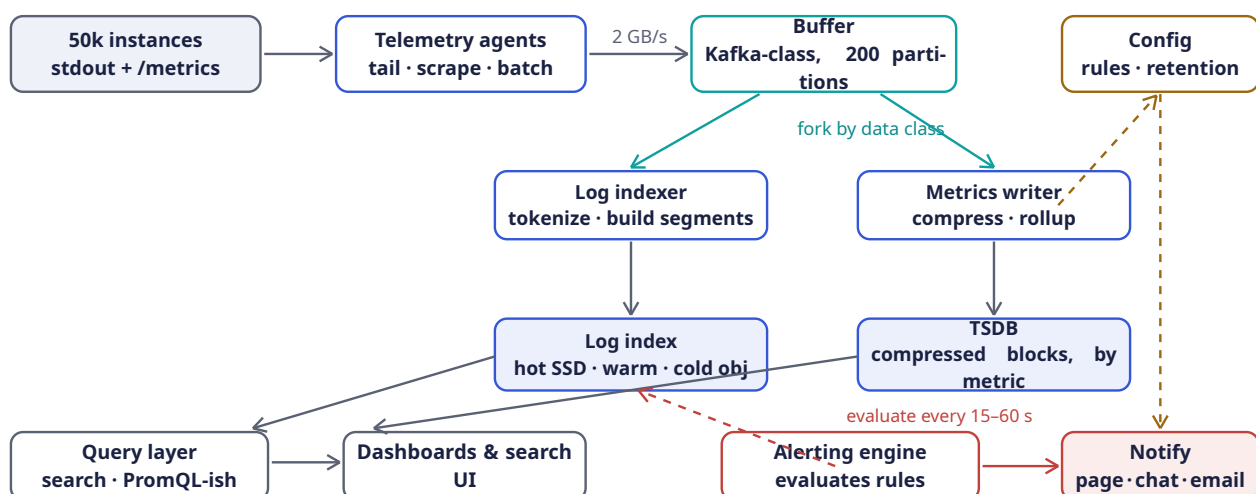
-- metrics: series selector + aggregation over a range
rate(http_requests_total{service="checkout", status=~"5.."}[5m])
|> sum by (region)
```

Read the two examples as the two data models asserting themselves in syntax. The log query is a **filter**: it selects records and shows them to a human, and the time range is a first-class citizen because log questions are always anchored in an incident's timeline. The metrics query is a **computation**: select series, take a rate over a window, then aggregate across a label dimension. Notice it returns one number per region — the raw samples are nobody's destination. If an interviewer asks you to design the query language itself, this contrast is your thesis: logs want search grammar, metrics want an algebra of aggregations.

Alert rules are records: `{ name, query, threshold, comparison, for_duration, severity, notify_channels }` — evaluated by the alerting engine of Section 4.12. Keep the `for_duration` field in view; it looks like a detail and is actually the load-bearing anti-flapping mechanism you are about to study.

4.11 High-Level Architecture

Every decision so far — agents everywhere, one durable buffer, two purpose-built stores, humans at the far end — assembles into a single pipeline. Here it is, drawn as the data experiences it:



One write path, two stores. The buffer absorbs incident storms; the query layer federates across both stores.

Walk the diagram in data order — write path first, read path second, control path last — because that ordering is also the correct order in which to **present** the architecture in an interview, and practicing the narration is half the value of the picture.

The write path runs along the top edge. At top-left, the 50,000 service instances emit the only two things FR-1 requires of them: text to stdout and a `/metrics` endpoint. The next box, **telemetry agents**, is where your platform first touches the data — tailing the log files, scraping the endpoints, batching and compressing. Then comes the arrow labeled “2 GB/s” into the **buffer**, and this is the moment to slow your narration down: that arrow is where the entire incident-storm insight of Section 4.4 lives. The buffer (a Kafka-class durable log, 200 partitions over 20–30 brokers per Section 4.5) absorbs whatever the fleet throws at it — including the 3× error-storm spike — and durably holds it, so that everything downstream can be slow, crashed, or mid-deploy without losing a byte. Immediately after the buffer, the diagram performs its central act: the two teal arrows forking downward, labeled “fork by data class.” Left fork: the **log indexer** consumes the log stream, tokenizes, and builds the immutable inverted-index segments of Section 4.8, writing them into the **log index cluster** with its hot/warm/cold tiers. Right fork: the **metrics writer** consumes the sample stream, compresses it with Section 4.9’s delta machinery, maintains rollups, and lands compressed blocks in the **TSDB**. One firehose in, two stores out — Section 4.6’s table, made spatial.

The read path is the bottom row, read right-to-left from the human’s perspective. An engineer opens the **dashboards & search UI**; their request hits the **query layer**, whose job is federation — a log search is routed to the log index, a PromQL-ish expression to the TSDB, and a dashboard mixing both panels fans out to both and merges. The two slate arrows angling up from the query layer to each store are those federated sub-queries. Note what the query layer also owns but the diagram cannot show: **caching** of dashboard panels, which is what absorbs the “two thousand engineers open the same dashboard at once” read spike from Section 4.4. And the **alerting engine** sits on this row too — deliberately drawn reading from the stores (dashed crimson arrow) exactly like a tireless engineer issuing a query every 15–60 seconds, because that is precisely what it is. When a rule fires, the solid crimson arrow carries the notification to **Notify** — page, chat, email — the platform’s only truly write-facing act toward humans.

The control path is the amber column at far right: the **config** service holds alert rules, retention policies, and quotas, and a dashed amber arrow runs down its side fanning config out to the components that need it — Chapter 3’s lesson (config is pushed, never fetched on a hot path) applied without modification.

Now the exercise that makes the picture yours: remove each box mentally and name what breaks. Remove the buffer and every downstream hiccup becomes upstream data loss — Section 4.7’s rigidity argument. Remove the fork and you are back in the one-store pitfall of Section 4.6. Remove the query layer and every dashboard client grows federation logic, duplicated and drifting across teams. Remove the alerting engine’s independent infrastructure (not a box, but a property Section 4.12 will stress) and the one outage that must page is the one that cannot. Every box is load-bearing; that is what makes it an architecture rather than a parts list.

Component	Responsibility	Why it is shaped this way
Telemetry agents	Tail logs, scrape/collect metrics, batch + compress, ship	Per-host daemon = zero per-service work (FR-1); local buffer rides out upstream blips
Buffer (Kafka-class)	Durable, replayable queue between agents and processors	The shock absorber of Section 4.7: spikes, replays, new consumers
Log indexer	Parse, tokenize, build inverted-index segments	Stateless consumers of the log topic; scale with partition count

Component	Responsibility	Why it is shaped this way
Log index cluster	Hot/warm/cold inverted indices, time-sharded	Search needs posting lists; retention needs cheap tiers (Section 4.8)
Metrics writer	Compress samples into blocks; maintain rollups	Stateless consumers of the metrics topic
TSDB cluster	Compressed time-series blocks; label index	Range scans over 2 B samples (Section 4.9)
Query layer	Federate queries across both stores; cache dashboard panels	One door for engineers; caches absorb dashboard refresh storms
Alerting engine	Evaluate rules continuously; notify without flapping	Section 4.12 — its correctness is the product's voice
Config service	Rules, retention, quotas; versioned, pushed	Chapter 3's lesson: config never fetched on the hot path

The right-hand column is your oral defense, one sentence per box. Two rows share a phrase worth noticing — the log indexer and metrics writer are both “stateless consumers” of buffer topics. That is not laziness of vocabulary; it is the scaling story of Section 4.14 in seed form. Stateless consumers scale by adding instances and rebalancing partitions, which means the processing tier’s capacity plan is “autoscale on consumer lag” — a metric the buffer gives you for free.

4.12 Deep Dive: The Alerting Engine

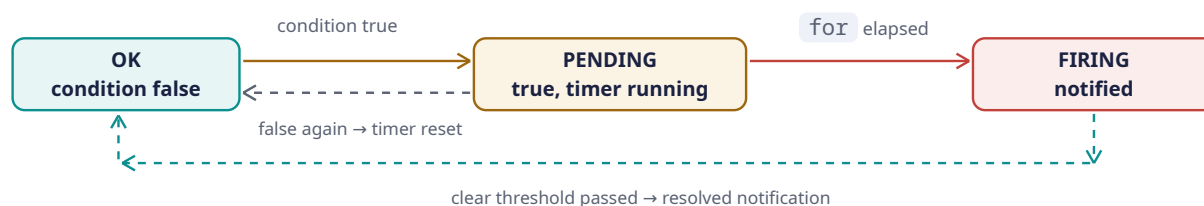
Alerting is where the metrics store earns its keep — and where the platform earns its **reputation**. Think about what an alert is from the receiving end: it is the only message in the company authorized to wake a human. Abuse that authority with noisy, flapping, repeating pages, and within a quarter your engineers will have muted the channel, at which point you have built a very expensive no-op. The engine’s loop is trivially stated — **every evaluation interval (15–60 s), run every rule’s query, compare against the threshold** — and the difficulty is entirely in not crying wolf.

DEFINITION — Flapping / hysteresis / “for” duration

Flapping: an alert that oscillates OK → FIRING → OK → FIRING as the metric jitters around the threshold, paging a human every oscillation until the rule is muted and the signal lost. The cures: a “for” duration — the condition must hold continuously for N minutes before firing (a PENDING state); and *hysteresis* — firing and clearing use **different** thresholds (fire at p99 > 500 ms for 5 min, clear only when p99 < 450 ms for 10 min), so a metric straddling one line cannot cross both. Alert fatigue is a **system failure mode**, and it is engineered against, exactly like throughput.

Understand **why** each cure works, not just what it is. The **for** duration exploits a statistical fact: real outages are **sustained** (a dependency is down, a deploy is bad), while threshold-crossing noise is **brief** (one garbage-collection pause, one slow retry). Requiring continuity is a cheap classifier that separates the two with almost no machinery. Hysteresis solves a different problem: a metric hovering **exactly at** the threshold crosses it every evaluation, so any single boundary line is a flap generator; splitting entry and exit into two different lines makes oscillation require a real swing, not a tremor. Borrowed from thermostats and Schmitt triggers, it is one of those ideas that proves the same mathematics keeps rescuing different fields.

The per-rule state machine — three states, and every arrow is a policy decision:



Trace it left to right, and then — this is the part candidates skip — trace the **backward** edges, because the backward edges are where the humanity lives. Forward: a rule sits in OK (teal, leftmost) while its condition is false. The first true evaluation moves it to PENDING (amber, center) — and crucially, **no notification is sent**; the state change only starts a timer. This is the `for` duration incarnate: PENDING is a waiting room where brief spikes go to be forgotten. If the condition holds continuously until the timer elapses, the crimson arrow to FIRING fires — and **that** transition, and only that transition, pages a human. Look at the label under the PENDING→FIRING arrow: “`for` elapsed.” The page is caused by the **timer**, not by the breach. A breach is cheap; a **sustained** breach is worth a human’s sleep.

Now the backward edges. From PENDING, a dashed slate arrow returns to OK labeled “false again → timer reset”: the spike ended, the waiting room empties, nobody was woken, and the timer forgets everything — the **continuity** requirement means a single healthy evaluation flushes all accumulated suspicion. From FIRING, the long dashed teal arrow sweeps down and all the way back left to OK, labeled “clear threshold passed → resolved notification”: when the metric recovers past the **clear** line (hysteresis — a different, stricter line than the fire line), exactly one more notification goes out, the resolution, and the machine returns to OK. Count notifications over a full incident lifecycle: one at firing, one at resolution. Two pages per incident, zero per evaluation. That count — not the states themselves — is the design’s entire reason to exist, and Section 4.13’s test suite proves the counting.

Three more engineering points, each one learned by somebody the hard way:

- **Deduplication and grouping.** One dying dependency fires forty service alerts; the engine groups by root labels (`region` , `dependency`) into **one** page with the forty attached as context. Notifications are facts about **state transitions**, not about every evaluation. Without grouping, the first casualty of a major outage is the on-call’s ability to think — forty simultaneous pages is information **destruction**, because the human cannot read faster than the pager can scream.
- **Evaluation must be independent of ingestion health of the thing it watches.** If the alerting engine shares a failure domain with the metrics pipeline, the one outage that must page is the one that can’t. It runs on separate infrastructure and — critically — can fire on **absence** of data (Section 4.15). Read that last property twice: an engine that only evaluates thresholds cannot distinguish “all services healthy” from “all telemetry stopped arriving.” Absence alerting is how the platform pages about its own blindness.
- **Self-service with guardrails (FR-6).** Teams write rules in config; the platform validates them (query parses, series cardinality bounded, threshold sane) and versions every change, exactly the Chapter 3 rule-lifecycle pattern. The guardrails are not bureaucracy — an unvalidated alert rule is a cron job that pages humans, and it deserves at least the review a cron job gets.

4.13 Rust Reference Implementations

Four distilled pieces, each with a deterministic test. They are teaching skeletons of what runs at platform scale: a mini inverted index for log search, timestamp compression for the TSDB, incremental rollups, and the alert state machine. Read each one as the executable form of a section you have already finished — Section 4.8, 4.9, 4.9, and 4.12 respectively — and the code will feel less like four programs and more like one argument.

4.13.1 A Mini Inverted Index for Log Search

Structured records are tokenized; each token maps to a sorted posting list of record ids. A two-term **AND** query intersects the lists in linear time.

```
use std::collections::HashMap;

/// One structured log record, as the indexer sees it after parsing.
#[derive(Debug, Clone)]
pub struct LogRecord {
    pub id: u64,
    pub timestamp_ms: u64,
    pub service: String,
    pub level: String,
    pub message: String,
}

/// Lower-case, split on anything that isn't alphanumeric:
/// "payment TIMEOUT after 3s" -> ["payment", "timeout", "after", "3s"].
fn tokenize(text: &str) -> Vec<String> {
    text.to_lowercase()
        .split(|c: char| !c.is_alphanumeric())
        .filter(|t| !t.is_empty())
        .map(str::to_string)
        .collect()
}

/// token -> sorted ids of records containing it ("posting list").
/// Also index the searchable fields under `field:value` tokens so that
/// `service:checkout` is just another posting-list lookup.
#[derive(Default)]
pub struct InvertedIndex {
    postings: HashMap<String, Vec<u64>>,
}

impl InvertedIndex {
    pub fn add(&mut self, rec: &LogRecord) {
        let mut tokens = tokenize(&rec.message);
        tokens.push(format!("service:{}", rec.service.to_lowercase()));
        tokens.push(format!("level:{}", rec.level.to_lowercase()));
        tokens.sort();
        tokens.dedup();
        for tok in tokens {
            let list = self.postings.entry(tok).or_default();
            // ids are appended in increasing order, so lists stay sorted;
            // a real engine would also store positions for phrase queries.
            list.push(rec.id);
        }
    }

    pub fn lookup(&self, token: &str) -> &[u64] {
        self.postings.get(token).map(Vec::as_slice).unwrap_or(&[])
    }

    /// Intersection of two sorted lists: the heart of `A AND B`.
    pub fn and(&self, a: &str, b: &str) -> Vec<u64> {
        let (xs, ys) = (self.lookup(a), self.lookup(b));
        let (mut i, mut j, mut out) = (0, 0, Vec::new());
        while i < xs.len() && j < ys.len() {
            match xs[i].cmp(&ys[j]) {
                std::cmp::Ordering::Less => i += 1,
            }
        }
    }
}
```

```

        std::cmp::Ordering::Greater => j += 1,
        std::cmp::Ordering::Equal => {
            out.push(xs[i]);
            i += 1;
            j += 1;
        }
    }
}
out
}
}

#[cfg(test)]
mod tests {
    use super::*;

    fn rec(id: u64, service: &str, level: &str, msg: &str) -> LogRecord {
        LogRecord { id, timestamp_ms: id, service: service.into(),
                    level: level.into(), message: msg.into() }
    }

    #[test]
    fn search_intersects_posting_lists() {
        let mut ix = InvertedIndex::default();
        ix.add(&rec(1, "checkout", "ERROR", "payment timeout after 3s"));
        ix.add(&rec(2, "checkout", "INFO", "payment authorized"));
        ix.add(&rec(3, "search", "ERROR", "timeout talking to index"));
        ix.add(&rec(4, "checkout", "ERROR", "payment timeout after 5s"));

        // free text
        assert_eq!(ix.lookup("timeout"), &[1, 3, 4]);
        // field query
        assert_eq!(ix.lookup("service:checkout"), &[1, 2, 4]);
        // the incident query: payment AND timeout, checkout only
        assert_eq!(ix.and("payment", "timeout"), &[1, 4]);
        assert_eq!(
            ix.and("service:checkout", "level:error")
                .into_iter()
                .collect::<Vec<_>>(),
            vec![1, 4]
        );
    }
}

```

Two design details to be able to narrate. First, the field-query trick: structured fields are indexed as **synthetic tokens** (`service:checkout` becomes just another key in the same map), so field filters ride the same machinery as full text with zero special cases — one structure, two query powers. Second, the `and` function is a classic merge join over sorted lists: two cursors, linear time, no hashing. And the test's final assertion encodes a whole incident workflow — “checkout errors involving payment and timeout” — answered by intersecting two lists, never by scanning records. That is Section 4.8's thesis with a green checkmark.

Production engines add positions (for phrase queries), compression of posting lists, skip pointers for faster intersection, and segment files instead of a `HashMap` — but the **algorithm** the interviewer wants to hear is the sorted intersection above, unchanged since the 1960s.

4.13.2 Delta + Zigzag + Varint: Compressing Timestamps

The honest core of Gorilla-style compression: store the first timestamp, then **delta-of-deltas**; encode each as a zigzag varint so small numbers — including small **negative** corrections — cost one byte.

Three tiny ideas compose here, so take them one at a time as the code introduces them: **delta** (store differences from the previous timestamp), **zigzag** (map signed values to unsigned so that small magnitudes of either sign become small unsigned ints), and **varint** (spend one byte on small values, more only when needed). None of the three is clever alone; stacked, they are why a steady 15-second scrape costs about a byte per point.

```

/// Zigzag maps signed -> unsigned so small magnitudes (either sign)
/// become small unsigned ints: 0->0, -1->1, 1->2, -2->3, 2->4, ...
fn zigzag(n: i64) -> u64 {
    ((n << 1) ^ (n >> 63)) as u64
}
fn unzigzag(z: u64) -> i64 {
    ((z >> 1) as i64) ^ -((z & 1) as i64)
}

/// Unsigned varint: 7 bits per byte, high bit = "more bytes follow".
fn write_varint(mut z: u64, out: &mut Vec<u8>) {
    loop {
        let byte = (z & 0x7f) as u8;
        z >>= 7;
        if z == 0 { out.push(byte); break; }
        out.push(byte | 0x80);
    }
}
fn read_varint(bytes: &[u8], pos: &mut usize) -> u64 {
    let (mut z, mut shift) = (0u64, 0);
    loop {
        let byte = bytes[*pos];
        *pos += 1;
        z |= ((byte & 0x7f) as u64) << shift;
        if byte & 0x80 == 0 { return z; }
        shift += 7;
    }
}

/// Compress a sorted series of millisecond timestamps:
/// first value verbatim, then delta-of-delta as zigzag varints.
pub fn compress_timestamps(ts: &[u64]) -> Vec<u8> {
    assert!(!ts.is_empty());
    let mut out = Vec::new();
    write_varint(ts[0], &mut out);
    let (mut prev_ts, mut prev_delta) = (ts[0] as i64, 0i64);
    for w in ts.windows(2) {
        let delta = w[1] as i64 - prev_ts;
        let dod = delta - prev_delta; // 0 for a steady 15s scrape
        write_varint(zigzag(dod), &mut out);
        prev_ts = w[1] as i64;
        prev_delta = delta;
    }
    out
}

pub fn decompress_timestamps(bytes: &[u8], count: usize) -> Vec<u64> {
    let mut pos = 0;
    let first = read_varint(bytes, &mut pos) as i64;
    let mut out = vec![first as u64];
    let (mut prev_ts, mut prev_delta) = (first, 0i64);
    for _ in 1..count {
        let dod = unzigzag(read_varint(bytes, &mut pos));
        let delta = prev_delta + dod;
    }
}

```



```

        prev_ts += delta;
        prev_delta = delta;
        out.push(prev_ts as u64);
    }
    out
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn zigzag_roundtrip_and_small_magnitudes() {
        for n in [-2, -1, 0, 1, 2, 15000, -15000] {
            assert_eq!(unzigzag(zigzag(n)), n);
        }
        assert_eq!(zigzag(-1), 1);
        assert_eq!(zigzag(0), 0);
    }

    #[test]
    fn steady_15s_scrape_compresses_to_about_a_byte_per_point() {
        let base = 1_700_000_000_000u64; // some epoch-ms
        let ts: Vec<u64> = (0..1000).map(|i| base + i * 15_000).collect();
        let packed = compress_timestamps(&ts);
        assert_eq!(decompress_timestamps(&packed, ts.len()), ts);
        // 1000 x 8-byte timestamps -> ~1007 bytes: ~1 byte per point.
        assert!(packed.len() < 1100, "got {} bytes", packed.len());
    }

    #[test]
    fn jitter_and_gaps_survive_roundtrip() {
        // scrape jitter (+/- 3ms) plus one 30s gap where a sample was lost
        let mut ts = vec![1_000u64, 16_003, 31_002, 61_001, 75_997];
        let packed = compress_timestamps(&ts);
        assert_eq!(decompress_timestamps(&packed, ts.len()), ts);
        ts.clear(); // prove the bytes alone reconstruct everything
        assert!(ts.is_empty());
    }
}

```

The middle test is the claim from Section 4.9, measured: a thousand perfect 15-second-interval timestamps — 8,000 bytes naively — pack into under 1,100. And the third test answers the skeptic’s question before it is asked: “what if the scrapes **aren’t** perfectly regular?” Jitter and even a dropped sample survive the round trip exactly, because delta-of-delta degrades gracefully — irregularity costs a few extra bytes, never correctness. Compression schemes that are optimal only for perfect input are toys; this one is exact for all input and merely **cheaper** for regular input, which is the property that matters in production.

Gorilla additionally XORs consecutive **values** and stores only meaningful bit-runs — the same philosophy: at scale, the difference between 16 bytes and 1.4 bytes per sample is the difference between a petabyte and a rack.

4.13.3 Incremental Rollups for Retention

A rollup bucket carries just enough state to be mergeable: `sum`, `min`, `max`, `count`. Raw samples stream in; the bucket answers the five aggregates without remembering any sample. Stop and appreciate what “mergeable” buys you, because it is the whole scalability story of Section 4.9’s retention ladder: if combining two buckets required their raw samples, rollup would be a batch job over

petabytes; because the aggregates combine in $O(1)$, an hour-bucket can become a day-bucket with four arithmetic operations and no memory of the past. Associativity — $(a + b) + c = a + (b + c)$ — is doing systems work here.

```
/// One downsampled bucket: mergeable in  $O(1)$  without raw samples.
/// This is why rollups scale: merging two 1-hour buckets is
/// adding sums, taking min/max, adding counts.
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct Bucket {
    pub sum: f64,
    pub min: f64,
    pub max: f64,
    pub count: u64,
}

impl Bucket {
    pub fn empty() -> Self {
        Bucket { sum: 0.0, min: f64::INFINITY, max: f64::NEG_INFINITY, count: 0 }
    }
    pub fn add(&mut self, value: f64) {
        self.sum += value;
        self.min = self.min.min(value);
        self.max = self.max.max(value);
        self.count += 1;
    }
    /// Merge another bucket's aggregates in  $O(1)$  — no raw samples needed.
    /// This associativity is exactly why rollups scale: two hour-buckets
    /// combine into a day-bucket with four cheap operations.
    pub fn merge(&mut self, other: &Bucket) {
        if other.count == 0 { return; }
        self.sum += other.sum;
        self.min = self.min.min(other.min);
        self.max = self.max.max(other.max);
        self.count += other.count;
    }
    pub fn avg(&self) -> Option<f64> {
        (self.count > 0).then(|| self.sum / self.count as f64)
    }
}

/// Assign a timestamp (ms) to its bucket of `width_ms`, then fold in.
pub fn rollup(samples: &[(u64, f64)], width_ms: u64)
    -> std::collections::BTreeMap<u64, Bucket>
{
    let mut buckets = std::collections::BTreeMap::new();
    for &(ts, v) in samples {
        buckets.entry(ts / width_ms).or_insert_with(Bucket::empty).add(v);
    }
    buckets
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn rollup_buckets_match_hand_computation() {
        // 15s samples across two 1-minute buckets; base aligned to a
        // minute boundary so the first four samples share bucket 0.
        let t = 1_700_000_040_000u64; // == 28_333_334 * 60_000
    }
}
```

```

let samples: Vec<(u64, f64)> = (0..8)
  .map(|i| (t + i * 15_000, (100 + i * 10) as f64))
  .collect();
let m = rollup(&samples, 60_000);
let keys: Vec<u64> = m.keys().copied().collect();
assert_eq!(keys.len(), 2);
let b0 = m[&keys[0]];
assert_eq!(b0.count, 4); // t, t+15s, t+30s, t+45s
assert_eq!(b0.sum, 100.0 + 110.0 + 120.0 + 130.0);
assert_eq!(b0.min, 100.0);
assert_eq!(b0.max, 130.0);
assert_eq!(b0.avg(), Some(115.0));
}

#[test]
fn buckets_merge_without_raw_samples() {
  let mut a = Bucket::empty();
  for v in [10.0, 20.0, 30.0] { a.add(v); }
  let mut b = Bucket::empty();
  for v in [5.0, 50.0] { b.add(v); }
  a.merge(&b);
  assert_eq!(a.count, 5);
  assert_eq!(a.sum, 115.0);
  assert_eq!(a.min, 5.0);
  assert_eq!(a.max, 50.0);
  assert_eq!(a.avg(), Some(23.0));
}
}

```

One choice in this listing deserves a question from you before the interviewer asks it: why does `avg` return an `Option`? Because an empty bucket has no average, and dividing `0.0` by `0` would produce `NaN` — a value that silently infects every aggregate it touches downstream. The type system is being used as a specification: **you cannot read an average without acknowledging the empty case**. Also note `min` starts at $+\infty$ and `max` at $-\infty$, the identity elements of their operations — the same algebra trick that makes `merge` need the early-return on `count == 0`. These are small lines, but they are the difference between code that is correct by inspection and code that is correct by luck.

4.13.4 The Alert Evaluator: OK → PENDING → FIRING

The state machine of Section 4.12, made executable. The test demonstrates the two failure modes it prevents: a single bad spike must **not** page (no `for` violation), and a sustained breach must page **once**, not on every evaluation. As you read the `match`, keep the diagram in view — each arm is one arrow, and the two `Some(Notification::...)` lines are the only two places in the entire system where a human gets woken.

```

/// States of Section 4.12's machine. `pending_since` is Some(ms) while
/// the condition has been continuously true but `for_ms` hasn't elapsed.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum AlertState {
  Ok,
  Pending { since_ms: u64 },
  Firing,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Notification {
  Fired,
  Resolved,
}

```

```

}

pub struct AlertRule {
    pub threshold: f64,
    pub for_ms: u64,
}

pub struct AlertEvaluator {
    pub rule: AlertRule,
    state: AlertState,
}

impl AlertEvaluator {
    pub fn new(rule: AlertRule) -> Self {
        AlertEvaluator { rule, state: AlertState::Ok }
    }

    /// Feed one evaluation (one query result) at time `now_ms`.
    /// Returns a notification only on *transitions*.
    pub fn evaluate(&mut self, breached: bool, now_ms: u64) -> Option<Notification> {
        use AlertState::*;
        match (self.state, breached) {
            (Ok, true) => {
                self.state = Pending { since_ms: now_ms };
                None
            }
            (Pending { since_ms }, true) => {
                if now_ms - since_ms >= self.rule.for_ms {
                    self.state = Firing;
                    Some(Notification::Fired)           // page once
                } else {
                    None                                // still waiting
                }
            }
            (Firing, true) => None,                      // already paged; stay quiet
            (Firing, false) => {
                self.state = Ok;
                Some(Notification::Resolved)
            }
            (Pending { .. }, false) => {
                self.state = Ok;
                None                                // spike ended: reset timer
            }
            (Ok, false) => None,
        }
    }

    pub fn is_breach(&self, value: f64) -> bool {
        value > self.rule.threshold
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn evaluator() -> AlertEvaluator {
        AlertEvaluator::new(AlertRule { threshold: 500.0, for_ms: 300_000 })
    }
}

```

```

#[test]
fn single_spike_never_pages() {
    let mut ev = evaluator();
    // p99 jumps over threshold for one 15s evaluation, then recovers
    assert_eq!(ev.evaluate(true, 0), None); // -> Pending
    assert_eq!(ev.evaluate(false, 15_000), None); // recovered: reset
    assert_eq!(ev.state, AlertState::Ok);
}

#[test]
fn sustained_breach_pages_once_then_resolves() {
    let mut ev = evaluator();
    let mut t = 0u64;
    let step = 60_000; // evaluate every minute
    let mut notes = Vec::new();
    // breach for 6 consecutive minutes, `for` = 5 minutes
    for _ in 0..6 {
        if let Some(n) = ev.evaluate(true, t) { notes.push((t, n)); }
        t += step;
    }
    // fired exactly once, when the timer elapsed (t = 5 min)
    assert_eq!(notes, vec![(300_000, Notification::Fired)]);
    // keep breaching: silence
    assert_eq!(ev.evaluate(true, t), None);
    // recovery -> one resolved notification
    assert_eq!(ev.evaluate(false, t + step), Some(Notification::Resolved));
    assert_eq!(ev.state, AlertState::Ok);
}

#[test]
fn threshold_boundary_is_not_a_breach() {
    let ev = evaluator();
    assert!(!ev.is_breach(500.0)); // strictly greater
    assert!(ev.is_breach(500.1));
}
}

```

Read the `(Firing, true) => None` arm until it feels profound, because that one line is the entire product promise of FR-4’s “once, not repeatedly”: while a breach continues, the machine says nothing. All the urgency lives in the **transitions**; steady states — good or bad — are silent. And the third test encodes a policy you should state explicitly in the room: the threshold comparison is **strictly greater**, so a metric sitting exactly on the line is not a breach. Which convention you pick matters less than that you picked one, documented it, and made the boundary a tested behavior rather than an accident of floating-point mood.

4.14 Scaling the Platform

Every tier scales a different axis by a different mechanism, and the table below is worth reading column-wise: first down the **scale axis** column to feel how heterogeneous the pressures are, then down the **mechanism** column to see how rarely the answer is “add more of the same.”

Layer	Scale axis	Mechanism
Agents	Fleet size (50k+)	Embarrassingly parallel: one per host; sizing is a per-host CPU/RAM budget, not a cluster problem

Layer	Scale axis	Mechanism
Buffer	Ingestion GB/s	Partitions (200) spread over 20–30 brokers; keyed by tenant+service so one team’s flood is contained and quota’d (Chapter 3 callback)
Log indexers	Events/s	Stateless consumers, one per partition lane; autoscale on consumer lag
Log index	Data volume + query QPS	Time-sharded indices spread over nodes; hot tier replicated; incident query storms absorbed by the query-layer cache
Metrics writers	Samples/s	Stateless consumers; 0.7M samples/s is small — headroom is free
TSDB	Series cardinality × retention	Shard by metric-name hash; rollups shrink old data 100× before it becomes a storage problem
Alerting engine	Rule count × eval rate	Partition rules across evaluators by hash; evaluations are independent, so this scales linearly

Notice which rows are **boring** — agents, metrics writers, alerting — and which row carries the chapter’s scaling anxiety: the TSDB, whose axis is not throughput at all but **cardinality × retention**. That is the cardinality nemesis of Section 4.9 surfacing in the capacity plan: samples/s is flat and small, but the number of **distinct series** is a product of human creativity in label design, and it grows whenever a team adds a label. You scale this axis with budgets and guardrails (rejecting write path label sets that exceed per-metric cardinality limits), not with hardware — a rare case where the scaling mechanism is a **policy enforced in code**.

KEY INSIGHT — The incident storm is the sizing case

Average load (2 GB/s) is not the design point; **everyone’s worst day at once** is. An outage multiplies error logs 3–5×, and two thousand engineers open dashboards simultaneously. The buffer absorbs the write spike; the query cache absorbs the read spike; per-tenant quotas keep one service’s panic from evicting the logs everyone else needs to debug it. A telemetry platform sized for the average is a platform that dies exactly when it is needed. When the interviewer asks “what’s your peak?”, do not quote the burst multiplier — explain the **correlation**: the load spike and the importance spike are the same event, so there is no such thing as an acceptable brownout.

4.15 Failure Modes & Degradation

The degradation contract from Section 4.2 — metrics may shed, logs degrade by sampling, silence is never acceptable — now gets worked out component by component. Read each row’s **Response** column and notice how often the answer references machinery you have already built; a good failure section should feel like reaping decisions, not making new ones.

Failure	Symptom	Response
Telemetry agent dies	A host goes silent	Agents buffer locally, so restarts lose nothing; and si-

Failure	Symptom	Response
		lence itself is a signal — a heartbeat-absence alert fires (the alerting engine evaluates absence , not just thresholds)
Buffer fills / brokers down	Agents' local buffers grow	Local disk spooling buys hours; on recovery, consumers replay the backlog. Durability is the buffer's whole job
Log indexer lags or crashes	Search results fall behind ingest	Lag is visible as a metric on the buffer (meta-observability, Section 4.17); stateless indexers are replaced and replay from last offset
Index node lost	A shard of one day's index unavailable	Replicas on the hot tier serve queries; the shard is rebuilt from the buffer while it is still retained
TSDB node lost	Gap in dashboards	Replicas; and because metrics are statistics, a brief gap degrades a chart without lying to an alert — for durations span it
Alerting engine down	The silent killer	It runs in an independent failure domain (separate infra, separate credentials), and a meta-monitor outside the platform watches its heartbeat — Section 4.17
Poison record crashes a parser	One consumer stuck in a crash loop	Dead-letter queue: N failed attempts → the record is parked aside, processing continues, an operator is notified
Storage budget blown	Hot tier at capacity	Enforced sampling on the noisiest tenants (never silent gaps — Section 4.2), accelerated warm/cold migration, quota renegotiation

Three rows deserve slow reading. The **index node lost** row hides an elegant property: the shard is rebuilt **from the buffer** — meaning the durable log is not only a shock absorber but a **backup** of everything still within its retention window. One mechanism, two jobs; this is why the buffer keeps winning every design argument. The **TSDB node lost** row shows the metrics-are-statistics permission slip in action: a gap in a dashboard is a cosmetic wound, and the `for` duration you designed in Section 4.12 — built to resist noisy metrics — turns out to also resist **missing** ones, because a brief gap cannot satisfy a continuity requirement. Designs that degrade gracefully usually do so because some earlier

decision is quietly paying a second dividend. And the **alerting engine down** row is labeled “the silent killer” deliberately: every other failure in this table announces itself in telemetry somewhere, but a dead alerting engine’s symptom is **the absence of pages**, which is indistinguishable from health — unless something outside the platform is watching, which is exactly the meta-observability problem Section 4.17 formalizes.

COMMON PITFALL — Monitoring the monitor

The most dangerous failure of an observability platform is **looking healthy while being blind**: dashboards render cached panels, no alerts fire, and meanwhile ingestion silently stopped. Every layer must therefore emit telemetry about itself into an **independent** pipeline — if the platform’s own metrics flow through the platform, its failure erases its own death certificate. Section 4.17 formalizes this as meta-observability. If you take one sentence from this pitfall into your career: a monitoring system that can die quietly trains everyone to trust a corpse.

4.16 Trade-offs & Alternatives

The receipt table, as always: what you bought, what you paid, and — this chapter’s twist — what you **rejected**, stated well enough that a skeptic could resurrect the debate:

Decision	Chosen	Alternative & why not (here)
Metrics collection	Pull (scrape) for services + push gateway for batch jobs	Pure push: loses scrape-time up/down detection and centralized scheduling; pure pull: can’t catch short-lived jobs
Log write path	Index at write time (inverted index)	Schema-on-read / index-at-query (cheap writes, brutal query scans) — fine for archive-only data; our FR-2 demands interactive search
Log retention	Hot/warm/cold tiers with time-sharded indices	Uniform hot storage: correct but 10× the cost for data that is queried <1% of the time
Metric resolution	Fixed 15 s + rollup tiers	High-res forever: cardinality × retention makes storage explode; variable/adaptive resolution: complex, inconsistent dashboards
Alert condition	Threshold + for duration + clear hysteresis	Fire on every breach: flapping burns trust; anomaly detection everywhere: powerful for exploration , too opaque as the paging contract
Buffer	Durable log (Kafka-class) between agents and processors	Direct agent→indexer RPC: no spike absorption, no replay, couples every producer to every consumer’s outages

Decision	Chosen	Alternative & why not (here)
Degradation under overload	Shed metrics first, sample logs, never silently	Hard block (429 to agents): telemetry backs up into application hosts — the observer becomes the outage

The last row is the one to be able to defend cold, because it inverts a reflex. “Why not just backpressure the agents with 429s, like any overloaded service?” Because the agent’s backpressure propagates into **application hosts** — disk-full from unsent logs, blocked scrapes, poisoned deploys — and suddenly the observability platform is the **cause** of the incident it exists to observe. Shedding your own least-valuable data (metrics samples first, then sampled logs) keeps the pressure inside a system that was designed to shed. Chapter 3’s limiter rejected **callers** to protect a **service**; this platform sheds **its own data** to protect **the callers’ services**. Same mathematics, opposite direction — knowing which direction applies to you is the engineering.

4.17 SLOs & Meta-Observability

The platform that measures everyone else’s SLOs needs its own — measured from **outside**, by a small independent “meta-monitor” in a separate failure domain:

Indicator	Definition	Target
Ingestion availability	Share of batches accepted by the buffer	$\geq 99.9\%$
Log searchability delay	Event timestamp $\rightarrow$ present in index, p95	≤ 60 s
Metric freshness	Scrape timestamp $\rightarrow$ queryable, p95	≤ 30 s
Query latency (hot)	Log search / dashboard render, p95	≤ 5 s / ≤ 3 s
Alert latency	Condition true $\rightarrow$ notification sent, p95	≤ 1 min + for
Data loss	Acked bytes later found missing, per month	≈ 0 (buffer replay)
Pipeline health coverage	Hosts whose agent heartbeat is watched	100%

Look at how each indicator closes a loop with an earlier promise. “Log searchability delay” is Section 4.2’s 60-second freshness negotiation, rendered measurable. “Alert latency” is careful enough to write ≤ 1 min + for — the SLO **excludes** the deliberate waiting room, because paging faster than the for duration would be a bug, not a success; an SLO that punishes correct behavior will be gamed or ignored, and both outcomes rot the culture. And “data loss ≈ 0 ” is scoped with legal precision — **acked** bytes found missing — because bytes never acknowledged are the buffer’s honest rejects, not its broken promises.

INTERVIEW TIP — State the SLO of the telemetry system in the interview

Candidates design observability for everyone else and forget the platform itself. Naming meta-observability — “who watches the watcher, and from which failure domain” — is a senior-level signal: it shows you have been paged by the monitoring system being down, not just by the systems it monitors. One sentence suffices: “and the platform’s own health is watched by a meta-monitor in a separate failure domain, because a watcher that shares your fate cannot report it.”

4.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **“Add distributed tracing — what changes?”** A third pillar with its own shape: traces are **trees** (spans with parent ids), stored per-trace-id with heavy head-based or tail-based sampling (1–5% head, or tail-sampling that keeps errors and slow outliers). Trace ids are **correlation keys**: log records carry `trace_id` so a log search pivots to the trace, and exemplars link metric spikes to example traces. The platform’s buffer and tiering lessons transfer directly. (This is the payoff of Section 4.2’s “don’t preclude it”: the fork architecture already has a slot for a third consumer of the buffer, and that sentence is the whole answer.)
2. **“How do you cut the log bill in half?”** Rank tenants by bytes; enforce structured logging; sample repetitive INFO lines at the agent (log 1-in-N, keep all WARN+); shorten hot retention for the noisiest services; negotiate quotas. The answer is organizational as much as technical — the top three noisiest services are usually 40% of the bill, and no compression scheme beats a conversation with the team logging every health check at INFO.
3. **“PII ends up in logs — now what?”** Scrubbing is cheapest at the agent (pattern rules before anything leaves the host), enforced again at the indexer; access to raw logs is itself logged; cold-tier encryption keys rotate. Treat telemetry as production data with a blast radius — a log archive is a copy of every secret your services ever touched, indexed for convenient search.
4. **“A team creates a label with user ids in it — what breaks?”** Cardinality explosion (Section 4.9): the TSDB’s series index balloons, queries slow globally, and you are now storing identifiers you may not be allowed to store. Guardrails: reject writes whose label sets exceed per-metric cardinality budgets, and alert **the platform team** on the growth itself.
5. **“Why not just use a vendor solution?”** Buy-versus-build: the architecture above is exactly what the vendors run internally; the interview question is whether you can reason about their trade-offs. At 170 TB/day the build case is real; at 170 GB/day it usually is not.

4.19 Summary & Further Reading

NOTE — Chapter summary

A logging-and-metrics platform is **one firehose, two shapes of data**. Telemetry agents collect from every host (pull for services, push for batch jobs); a durable buffer absorbs the incident storm and decouples ingestion from processing; then the data forks: logs into a time-sharded **inverted index** (tokenization, posting lists, immutable segments, hot/warm/cold tiers that make retention a file deletion), metrics into a **time-series store** (delta-of-delta compression, label cardinality as the nemesis, mergeable rollups). Alerting is a per-rule state machine — OK → PENDING → FIRING — that pages on transitions, never on repetitions, and runs in its own failure domain. The platform watches itself from outside. The sizing insight: design for everyone else’s worst day, because that is precisely when this system becomes the most important one in the building.

Further reading.

- The source video: “14: Distributed Logging & Metrics Framework — Systems Design Interview Questions With Ex-Google SWE” (Jordan has no life): https://www.youtube.com/watch?v=p_q-n09B8KA
- Pelkonen, Franklin, Teller, Cavallaro, Huang, Meza, Veeraraghavan — “Gorilla: A Fast, Scalable, In-Memory Time Series Database” (VLDB 2015) — the delta-of-delta / XOR compression scheme Section 4.9 distills.
- The Prometheus documentation — the pull model, the label model, and the `for` / alerting-rule semantics this chapter borrows.
- McCandless, Hatcher, Gospodnetić — *Lucene in Action* — inverted indices, segments, and merges in production depth.

- *Google SRE Book*, chapters “Monitoring Distributed Systems” and “Alerting on SLOs” — symptoms vs. causes, and the philosophy behind Section 4.12.

4.20 Chapter Glossary

Every term this chapter defined, collected for review. If a row feels unfamiliar, the section that defines it is worth a re-read before the next chapter — these words recur throughout the rest of the book.

Term	Meaning
Alert fatigue	The learned ignoring of alerts caused by excessive false or repeated pages; a system failure mode, engineered against with for durations and hysteresis
Backpressure	Propagation of slowness upstream through a pipeline, absorbed by queues, throttling, or deliberate shedding
Cardinality	The number of distinct time series a metric’s label combinations produce; the metric platform’s primary scaling constraint
Cold / warm / hot tier	Storage classes of decreasing cost and speed holding progressively older telemetry; retention is implemented by migrating and deleting time-sharded indices
Dead-letter queue (DLQ)	A side channel where records that repeatedly fail processing are parked so the pipeline proceeds
Delta-of-delta encoding	Compressing a timestamp series by storing the difference of consecutive deltas — zero for a steady scrape, near-zero with jitter
Downsampling / rollup	Replacing old high-resolution samples with precomputed mergeable aggregates (sum/min/max/count) per bucket
Failure domain	The set of components that can fail together; the alerting engine and meta-monitor must live outside the domain they watch
Flapping	An alert oscillating between firing and clearing as a metric jitters around its threshold
Hysteresis	Using different thresholds to enter and leave a state, preventing oscillation at a single boundary
Inverted index	Map from token to the sorted list of records containing it; the data structure behind text search
Label	A key=value dimension on a metric series; dimensions, never identifiers
Meta-observability	Telemetry about the telemetry platform itself, emitted into an independent pipeline so the platform’s death cannot erase its own certificate
Observability / telemetry	The practice (and data) of inferring a system’s internal state from its outputs; pillars: logs, metrics, traces
Posting list	The sorted list of record ids attached to one token in an inverted index; queries intersect posting lists

Term	Meaning
Pull (scrape) model	Metrics collection where the collector periodically requests samples from each target's endpoint, gaining up/down detection for free
Push model	Metrics collection where sources send samples upstream; suits short-lived jobs and outbound trust boundaries
Sample	One (timestamp, value) pair of a time series
Sampling (logs)	Keeping a deterministic 1-in-N subset of a repetitive log class; degrades volume without creating silent gaps
Schema-on-read	Indexing little at write time and interpreting data at query time; cheap writes, expensive queries
Segment (index)	An immutable shard of an inverted index; small segments are merged in the background
Structured logging	Emitting logs as fielded records (JSON) rather than free text, enabling field queries and smart sampling
Telemetry agent	The per-host daemon that tails logs, collects metrics, batches, buffers, and ships telemetry upstream
Time series	A named, labeled sequence of timestamped values; the atom of the metrics pillar
Tokenization	Splitting text into searchable terms at index time
Zigzag encoding	A signed-to-unsigned integer mapping that makes small magnitudes of either sign varint-cheap

— End of Chapter 4 · Next: Chapter 5, Designing a Comments System —

Designing a Hierarchical Comment System

NOTE — Problem source

This chapter solves the problem posed in the talk “15: Reddit Comments” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the commenting system of a Reddit-scale discussion site — nested reply threads, up/down voting, and the several ways users can sort a thread. The interesting parts are the **data shape** (a forest of unbounded trees) and the **ranking mathematics** (votes plus time decay plus statistical confidence), not raw storage volume — a deliberate contrast with Chapter 4’s firehose. All terms are defined before use; all reference code is Rust with deterministic tests.

5.1 The Problem Statement

The interviewer draws a post with a familiar indented conversation underneath it and says:

“Users comment on posts and reply to each other, arbitrarily deep. They upvote and downvote comments. A thread can be sorted by ‘best’, ‘top’, ‘new’, or ‘controversial’. Popular posts get tens of thousands of comments, so users page through them and expand subtrees on demand. Design the system that stores, ranks, and serves these comment threads.”

After Chapter 4’s infrastructure marathon, this prompt feels pleasantly concrete — you have used this product; you can picture the indentation lines. Do not let the familiarity sedate you. The previous chapters designed systems whose value was **throughput** or **correctness**; this one’s entire value is **ordering**. Think about what that means: the same fifty thousand comments, in one order, are a delightful conversation; in another order, an unreadable swamp. The product is the permutation. Two hard problems hide inside that observation, and the chapter is organized around them — representing arbitrarily deep trees so that subtrees can be fetched and paged cheaply, and ranking votes in a way that is **statistically honest** and **time-aware** while thousands of votes per second stream in. Notice what is **not** on that list: capacity. Storage here is trivial by Chapter 4’s standards, which should make you suspicious that the interview is really about something else — it is.

DEFINITION — Comment thread / tree

The replies under one post form a *tree*: the post is the root context, each comment has exactly one parent (the post or another comment), and replies fan out to arbitrary depth and breadth. The set of trees under all posts is a *forest*. “The thread” usually means one post’s whole tree, or a rendered window into it.

DEFINITION — Score / vote tally

A comment's *score* is $\text{upvotes} - \text{downvotes}$, the single number users see. Votes serve two masters: they produce the visible score, and they feed the **ranking functions** (Section 5.8) that decide display order — and those two consumers want different things from the same data, which is where the design gets interesting.

Pause on that tension, because it is the seed of the chapter's best interview moment. The **score** wants to be simple — one integer a user can watch tick upward. The **ranking** wants to be honest — and honesty, it turns out, requires statistics, because a comment with one upvote and a comment with two hundred upvotes and ten downvotes are not the same kind of object no matter how similar their scores might get. Section 5.8 is the machine that reconciles the two: the score stays simple for humans while the ranking gets to be smart for the product.

5.2 Scope & Clarifying Questions

The prompt says “arbitrarily deep” and “tens of thousands” — two phrases that will dominate the design — but leaves almost everything else open. Run the negotiation; the table below compresses a longer conversation into its load-bearing answers:

Candidate asks	Interviewer answers
How deep can nesting go?	Unbounded in principle; the UI collapses past 10 levels with a “continue this thread” link
Edit and delete?	Edits allowed briefly (mark “edited”); deletes are soft — the comment becomes a tombstone but its replies survive
Are votes final?	No — users can change or retract votes; one vote per user per comment, enforced
Real-time?	New comments should appear within seconds on refresh; no live push needed. Counts can lag a little
Moderation?	Assume a separate moderation pipeline flags/removes; we must honor removals fast. Don't design moderation itself
Sort orders?	best, top, new, controversial — “best” is the default and the tricky one
Scale?	Reddit-scale: 70M daily users, viral posts with 10^5 comments
Anonymous reading?	Yes — most reads are logged-out; that population is cache-friendly
Media in comments?	Text plus links only; media uploads are the Maps chapter's CDN problem, already solved

Read the answers as a list of permissions, because several of them are quietly generous, and noticing **permissions** is as important as noting constraints. “No live push needed” frees you from WebSocket fleets and subscription fanout — the whole real-time machinery of Chapter 1 stays on the shelf. “Counts can lag a little” legalizes asynchronous counters, which Section 5.14 will cash in to keep comment writes off a hot post row. “Most reads are logged-out” is the single most valuable sentence in the table for your infrastructure bill: logged-out readers all see the **same** page, and identical pages are what caches eat for breakfast. And “deletes are soft, replies survive” is not a detail at all — it is the constraint that forces the tombstone design of Section 5.7, because you can no longer just

DELETE

FROM comments without orphaning a subtree of conversation other people are still having.

NOTE — Agreed scope

1. **Post and reply** to arbitrary depth; soft-delete with surviving replies.
2. **Vote** up/down/retract; exactly one vote per user per comment.
3. **Sort** a thread by best / top / new / controversial, with sound mathematics behind each.
4. **Page** through large threads: windowed top-level listing, expandable subtrees, collapsed deep chains.
5. **Counts**: post-level comment counts and comment scores may be seconds stale, never lost.
6. **Out**: live push updates, moderation tooling, rich media, recommendations.

5.3 Functional Requirements

One definition first, because the four sort orders are the product's whole personality and you need their precise meanings before you can promise to build them:

DEFINITION — Ranking mode

One of the supported orderings of a thread's comments. *Top*: highest score. *New*: most recent first. *Best*: highest **statistical confidence** of being liked (Section 5.8's Wilson score — not the same as top!). *Controversial*: most balanced **and** voluminous disagreement. Each mode is a different pure function of the same vote data — an important honesty property.

That last sentence deserves emphasis before the requirements list: four orderings, **one** dataset. There is no “best” flag in the database, no separate controversial tally — every mode is a deterministic function of `(up, down, created_utc)`. This is what makes the modes **consistent with each other** (a comment cannot be simultaneously excellent and terrible depending on which tab you view it through — only differently **sorted**), and it is what makes the write path tractable, because one vote update feeds all four rankings at once.

1. **FR-1 — Create**. Post a top-level comment on a post, or a reply to any existing comment, recording parent, author, timestamp, and body.
2. **FR-2 — Vote**. Cast, change, or retract one up/down vote per user per comment; the score reflects the change within seconds.
3. **FR-3 — Read a thread window**. Given a post, a ranking mode, and a cursor, return the next page of the ranked tree — top-level comments each with the first few levels of their best children.
4. **FR-4 — Expand a subtree**. Given any comment, return the next window of its children (“more replies”, “continue this thread”).
5. **FR-5 — Soft-delete and removal**. Deleted comments render as a tombstone that preserves thread structure; moderator removals disappear entirely, quickly.
6. **FR-6 — Post-level counters**. Comment counts per post for listing pages, eventually consistent within seconds.

Read FR-3 and FR-4 together and notice what they **don't** say: nowhere does anyone ask for “the thread.” The product surface is a sequence of **windows** — a page of top-level comments, a burst of children, an expansion on demand — and that phrasing, which the scope dialogue's depth-collapse answer hinted at, is doing enormous work. A 50,000-comment thread is not a document to be transferred; it is a territory to be **explored** through a viewport. Every systems consequence you care about — cursor pagination, shallow children, collapse tokens — descends from reading the FRs as window-shaped rather than tree-shaped. Section 5.12's tip box will name this principle (“window, don't tree”); watch for it forming here.

5.4 Non-Functional Requirements

One definition before the table, because it names the single number that shapes this architecture more than any latency target:

DEFINITION — Read-to-write ratio

The proportion of read requests to write requests a system serves. It dictates architecture: high ratios justify aggressive caching and denormalization (precomputing for reads at write time), because every write is amortized across thousands of reads. Comment systems are extreme — Section 5.5 derives roughly 35:1 on requests, and far higher on bytes.

Hold the logic of that definition clearly, because it is an argument template you will reuse for the rest of your career: **the ratio licenses the asymmetry**. If reads outnumber writes 35:1, then any work you move from the read side to the write side is work done once instead of thirty-five times — precomputed rank keys, pre-rendered windows, denormalized tallies. You are not “over-engineering the write path”; you are investing where the return is 35×. Watch for this pattern: Section 5.8 stores ranking as a column, and Section 5.12 caches rendered windows, and both are the ratio argument wearing different clothes.

Requirement	Target & reasoning
Thread read latency	p95 $\leq$ 200 ms for a cached window, $\leq$ 700 ms uncached; this is the product’s main surface
Write latency	p95 $\leq$ 300 ms for comment and vote writes; users tolerate a beat before their own comment appears
Availability	Reads $\geq$ 99.99% (stale cache is acceptable degradation); writes $\geq$ 99.9%
Durability	A confirmed comment or vote is never lost — it is social content, not telemetry (contrast Chapter 4’s metrics)
Vote integrity	One-vote-per-user enforced exactly; tallies reconcile to the true vote log eventually
Consistency	Read-your-own-write for the author; seconds of staleness acceptable for everyone else
Ranking freshness	A vote visibly affects ordering within 1 minute on hot threads; scores tick visibly on refresh
Cost	Storage is cheap here (11 TB/year); spend engineering on read shape and ranking, not capacity

Compare the durability row with Chapter 4 and say the contrast out loud in the interview — it proves the NFRs are **derived**, not recited. Chapter 4’s metrics could shed samples under load because statistics tolerate gaps; a comment is someone’s words, and losing it is a small but permanent betrayal of a human who trusted the text box. Same industry, opposite answer, because the **data’s relationship to its owner** differs. Also savor the availability row’s built-in degradation plan: “stale cache is acceptable” means a read outage and a write outage have different blast radii — if the write path dies, readers happily consume yesterday’s rankings for hours; if the read path dies, the product is simply down. Protect the read path first.

KEY INSIGHT — This is a shape problem, not a size problem

Chapter 4 drowned in bytes (170 TB/day); this chapter stores 30 GB/day — three orders of magnitude less. The difficulty is that the data is a forest of unbounded trees that must be **ranked four ways, paged stably, and re-ranked continuously** as votes arrive. Interviewers use this problem to test data modeling and algorithmic reasoning, not capacity arithmetic. Recalibrate your instincts accordingly:

the wins here come from schema and statistics, and every minute you spend on sharding arithmetic past Section 5.5 is a minute not spent where the interview is actually scored.

5.5 Back-of-the-Envelope Estimation

The discipline continues even in a shape problem — because “shape, not size” is a conclusion you have to **earn** with numbers before you are allowed to believe it.

Assumptions (stated, then derived from): 70M daily active users; 20% read comment threads, averaging 10 thread views; 1 in 35 thread readers writes a comment; every comment author and 10 lurkers vote on comments per reader; average comment body 500 bytes.

Quantity	Estimate	Derivation
Thread views	700M/day $\approx$ 8k/s avg, 40k/s peak	14M readers $\times$ 10 views; peak $\times$ 5 for a viral moment
Comments written	20M/day $\approx$ 230/s avg, 1.2k/s peak	20M posts’ worth of discussion; 1-in-35 readers comment
Votes cast	200M/day $\approx$ 2.3k/s avg, 12k/s peak	10 votes per active voter; the write-heavy path
Comment storage	30 GB/day $\rightarrow$ 11 TB/year	20M $\times$ 1.5 KB with ids, indexes, overhead
Vote records	40 GB/day raw, compacted small	200M $\times$ 50 B (user, comment, value, time); idempotency needs them
Hot cache	50 GB	Top 1M threads’ first ranked windows $\times$ 50 KB
Read bandwidth	800 MB/s at avg load	8k views/s $\times$ 100 KB rendered window

Three rows earn their keep immediately. The **comments written** and **votes cast** rows, read side by side, deliver the ratio insight: 230 comments/s versus 2,300 votes/s — the vote path is ten times hotter than the comment path, so Section 5.11 will spend its entire engineering budget there. The **hot cache** row says the entire working set of rendered first-pages fits in fifty gigabytes — one modest cache tier stands between a viral post and your database. And the **comment storage** row confirms the “shape not size” verdict: 11 TB a year is a rounding error against Chapter 4’s 170 TB a **day**; nobody needs exotic storage here, they need correct modeling.

KEY INSIGHT — Votes, not comments, are the write storm

Comments arrive at 230/s; **votes** at 2.3k/s average and 12k/s peak — and each vote is a read-modify-write on **at least four** things: the voter’s idempotency record, the comment’s up/down tallies, the cached score, and eventually the ranked order. Naively, that is a hot-row update per vote on viral comments. The vote pipeline (Section 5.11) exists to make the busiest path in the system also the cheapest. Remember Chapter 3’s lesson while you read it: a read-modify-write under concurrency is where races breed, so the pipeline’s first job is making that operation indivisible.

The pipeline of the rest of the chapter: Section 5.6 isolates the two hard problems; 5.7 models the trees; 5.8 does the ranking mathematics; 5.9–5.11 design APIs, architecture, and the vote pipeline; 5.12 serves threads; 5.13 implements all of it in Rust; 5.14–5.20 scale, harden, and review.

5.6 The Core Challenge: Trees and Honest Ordering

Two problems carry this design, and neither is capacity. Naming them precisely — and noticing that they are **independent**, solvable one at a time — is the first senior move of the interview.

Problem one — represent a forest of unbounded trees so that windows of it are cheap to read. Every read asks a deceptively simple question: “give me top-level comments 41–60 by best, each with its first two levels of best children.” Sit with that sentence and count the hazards inside it: there is a **ranked page** over a filtered subset (top-level only), a **subtree expansion** per page item, and a **depth budget** — three different tree operations fused into one user-visible response. A relational row per comment answers that badly unless the schema is designed for it; Section 5.7 compares the three classical tree encodings and picks a hybrid.

Problem two — rank honestly. “Best” cannot mean “highest score”, and the fastest way to see why is to play the adversary against yourself: a comment at +2 (2 up, 0 down) sits above a comment at +190 (200 up, 10 down) under raw-score ordering. Is that right? Your product sense says no — the second comment has survived two hundred independent judgments with a 95% approval rate; the first has survived two. But flip to the other naive answer, sorting by **ratio** of upvotes, and it fails just as embarrassingly in the opposite direction: $1/1 = 100\%$ beats $200/210 \approx 95\%$, so a comment one person liked outranks a comment two hundred people mostly loved. Raw difference ignores sample size; raw ratio worships it. Ranking must be a statistical statement about votes, plus a time statement (freshness), and it must be recomputed continuously as votes stream in. Section 5.8 derives the three functions used in production practice.

COMMON PITFALL — Score is not rank

Sorting “best” by `up - down` is the most common wrong answer in this interview. It ranks a 1-up-0-down comment above a 200-up-10-down one (+1 vs +190 is fine, but +2 vs +190 is not — and +1, 0 down beats +190 never, yet **percentage** sorting fails oppositely: $1/1 = 100\%$ beats $200/210 = 95\%$). Both naive orders are statistically illiterate; the fix is a confidence bound (Section 5.8). Say this early in the interview — it is the question the interviewer is waiting for, and volunteering it reads as “has thought about ranking before” rather than “was caught by the standard trap.”

5.7 Data Model: Encoding Comment Trees

Problem one first: how do you store a tree in a database that thinks in rows? This is a classic question with three classical answers, and the interview expects you to know all three well enough to **reject** two of them with reasons. So learn them as a set:

DEFINITION — Adjacency list / materialized path / nested sets

The three classical ways to store trees in a relational store. *Adjacency list*: each row stores `parent_id` — trivial writes, but fetching a subtree needs recursive queries. *Materialized path*: each row stores its whole ancestor chain (`/a1/b7/c3`), so a subtree is one `WHERE path LIKE '/a1/b7/%'` prefix scan and depth is the path length — at the cost of longer keys and painful subtree **moves**. *Nested sets*: each row stores `lft / rgt` bounds from a depth-first numbering, making subtrees a range scan but every **insert** renumbers half the tree — catastrophic at comment write rates.

Build the intuition for each by asking the same question three ways: “given comment c42, find everything under it.” Under **adjacency list**, you fetch c42’s children, then each child’s children, and so on — one query per generation, the N+1 trap in tree clothing. Under **materialized path**, c42’s path is `/c1/c7/c42`, so its entire subtree is every row whose path starts with that prefix — one indexed range scan, done. Under **nested sets**, c42 carries two numbers such that every descendant’s numbers

fall strictly inside them — also one range scan. So the two modern contenders both read subtrees beautifully. The difference only appears on **writes**, and there it is brutal: inserting a reply under nested sets requires shifting the `rgt` boundary of every ancestor and renumbering potentially half the forest, while a materialized-path insert writes one row whose path is just the parent’s path plus its own id. Comments are write-heavy (230/s) and **append-only** — replies are never moved between parents in this product — so the encoding whose weakness is subtree moves gives up a weakness you never exercise.

Property	Adjacency list	Materialized path	Nested sets
Insert a reply	One row	One row (path = parent’s + own id)	One row + mass renumbering
Fetch a subtree	Recursive	One prefix/range scan	One range scan
Fetch top-level page	Indexed <code>parent_id IS NULL</code> scan	depth = 1 or path prefix	Range union
Depth of a comment	Unknown without walking	Path segment count	Stored or derived
Move a subtree	One update	Rewrite all descendant paths	Renumber the tree
Fit for comments	Good bones	Best fit — subtrees are read, rarely moved	Unacceptable writes

Read the last row as the verdict and the rows above it as the evidence. Then notice the hybrid hiding in plain sight: adjacency’s `parent_id` is nearly free to keep alongside a materialized path, and it buys you the operations paths are bad at — “who is my direct parent?” for reply validation, “list my siblings” for certain expansions — without walking strings. Bones from one encoding, muscle from another.

The chosen schema — adjacency bones, path muscle, denormalized tallies:

```

comments
  comment_id    bigint pk
  post_id       bigint      -- shard key (Section 5.14)
  parent_id     bigint null  -- adjacency: direct parent
  path          text        -- materialized: /c1/c7/c42 (segment per ancestor)
  depth         smallint    -- = segments in path; drives collapse rules
  author_id     bigint
  body          text        -- null when tombstoned
  created_utc   timestamp
  up / down     bigint      -- denormalized tallies (vote pipeline owns them)
  state         enum        -- live | tombstone | removed
  index (post_id, path)    -- serves every window query in Section 5.12

```

Study the index line until it feels inevitable: `(post_id, path)` means “within one post, ordered by path” — and since a path is the ancestor chain, ordering by path is a depth-first traversal order. One index gives you top-level pages (path has one segment — or filter on the stored `depth`), subtree expansion (path prefix), and DFS ordering for rendering, all from a single B-tree. The schema is not a list of columns; it is a promise about which questions will be cheap.

`votes` is a separate table keyed `(user_id, comment_id)` — the idempotency backbone of Section 5.11. A post’s `comment_count` lives denormalized on the `posts` row, maintained asynchronously (FR-6). Keep the separation straight: `comments.up/down` are **derived display state** (fast to read, rebuilt by the

pipeline), while `votes` is **ground truth** (the legal record of who voted what). Which table you trust in a dispute is a design decision; here the small, write-once-style table wins custody of truth, and the big, read-hot table is allowed to drift and be repaired. Section 5.11's reconciliation paragraph closes that loop.

DEFINITION — Tombstone (soft delete)

A deletion that erases content but keeps structure: `body` and `author` are blanked, `state` becomes `tombstone`, and the row **stays** so its replies still have a parent. Rendering shows “[deleted]”. Hard deletion would orphan entire subtrees; moderation removals (`removed`) are the exception — they may take the subtree with them per policy.

The scope dialogue forced this on you (“replies survive”), but it is worth seeing that the requirement is also **good product**: conversations are shared artifacts, and letting one author vaporize the context of other people's words would make every heated thread self-destruct at its most interesting moment. The tombstone is the schema's opinion that a conversation belongs to its participants, not to any single speaker.

5.8 Deep Dive: The Ranking Mathematics

Now problem two, and this section is the interview's centerpiece — the twenty minutes where you either recite rankings you have seen or derive rankings you understand. Three pure functions over the same two integers (`up` , `down`) plus time. Each answers a different question honestly, and learning to say **which question** each answers is worth more than the formulas.

5.8.1 “Top” and “New” — trivial orders

`top` sorts by score `up - down` descending (ties by age, newest first); `new` sorts by `created_utc` descending. Both are indexable directly. Their weakness is statistical, which is exactly why “best” exists: `top` treats +2 (two votes) and +190 (210 votes) as two points apart, and `new` ignores votes entirely. Neither is **wrong** — they are honest answers to questions nobody asked. Users did not ask “which comment has the largest arithmetic difference?”; they asked “which comments will I be glad I read?” Those are different questions, and the gap between them is where the next two functions live.

5.8.2 “Best” — the Wilson lower confidence bound

DEFINITION — Wilson score interval (lower bound)

Treat each comment's votes as $n = \text{up} + \text{down}$ Bernoulli trials estimating the **true probability** p that a random viewer upvotes it. A confidence interval around the observed ratio $\hat{p} = \text{up}/n$ narrows as n grows; the *lower bound* of the Wilson interval (at $z = 1.96$, $\approx 95\%$) is a pessimistic estimate of p that fairly compares comments with **different sample sizes**. Small-sample comments are penalized toward 0; as $n \rightarrow \infty$ the bound approaches $\hat{p}$. It needs only `up` and `down`, is monotonic in both, and costs a few flops — computable at vote time and stored.

Unpack the strategy, because the formula below is just the strategy frozen into arithmetic. You cannot know a comment's **true** quality p ; you only have the votes so far. So instead of pretending $\hat{p}$ is the truth (ratio ranking's sin) or discarding the ratio entirely (score ranking's sin), you ask: “given this much evidence, what is the **lowest** p still consistent with the data at 95% confidence?” Rank by that pessimistic bound and two beautiful behaviors emerge for free: new comments with few votes are penalized in proportion to their uncertainty (so 1/1 no longer beats 200/210 — the 1/1 bound is dragged toward zero by doubt), and as evidence accumulates the bound relaxes toward the observed ratio (so proven comments rise to exactly where their record deserves). You are not choosing between ratio and sample size; you are **charging each comment for its uncertainty**, and letting the charge evaporate as evidence arrives.

$$\text{wilson}(u, d) = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p}) + \frac{z^2}{4n}}{n}}}{1 + \frac{z^2}{n}}, \quad n = u + d, \hat{p} = u/n$$

You will never need to derive this at the whiteboard; you need to be able to **interrogate** it. Three numbers do that job better than any derivation: a comment with 1 up / 0 down scores ≈ 0.21 ; with 10 / 0, ≈ 0.72 ; with 100 / 0, ≈ 0.96 . Same 100% approval, three wildly different ranks — that is the uncertainty charge, visible. And 100 up / 100 down ($\hat{p} = 50\%$ but $n = 200$, ≈ 0.42) correctly beats 1 / 0 — volume of evidence matters more than a lone perfect record. Section 5.13 implements and tests exactly these numbers, so if your memory of the formula blurs in the interview, reconstruct the behavior table and describe the curve; it lands the same.

5.8.3 “Controversial” — balanced disagreement, weighted by volume

$$\text{controversial}(u, d) = \begin{cases} 0 & \text{if } \min(u, d) = 0 \\ (u + d)^{\min(u, d) / \max(u, d)} & \text{otherwise} \end{cases}$$

Read the formula as a two-factor test, because both factors are necessary and neither is sufficient alone. The exponent `min/max` measures **balance**: it is 1 only for a perfect 50/50 split and decays toward 0 as the split lopsides, so one-sided comments are exponentially demoted. The base `u + d` measures **volume**: among equally balanced comments, the one with more total votes ranks higher. A 50/50 split on 100 votes (`balance` = 1, magnitude 100) crushes a 105-vote comment with a 100/5 split (`balance` = 0.05 $\rightarrow$ magnitude ≈ 1.26): near-even splits dominate, and among equal splits the **louder** argument wins. Also notice the guard clause — zero on the weak side means zero controversy, because disagreement requires two camps, and a camp of zero is a monologue.

5.8.4 “Hot” — score with time decay (for posts and fast threads)

DEFINITION — Time-decay ranking (“hot”)

“hot” = $\log_{10} \max(|s|, 1) + \text{“sign”}(s) \cdot (t - t_0) / 45000$, with $s = \text{up} - \text{down}$, t the comment’s epoch seconds, and t_0 an arbitrary fixed epoch. The log makes the first 10 votes worth as much as the next 100 (diminishing returns); dividing age by 45,000 s (12.5 h) means a post must be **10× more popular** to tie one 12.5 hours newer. Fresh content cycles through the front page; score advantages decay gracefully instead of cliffing.

Both halves of this formula are product decisions disguised as math, and naming them as such is the interview answer. The `log10` term encodes **diminishing returns on popularity**: the jump from 1 to 10 votes moves the rank as much as 10 to 100, because early traction is the scarce signal that a submission is worth showing anyone. The linear age term encodes **front-page turnover**: every 12.5 hours costs one order of magnitude of score, so yesterday’s thousand-vote story must be ten times as beloved as this morning’s hundred-vote story to keep its slot — the front page renews itself on a schedule you can tune by changing one constant. Neither behavior is a law of nature; both are choices about what the product should feel like, frozen into arithmetic where they can be tested, tuned, and defended.

KEY INSIGHT — Rank is a stored, recomputed column — not a query-time sort

All four functions are pure over `(up, down, created_utc)`. So the vote pipeline recomputes a comment’s rank keys **on every vote** and stores them; reads are then index scans, never sorts. “Best” order changes only when votes change — and votes arrive at 2.3k/s, exactly the pipeline built in Section 5.11. Sorting 50k comments per page-view would be malpractice. Hold the general principle once you see it here: when a read asks for a **function** of data that changes far more slowly than reads arrive,

compute the function on write and store the answer. You will meet this principle again in Chapter 6's leaderboard and Chapter 7's recommendations — it may be the single most reusable move in this book.

5.9 API Design

Seven endpoints cover the whole product, and their shapes are all forced by decisions you have already made — read the table as a summary of the chapter so far rather than a new topic:

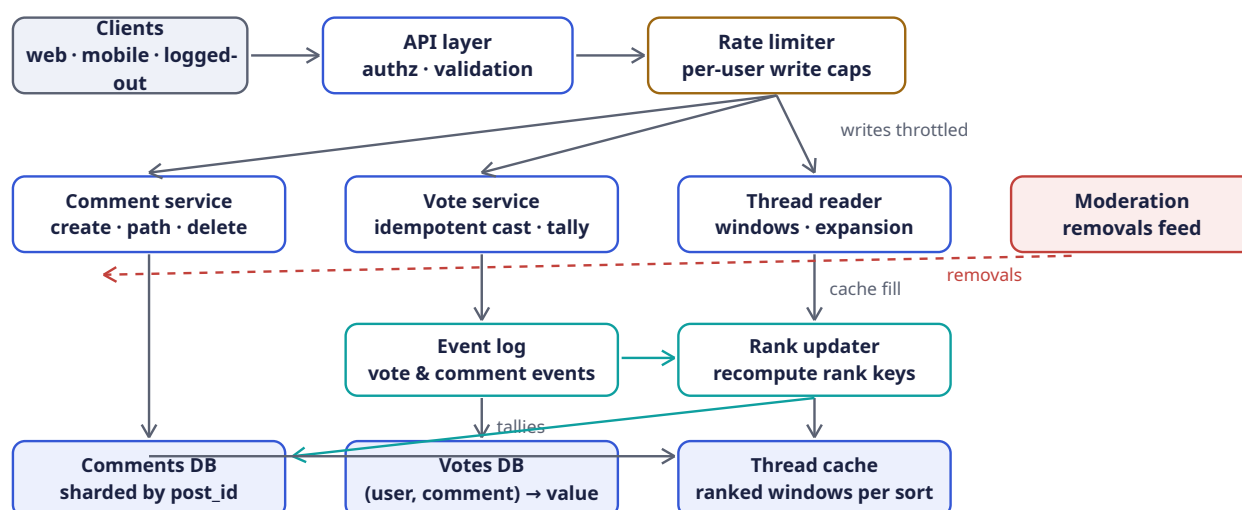
Method	Path	Purpose
POST	<code>/v1/posts/{id}/comments</code>	Top-level comment (FR-1); body, author
POST	<code>/v1/comments/{id}/replies</code>	Reply to a comment (FR-1); server computes path, depth
PUT	<code>/v1/comments/{id}/vote</code>	Cast/change/retract vote: {value: 1 -1 0} — idempotent (FR-2)
GET	<code>/v1/posts/{id}/comments?sort=&cursor=&limit=</code>	Ranked thread window with shallow children (FR-3)
GET	<code>/v1/comments/{id}/children?cursor=&limit=</code>	Expand a subtree window (FR-4)
DELETE	<code>/v1/comments/{id}</code>	Soft-delete → tombstone (FR-5); author-only
GET	<code>/v1/posts/{id}/comments/count</code>	Post-level counter (FR-6); cacheable, seconds-stale

Two deliberate choices deserve their own paragraphs, because each is a famous trap disarmed in advance:

- **Opaque cursors, never offsets.** A cursor encodes the last seen rank key and comment id (`base64(wilson, id)`); offsets break the moment a new comment inserts above the reader's position — Chapter 1's cursor lesson applied to ranked lists. Feel the failure mode concretely: a reader is on page 2 (rows 21–40 by offset) when a burst of votes promotes three new comments into the top 20; every offset shifts, and rows 21–40 now overlap page 1 — the reader sees duplicates and silently **skips** three comments they never knew existed. Cursor pagination cannot do this, because the cursor names a **position in the ordering** (“everything after (0.94, c55)”) rather than a **count of rows to skip**, and positions stay meaningful as the list churns. Ties are impossible: `(rank_key, comment_id)` is a total order.
- **Votes are PUT, not POST.** Repeating or flipping a vote is one idempotent operation on a `(user, comment)` resource, not an event stream the client can double-fire. Retries are free. This is FR-2's “exactly one vote per user” enforced **at the protocol level**: the HTTP method itself promises that sending the same request twice is safe, which licenses clients to retry aggressively on flaky mobile networks — exactly the networks your voters are on.

5.10 High-Level Architecture

The pieces you have been assembling — a write path that must be idempotent, a ranking pipeline that recomputes continuously, a read path that serves cached windows — now assemble into one picture. Draw it in three horizontal bands: the edge at top, services in the middle, storage at the bottom, with the ranking pipeline as the connective tissue between write and read:



Now the walkthrough — and this diagram repays a careful narration, because its real content is not the boxes but **which paths are fast and which are allowed to lag**.

Follow a **vote** first, since Section 5.5 told you it is the hottest write. The client (top-left) hits the API layer, which authenticates and validates; then the amber **rate limiter** — Chapter 3’s design, recycled without modification — enforces per-user write caps, with the “writes throttled” label reminding you this is the first anti-brigading wall. From there the request descends to the **vote service** in the middle band. The vote service does two synchronous things, and the diagram’s arrows show both: down to the **votes DB** (bottom-center-left) for the idempotent upsert that is the write’s legal record, and down to the **event log** (teal, middle-low) to emit a `VoteChanged` event. Then — this is the crucial beat — the request **returns**. The user is not waiting for rankings; they are waiting only for two durable writes. Everything else happens behind them.

Now follow the event, not the request. The teal arrow from event log to **rank updater** is the asynchronous handoff: the updater consumes vote events, coalesces bursts per comment, recomputes tallies and all four rank keys, and then fires the two arrows you see leaving it downward — one into the **comments DB** (writing the fresh tallies and rank keys back onto the comment rows, the “tallies” label marking where they land) and one long teal arrow sweeping left into the **thread cache**, version-stamping the affected post’s windows so the next read repopulates. Study the shape of this loop: votes enter synchronously, rankings leave asynchronously, and the event log between them is Chapter 4’s buffer lesson transplanted — spikes become lag, never loss.

Follow a **read** now, and notice how short its path is. The **thread reader** service (middle-right) checks the **thread cache** for the requested `(post_id, sort, cursor)` window — the “cache fill” arrow runs from the reader down past the cache — and on a hit, which is nearly always, the response is already-rendered JSON. On a miss, the long slate arrow from the cache area back to the **comments DB** shows the fallback: run Section 5.12’s two window queries against the shard that owns the post, render, cache, serve. The read path never touches the votes DB, never computes a Wilson score, never sorts anything — Section 5.8’s “rank is a stored column” insight made visible as the **absence** of arrows.

Two more elements complete the tour. The crimson **moderation** box at the right edge fires a dashed “removals” arrow across the whole diagram into the comment service: removals arrive on their own pipeline (designed by another team, per the scope) and must propagate to tombstone/removed states and then to caches within seconds — hence the arrow’s path through, not around, the write side. And the comments DB label — “sharded by post_id” — encodes Section 5.14’s key decision in the picture: one post’s entire tree lives on one shard, so every window query you saw in the read path stays single-shard by construction.

Component	Responsibility	Why it is shaped this way
API layer	Auth, validation, routing	Stateless; scales horizontally behind a load balancer
Rate limiter	Per-user caps on comment & vote writes	Chapter 3 verbatim: comments/hour, votes/minute — the first anti-brigading wall
Comment service	Create replies (computing path / depth), edits, tombstones	Owns the tree invariants; the only writer of comments rows
Vote service	Idempotent cast/change/retract; owns votes table	One writer per vote record; the idempotency enforcer (Section 5.11)
Event log	Durable stream of vote & comment events	Decouples tally/rank recomputation from the write path — Chapter 4's buffer lesson
Rank updater	Recompute tallies, Wilson, hot, controversial; write rank keys; bump post counters	Turns 2.3k votes/s into batched, coalesced rank updates
Comments DB	Tree rows, sharded by post_id	One post's whole thread lives on one shard — window queries stay single-shard
Votes DB	(user, comment) → value	Sharded by comment_id; the ground truth for reconciliation
Thread cache	Pre-rendered ranked windows per post × sort	35:1 read ratio + logged-out majority = the tier that actually serves traffic
Moderation feed	Removals pushed to comment service	Honored in seconds (agreed scope); separate pipeline, not designed here

Two rows articulate single-writer disciplines that keep the whole system sane. The comment service is **the only writer of comments rows** — so tree invariants (path equals parent's path plus id, depth equals segment count) have exactly one enforcer, and corruption has exactly one place to look. The vote service is the one writer of votes records — so idempotency rules live in one codebase, not in every client. Distributed systems rot when invariants are maintained by convention across many writers; they stay healthy when each invariant has a single gatekeeper. This diagram has two, and both are deliberate.

5.11 Deep Dive: The Vote Pipeline

The busiest path in the system deserves the most engineering — Section 5.5 proved the votes outnumber comments ten to one, so this is where the chapter spends its care. Requirements, restated as a checklist you can design against: one vote per user (exactly), changeable, never lost, score visible in seconds, rank keys refreshed continuously. The first requirement sounds like a business rule; it is actually a **concurrency** problem, because two simultaneous votes from the same user (double-tap on a laggy phone, retried request after a timeout) must not count twice.

DEFINITION — Idempotent write / natural idempotency key

A write that can be applied any number of times with the same effect. Retries, double-clicks, and client bugs make **at-least-once** delivery the honest assumption, so the handler must deduplicate. Here the key is free: `(user_id, comment_id)` is the natural primary key of a vote — one row per pair; upserted, is the deduplication mechanism itself. No tokens, no expiry windows (contrast Chapter 3’s idempotency keys on payment APIs).

The phrase “the key is free” is the insight to carry away. Payment APIs need client-generated idempotency tokens because “charge this card” has no natural deduplication identity — two identical charges might both be legitimate. A vote is different: its **meaning** already includes its uniqueness constraint (“user 7’s vote on comment 42” is one thing, not a count of things). When your write has a natural idempotency key, the database’s unique constraint becomes your entire deduplication infrastructure. Train yourself to look for natural keys before reaching for token machinery; the best idempotency mechanism is the one that was always there.

The write path for `PUT /v1/comments/c42/vote {value: -1}` :

1. **Rate-limit** the user (Chapter 3: sliding window, votes/minute).
2. **Upsert** `votes[(u, c42)] = -1` in one statement returning the previous value — the atomic read-modify-write that Chapter 3’s TOCTOU section warns about; the unique key makes it safe:

```
-- one atomic statement; all clauses share one snapshot, so `old`
-- reads the pre-upsert value
WITH old AS (
  SELECT value FROM votes WHERE user_id = $1 AND comment_id = $2
)
INSERT INTO votes (user_id, comment_id, value, updated_utc)
VALUES ($1, $2, $3, now())
ON CONFLICT (user_id, comment_id)
DO UPDATE SET value = $3, updated_utc = now()
RETURNING (SELECT value FROM old) AS prev_value;
```

3. **Emit** a `VoteChanged{user, comment, prev, new}` event to the log. The request returns here — the write path is **two** durable operations.
4. **Rank updater** consumes events, **coalesces** bursts per comment (200 votes in a second become one recompute), recomputes tallies `up += (new==1) - (prev==1)`, `down += (new==-1) - (prev==-1)`, re-derives Wilson/hot/controversial, writes them back, and **version-stamps** the thread cache so the next read repopulates.

Walk the SQL once, because it is doing something subtle enough to deserve your respect. A naive vote update is two statements — “read the old vote, then write the new one” — which is precisely the check-then-act pattern Chapter 3’s TOCTOU diagram destroyed: two concurrent flips by the same user could read the same old value and both mis-compute the tally delta. The statement above fuses read and write into one atomic upsert whose `RETURNING` clause hands back the **pre-update** value, so the caller learns “this user’s previous vote was X, and now it is Y” as a single indivisible fact. The tally delta the rank updater needs — `(new==1) - (prev==1)` — is computable from that fact alone, which is why the event emitted in step 3 carries both `prev` and `new`: the event is the delta, serialized.

KEY INSIGHT — Coalescing is the whole trick

A viral comment can take thousands of votes per minute, but its **rank key** only needs recomputing a few times a minute — readers cannot perceive faster change, and sorting is stable between recomputes. Treating votes as a stream to be compacted (latest-wins per user; batch-sum per comment) turns the hottest write path into a trickle of rank updates. The score users see may lag the truth by seconds — the agreed scope allows exactly that, and spends the savings on reads. Notice the pattern, because it is the

second time this book has used it: Chapter 4’s buffer absorbed spikes in **bytes**; this coalescer absorbs spikes in **recomputation**. Durable log plus batching consumer is the universal shock absorber.

Reconciliation. Tallies are derived state; `votes` is ground truth. A nightly job (and a per-comment repair tool) recomputes tallies from `votes` and fixes drift — the same discipline as Chapter 4’s buffer replays: **derived numbers must be rebuildable from the event record**. Say the principle in the interview before anyone asks, because it preempts the sharp follow-up (“what if a tally and the vote log disagree?”) with a framework: you have already decided which store wins, how drift is detected, and how it heals. Systems that can answer “how would you know you’re wrong?” are the ones that survive contact with production.

5.12 Deep Dive: Reading a Thread Window

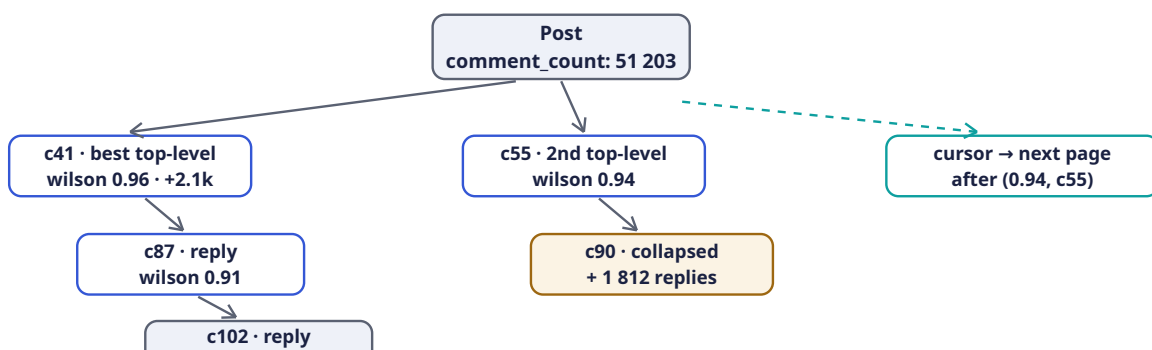
Everything so far — the hybrid schema, the stored rank keys, the vote pipeline — converges on the read path, and here you get to collect the dividends. A thread read assembles a **window** of the tree: a page of top-level comments, each carrying its first levels of best children, with everything past the depth cap collapsed behind an affordance. With the schema of Section 5.7 the whole window is two queries:

1. **Top-level page:** `WHERE post_id = $1 AND depth = 1 ORDER BY rank_key DESC, comment_id DESC LIMIT $2` with the cursor supplying the rank-key boundary — a single index range scan on `(post_id, rank)` per sort mode.
2. **Children fill:** for the ≤ 20 parents in the page, one batched prefix scan `WHERE post_id = $1 AND path ~ ANY ('/c41/%', '/c87/%', ...) AND depth <= 4 ORDER BY path` — materialized paths turn “first two levels of every parent on the page” into **one** query instead of N recursive ones (the N+1 that sinks the naive adjacency-list design).

Count the queries: two, constant, regardless of how many parents the page contains or how deep the forest runs. That is the materialized path earning its storage cost — the same operation under a pure adjacency list is one recursive walk **per parent**, and the interviewer has seen a hundred candidates discover this in production. Also note the `depth <= 4` clause working alongside the depth column: the collapse rule from the scope dialogue is enforced **in the query**, not in application code, so the database never reads a row the UI will never render.

Assembly is in memory: parents in rank order, children nested under them by path prefix, tombstones rendered as “[deleted]” placeholders, subtrees beyond depth 10 replaced by a `{continue_thread: comment_id}` token the client expands through `/children` (FR-4).

Here is the assembled window drawn as the user experiences it — one post, its best conversation, and all three of the design’s surgical instruments (ranking, tombstoning, collapse) visible at once:



Take the tour top to bottom. The post node at the top carries its denormalized `comment_count: 51 203` — FR-6’s asynchronous counter, good enough for display precisely because the scope said “seconds stale, never lost.” Two slate arrows descend from it to the page’s two top-level comments, and the left branch tells the design’s whole story in three boxes. First, **c41**, the best top-level comment: its label shows the stored rank key (wilson 0.96) and the visible score (+2.1k) — the two consumers of Section 5.1’s vote data, side by side, finally reconciled. Below and indented, its best reply **c87** (wilson 0.91) — the shallow-children fill, one level of conversation context included so the reader can feel the thread’s texture without loading it. Below that, **c102** rendered as “[deleted]”: the tombstone, still occupying its structural slot so that c102’s own replies (off-screen below) keep a parent to hang from. The right branch shows the other two instruments: **c55**, the second top-level comment, with its child **c90** collapsed into an amber affordance labeled “+ 1 812 replies” — the depth/breadth budget made visible, one box standing in for a subtree larger than most websites’ entire databases. And at the far right, the dashed teal arrow leads to the cursor box: `after (0.94, c55)` — the next page’s starting position encoded as a (rank key, id) pair, immune to the vote churn that would corrupt an offset.

Look at what the picture proves: a reader can scroll a 51,203-comment thread forever, and every page they ever request is two indexed queries against one shard, mostly served from cache. **That** is what all the mathematics bought.

Caching. The rendered window JSON is cached under `(post_id, sort, cursor)` for logged-out readers — the majority (Section 5.2) — and the rank updater **version-stamps** the post on every coalesced update, so stale windows age out within seconds without any invalidation fanout. Logged-in readers get the same window plus a per-user vote overlay

```
SELECT comment_id, value FROM
(votes WHERE user_id = $1 AND comment_id = ANY(...)) merged at read time — personalization over a shared cache, instead of a cache per user.
```

The overlay pattern deserves a pause, because it dissolves a dilemma that otherwise eats designs like this. Votes make every user’s view **personal** — you see your own upvotes highlighted — and naive personalization multiplies cache entries by user count, destroying the cache entirely. The overlay splits the response into the expensive shared part (the rendered window: bodies, structure, scores — identical for everyone) and the cheap personal part (your votes on the visible comments — a handful of rows fetched by a primary key lookup). Shared part cached once for the world; personal part fetched fresh per user. The general move — **separate the response into shared skeleton and personal overlay, cache only the skeleton** — applies to feeds, timelines, and dashboards everywhere, and interviewers recognize it as a mark of someone who has actually scaled a read path.

INTERVIEW TIP — Window, don’t tree

The API never returns “the thread” — only windows: ranked top-level pages, shallow children, explicit expansions. A 50k-comment thread is 50k rows the user will never scroll; serving the **shape** of the conversation (best subthreads first, depth capped) is both the product-correct and the systems-correct answer. Candidates who serialize whole trees lose the room.

5.13 Rust Reference Implementations

Four pieces with deterministic tests: the ranking trio, the tree model with windowing, the idempotent vote service, and cursor pagination. Each is the executable proof of a claim you have already made in prose — Section 5.8’s statistics, Section 5.7’s tree encoding, Section 5.11’s idempotency, and Section 5.9’s cursor stability — so read them as evidence, not as appendix.

5.13.1 The Ranking Functions


```

/// z = 1.96 -> ~95% confidence. The lower Wilson bound of the true
/// upvote probability, from (up, down) alone. Pessimistic for small n.
pub fn wilson_lower(up: u64, down: u64) -> f64 {
    let n = (up + down) as f64;
    if n == 0.0 { return 0.0; }
    let z: f64 = 1.96;
    let phat = up as f64 / n;
    let denom = 1.0 + z * z / n;
    let centre = phat + z * z / (2.0 * n);
    let margin = z * ((phat * (1.0 - phat) + z * z / (4.0 * n)) / n).sqrt();
    (centre - margin) / denom
}

/// Reddit's "hot": log score + age bonus. 45_000s = 12.5h; a comment must
/// be 10x more popular to tie one 12.5h newer. t0 is a fixed epoch.
pub fn hot(up: u64, down: u64, epoch_secs: i64) -> f64 {
    const T0: i64 = 1_134_028_003; // Reddit's original launch epoch
    let s = up as i64 - down as i64;
    let order = s.unsigned_abs().max(1) as f64;
    let order = order.log10();
    let sign = (s > 0) as i8 as f64 - (s < 0) as i8 as f64;
    order + sign * (epoch_secs - T0) as f64 / 45_000.0
}

/// Balanced disagreement weighted by volume: (u+d)^(min/max).
pub fn controversial(up: u64, down: u64) -> f64 {
    let (lo, hi) = (up.min(down), up.max(down));
    if lo == 0 { return 0.0; }
    ((up + down) as f64).powf(lo as f64 / hi as f64)
}

#[cfg(test)]
mod ranking_tests {
    use super::*;

    #[test]
    fn wilson_grows_with_sample_size_at_same_ratio() {
        // all 100% approval, increasing n: 0.21 -> 0.72 -> 0.96
        let (w1, w10, w100) =
            (wilson_lower(1, 0), wilson_lower(10, 0), wilson_lower(100, 0));
        assert!((w1 - 0.2065).abs() < 1e-3, "w1 = {w1}");
        assert!((w10 - 0.7225).abs() < 1e-3, "w10 = {w10}");
        assert!((w100 - 0.9630).abs() < 1e-3, "w100 = {w100}");
        assert!(w1 < w10 && w10 < w100);
    }

    #[test]
    fn wilson_prefers_proven_mediocre_to_lucky_perfect() {
        // 100/100 (phat 50%, n 200) beats 1/0: evidence over luck.
        let proven = wilson_lower(100, 100);
        assert!((proven - 0.4307).abs() < 1e-3, "{proven}");
        assert!(proven > wilson_lower(1, 0));
        assert!(wilson_lower(0, 0) == 0.0);
        assert!(wilson_lower(0, 5) < wilson_lower(1, 5));
    }

    #[test]
    fn hot_decay_needs_10x_score_per_12h30m() {
        let t = 1_700_000_000;
        let older = hot(100, 0, t);
    }
}

```



```

    let newer = hot(10, 0, t + 45_000);
    assert!((older - newer).abs() < 1e-9);
    assert!(hot(0, 10, t) < hot(0, 1, t)); // downvotes push below zero-side
}

#[test]
fn controversial_needs_both_balance_and_volume() {
    let even_split = controversial(50, 50); // 100^1.0
    let loud_blowout = controversial(100, 5); // 105^0.05
    assert!((even_split - 100.0).abs() < 1e-9);
    assert!((loud_blowout - 1.2619).abs() < 1e-3, "{loud_blowout}");
    assert!(even_split > loud_blowout);
    assert_eq!(controversial(0, 100), 0.0); // no disagreement
}
}

```

Notice how the tests encode **opinions**, not just arithmetic. The second Wilson test is named `wilson_prefers_proven_mediocre_to_lucky_perfect` — that name is the entire Section 5.8 argument compressed into an identifier, and the assertion `proven > wilson_lower(1, 0)` is the sentence “evidence beats luck” with a compiler behind it. The hot-ranking test asserts the 12.5-hour exchange rate **exactly** (equality within 1e-9): one order of magnitude of score buys precisely one decay period of age, no more, no less — if someone “tunes” the constant from 45,000 to 44,000, this test fails loudly instead of letting the front page quietly age faster. Tests like these are how product decisions survive refactors.

5.13.2 The Tree Model: Paths, Depth, Tombstones

```

use std::collections::HashMap;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum State { Live, Tombstone }

#[derive(Debug, Clone)]
pub struct Comment {
    pub id: u64,
    pub parent_id: Option<u64>,
    pub created_ms: u64,
    pub up: u64,
    pub down: u64,
    pub state: State,
    pub body: String,
}

impl Comment {
    pub fn score(&self) -> i64 { self.up as i64 - self.down as i64 }

    /// Materialized path segment: parent path + own id.
    pub fn path(&self, parent_path: &str) -> String {
        format!("{}/{}/{}", parent_path, self.id) // "" -> "/1" -> "/1/2" -> ...
    }

    pub fn render_body(&self) -> &str {
        match self.state {
            State::Live => &self.body,
            State::Tombstone => "[deleted]",
        }
    }
}
}

```

```

/// Depth-first flatten of a thread for rendering: parents before children,
/// siblings ranked by score (stand-in for "best" at read time). Tombstones
/// keep their position so replies are not orphaned.
pub fn flatten_thread(rows: &[Comment]) -> Vec<(usize, &Comment)> {
    let mut children: HashMap<Option<u64>, Vec<&Comment>> = HashMap::new();
    for c in rows {
        children.entry(c.parent_id).or_default().push(c);
    }
    for kids in children.values_mut() {
        kids.sort_by(|a, b| {
            b.score()
                .cmp(&a.score())
                .then(b.created_ms.cmp(&a.created_ms)) // newer wins ties
                .then(a.id.cmp(&b.id)) // total order: no ambiguity
        });
    }
    let mut out = Vec::with_capacity(rows.len());
    let mut stack: Vec<(usize, &Comment)> = Vec::new();
    if let Some(roots) = children.get(&None) {
        for c in roots.iter().rev() {
            stack.push((1, *c)); // top-level comments have depth 1
        }
    }
    while let Some((depth, c)) = stack.pop() {
        out.push((depth, c));
        if let Some(kids) = children.get(&Some(c.id)) {
            for k in kids.iter().rev() {
                stack.push((depth + 1, *k));
            }
        }
    }
    out
}

#[cfg(test)]
mod tree_tests {
    use super::*;

    fn c(id: u64, parent: Option<u64>, up: u64, state: State) -> Comment {
        Comment { id, parent_id: parent, created_ms: id * 1000,
            up, down: 0, state, body: format!("body {id}") }
    }

    #[test]
    fn window_is_dfs_with_ranked_siblings() {
        let rows = vec![
            c(1, None, 10, State::Live),
            c(2, Some(1), 5, State::Live),
            c(3, Some(1), 8, State::Live),
            c(4, Some(2), 1, State::Live),
            c(5, None, 20, State::Tombstone),
            c(6, Some(5), 3, State::Live),
        ];
        let flat = flatten_thread(&rows);
        let ids: Vec<u64> = flat.iter().map(|(_, c)| c.id).collect();
        // roots by score: 5 (20) then 1 (10); children of 1: 3 (8) then 2 (5)
        assert_eq!(ids, vec![5, 6, 1, 3, 2, 4]);
        // depth tracks nesting
        assert_eq!(flat.iter().find(|(_, c)| c.id == 4).unwrap().0, 3);
    }
}

```

```

    assert_eq!(flat.iter().find(|(_, c)| c.id == 1).unwrap().0, 1);
}

#[test]
fn tombstone_hides_body_but_keeps_subtree() {
    let rows = vec![c(5, None, 20, State::Tombstone),
                    c(6, Some(5), 3, State::Live)];
    let flat = flatten_thread(&rows);
    assert_eq!(flat[0].1.render_body(), "[deleted]");
    assert_eq!(flat[1].1.render_body(), "body 6");
}

#[test]
fn materialized_paths_compose() {
    let (c1, c2, c4) = (c(1, None, 0, State::Live),
                       c(2, Some(1), 0, State::Live),
                       c(4, Some(2), 0, State::Live));
    let p1 = c1.path("");
    let p2 = c2.path(&p1);
    assert_eq!(c4.path(&p2), "/1/2/4");
    assert_eq!(p2.matches('/').count(), 2); // segment count == depth
}
}

```

The `flatten_thread` function is worth one careful read because it answers a question every tree-rendering system faces: how do you turn a pile of rows into a depth-first **display order** without recursion blowing the stack on a 10^4 -deep chain? The answer here is an explicit `Vec` used as a stack — recursion converted to iteration, with children pushed in reverse so they pop in ranked order. The sibling sort has three keys, and the third (`.then(a.id.cmp(&b.id))`) is the one candidates forget: score ties broken by recency can **still** tie, and a sort without a total order is a sort whose output can differ between runs — which means cursors over it can skip or repeat rows. The comment says it: “total order: no ambiguity.” And the tombstone test locks in Section 5.7’s contract: the body is hidden, the child survives, the thread’s shape outlives any single comment’s content.

5.13.3 The Idempotent Vote Service

```

use std::collections::HashMap;

/// Natural idempotency key: (user, comment). One row per pair; upserts.
#[derive(Default)]
pub struct VoteService {
    votes: HashMap<(u64, u64), i8>, // value in {-1, 0, +1}; 0 == retracted
}

impl VoteService {
    /// Cast / change / retract a vote. Returns (delta_up, delta_down) for
    /// the rank updater to apply to the comment's tallies – the whole
    /// effect of the vote in two integers.
    pub fn cast(&mut self, user: u64, comment: u64, value: i8) -> (i64, i64) {
        assert!(value == -1 || value == 0 || value == 1);
        let prev = self.votes.insert((user, comment), value).unwrap_or(0);
        let delta = |old: i8, new: i8, side: i8| {
            ((new == side) as i64) - ((old == side) as i64)
        };
        (delta(prev, value, 1), delta(prev, value, -1))
    }
}

```

```
#[cfg(test)]
mod vote_tests {
    use super::*;

    #[test]
    fn repeat_and_flip_and_retract() {
        let mut svc = VoteService::default();
        assert_eq!(svc.cast(7, 42, 1), (1, 0)); // new upvote
        assert_eq!(svc.cast(7, 42, 1), (0, 0)); // retry: no double count
        assert_eq!(svc.cast(7, 42, -1), (-1, 1)); // flip: score swings by 2
        assert_eq!(svc.cast(7, 42, 0), (0, -1)); // retract
        assert_eq!(svc.cast(8, 42, 1), (1, 0)); // another user unaffected
    }

    #[test]
    fn folding_deltas_reproduces_true_tallies() {
        let mut svc = VoteService::default();
        let deltas = [svc.cast(7, 42, 1), svc.cast(7, 42, -1),
                      svc.cast(8, 42, 1)];
        let (up, down) = deltas
            .iter()
            .fold((0i64, 0i64), |(u, d), &(du, dd)| (u + du, d + dd));
        assert_eq!((up, down), (1, 1)); // 7's flip + 8's upvote
    }
}
```

The whole vote pipeline’s correctness lives in the four-line `delta` closure, so make sure you can narrate it. Every vote event reduces to two integers — how much `up` changed, how much `down` changed — computed by comparing the previous stored value with the new one. A retry of the same vote yields `(0, 0)` and the tallies do not twitch; a flip from up to down yields `(-1, +1)`, which is why the comment notes the score swings by **two** — flipping is not removing, it is defecting to the other camp. And the second test demonstrates the property that makes reconciliation possible: folding the emitted deltas reproduces the true tallies exactly, which is Section 5.11’s “derived numbers must be rebuildable from the event record,” proven in seven lines.

5.13.4 Cursor Pagination over a Ranked List

```
/// Map an f64 to bits whose unsigned order matches the float's order -
/// so rank keys can live in opaque cursors and database indexes.
pub fn ordered_bits(f: f64) -> u64 {
    let b = f.to_bits();
    if b & (1 << 63) == 0 { b | (1 << 63) } else { !b }
}

/// Total order per comment: (rank desc, id desc). No ties, ever.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub struct RankKey {
    pub wilson: u64, // ordered_bits(wilson_lower(up, down))
    pub id: u64,
}

pub type Cursor = RankKey; // the client treats it as opaque base64

/// Next page strictly after `after` from a list sorted descending.
pub fn page_after(
    sorted_desc: &[RankKey],
    after: Option<Cursor>,
    limit: usize,
```

```

) -> (Vec<RankKey>, Option<Cursor>) {
    let start = match after {
        None => 0,
        Some(c) => sorted_desc
            .iter()
            .position(|&k| k == c)
            .map(|i| i + 1)
            .unwrap_or(sorted_desc.len()),
    };
    let page: Vec<RankKey> =
        sorted_desc.iter().skip(start).take(limit).copied().collect();
    let next = match page.last() {
        Some(&last) if start + page.len() < sorted_desc.len() => Some(last),
        _ => None,
    };
    (page, next)
}

#[cfg(test)]
mod page_tests {
    use super::*;
    use crate::wilson_lower; // from the ranking listing (same crate)

    fn key(up: u64, down: u64, id: u64) -> RankKey {
        RankKey { wilson: ordered_bits(wilson_lower(up, down)), id }
    }

    #[test]
    fn ordered_bits_preserves_float_order() {
        assert!(ordered_bits(0.72) > ordered_bits(0.21));
        assert!(ordered_bits(-1.0) < ordered_bits(0.0));
    }

    #[test]
    fn pages_cover_everything_once_and_are_stable() {
        let mut keys = vec![
            key(100, 0, 1), // 0.963
            key(10, 0, 2),  // 0.722
            key(10, 0, 3),  // 0.722 - tie with id 2, broken by id desc
            key(1, 0, 4),   // 0.207
            key(0, 1, 5),   // 0.0
        ];
        keys.sort_by(|a, b| b.cmp(a)); // (wilson desc, id desc)

        let (p1, c1) = page_after(&keys, None, 2);
        let (p2, c2) = page_after(&keys, c1, 2);
        let (p3, c3) = page_after(&keys, c2, 2);
        let ids: Vec<u64> =
            [p1.as_slice(), &p2, &p3].concat().iter().map(|k| k.id).collect();
        assert_eq!(ids, vec![1, 3, 2, 4, 5]); // no skips, no duplicates
        assert_eq!(c1.unwrap().id, 3);       // cursor = last item of page
        assert!(c3.is_none());               // exhausted
    }
}

```

Two details here reward a second look. The `ordered_bits` function solves a problem you only discover when you try to store float ranks in systems that sort **bytes**: IEEE-754 floats do not sort correctly as unsigned integers once negatives appear (their sign bit flips the wrong way). The transform — flip the sign bit for non-negatives, complement everything for negatives — maps floats to u64s whose plain integer order matches the float order, so Wilson scores can live in B-tree indexes and base64

cursors without any float-awareness downstream. And the final test's assertion list is a checklist of everything pagination can get wrong: every item appears exactly once (no skips, no duplicates), the cursor equals the page's last item, and exhaustion is signaled by `None` rather than by an empty page — three different bugs, three assertions, one permanent guard.

5.14 Scaling the Platform

The scaling story here is refreshingly one-dimensional — everything hinges on the shard key, so choose it the way Section 3.14 taught you: by asking which entity every hot operation is scoped to.

Layer	Scale axis	Mechanism
API / services	Request rate	Stateless; horizontal scaling behind load balancers
Comments DB	Posts × comments	Shard by <code>post_id</code> — a whole thread on one shard, so every window query is single-shard; viral posts are one hot shard, handled by cache, not resharding
Votes DB	Users × votes	Shard by <code>comment_id</code> — tallies per comment stay local; lookup by user is the rare path (vote overlay), served by a secondary index
Event log / rank updater	Vote rate 2.3k/s avg	Partitioned by <code>comment_id</code> ; coalescing collapses bursts — consumer count tracks partitions, not votes
Thread cache	Concurrent readers	Window JSON is immutable between version stamps → any number of replicas; logged-out traffic is fully cacheable
Read bandwidth	800 MB/s avg	Edge/CDN for logged-out windows; the origin only serves logged-in overlays and cache misses

Notice that the two databases shard on **different** keys, and be ready to defend the asymmetry — it looks inconsistent and is actually precise. Comments shard by `post_id` because every comment query is scoped to one post's thread. Votes shard by `comment_id` because every vote operation is scoped to one comment, and tally locality matters more than user history (the per-user vote overlay is one small `ANY(...)` lookup per read, served by a secondary index). Each table shards on the key its **hot path** filters by; the cold path pays the secondary-index tax. That is the entire principle: you get to choose one cheap direction per table, so spend it on the path that runs a thousand times more often.

KEY INSIGHT — The viral-post problem is a cache problem, not a database problem

A post that makes the front page concentrates a meaningful share of the **entire site's** read QPS onto one shard's rows. Sharding cannot help — one post has one home. What helps: the ranked-window cache (one fill serves millions of reads between version stamps), read replicas of the hot shard for

cache misses, and coalesced rank updates so the vote storm does not translate into row-write storms. Measure success as **origin QPS during a viral event** staying flat. The broader lesson generalizes: when load concentrates on one **key**, your tools are caching, replication, and write coalescing — resharding a hot key does nothing, because the key's heat moves with it.

5.15 Failure Modes & Degradation

Failure	Symptom	Response
Tally drift	Displayed score $\neq$ vote log sum	Expected in small doses (coalescing, replays); nightly reconciliation recomputes tallies from votes ; a repair endpoint fixes single comments
Rank updater lag	Scores update, order doesn't	Visible as consumer lag (Chapter 4's meta-metrics); degraded mode = windows re-sorted by stored (stale) rank keys — the product blurs, never breaks
Event log outage	Writes accepted, ranks frozen	Vote service buffers events locally (Chapter 4's agent pattern); on recovery the backlog replays and coalesces
Thread cache loss	Cache cold on hot posts	Stampede protection: single-flight fill per window (one miss rebuilds, the rest wait), stale-while-revalidate serving
Replica lag on reads	User posts, refreshes, comment missing	Read-your-own-write for authors: route their reads to the primary, or merge their pending writes client-visible via token
Votes DB shard loss	Votes on one slice of comments fail	Fail closed for integrity (a queued vote is better than a lost vote); tallies already computed remain served
Brigading wave	Hundreds of votes/s on one thread from fresh accounts	Chapter 3's rate limiter plus velocity anomaly detection (Chapter 4's metrics!); flagged threads shift to slower rank recompute while investigated

Read the degraded modes and notice how deliberately each one picks **which property to sacrifice**. Rank updater lag sacrifices **freshness** (stale order) to protect availability — the product blurs, never breaks. Votes DB loss sacrifices **write availability** (fail closed) to protect integrity — because a vote that is queued and retried is recoverable, while a vote silently dropped is a lie the system told its user. Replica lag sacrifices **everyone else's view** (stale) while routing the one user who would notice — the author — to the primary. Failure engineering is triage: you cannot save every property in every

failure, so you decide in advance which property each failure is **allowed** to wound, and you write it down where a reviewer can argue with it.

5.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Tree encoding	Adjacency + materialized path hybrid	Pure adjacency: recursive subtree reads; nested sets: write amplification on every insert
“Best” ranking	Wilson lower bound on up/(up+down)	Raw score: statistically illiterate at small n ; ratio: worse — 1/1 beats 200/210; Bayesian average: defensible, but needs a global prior and is harder to explain to users
Rank computation	Recompute on vote, store rank keys	Sort at query time: 50k-row sorts per page view; scheduled recompute: stale beyond tolerance on fast threads
Vote writes	Synchronous upsert + async event	Fully async queue-first: lowest latency, but integrity (one-vote-per-user) becomes eventual — not acceptable
Tallies	Denormalized, reconciled nightly	Count from votes per read: honest but scans the largest table on the hottest path
Thread caching	Version-stamped windows, per-user overlay	Per-user full windows: personalization cost multiplies cache by user count; no cache: origin melts on viral posts
Deep nesting	Depth cap + “continue this thread”	Unbounded inline rendering: pathological subtrees (10^4 -deep chains) DoS the renderer and the reader
Comment count on posts	Async maintained counter	Synchronous increment on the post row: every comment write contends on one hot row — the exact anti-pattern Section 5.5 warned about

The “Best” ranking row deserves one more breath, because its rejected alternative is not obviously wrong: a Bayesian average (blend each comment’s ratio toward the global mean, weighted by sample size) is statistically respectable and some real platforms use it. Why Wilson instead? Two reasons you can say aloud: Wilson needs no **prior** — the Bayesian version imports a global average that must be estimated, versioned, and defended — and Wilson has a one-sentence explanation a product manager can repeat (“we rank by how good a comment **at least probably** is”), which matters when the ranking

function has to survive meetings, not just mathematics. Choose the statistic your organization can govern.

5.17 Observability & SLOs

Indicator	Definition	Target
Thread read latency	Window fetch, cached / uncached, p95	≤ 200 ms / ≤ 700 ms
Comment write latency	Create reply, p95	≤ 300 ms
Vote latency	PUT vote, p95	≤ 150 ms
Rank freshness	Vote $\rightarrow$ reflected in stored rank keys, p95	≤ 60 s
Read-your-own-write	Author sees own comment after create, p99	≤ 1 s
Tally accuracy	Comments whose tally $\neq$ vote-log sum, sampled	$\leq 0.1\%$, self-healing nightly
Removal propagation	Moderation removal $\rightarrow$ gone from all caches	≤ 60 s
Availability	Thread reads / writes	99.99% / 99.9%

Every one of these is a metric with labels and an alert with a `for` duration — Chapter 4’s platform, pointed at this chapter’s system. The reconciliation job emits tally-drift as a metric; brigading detection is a velocity alert on votes per thread. The book’s chapters compose, and saying so in the interview is a senior signal. Notice also which SLO is **not** in the table: there is no “rank accuracy” target, because rank keys are a pure function of vote state — correctness there is enforced by construction and tested in Section 5.13, not measured at runtime. The best SLOs are the ones the architecture makes unnecessary.

5.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **“Make it real-time — new comments appear without refresh.”** Add a subscription tier: Web-Socket/SSE gateways subscribed per post; the event log fans `CommentCreated` events to gateways. Key decision: live comments arrive **appended** (by time), not re-sorted — re-ranking a live view under the reader’s eyes is a product bug; re-sort applies on next window load. (A comment that leaps over three others mid-read is the UI equivalent of someone rearranging a book while you read it.)
2. **“How do you detect vote manipulation?”** Signals: velocity anomalies on single threads, account-age and IP clustering of voters, vote-ring graph analysis (accounts that only vote each other). Enforcement: shadow-exclude flagged votes from **rank keys** while keeping them in the visible score — manipulation loses its feedback signal. That last move is the elegant one: a manipulator who cannot observe their own effect cannot tune their attack.
3. **“Users edit comments to bait-and-switch after they rank.”** Keep edit history; reset or decay a comment’s rank confidence on substantive edits (re-open the Wilson interval); show “edited” markers. Ranking trust is the product — this is an integrity issue, not a UX nicety.
4. **“Sharding: what about a post with 10^6 comments?”** One shard holds it — $10^6 \times 1.5$ KB ≈ 1.5 GB, fine; reads are cached windows; votes partition by comment. The real ceiling is per-shard write QPS, and comment writes even on viral posts stay 10^2 /s. If it ever breaks, sub-shard the thread by top-level comment id.

5. **“Design comment search.”** Chapter 4’s inverted index, fed from the same event log: tokenize bodies, index `(post_id, comment_id, tokens)`, rank results by stored Wilson score. Pipelines compose; nothing new is needed.

5.19 Summary & Further Reading

NOTE — Chapter summary

A comment system is a **shape** problem wearing a scale costume. The data is a forest of unbounded trees: store it as an adjacency list with materialized paths, so any subtree is one prefix scan and depth is a stored fact; delete with tombstones so replies are never orphaned. Ranking is statistics, not arithmetic: “best” is the Wilson lower confidence bound (evidence beats luck), “controversial” is balanced volume, “hot” is log score plus 12.5-hour decay — all pure functions of `(up, down, time)`, recomputed on every vote and **stored**, so reads are index scans. Votes are the write storm: idempotent upserts keyed by `(user, comment)`, coalesced rank recomputation, tallies reconciled against the vote log nightly. Reads are windows — ranked pages with shallow children — cached per sort mode and version-stamped. The thread is never serialized; the conversation’s shape is what ships.

Further reading.

- The source video: “15: Reddit Comments — Systems Design Interview Questions With Ex-Google SWE” (Jordan has no life): <https://www.youtube.com/watch?v=B02gRisnBcA>
- Randall Munroe / Reddit — “How Reddit ranking algorithms work” (redditblog, 2009) — the original public write-up of hot/best/controversial.
- Evan Miller — “How Not To Sort By Average Rating” — the Wilson score interval derivation Section 5.8 distills, with the canonical worked numbers.
- Joe Celko — *Trees and Hierarchies in SQL for Smarties* — the encyclopedic treatment of adjacency lists, materialized paths, and nested sets.
- Amir Salihefendić — “How Reddit ranking algorithms work” commentary and Hacker News ranking write-ups — production variations on time decay.

5.20 Chapter Glossary

Every term this chapter defined, gathered for review — read the list once through before moving on, because Chapters 6 and 7 will reuse the ranking vocabulary without re-deriving it.

Term	Meaning
Adjacency list	Tree encoding where each row stores its parent’s id; cheap writes, recursive reads
Coalescing	Collapsing many updates to the same entity into one recomputation; how vote bursts become trickles of rank writes
Comment count	Denormalized per-post tally of comments, maintained asynchronously for listing pages
Confidence interval	A range estimating a true probability from samples; wider (more pessimistic) when data is scarce
Controversial ranking	$(u + d)^{\min / \max}$ — balanced disagreement weighted by volume
Cursor (pagination)	Opaque token encoding the last seen sort key; stable under insertions, unlike offsets

Term	Meaning
Denormalization	Storing derived values (tallies, rank keys) to make reads cheap; the derived value must be rebuildable from truth
Depth cap	Maximum inline nesting rendered before collapsing behind “continue this thread”
Forest	The set of all comment trees, one rooted at each post
Hot ranking	$\log_{10} \max(s , 1) + \text{sign}(s) \cdot (t - t_0) / 45000$ — log-score with 12.5-hour decay
Idempotency key	A deduplication token making retried writes safe; for votes it is the natural key (user, comment)
Materialized path	Tree encoding storing each row’s full ancestor chain; subtrees become prefix scans
Nested sets	Tree encoding with depth-first lft/rgt bounds; range-scan reads, catastrophic insert renumbering
N+1 query	One query per parent to fetch children; the adjacency-list trap that batched path scans eliminate
Rank key	A stored, indexed, orderable encoding of a comment’s rank per sort mode; recomputed on vote
Read-to-write ratio	Reads per write; extreme ratios justify aggressive caching and denormalization
Read-your-own-write	Guarantee that a writer immediately sees their own write; routed to primary or merged client-side
Score	<code>up - down</code> , the visible number; deliberately not the “best” ordering
Single-flight fill	One cache miss rebuilds a value while concurrent misses wait; stam-pede protection
Soft delete / tombstone	Content erased, structure kept, so replies stay threaded under a “[deleted]” placeholder
Time decay	Ranking term that ages content out, forcing continuous freshness
Version stamp	A per-post counter bumped by rank updates; cached windows keyed by it age out without invalidation fanout
Vote overlay	Per-user vote values merged over a shared cached window at read time
Wilson score interval	Binomial confidence interval on the true upvote probability; its lower bound ranks “best” honestly across sample sizes

— End of Chapter 5 · Next: Chapter 6, Designing a Real-Time Leaderboard —

Designing a Top-K Leaderboard

NOTE — Problem source

This chapter solves the problem posed in the talk “17: Top K Leaderboard” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the leaderboard system of a hit online game — millions of players submit scores after every match, and the service answers “who are the top K?” and “what is **my** rank?” in milliseconds, on boards that reset daily, weekly, and run all-time. The challenge is not volume but a very specific primitive — **rank as a first-class query** over an ordered set that mutates 25,000 times a second. All terms are defined before use; all reference code is Rust with deterministic tests.

6.1 The Problem Statement

The interviewer sketches a phone screen — a podium of three avatars, a scrolling list below, and a highlighted row pinned at the bottom: “**You: rank 1 247 003**” — and says:

“Players finish matches and submit scores. Players can view the global top 100, page deeper, and always see their own rank and neighborhood even when they are a million places down. Boards exist per season and all-time. Ties go to whoever got the score first. Design it.”

This problem looks small next to Chapters 2–5 — and that is the trap, so let’s spring it deliberately and see what’s inside. The arithmetic is gentle (Section 6.5 will show the entire dataset fits in one machine’s RAM); no firehose, no petabytes, no exotic geography. But look at the phone screen again, specifically at that pinned bottom row: “**You: rank 1 247 003.**” Somebody a million places from the podium opens the app and expects their **exact** position — not an estimate, not a percentile band — in the time it takes the screen to render. Now combine that with the other headline query, top-100, and with the write side mutating the ordering 25,000 times a second. The difficulty is that both queries are **positional** — they are about where an entry sits inside a total order — and positional queries over a hundred-million-entry order that never stops moving are exactly the queries ordinary databases are worst at. Choose the wrong structure and either the reads or the writes eat the system. Choose the right one and most of the chapter becomes careful engineering around it: windows, ties, sharding, and cheating.

DEFINITION — Leaderboard / rank / top-K

A *leaderboard* is an ordering of players by score within some scope (a game, a season, a region, a friend group). A player’s *rank* is their 1-based position in that ordering. A *top-K* query returns the first K entries in order. The pair {top-K, rank} is the complete query surface — every UI screen is one or the other.

That last claim is worth testing against the sketched screen, because it collapses the product into two primitives. The podium and the scrolling list: top-K with paging. The pinned “you are here” row: rank,

plus a small walk in both directions (the **neighborhood**, FR-4). Profile badges like “top 2.3%”: rank divided by board size — a derived form of rank. There is no third query. When a whole product surface reduces to two operations, your entire design conversation can — and should — be about serving those two operations well. Scope discipline starts with noticing how small the real surface is.

DEFINITION — Best-score vs. every-run boards

Two admission semantics. *Best-score*: a player occupies exactly one slot, holding their personal best; a worse new score changes nothing. *Every-run* (arcade-style): each match is its own entry and one player may hold many slots. We design best-score — the common default — and note where every-run differs (Section 6.10).

6.2 Scope & Clarifying Questions

The prompt is short but leaves four genuinely load-bearing ambiguities — score semantics, tie-breaking, freshness, and board lifecycles — and the dialogue below closes each one. Watch for the tie-break answer especially: “whoever got the score first” sounds like a footnote and will end up **inside the ordering key** in Section 6.7.

Candidate asks	Interviewer answers
Score semantics?	Personal best per player per board; ties broken by who achieved it first
Which boards?	Global all-time, plus daily and weekly; same game, one platform
Rank granularity?	Exact rank for the querying player; “top X%” displays for everyone are fine approximate
Freshness?	A submitted score should be reflected in rank/top-K within 5 seconds
Live updates?	No push; clients poll or refresh. The list may change between pages — acceptable
Anti-cheat?	Hook it in; don’t design it. Suspicious scores can be quarantined
Scale?	20M daily players, 200M matches/day, 200M registered accounts
Friends boards?	Yes — a player vs. their friends ($\leq$ a few hundred)
Score range?	Non-negative integers, 0 to 10^9

Three of these answers are quiet gifts. “Rank granularity: exact for the querying player, approximate for displays” splits the rank query into a **trusted** variant (yours, exact, screenshotted, argued about) and a **glanced-at** variant (percentile badges nobody audits) — Section 6.11 will turn that split into an enormous cost saving. “No push; polling is fine” keeps Chapter 1’s entire real-time machinery on the shelf, again. And “5 seconds of freshness” is the permission slip for the asynchronous ingestion pipeline of Section 6.9 — a score that must be visible **instantly** would force synchronous coupling between match servers and rank storage; five seconds buys you a queue, and with the queue comes burst absorption, replayability, and an anti-cheat tap, all for free.

NOTE — Agreed scope

1. **Submit score** after each match; best-score semantics per player per board; ties to the earliest achiever.
2. **Top-K** with cursor paging ($K \leq 100$ per page).
3. **My rank** — exact, fast, at any depth, plus a neighborhood view (the $\pm r$ players around me).
4. **Time-windowed boards** — daily/weekly boards rotate; all-time persists.

5. **Friends leaderboard** — my standing among my friend list.
6. **Anti-cheat quarantine** hook on the ingestion path.
7. Out: live push, team boards, tournaments, cross-game boards.

6.3 Functional Requirements

One definition first — the query that makes this problem famous, and the one that quietly demands the most of your data structure:

DEFINITION — Neighborhood query

The $\pm r$ entries around a given player's rank — the “you are here” view. It is the query that makes the problem famous: it requires **rank** (to locate the player) and **ordered iteration** (to walk r places in both directions), at any depth in the ordering, in milliseconds.

Read that definition as a two-stage rocket, because each stage eliminates different candidate structures. Stage one: locate the player — that needs a **by-key lookup** **and** their position in the order, two different kinds of access fused. Stage two: walk r places outward — that needs the order to be **iterable** from an arbitrary interior point, not just from the top. A hash map fails stage one's second half (it has no positions). A heap fails stage two entirely (it can iterate only by destroying itself). Keep both stages in mind when Section 6.6 walks the candidate structures — the neighborhood query is the examiner.

1. **FR-1 — Submit score.** (`player_id`, `score`, `achieved_at`, `match_id`) per board scope; stored if it beats the player's current best; idempotent under retries and duplicate match reports.
2. **FR-2 — Top-K page.** Ordered entries (`rank`, `player`, `score`) from a cursor, per board.
3. **FR-3 — Player rank.** Exact rank and score for one player on one board.
4. **FR-4 — Neighborhood.** The $\pm r$ window around a player, ranks included.
5. **FR-5 — Windowed boards.** Daily/weekly/all-time boards of the same game, rotating automatically, old windows archived read-only.
6. **FR-6 — Friends board.** Top-K/rank restricted to a player's friend set.

Notice how FR-1 packs three guarantees into one line, and each will resurface as machinery. “Stored if it beats the player's current best” is **best-score admission** — Section 6.10 shows it makes the whole write path monotonic. “Idempotent under retries” is Chapter 5's lesson arriving on schedule — but with a twist: here the idempotency comes from the **semantics** (`max` is naturally idempotent), not from a dedup table, and Section 6.10 will make you appreciate the difference. And `match_id` riding along in the tuple is the dedup hook for the one failure mode `max` cannot absorb: the **same match** reported twice with the **same** score.

6.4 Non-Functional Requirements

The NFRs for this problem hinge on a vocabulary word most candidates have never needed — so define it before the table, because the table assumes it:

DEFINITION — Order-statistics query

A query about **position** in an ordering — “rank of X”, “the k-th element”, “count of entries above Y” — as opposed to a lookup by key. Supporting order statistics efficiently under inserts/deletes requires an ordered structure augmented with subtree/span counts (Section 6.7); plain hash indexes cannot answer them at all, and plain sorted scans answer them at $O(n)$.

This definition is the chapter in embryo. Hash indexes answer “what is X's value?” — they are blind to position. Sorted arrays know position but charge $O(n)$ to mutate. The question the whole interview

pivots on is whether you know there exists a family of structures that tracks position **as an invariant**, maintained incrementally on every insert and delete, so that rank queries read out a precomputed answer instead of recomputing one. Augmentation — storing extra derived data inside a structure’s nodes and repairing it locally on each mutation — is the technique, and Section 6.7’s skip list is its friendliest form.

Requirement	Target & reasoning
Score ingest throughput	25k updates/s peak sustained; a launch event may 5× that briefly
Top-K read latency	p95 ≤ 50 ms (cached pages: ≤ 10 ms); it is a hot UI surface
Rank / neighborhood latency	p95 ≤ 100 ms at 10 ⁸ -entry boards — this kills naive designs
Freshness	Score → visible in rank/top-K ≤ 5 s; rank need not be transactional with the match
Durability	A confirmed personal best must survive node loss (replication + journaling)
Availability	Reads ≥ 99.99% (stale board pages acceptable); writes ≥ 99.9%
Integrity	One slot per player per board; ties deterministic; quarantined scores never render

The third row carries the phrase “kills naive designs” — take it seriously as a filtering device. A hundred milliseconds at 10⁸ entries sounds generous until you price the naive approach: computing rank by counting better entries scans, on average, half the board — fifty million rows — per query, thousands of times per second. You are not 10× off; you are **five orders of magnitude** off. When an NFR misses feasibility by that margin, the fix is never tuning — it is a different data structure. That is why this chapter’s architecture section is shorter than its data-structure section, inverting the book’s usual proportions.

KEY INSIGHT — Reads and writes are both modest — the operation mix is the design

Nothing here rivals Chapter 4’s firehose or Chapter 5’s vote storm. What is unusual: the system must maintain a **total order** over 10⁸ entries, mutate it 25k times a second, and answer positional queries — top-K, rank, neighborhood — in milliseconds. Data structure choice **is** the architecture; everything else is distribution plumbing around one ordered set.

6.5 Back-of-the-Envelope Estimation

Run the numbers anyway — gently, this time — because two of them will hand you the central design license.

Assumptions: 20M daily players; 10 matches per player per day; every match submits one score; each player opens leaderboard surfaces 5×/day; a board entry is 100 B with index overhead.

Quantity	Estimate	Derivation
Score submissions	200M/day ≈ 2.3k/s avg, 25k/s peak	20M players × 10 matches; evening × event multiplier ≈ ×10
Leaderboard reads	100M/day ≈ 1.2k/s avg, 12k/s peak	20M × 5 views; top-K pages + rank lookups

Quantity	Estimate	Derivation
Global all-time board	20 GB in RAM	200M players × 100 B (id, score, ts, structure overhead)
Daily board	2 GB	20M active × 100 B; born and dies daily
Top-100 page	5 KB JSON	100 × (id, name ref, score, rank)
Update cost	27 pointer hops	$\log_2(2 \times 10^8) \approx 27$ — the ordered set makes writes cheap
Snapshots	20 GB/board, minutes to write	Sequential dump; recovery = snapshot + journal replay

The update-cost row is worth a slow read: $\log_2(2 \times 10^8) \approx 27$ means that even on the all-time board, applying a score change touches about twenty-seven nodes — microseconds of work. Writes are not the problem. Reads of the podium are not the problem either (5 KB from the top of an ordered structure). The **only** expensive questions are the positional ones — which is the estimation table independently confirming Section 6.4’s warning from a second direction. When your arithmetic and your NFR analysis point at the same bottleneck, you have found the real problem.

KEY INSIGHT — The whole working set fits in RAM — spend it

20 GB for the all-time board is one machine’s RAM, or a few shards’ worth. That licenses the chapter’s central decision: keep boards **in memory** in a purpose-built ordered structure with $O(\log n)$ everything, replicate for reads and failover, journal for durability. A disk-first database would answer the same queries 10–100× slower for zero capacity benefit. Capacity is solved; latency and structure are the game. Say this sentence in the interview exactly as written — “the working set fits in RAM, so the design should be memory-native with durability bolted on, not disk-native with caching bolted on” — and watch the interviewer nod: it is the correct first move for a whole family of problems (session stores, real-time bidding, matchmaking) that candidates routinely over-persist.

The flow of the chapter: Section 6.6 isolates why rank is the crux; 6.7 picks the data structure (with the skip-list mechanics drawn out); 6.8–6.9 design APIs and architecture; 6.10–6.12 go deep on ingestion semantics, sharded top-K, and windowed/friends boards; 6.13 implements it all in Rust; 6.14–6.20 scale, harden, and review.

6.6 The Core Challenge: Rank Is Not a Lookup

Strip the product away and the service is one abstract data type with four operations: `upsert(player, score)`, `top(k, cursor)`, `rank(player)`, `neighborhood(player, r)` — under 25k mutations/s and 12k queries/s, on a collection of 10^8 . Now do the exercise that this entire interview exists to elicit: walk the candidate structures you already know, and **fail them in order**, saying out loud why each one dies. The walk is the answer — the destination is just the last structure standing:

Structure	upsert	top-K	rank(player)	Verdict
Hash map	$O(1)$	$O(n \log n)$	$O(n \log n)$	Rank does not exist; every query re-sorts 10^8 entries
Sorted array	$O(n)$	$O(K)$	$O(\log n)$ by score	Reads fine; $O(n)$ write shifts ruin 25k updates/s

Structure	upsert	top-K	rank(player)	Verdict
B-tree index (DB)	$O(\log n)$	$O(\log n + K)$	$O(n)$ count scan	The classic wrong answer — see below
Heap	$O(\log n)$	$O(K \log n)$	unsupported	No rank, no paging; wrong shape entirely
Skip list + hash map, spans	$O(\log n)$	$O(\log n + K)$	$O(\log n)$	Ordered, rank-augmented, in-memory — Section 6.7

Narrate the eliminations. The **hash map** fails not on speed but on expressiveness: it has no concept of order, so every rank or top-K question degenerates into “sort the universe, then look.” The **sorted array** is the mirror image — position is free (binary search!), but a single score update can shift half the array’s elements one slot over, and 25,000 such shifts per second is an earthquake that never stops. The **heap** is the structure people reach for when they hear “top-K” — and it answers exactly that one query, at the cost of forgetting everything else: no interior access, no rank, no neighborhood, no deletion of a quarantined cheater without a rebuild. And the **B-tree** deserves its own pitfall, because it is the answer that sounds most like production experience:

COMMON PITFALL — The `COUNT(*)` rank trap

The most common wrong answer: store scores in a relational table, index the score column, and compute rank as `SELECT COUNT(*) WHERE score > :mine`. The index makes top-K fine, but rank-by-count touches **every entry above the player** — for rank 1.2M of 200M, that is a 198.8M-row index scan per query, thousands of times a second. Rank must be **maintained by the structure**, not computed by counting. This is the sentence the interviewer is listening for.

Do not just memorize the trap — understand why it is seductive, because the same shape of mistake recurs everywhere. The database **feels** like it knows the rank: it has a B-tree on score, and a B-tree is an ordered structure, and ordered structures “know” positions. But a B-tree index maintains order **locally** (each node sorted, children in ranges) without maintaining **counts** — no node records how many keys live beneath it — so the only way to compute “how many entries beat me” is to visit them all. Augmented structures exist precisely to close this gap: store the subtree counts, repair them on the rotation path, and rank becomes a root-to-leaf walk. Chapter 8 will show you B-trees earning their keep where they belong; this chapter is where you learn they are not positionally aware by default.

6.7 Deep Dive: The Data Structure — Skip List with Spans

Here is the last structure standing from Section 6.6’s walk, and it deserves a proper introduction rather than a magic invocation, because “use a skip list” is only an answer if you can explain **what makes rank $O(\log n)$** .

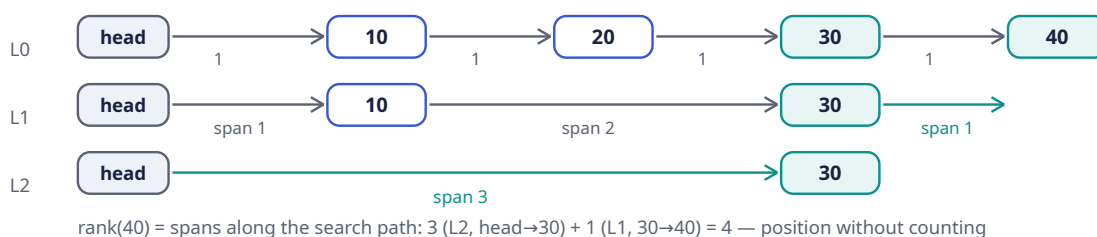
DEFINITION — Skip list / span

A *skip list* is a probabilistically balanced ordered structure: a sorted linked list (level 0) with express lanes above it — each node is promoted to the next level with probability p (e.g. $1/4$), so level i holds p^i of the nodes. Search starts on the top express lane and drops down, skipping $1/p$ nodes per hop: $O(\log n)$ expected search, insert, delete. A *span* is an integer on every forward pointer counting how many level-0 steps it covers; spans turn the skip list into an **order-statistics** structure: summing spans along a search path yields the rank directly, and rank + forward walks yield neighborhoods — still $O(\log n)$.

Build the intuition in two passes, because the two halves are independent ideas that happen to compose perfectly. **Pass one — the skip list itself.** A plain sorted linked list already has the property

you need for reads from the top (top-K is “walk K steps”) but searching it is $O(n)$: to find your player you must visit every node before them. The skip list fixes this with an almost childlike idea — add **express lanes**. Level 0 is the full list. Each node, at birth, flips a weighted coin: with probability $\frac{1}{4}$ it also appears at level 1; of those, a quarter also appear at level 2; and so on. The result is a stack of ever-sparser copies of the list, each an express lane that skips about three quarters of the nodes below it. Searching starts at the top lane — giant strides — and drops a level whenever the next stride would overshoot the target, until level 0 delivers the exact node. Each drop halves-ish the remaining distance in expectation, hence $O(\log n)$. Nothing is ever rotated or rebalanced; balance is **statistical**, maintained by the coin flips at insert time, which is why concurrent and in-memory implementations of skip lists are so much simpler than their tree cousins.

Pass two — spans, the augmentation that creates rank. On every forward pointer, at every level, store one extra integer: how many level-0 steps that pointer covers. A level-0 pointer always spans 1 (it goes to the next node); a level-2 pointer might span 9 (it leaps nine real entries). Now search for your player and **add up the spans of every pointer you traverse**: each pointer you use is, by construction, a leap over entries that all rank **ahead** of your player, and the sum of the leaps is exactly the count of those entries — which is the rank minus one. Position falls out of the search path itself. No counting scan, no per-query work proportional to anything but the path length. **That** is what “maintained by the structure” means concretely, and the diagram below makes it visible.



Walk this picture the way the algorithm walks it, because the caption’s arithmetic only makes sense once you see the path. The drawing shows four entries — 10, 20, 30, 40 — on level 0 (the bottom row of boxes, chained by the gray arrows labeled “1”: every level-0 step covers exactly one real entry). Above them, fate has been uneven: node 10 was promoted to level 1, node 30 all the way to level 2 (its three stacked boxes are shaded teal, the tallest tower in the picture), and nodes 20 and 40 never got promoted at all. The leftmost stack of three “head” boxes is the sentinel every search starts from.

Now answer the caption’s question: what is the rank of entry 40? Start at the head’s top box, L2. The only L2 arrow in the structure leaps from head all the way to node 30 — a giant teal arrow spanning the top of the picture, labeled “span 3” because it jumps over three level-0 entries (10, 20, and lands on 30). You take it, and you record: 3 entries are now behind you. From node 30 you would like to continue on L2, but 30’s L2 pointer goes nowhere useful (40 was never promoted), so you **drop a level** — the search steps down inside node 30’s tower, free of charge. On L1, node 30 has a pointer to node 40, drawn in teal and labeled “span 1”: one more real entry traversed. Running total: $3 + 1 = 4$ — and entry 40 is indeed the 4th in the list. Rank computed from two pointer hops, without visiting nodes 10 or 20 at all. At 10^8 entries the same computation takes about 27 hops instead of 2 — that is the entire claim of the structure, and you have just executed it by hand.

Notice also what the spans buy **deletion** and **insertion**, since the interviewer may ask: when a node enters or leaves, only the pointers on its search path change, and each changed pointer’s span is repaired by adding or subtracting the new local step counts — constant work per level, no global recount. The invariant “span = level-0 steps covered” is maintained locally, edge by edge, which is exactly the augmentation discipline Section 6.4 described abstractly.

Why this over a balanced tree with subtree counts: same asymptotics, but level-promotion makes inserts/deletes local (no rotations), concurrent implementations are far simpler (only level 0 needs careful linking), and span bookkeeping is an 10-line addition. It is exactly the structure at the core of the industry’s canonical sorted-set implementations — a hash map from player→node for $O(1)$ “where is this player?” lookup, plus the skip list for order and rank. Two structures, one object: the hash map answers **identity**, the skip list answers **position**, and the pair of them is the “ordered set” the rest of the chapter treats as a single component.

Ties slot in cleanly — and this is where the scope dialogue’s footnote (“ties to whoever got it first”) becomes structure. The ordering key is not `score` but the triple `(score desc, achieved_at asc, player_id)`: deterministic total order, no equal keys ever, and “first to achieve” falls out of the comparison itself rather than being special-cased anywhere. Design lesson worth saying aloud: when a product rule says “break ties by X,” the robust implementation is almost always to **fold X into the sort key**, not to handle ties at query time — a total order cannot forget its tie-break rule, because it does not have ties. Section 6.13 implements the triple on Rust’s ordered map (same complexity story; Rust’s std ordered map lacks spans, and the listing says so honestly).

6.8 API Design

Six endpoints, and by now you can predict most of their shapes from the FRs — which is the point of doing requirements first:

Method	Path	Purpose
POST	<code>/v1/scores</code>	Submit <code>match</code> <code>score</code> <code>{player_id, board, score,</code> <code>achieved_at, match_id}</code> — best-score + idempotent (FR-1)
GET	<code>/v1/leaderboards/{board}/top?cursor=&limit=</code>	Top-K page (FR-2); cursor = last <code>(score, ts, id)</code> triple
GET	<code>/v1/leaderboards/{board}/players/{id}</code>	Exact rank + score (FR-3)
GET	<code>/v1/leaderboards/{board}/players/{id}/around?r=</code>	Neighborhood window (FR-4)
GET	<code>/v1/leaderboards/{board}/players/{id}/friends</code>	Friends board (FR-6)
POST	<code>/v1/admin/quarantine</code>	Anti-cheat hook: remove/hold entries (scope)

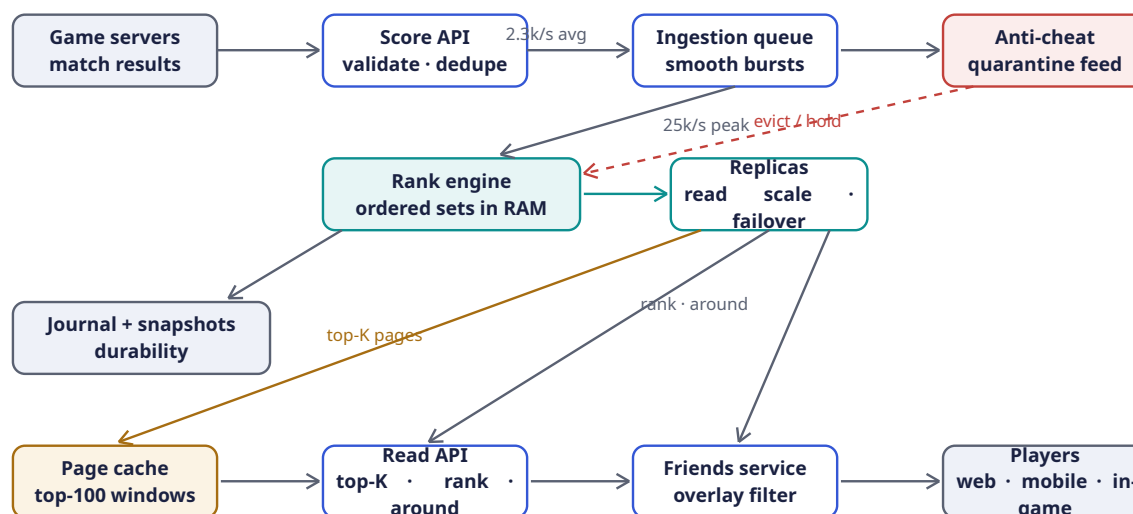
Notes: `board` encodes window and scope (`global:alltime`, `global:2026-W36`, `region:eu:daily:2026-09-05`) — Section 6.12 will show that making the window **part of the key** is what turns board rotation from a migration into a string change. Cursors are the rank triple, opaque and base64’d — Chapter 5’s pagination lesson, and here the cursor **is** the ordering key, so paging is a pure `range()` from the cursor position: no translation layer, no drift, no ambiguity. Submission responses return `{accepted, improved, current_best}` so clients can celebrate personal bests without a second call — a small product flourish that costs you nothing because the best-score check computed exactly those booleans on the way through.

6.9 High-Level Architecture

You now have every ingredient: an ordering structure that fits in RAM (Sections 6.5, 6.7), a write path that must absorb $10\times$ event-day bursts (Section 6.4), a podium that is read three orders of magnitude more often than it changes, and an integrity requirement that says cheaters must vanish from the

board without the honest players noticing anything. Assemble them and the architecture almost draws itself — the whole system is really just **one in-memory data structure with a well-guarded front door and a very wide read porch**:

Writes: validate → queue → in-memory ordered sets (replicated, journaled). Reads: cached pages for the top, live rank queries for everything else



The caption above the diagram is the entire architecture in one sentence; now walk the picture slowly enough that every box earns its place. **The write story** flows left to right along the top row. A match ends and the game server — your only trusted caller, the gray box at top-left — submits the result to the **Score API**. This box is deliberately narrow: it authenticates the caller, validates the event's shape, and dedupes by `match_id` so a retried submission cannot double-count. Everything that will ever enter the system squeezes through this one auditable door, which is exactly what you want when integrity is an NFR: one place to log, one place to rate-limit, one place to change validation. From there the event joins the **ingestion queue** — Chapter 4's shock absorber, recycled without shame. The two rate labels tell its story: 2.3k/s flows on an average day, but an event day (Section 6.5) multiplies that by ten, and the queue's job is to make the 25k/s peak somebody else's problem — the rank engine consumes at its own pace and the burst becomes **lag**, not loss.

The diagonal arrow from the queue down into the teal **rank engine** is where the design's center of gravity sits. This box holds every board as an in-memory ordered set — the skip-list-plus-hash-map pair from Section 6.7 — and applies each update as a best-score `max`. Twenty gigabytes fits in RAM; there is no database on the write path, no disk in the latency budget, no cache-invalidation puzzle because the structure **is** the system of record. Follow the arrows out of it. To the right, the engine streams every applied update to its **replicas**: they exist for two reasons the diagram shows honestly — reads fan out across them (12k rank-reads/s would rather not share a process with 25k writes/s), and if the engine dies a replica is already warm for promotion. Down and to the left, the gray **journal + snapshots** box is the durability story: every update is appended to a log before the caller is told it stuck, and periodic dumps of the whole board bound recovery to “load latest snapshot, replay the tail.” The board is rebuildable and therefore never lost — and notice that because updates are idempotent (next section), replaying the journal twice is **harmless**, which makes recovery procedures blessedly un-subtle.

The dashed crimson arrow deserves its own paragraph, because it is the integrity NFR drawn as plumbing. The **anti-cheat** service taps the same ingestion stream (the short arrow from queue to its box — it is a consumer like any other, no special coupling to the write path), runs whatever detection heuristics the trust-and-safety team dreams up, and when it convicts someone it issues an **evict / hold**: delete the player's entries from every board and park their future submissions until cleared.

Two properties make this elegant rather than bolted-on. First, removal is just another update — a delete flows through the same ordered set, the same journal, the same replicas, so there is no second code path to keep consistent. Second, because the leaderboard never renders a quarantined score, the podium’s integrity survives even a slow detection pipeline: the worst case is a cheater visible for minutes, then gone everywhere at once, including from cached pages at their next TTL tick.

The read story is the bottom row, and it is shaped by the asymmetry from Section 6.4: the podium is read enormously more often than any individual rank. So the two read paths are deliberately different. The amber arrow carrying **top-K pages** from the replicas down to the **page cache** is the hot path for the masses: the top-100 window of each popular board is materialized as flat pages with a few-second TTL, so a million users hammering the season leaderboard hit cheap cache reads, and the slight staleness is invisible because — as Section 6.14 will quantify — the top 100 churns far slower than the board as a whole. The plain arrow carrying **rank · around** is the personal path: when a player asks “where do I stand,” that query cannot be served from a cached podium (they are rank 1 247 003, remember), so it goes live to a replica, which answers in one ordered-set lookup and one span-summing search. Both paths converge on the **Read API**, which also consults the **friends service** — an overlay filter that computes friends boards on the fly (Section 6.12 explains why they are not materialized) — and the merged result flows right into **players** on web, mobile, and in-game clients. Reads never touch the journal, never touch the write door, and mostly never touch the engine itself; the porch is wide precisely because the write room is small.

Component	Responsibility	Why it is shaped this way
Score API	Authenticate game servers, validate, dedupe by <code>match_id</code>	Writes enter through one narrow, auditable door
Ingestion queue	Buffer score events; absorb event-day bursts	Chapter 4’s shock absorber; ingestion lag becomes a metric, not an outage
Rank engine	Holds all boards as in-memory ordered sets; applies best-score updates; serves rank/top-K/around	The structure of Section 6.7 is the system; 20 GB fits in RAM (Section 6.5)
Replicas	Read scale-out + hot standby	Rank reads $\times 12k/s$ spread; promotion covers node loss
Journal + snapshots	Append-only update log; periodic board dumps	Recovery = latest snapshot + replay; the board is rebuildable, never lost
Anti-cheat	Consumes the same stream; quarantines suspects	A consumer like any other — removal is just a delete from the set
Page cache	Top-100 pages per board, few-second TTL	The podium is read $10^3\times$ more often than it changes (top-100 churn is slow; Section 6.14)
Friends service	Intersection of friend list with board entries	Small overlays computed on read; Section 6.12

6.10 Deep Dive: Score Ingestion Semantics

Before you shard anything or cache anything, it is worth pausing on the single most convenient property this system has, because several later decisions quietly depend on it. Best-score admission makes the write path almost suspiciously simple:

DEFINITION — Monotonic (idempotent) update

An update whose repeated application changes nothing after the first time. Best-score submission is monotonic: `board[player] = max(old, new)` absorbs retries, duplicates, and out-of-order redeliveries for free — no dedup tokens, no exactly-once machinery on the hot path. (Chapter 5’s votes needed an upsert table; here the semantics do the work.) The one non-monotonic case — same player, same score, earlier timestamp stealing the tie-break — is excluded by rule: a player’s slot keeps its first achieving timestamp.

Stop and appreciate what this buys you, because “idempotent” is easy to say and rarely this complete. A client retries because the response was lost? `max` shrugs. The queue redelivers after a consumer crash? `max` shrugs. Two game servers somehow report the same match? The `match_id` dedupe at the door catches the obvious case, and `max` catches everything else. The journal replays a day of updates onto a stale snapshot? Every entry converges to the same value it would have had anyway. You get at-least-once delivery — the cheap, reliable kind — and the **semantics** upgrade it to exactly-once effects. The single sharp edge is the tie-break: if the same score arrives twice with different timestamps, blind `max` on the triple could let a later redelivery steal the “earlier `achieved_at` wins” slot, which is why the rule keeps the first timestamp and never rewrites it. One sentence of policy, enforced in one comparison function, and the whole exactly-once debate evaporates.

The ingestion contract, end to end:

1. Game server submits `(player, board, score, achieved_at, match_id)`; the Score API dedupes by `match_id` briefly (same match reported twice) and validates shape.
2. The queue buffers; the rank engine applies `max` updates in stream order. Bursts only stretch the freshness budget (≤ 5 s) — never correctness.
3. Anti-cheat consumes the same stream independently; a quarantine verdict deletes the player’s entries from every board (a delete is just another update) and parks their submissions until cleared. The leaderboard never renders a quarantined score — integrity requirement, Section 6.4.
4. Every-run variant (arcade boards): entries keyed `(match_id)` instead of `player`; dedupe by `match_id` is still free; top-K is unchanged; “my rank” becomes “my best run’s rank.” One sentence in the interview shows the abstraction holds.

That fourth step is worth dwelling on for a moment, because interviewers love to poke it. If the product pivots from “your best score ever” to “every run on the board” — arcade style, where one player can occupy five slots of the top ten — notice how **little** of the design moves. The ordered set does not care whether its members are keyed by `player` or by `match`; the ingestion contract’s dedupe-by-`match_id` becomes the primary key instead of a guard; the podium, paging, and caching are untouched. The only real casualty is “my rank,” which must now be defined as “the rank of my best run” — a product sentence, not an engineering one. When a one-line key change absorbs a product pivot, you know the abstraction was drawn at the right joint.

6.11 Deep Dive: Sharding and the Global Top-K

One node holds 20 GB and applies 25k skip-list updates per second — a single beefy rank engine covers the numbers from Section 6.5, with replicas. Say that out loud in the interview, because “one machine is enough” is a **finding**, not a cop-out, and it is backed by arithmetic you did in front of them. But boards grow — new regions, new games, new seasons, and the proud-but-terrifying day when 20 GB stops fitting — so the sharding answer must be ready in your pocket even if you never deploy it. The design is short and the correctness argument is the part worth rehearsing.

Shard by `hash(player_id)`. Each shard owns a complete ordered set of its players — every board exists on every shard, holding exactly that shard’s players. Then:

- **Top-K, exact:** fetch the top K from **every** shard and k-way merge. Correctness: any member of the global top-K has at most K-1 players outranking it globally, hence at most K-1 on its own shard — so it must appear in its shard's top-K. Fetching K per shard is sufficient **and** necessary. Cost: shards × K entries per query — 8 shards × 100 = 800 entries per page, trivial.
- **My rank, exact:** $1 + \sum \text{shards count}(\text{better than me})$ — a scatter-gather count across all shards, $O(\log n)$ per shard with spans, but fan-out per query. Fine at 1.2k rank-reads/s; painful at $10^5/s$.
- **“Top X%” displays, approximate:** each shard also maintains a fixed-boundary histogram of its scores. Histograms are **mergeable by addition** — sum the bucket counts and interpolate (Section 6.13) — so a global percentile is a cheap aggregate with no global ordering at all. This is how “top 0.4%” renders without ever computing 200M ranks.

The first bullet's argument repays a slow reading, because it is the kind of proof interviewers lean on. Suppose some player **is** in the global top 100 but **not** in their own shard's top 100. Then 100 players on their shard outrank them — and those 100 are also globally ahead of them — so at least 100 players outrank them globally, contradicting membership in the global top 100. Therefore every global-top-100 member appears in its own shard's top 100, the merge sees every candidate, and nothing is missed. Necessity is the mirror image: an adversarial score distribution can put a global top-K member at exactly rank K on its shard, so fetching fewer than K per shard can miss one. Sufficient and necessary, one paragraph each — you will reuse this exact argument in Chapter 7's candidate merging and Chapter 12's nearby-driver scatter-gather.

KEY INSIGHT — Exact where it is read, approximate where it is glanced

The podium (top-100) must be exact — it is the product. A player's own rank must be exact — they will screenshot it. The “top 2.3%” badge on a profile is glanced at, not audited — approximate it with mergeable histograms and save the global scatter-gather entirely. Precision is a budget; spend it only on surfaces where users check the math.

6.12 Deep Dive: Windowed and Friends Boards

Two product features remain from the scope list — “this season,” “today,” and “my friends” — and both dissolve into key-naming decisions once the core is right. That is not a coincidence; it is what a well-chosen core abstraction does to feature requests.

Time windows are just keys. A board is identified by (scope, period) — `global:alltime`, `global:weekly:2026-W36`, `global:daily:2026-09-05`. Every score submission writes to **all live windows** for its scope (daily + weekly + all-time = three ordered-set updates, still cheap at our write rates). Rotation is a scheduler flipping the live key: yesterday's daily board stops accepting writes, gets snapshotted to cold storage, and stays readable as a frozen archive. No migration, no reindexing — the empty new window is created lazily by the first write. Contrast this with the alternative you might have sketched naively: one board with timestamps, filtered at query time. That version pays a filter cost on **every read forever** to support a rotation that happens **once a day**; the key-naming version pays three writes instead of one (a constant, tiny) and makes rotation a string flip. When a periodic event meets a hot read path, prefer making the period part of the addressing over making it part of the query — this is Chapter 4's roll-up lesson wearing a different hat.

Friends boards are overlays, not boards. A friend list is at most a few hundred players. On read: fetch each friend's current (score, ts) by direct player lookup (the hash-map side of the ordered set, $O(1)$ each), sort in memory, rank locally. Milliseconds, zero storage, and always consistent with the main board because it **is** the main board's data, viewed through a filter. The alternative — materializing a per-user friends board at write time — multiplies every score update by the player's appearance in friend lists: a Chapter 2-style fanout write amplification, where one match result becomes hundreds of ordered-set updates, all to answer a query Section 6.5 sized at a trickle compared

to podium reads. Compute-on-read wins decisively here — but notice **why** it wins, because the general skill is the deciding, not the answer. The friend set is small (bounded by a few hundred), the overlay query is rare relative to the podium, and the underlying data already lives in a structure with $O(1)$ player lookup. If any of those three flipped — friend sets of millions, friends boards as the primary surface, lookups that require a search — the decision flips too, and you should say so in the room.

6.13 Rust Reference Implementations

Time to make the design concrete. Three pieces, each with deterministic tests you can run in your head as you read: the leaderboard itself (the ordered-set-plus-index pair from Section 6.7), the sharded top-K merge (whose correctness argument you rehearsed in Section 6.11), and the mergeable percentile histogram that powers the “top X%” badges. As in every chapter, the code is not a toy — it is the interview whiteboard answer made executable, and the tests encode the exact claims the prose made.

The leaderboard: ordered set + player index. Read the doc comments carefully — they carry an honesty the interview room rewards. Rust’s standard `BTreeMap` gives you the ordered set with the right interface and the right big-O for everything **except** rank: it keeps no order statistics, so `rank_of` below counts the better half in $O(\text{rank})$ rather than the span skip list’s $O(\log n)$. The code says so in a `NOTE`: rather than pretending otherwise. That is the right trade for a reference implementation — the skip list’s span arithmetic is the **deployable** answer, the `BTreeMap` is the **explainable** one, and the difference is confined to a single method body. Notice too how the tie-break rule from Section 6.7 becomes literally one type alias: `Key = (Reverse<u64>, u64, u64)` — score descending, timestamp ascending, player id last. A total order, so entries never tie, and “first to achieve” is free. The `submit` method is the monotonic-update contract from Section 6.10 written in fifteen lines: strictly-better-scores-only admission, old key removed before the new key is inserted, `false` returned whenever nothing changed — which is precisely what makes retries and duplicate submissions safe by construction.

```
use std::cmp::Reverse;
use std::collections::{BTreeMap, HashMap};

/// Ordering key: score desc, then earliest achieved_at, then player id –
/// a total order, so entries never tie and "first to achieve" is free.
type Key = (Reverse<u64>, u64, u64);

/// In-memory board. `entries` is the ordered set (a stand-in for the
/// span-augmented skip list of Section 6.7 – same interface and big-O,
/// except rank, noted below); `by_player` is the  $O(1)$  side index.
#[derive(Default)]
pub struct Leaderboard {
    entries: BTreeMap<Key, ()>,
    by_player: HashMap<u64, (u64 /*score*/, u64 /*achieved_at*/)>,
}

impl Leaderboard {
    /// Best-score admission: strictly better scores only. Monotonic, so
    /// retries and duplicate submissions are safe by construction.
    pub fn submit(&mut self, player: u64, score: u64, achieved_at: u64) -> bool {
        if let Some(&(old, _)) = self.by_player.get(&player) {
            if score <= old { return false; }
        }
        if let Some(&(old, old_ts)) = self.by_player.get(&player) {
            self.entries.remove(&(Reverse(old), old_ts, player));
        }
        self.entries.insert((Reverse(score), achieved_at, player), ());
        self.by_player.insert(player, (score, achieved_at));
    }
}
```

```

        true
    }

    pub fn top_k(&self, k: usize) -> Vec<(u64, u64)> {
        self.entries
            .keys()
            .take(k)
            .map(|&(Reverse(s), _, id)| (id, s))
            .collect()
    }

    /// 1-based rank. NOTE: Rust's ordered map keeps no order statistics, so
    /// this counts the better half - O(rank). The span skip list answers the
    /// same call in O(log n); only the implementation changes.
    pub fn rank_of(&self, player: u64) -> Option<u64> {
        let &(s, ts) = self.by_player.get(&player)?;
        let better = self.entries.range(..(Reverse(s), ts, player)).count();
        Some(better as u64 + 1)
    }

    /// ±r window around a player: (rank, player, score), edges clamped.
    pub fn neighborhood(&self, player: u64, r: usize) -> Vec<(u64, u64, u64)> {
        let rank = match self.rank_of(player) {
            Some(r) => r as usize,
            None => return vec![],
        };
        let lo = rank.saturating_sub(r + 1); // 0-based start
        let hi = (rank + r).min(self.entries.len()); // exclusive end
        self.entries
            .keys()
            .enumerate()
            .skip(lo)
            .take(hi - lo)
            .map(|(i, &(Reverse(s), _, id))| (i as u64 + 1, id, s))
            .collect()
    }
}

#[cfg(test)]
mod board_tests {
    use super::*;

    #[test]
    fn best_score_ties_and_rank() {
        let mut lb = Leaderboard::default();
        assert!(lb.submit(1, 500, 100)); // alice
        assert!(lb.submit(2, 900, 100)); // carol
        assert!(lb.submit(3, 500, 200)); // bob: same score, later ts
        assert!(lb.submit(4, 100, 100)); // dave
        // board order: 2 (900), 1 (500@100), 3 (500@200), 4 (100)
        assert_eq!(lb.top_k(2), vec![(2, 900), (1, 500)]);
        assert_eq!(lb.rank_of(1), Some(2)); // tie broken by earlier ts
        assert_eq!(lb.rank_of(3), Some(3));
        assert_eq!(lb.rank_of(4), Some(4));
        assert_eq!(lb.rank_of(99), None);
        // monotonic admission: worse or equal scores change nothing
        assert!(!lb.submit(1, 400, 300));
        assert!(!lb.submit(1, 500, 50));
        assert_eq!(lb.rank_of(1), Some(2));
        // improvement repositions
    }
}

```

```

    assert!(lb.submit(1, 950, 400));
    assert_eq!(lb.top_k(2), vec![(1, 950), (2, 900)]);
}

#[test]
fn neighborhood_windows_clamp_at_edges() {
    let mut lb = Leaderboard::default();
    for i in 0..10u64 {
        lb.submit(i, 1000 - i * 10, 1); // id i -> rank i+1
    }
    let w = lb.neighborhood(4, 2); // rank 5 -> ranks 3..=7
    let ranks: Vec<u64> = w.iter().map(|e| e.0).collect();
    let ids: Vec<u64> = w.iter().map(|e| e.1).collect();
    assert_eq!(ranks, vec![3, 4, 5, 6, 7]);
    assert_eq!(ids, vec![2, 3, 4, 5, 6]);
    let w0 = lb.neighborhood(0, 2); // top edge
    assert_eq!(w0.iter().map(|e| e.0).collect():<Vec<_>>(), vec![1, 2, 3]);
    assert!(lb.neighborhood(99, 2).is_empty());
}
}

```

Walk the first test the way you would trace it on the whiteboard, because every assertion is a sentence from earlier sections made executable. Alice (player 1) submits 500, Carol (2) submits 900, Bob (3) submits the **same** 500 but one hundred seconds later; Dave (4) submits 100. The board order asserted by `top_k(2)` — Carol, then Alice — is the tie-break triple doing its quiet work: Alice and Bob have equal scores, and Alice’s earlier timestamp ranks her ahead, which `rank_of(3) == Some(3)` confirms from the other direction. Then the monotonicity block: Alice resubmits a **worse** score and a **duplicate** of her best with an earlier timestamp, both return `false`, and her rank is unmoved — that second case is the tie-break theft from Section 6.10 being refused entry. Finally Alice legitimately improves to 950 and leapfrogs Carol. If you can narrate this test in the room, you have demonstrated best-score admission, total-order tie-breaking, and idempotent writes without saying any of those phrases — and then you get to say the phrases.

The second test guards a boundary that product demos always hit: the neighborhood window near the edges. A player at rank 5 with radius 2 sees ranks 3 through 7 — but the player at rank 1 has no ranks `-1` and `0` to show, so the window clamps to 1, 2, 3 instead of erroring or padding. Clamp, do not crash: the podium’s neighbors view is a UI surface, and UIs should never learn about `saturating_sub` the hard way.

The sharded top-K merge implements Section 6.11’s proof as a heap dance. The classic k-way merge: seed a max-heap with each shard’s current leader, repeatedly pop the global leader, and push that shard’s next entry in its place. Two details carry the correctness. The heap orders by `(score, Reverse(ts))` — the same tie-break rule as the board itself, so a merge of ordered streams stays honest about “first to achieve.” And the function asks each shard for **its** top-k because of the sufficiency argument you proved: any global top-k member has fewer than k entries outranking it on its own shard, so it is always inside its shard’s contribution. The second test constructs the adversarial case on purpose — **all three** global leaders stacked on one shard — and confirms that asking k=3 per shard still recovers the exact global podium. When an interviewer asks “how do you know K per shard is enough?”, you point at that test and retell the proof.

```

use std::cmp::Reverse;
use std::collections::BinaryHeap;

/// One shard's top entries, already in board order (score desc, ts asc).
pub type ShardTop = Vec<(u64 /*player*/, u64 /*score*/, u64 /*ts*/)>;

```

```

/// Global top-k by merging each shard's top-k. Sound because any global
/// top-k member has < k entries outranking it globally, hence < k on its own
/// shard — so it is always inside its shard's top-k contribution.
pub fn merge_top_k(shards: &[ShardTop], k: usize) -> Vec<(u64, u64, u64)> {
    // max-heap on (score, Reverse(ts)): highest score, earliest ts pops first
    let mut heap = BinaryHeap::new();
    for (s, shard) in shards.iter().enumerate() {
        if let Some(&(p, sc, ts)) = shard.first() {
            heap.push((sc, Reverse(ts), p, s, 0usize));
        }
    }
    let mut out = Vec::with_capacity(k);
    while let Some((sc, Reverse(ts), p, s, i)) = heap.pop() {
        out.push((p, sc, ts));
        if out.len() == k { break; }
        if let Some(&(p2, sc2, ts2)) = shards[s].get(i + 1) {
            heap.push((sc2, Reverse(ts2), p2, s, i + 1));
        }
    }
    out
}

#[cfg(test)]
mod merge_tests {
    use super::*;

    #[test]
    fn merge_matches_brute_force_global_sort() {
        let shards: Vec<ShardTop> = vec![
            vec![(1, 900, 5), (4, 300, 5)],
            vec![(2, 950, 5), (5, 250, 5)],
            vec![(6, 500, 1), (3, 500, 5)], // tie at 500: earlier ts first
        ];
        let merged = merge_top_k(&shards, 4);
        let ids: Vec<u64> = merged.iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![2, 1, 6, 3]);

        let mut all: Vec<(u64, u64, u64)> = shards.concat();
        all.sort_by(|a, b| b.1.cmp(&a.1).then(a.2.cmp(&b.2)));
        let oracle: Vec<u64> = all.into_iter().take(4).map(|e| e.0).collect();
        assert_eq!(ids, oracle);
    }

    #[test]
    fn k_per_shard_is_sufficient() {
        // all three global leaders live on one shard; asking only k=3
        // per shard still recovers the exact global top-3
        let shards: Vec<ShardTop> = vec![
            vec![(1, 100, 1), (2, 90, 1), (3, 80, 1), (4, 70, 1)],
            vec![(5, 60, 1)],
        ];
        let per_shard: Vec<ShardTop> =
            shards.iter().map(|s| s.iter().take(3).cloned().collect()).collect();
        let ids: Vec<u64> =
            merge_top_k(&per_shard, 3).iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![1, 2, 3]);
    }
}

```


The first test deserves a nod for its technique: it checks the heap merge against a **brute-force oracle** — concatenate every shard, sort globally, take four — and demands identical output, tie-breaks included. When the clever algorithm and the dumb algorithm agree, you can trust the clever one. That pattern (fast path + oracle in tests) is worth stealing for any system where you replace an obvious implementation with a subtle one.

The **mergeable percentile histogram** is the piece that makes “top 0.4%” badges cheap. The design constraint comes straight from Section 6.11’s insight: the aggregate must be **mergeable by addition**, which forces fixed bucket boundaries chosen up front (log-scaled here — score distributions in games are heavy-tailed, so the buckets stretch the same way). `add` is a `partition_point` binary search into the bounds; `merge` asserts the bounds match and adds bucket-by-bucket; `percentile_of` accumulates whole buckets below the query score and interpolates linearly inside the straddling one. The approximation’s error lives entirely inside that one bucket — which is why log-scaled bounds give you fine resolution at the elite end, where players actually care, and coarse resolution among the masses, where nobody checks.

```
/// Fixed-boundary histogram for approximate "top X%" displays.
/// counts[i] = scores in [bounds[i-1], bounds[i]]; the last bucket catches
/// everything >= the final bound. Histograms merge by adding counts —
/// that is what makes global percentiles cheap across shards.
pub struct ScoreHistogram {
    bounds: Vec<u64>,
    counts: Vec<u64>,
    total: u64,
}

impl ScoreHistogram {
    pub fn new(bounds: Vec<u64>) -> Self {
        assert!(!bounds.is_empty());
        let m = bounds.len();
        ScoreHistogram { bounds, counts: vec![0; m + 1], total: 0 }
    }

    pub fn add(&mut self, score: u64) {
        let i = self.bounds.partition_point(|&b| score >= b);
        self.counts[i] += 1;
        self.total += 1;
    }

    /// Merge another shard's histogram (identical bounds) by addition.
    pub fn merge(&mut self, other: &ScoreHistogram) {
        assert_eq!(self.bounds, other.bounds);
        for (a, b) in self.counts.iter_mut().zip(&other.counts) {
            *a += b;
        }
        self.total += other.total;
    }

    /// Estimated fraction of players scoring below `score`, with linear
    /// interpolation inside the straddling bucket.
    pub fn percentile_of(&self, score: u64) -> f64 {
        if self.total == 0 { return 0.0; }
        let (mut below, mut lower) = (0.0f64, 0u64);
        for (i, &upper) in self.bounds.iter().enumerate() {
            if score >= upper {
                below += self.counts[i] as f64;
                lower = upper;
            } else {

```



```

        if score > lower {
            let frac = (score - lower) as f64 / (upper - lower) as f64;
            below += frac * self.counts[i] as f64;
        }
        return below / self.total as f64;
    }
}
// strictly below `score` (bucket contents are >= the last bound)
if score > *self.bounds.last().unwrap() {
    below += self.counts[self.bounds.len()] as f64;
}
below / self.total as f64
}
}

#[cfg(test)]
mod hist_tests {
    use super::*;

    #[test]
    fn percentile_tracks_uniform_distribution() {
        let mut h =
            ScoreHistogram::new(vec![100, 1_000, 10_000, 100_000, 1_000_000]);
        for i in 0..1000u64 { h.add(i * 1_000); } // 0, 1000, ..., 999_000
        assert_eq!(h.total, 1000);
        assert_eq!(h.percentile_of(0), 0.0);
        let p50 = h.percentile_of(500_000); // interpolates in [100k, 1M)
        assert!((p50 - 0.5).abs() < 0.05, "p50 = {p50}");
        assert_eq!(h.percentile_of(2_000_000), 1.0);
        // "top X%" badge: the 999_500 score is elite
        assert!((1.0 - h.percentile_of(999_500)) * 100.0 < 1.0);
    }

    #[test]
    fn histograms_merge_by_addition() {
        let bounds = vec![100, 1_000];
        let (mut a, mut b) =
            (ScoreHistogram::new(bounds.clone()), ScoreHistogram::new(bounds));
        for s in [0, 50, 500] { a.add(s); }
        for s in [5_000, 9_999] { b.add(s); }
        a.merge(&b);
        assert_eq!(a.total, 5);
        assert!((a.percentile_of(1_000) - 0.6).abs() < 1e-9); // 3 of 5 below
    }
}

```

The tests pin the two properties the architecture actually relies on. `percentile_tracks_uniform_distribution` feeds a perfectly uniform spread of scores and demands the median land within five points of 0.5 — the interpolation doing its job — and then checks the badge math directly: a score of 999 500 must render as “top < 1%,” which is the literal product feature from Section 6.11 expressed as an assertion. `histograms_merge_by_addition` is the sharding enabler: two independent shards, merged by addition, produce the same percentile as if the data had never been split — 3 of 5 below 1 000, exact to 10^{-9} . That assertion is why the global scatter-gather can be skipped for badges: the merge is not approximately associative, it **is** associative, and the only approximation is the interpolation you already accepted.

6.14 Scaling the Platform

The scaling story here is refreshingly undramatic, and you should present it that way: each layer scales on its own axis, and no layer’s mechanism leaks into another’s. Read the table top to bottom and notice that every “mechanism” cell is something you have already justified — the table is a summary of decisions, not a list of new ones.

Layer	Scale axis	Mechanism
Score API	Submission rate	Stateless; horizontal
Ingestion queue	25k/s avg→peak bursts	Partitioned by board+shard key; lag is a metric, not an outage (Chapter 4 pattern)
Rank engine	Board entries × update rate	One node per board-group until RAM or write QPS demands sharding by $\text{hash}(\text{player})$; $O(\log n)$ updates mean a single core does 10^6 ops/s — 40× headroom over peak
Read path	Rank/top-K reads	Replicas + the page cache; the podium page is shared by everyone and changes slowly — a few-second TTL absorbs the viral majority
Storage	200M-entry all-time board	20 GB RAM per replica; snapshots to object storage; journal retention days
Windows	Board count = scopes × live windows	Lazy creation; frozen archives are read-only snapshots — zero live cost

The rank-engine row contains the chapter’s most quietly important number: **40× headroom over peak**. $O(\log n)$ updates mean a single core sustains roughly a million ordered-set operations per second, and the event-day peak is 25 000. This is why “one beefy node with replicas” is an engineering conclusion rather than a laziness — and it is also your cue to say the grown-up sentence: **we shard when the RAM or the write rate forces us to, not because sharding is what scalable architectures do**. Premature distribution is still premature.

KEY INSIGHT — Top-100 churn is glacial compared to mid-table churn

The millionth-best player changes rank with every match; the podium changes a few times an hour. So cache top pages aggressively (they barely move), serve rank/neighborhood live from the ordered set (they move constantly and are queried rarely per position), and never cache mid-table pages — they are wrong the moment they are written. The cache policy mirrors the physics of the data.

That insight is really a **cache-validity** argument, and it generalizes far beyond leaderboards: a cache is only as good as the ratio of read rate to change rate at the exact slice being cached. The podium slice has reads in the millions and changes per hour — cache it. A personal rank has one reader and changes per match — computing it live costs less than invalidating it. Mid-table pages have the worst of both worlds — many potential readers, constant change — which is why the API from Section 6.8 offers `top` and `around` but pointedly no `page 12 470` endpoint. The architecture does not merely tolerate the physics; it **declines to build** the surface the physics would punish.

6.15 Failure Modes & Degradation

The failure table follows the book’s discipline — walk the diagram, kill each box, and ask what the user sees and what the system does about it. Two rows deserve special attention: the journal row, because it is where durability claims go to be tested, and the anti-cheat row, because it is where the chapter’s **integrity beats availability** stance stops being a slogan.

Failure	Symptom	Response
Rank engine node loss	Board reads fail over, writes pause seconds	Promote a replica; it is already warm. Recovery to a fresh node = snapshot + journal replay
Journal gap / corruption	Replay inconsistency on recovery	Checksummed journal frames; on gap, rebuild the board from match-history re-ingest (slow but total) — the board is derived state over score facts
Queue backlog on event day	Freshness degrades past 5 s	Degrade freshness, never integrity: clients show “updating...” affordance; Chapter 3’s limiter sheds anonymous poll traffic first
Replica lag	Stale ranks on some reads	Version the board (update count); reads report the version so clients never go backwards within a session
Hot shard	One shard’s players dominate writes	Reshard by splitting the hash range; top-K merge is shard-count-agnostic, so reads need no coordination
Anti-cheat pipeline down	Suspect scores render	Fail toward quarantine for new suspicious velocity; previously cleared scores unaffected. Integrity beats availability here — Section 6.4
Clock skew in <code>achieved_at</code>	Wrong tie-breaks	Ties are rare and low-stakes; still, trust server receipt time over client clocks, accept <code>achieved_at</code> only within a tolerance window

The journal row hides the deepest idea in this table: **the board is derived state**. The ordered sets in RAM, the snapshots, even the journal itself are all reconstructions of one ground truth — the score facts that game servers submitted. As long as match history exists somewhere (and it does, in the game’s own records), the worst-case recovery is re-ingesting it through the same monotonic pipeline, which converges to exactly the same board. This is the same philosophical move Chapter 4 made with metrics and Chapter 5 with vote tallies, and it is worth internalizing as a pattern: when your hot state is a **function** of an append-only fact stream, no failure of the hot state is permanent. Expensive, slow, embarrassing — yes. Permanent — no.

The clock-skew row is the table’s honest confession. The tie-break rule trusts `achieved_at`, and client clocks lie. The mitigation is deliberately boring — prefer server receipt time, accept client timestamps only within a tolerance window — and the **reason it is acceptable** is the rarity argument: ties require equal scores, equal scores at decision-relevant positions are uncommon, and the cost of a wrong

tie-break is one podium slot's ordering between two equally-scored players. Spend engineering in proportion to blast radius; a fairness edge case affecting one tie in a million gets a tolerance window, not a distributed clock protocol.

6.16 Trade-offs & Alternatives

Every decision this chapter made has a respectable alternative, and the table's "why not" column is written from this workload's specific physics — re-read it as a set of **conditional** statements, not universal laws. The same alternatives win in other buildings; Chapter 11 will happily choose a disk-first database, because its workload's working set does not fit in RAM and its correctness bar is higher.

Decision	Chosen	Alternative & why not (here)
Storage of boards	In-memory ordered sets + journal	Disk-first DB: 10–100× slower rank ops for zero capacity gain — the working set fits in RAM
Rank computation	Structure-maintained (spans)	Rank-by-count SQL: $O(\text{better-entries})$ per query — melts (Section 6.6)
Score semantics	Best-score, monotonic	Every-run: right for arcade boards; same machinery, different key (<code>match_id</code>)
Global top-K	Per-shard top-K + k-way merge	One global sorted set across shards: distributed ordering is a consensus problem nobody needs for a leaderboard
Global percentile	Mergeable histograms	Exact scatter-gather counts: fine at $10^3/s$, wrong tool for every profile badge
Friends board	Computed on read from the friend list	Materialized per-user boards: write fanout per score — Chapters 2/5's fanout lesson
Freshness	≤ 5 s via async apply	Synchronous end-to-end: turns every match-end into a distributed transaction
Live push	Out of scope; polling + short TTL	WebSocket fanout of rank changes: a fine Chapter 1-style add-on, but rank updates at 25k/s make push storms the default state

Two rows repay discussion. **Global top-K** — the rejected alternative, one giant globally-ordered set spanning shards, sounds clean until you notice that maintaining a total order across machines means every insert must coordinate with every machine that might hold a neighbor: you have reinvented distributed consensus to answer a question (who is 47th?) that a 800-entry merge answers in microseconds without any coordination at all. The chapter's recurring pattern shows up again here: **keep the hot shared thing small and local; reconstruct the global view at read time from provably sufficient parts**. And **live push** — the rejected WebSocket fanout — fails on arithmetic, not taste: 25k score updates per second each reorder thousands of mid-table ranks, so a truthful push stream is a 10^8 -events/s firehose of ranks nobody is looking at. Polling with a short TTL shows the user the same podium a few seconds later at a ten-thousandth of the cost. When a feature request fails the physics, the polite word is "add-on."

6.17 Observability & SLOs

Indicator	Definition	Target
Ingest freshness	Score accepted → reflected in rank engine, p95	≤ 5 s
Top-K latency	Page read, cached / live, p95	≤ 10 ms / ≤ 50 ms
Rank latency	rank + neighborhood, p95	≤ 100 ms
Board integrity	Sampled players whose stored slot ≠ submitted best	≈ 0 (journal-rebuildable)
Quarantine latency	Verdict → entries removed from all boards	≤ 60 s
Availability	Reads / writes	99.99% / 99.9%

Read the availability row's asymmetry: four nines for reads, three for writes. That is the chapter's priority order rendered as an SLO — a leaderboard that renders slightly stale is a leaderboard; one that cannot be viewed is a blank screen in a product whose entire job is being viewed. Writes pausing for seconds during an engine failover (Section 6.15's first row) is priced into the three nines and covered by the queue; reads have no such buffer, hence replicas plus the page cache absorbing the viral majority.

Chapter 4's platform carries all of it: ingestion lag as a consumer-lag metric, freshness as a histogram, an alert with a `for` duration on replica version drift. The reconciliation sampler — comparing stored slots against recomputed bests from the journal — is this chapter's version of Chapter 5's tally audit, and the **board integrity** SLO row is its output: a sampled disagreement rate that should sit at approximately zero forever, with the journal guaranteeing that any nonzero reading is repairable by replay. This is the observability pattern worth naming in the room: for every derived structure you operate, run a permanent, sampled comparison against the facts it was derived from. Dashboards tell you the system is busy; the reconciler tells you it is **right**.

6.18 Interview Wrap-Up

Likely follow-ups, and the shape of strong answers:

1. **“Push live rank changes to clients.”** This is a fan-out problem in disguise: 25k updates per second each reorder thousands of ranks — and nobody's **visible** rank actually moved. Push only material changes: podium swaps, personal bests, crossing a friend. Everything else is noise dressed as real-time. The answer demonstrates that you evaluate features with the workload's physics, not with enthusiasm.
2. **“Team leaderboards.”** Team score = aggregate of members (sum, or an ELO-style rating); the ordered set is identical, keyed by team. The interesting part is member-change handling and anti-stack incentives, not the structure — say so, then discuss those.
3. **“Regional boards at 50 regions × 3 windows?”** Boards are cheap keys — 150 extra ordered sets, each small. Writes fan out per applicable board (a player's score lands on their region's set + global); reads unchanged. Section 6.12's windows-as-keys insight pays off verbatim.
4. **“How do you catch cheaters?”** Server-side authority (the game server signs match results; clients never self-report), statistical anomaly detection on score distributions per level (a Chapter 4 alert on percentile outliers), velocity checks (Chapter 3's limiter: matches/hour per player), and quarantine-not-delete so false positives recover. Notice the shape: three earlier chapters each donate one mechanism.
5. **“10⁹ players — what breaks first?”** Single-node RAM (100 GB is still one node, but uncomfortable) and the top-K merge fan-out staying flat — the design already shards; the rank scatter-

gather for exact global rank is what you would approximate next (histograms), keeping exact rank only within a player's region shard. A strong answer names the **first** casualty and the **ordered** list of fallbacks, not a vague “shard more.”

6.19 Summary & Further Reading

NOTE — Chapter summary

A leaderboard is one primitive — rank over a mutating total order — and the chapter is the engineering that protects it. The structure is an ordered set: hash map for player lookup, skip list with spans for $O(\log n)$ rank, top-K, and neighborhoods; ties are a total ordering key `(score, achieved_at, id)`. Writes are monotonic best-score updates — idempotent by construction, journaled for durability, replicated for reads. Reads split by physics: the podium is cached (it barely moves), personal rank is live (it always moves), percentiles are approximated with mergeable histograms (they are glanced at, not audited). Sharding by player hash keeps top-K exact through a k-way merge — any global top-K member is provably inside its shard's top-K. Windows are keys; friends boards are overlays; cheating is a stream consumer with the power to delete. The lesson that transfers: **when the query is positional, the data structure is the architecture.**

Further reading.

- The source video: “17: Top K Leaderboard — Systems Design Interview Questions With Ex-Google SWE” (Jordan has no life): <https://www.youtube.com/watch?v=nQpkRONzEQI>
- William Pugh — “Skip Lists: A Probabilistic Alternative to Balanced Trees” (1990) — the structure of Section 6.7, including span-based ranks.
- Redis sorted sets documentation (`ZADD` / `ZRANK` / `ZRANGE`) — the canonical production embodiment of this chapter's ordered set.
- Cormen et al., *Introduction to Algorithms* — the order-statistics tree chapter (subtree sizes) for the balanced-tree equivalent of spans.
- “Elo rating system” and TrueSkill papers — for the follow-up where the board ranks skill rather than scores.

6.20 Chapter Glossary

Term	Meaning
Best-score semantics	One slot per player per board holding their personal best; worse scores are no-ops
Cursor	Opaque paging token; here literally the ordering key triple <code>(score, achieved_at, id)</code> of the last entry
Every-run semantics	Arcade-style admission where each match is a separate entry and a player may hold many slots
Exact rank	A player's precise 1-based position; required for self-queries, wasteful for global percentiles
Friends board	A leaderboard restricted to a player's friend set; computed as a read-time overlay, not stored
Histogram (mergeable)	Fixed-bucket score distribution; bucket counts add across shards, giving cheap approximate percentiles

Term	Meaning
Idempotent / monotonic update	An update whose repetition changes nothing; best-score _{max} admission has this property for free
Journal (write-ahead)	Append-only record of updates enabling replay after snapshot restore
K-way merge	Merging k-sorted shard outputs with a heap; sound for top-K because K entries per shard always suffice
Leaderboard	An ordering of players by score within a scope (window, region, friends)
Neighborhood query	The $\pm r$ window around a player's rank — needs both rank and ordered iteration
Order-statistics query	A positional query (rank, k-th, count-above); needs an augmented ordered structure, not a plain index
Ordered set	The abstract data type: map from member to score plus rank-ordered iteration; hash map + skip list in practice
Percentile / “top X%”	Approximate positional display computed from merged histograms instead of exact ranks
Quarantine	Anti-cheat state: scores held out of all boards pending verdict; deletable, recoverable
Rank	1-based position in a leaderboard's ordering
Shard sufficiency (top-K)	The theorem that global top-K needs only K entries per shard, since a global top-K member is outranked by $< K$ entries anywhere
Skip list	Probabilistically balanced ordered list with express lanes; $O(\log n)$ search/insert/delete without rotations
Span	Count on a skip-list forward pointer of level-0 steps covered; summing spans along a search path yields rank
Tie-break	Deterministic total order extension: (score desc, <code>achieved_at</code> asc, id) — first to achieve wins
Top-K	The first K entries of a board in order; the podium query
Windowed board	A board scoped to a time period (daily/weekly); rotation is a key change, archives are frozen snapshots

— End of Chapter 6 · Next: Chapter 7, Designing a Recommendation Engine —

Designing a Recommendation Engine

NOTE — Problem source

This chapter solves the problem posed in the talk “22: Recommendation Engine (YouTube, TikTok)” from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the personalized home feed of a video platform — hundreds of millions of users, a billion-item catalog, and 300 milliseconds to decide which twenty items a given user sees next. It is the chapter where the book’s infrastructure (event pipelines, ranking, caching) meets machine learning as a **systems** concern: feature stores, model freshness, and feedback loops. All terms are defined before use; all reference code is Rust with deterministic tests.

7.1 The Problem Statement

The interviewer opens a video app’s home screen — an endless, uncannily well-chosen scroll — and says:

“When a user opens the app, we show them a personalized feed of videos. We know everything they’ve watched, liked, skipped, and how long they lingered. A million new videos arrive every day. Design the system that decides what goes on this screen.”

Notice how different this prompt feels from everything before it, and let that difference guide your whole posture. Every prior chapter had a crisp correct answer to compute: the rate limiter either allows or rejects, the leaderboard has one true ranking, the comment count is a fact. This system computes a **guess**. There is no test that tells you the feed was correct — only a business metric, watched weeks later through aggregated behavior, whispering that the guesses seem to be getting better or worse. You might expect that to make the engineering softer. It does the opposite. Because the output is a guess, the system must be built to **guess well under constraints**: take a billion-item catalog and one user’s entire history, spend no more than 300 milliseconds, and produce an ordering that maximizes a metric nobody can measure directly at request time. The design that emerged across the industry — a **funnel** of candidate generation followed by ranking — is the spine of this chapter, and understanding **why** the funnel has that shape is most of the interview. The rest of the machinery you have already built: the event pipeline is Chapter 4 again, the trending board is Chapter 6 again, the caching debates are Chapter 5 again. What is new is how machine learning enters the design — not as magic, but as a component with a latency budget, a freshness SLO, and a supply chain.

DEFINITION — Recommendation / the feed

A *recommendation* is a predicted-relevance ordering of catalog items for a specific user at a specific moment. The *feed* is its product form: the ranked, paginated, infinite scroll. Unlike search (Chapter 4’s inverted index answers “what matches this query”), the feed answers “what does this user want, having asked nothing” — the query is the user’s history.

DEFINITION — Implicit vs. explicit feedback

Explicit feedback is deliberate: likes, ratings, follows, “not interested”. *Implicit* feedback is behavioral exhaust: impressions (the item was shown), plays, watch time, skips, rewatches. Implicit feedback is 1000× more abundant and is what actually trains the system — watch time and skips are votes every user casts on every item, whether they mean to or not (Chapter 5’s votes, but continuous and uninvited).

Hold the implicit-feedback idea still for a moment, because everything in this chapter drinks from it. When the prompt says “we know everything they’ve watched, liked, skipped, and how long they lingered,” it is describing a **data advantage** that only implicit feedback provides: every user, on every visit, generates dozens of training labels for free. An explicit-feedback-only system (star ratings, say) would see a few labels per user per week and starve. The system’s first job, before any model exists, is therefore **capture** — recording that exhaust faithfully, at scale, without making the product slower. Keep that ordering in mind: telemetry first, model second. It will be reflected in the architecture diagram, where the event log sits at the same rank as the ranker.

7.2 Scope & Clarifying Questions

The prompt is one sentence long and hides a dozen systems. Before drawing anything, find out which dozen. The dialogue below is the one you want to have — each answer deletes an ambiguity that would otherwise surface at the worst possible moment (usually while you are mid-whiteboard on something else):

Candidate asks	Interviewer answers
What surfaces?	Just the home feed. Search, subscriptions page, and ads are separate systems
What do we optimize?	Watch time / session satisfaction — assume a business metric exists; your job is the machinery
Freshness requirements?	A great new video should be discoverable within minutes; user actions should affect their feed within the session
How personalized?	Fully — no two users’ feeds need overlap. But logged-out users get a fallback (trending)
Model training?	Not the focus — data scientists own model internals. You own data flow, features, serving, latency
Content safety?	A policy filter gate exists; respect it, don’t design it
Scale?	500M daily users, 1B public videos, 2B feed loads/day
Cold start?	Must have an answer for new users and new videos

Read the answers as a map of where your effort goes. “What surfaces — just the home feed” deletes search and ads and gives you a single, obsessable latency target. “What do we optimize — assume a business metric exists” is permission to stay an engineer: you will not design loss functions today, but you **will** design the pipeline that delivers labels and features, and you should say so explicitly — that boundary, named out loud, is itself a senior signal. “Model training — not the focus” works the same way in reverse: you own the model’s **supply chain** (data in, features in, versioned models out, rollback when bad) while its internals belong to someone else. And the last two rows quietly hand you the two hardest sub-problems: the scale row gives you numbers that will force the funnel shape (Section 7.6 derives this), and the cold-start row promises the interviewer will ask how a day-old video

or a brand-new user ever gets a fair chance — a question with a structural answer (Section 7.9), not a shrug.

NOTE — Agreed scope

1. **Feed generation:** personalized, ranked, paginated home feed per user.
2. **Feedback loop:** impressions/plays/watch-time/likes recorded and reflected — in-session effects within seconds, model retraining daily.
3. **Candidate diversity:** multiple retrieval channels (follows, similarity, embeddings, trending), blended.
4. **Freshness & cold start:** new items get an exploration quota; new users get trending + rapid personalization from first signals.
5. **Guardrails:** dedup, policy filter, diversity caps in the final mix.
6. Out: search, ads ranking, model architecture design, content moderation itself.

7.3 Functional Requirements

Six requirements, and the first thing to notice is that they come in two voices: three describe what the **user** experiences, and three describe what the **system** must do so those experiences are possible. The vocabulary split that makes the whole chapter legible arrives first, so define it before any requirement uses it:

DEFINITION — Candidate generation (retrieval) vs. ranking (scoring)

The two mandatory stages of any industrial recommender. *Candidate generation* cheaply selects 10^3 – 10^4 plausibly-relevant items from the billion (per-channel: what your follows posted, items similar to what you loved, embedding neighbors, trending). *Ranking* expensively scores those few thousand with the full feature set and orders them. Retrieval is recall-shaped (“don’t miss anything they’d love”); ranking is precision-shaped (“order these twenty slots perfectly”). Neither can do the other’s job at scale — Section 7.6 derives this.

1. **FR-1 — Feed.** `GET /feed` returns the next page of ranked items for the user, with an opaque cursor; the first page is the home screen.
2. **FR-2 — Event intake.** Impressions, plays, watch time, likes, skips, “not interested” recorded durably and at scale.
3. **FR-3 — Session adaptation.** Signals from **this** session (last N watches) influence the next page within seconds.
4. **FR-4 — Channel blend.** Every page mixes sources: followed creators, similar-to-history, embedding neighbors, trending, and an exploration quota for new items.
5. **FR-5 — Negative feedback.** “Not interested” removes the item and down-weights its ilk immediately.
6. **FR-6 — Cold start.** New users receive trending + category picks, personalizing from the very first events; new videos receive a small, guaranteed stream of test impressions.

Walk them once in the order a user meets them. FR-1 is the front door — one endpoint, twenty items, a cursor for the infinite scroll. FR-2 is the bill that pays for everything: every pixel the feed shows generates events, and those events are the only fuel the personalization engine will ever have — lose them and the system goes blind while looking healthy. FR-3 is the one users **feel**: watch two cooking videos and the third page leans culinary. It is also an engineering provocation, because “within seconds” is far faster than any model retraining — the resolution (update **features** in seconds, keep the **model** fixed) is one of the chapter’s core ideas, and Section 7.8 makes it concrete. FR-4 protects the feed from monoculture; FR-5 gives the user a steering wheel; FR-6 is the scope dialogue’s cold-

start promise written as a requirement, and notice it has **two** halves — new users and new videos are different problems with different mechanisms, and conflating them is a common interview stumble.

7.4 Non-Functional Requirements

DEFINITION — Latency budget

The total time a request may consume, allocated across stages. Ours: 300 ms p95 end-to-end for a feed page — 50 ms retrieval fan-out, 150 ms ranking 5k candidates, 50 ms re-rank + assemble, 50 ms network and slack. The budget **dictates the architecture**: anything that cannot fit its allocation must be precomputed offline.

That last sentence is the one to internalize, because it inverts how you design. In CRUD systems you start from the data model and let latency fall where it lands. Here you start from the budget and let it decide what is allowed to exist at request time at all. Anything too slow for its allocation — similarity computations, model training, most of the catalog — does not get optimized; it gets **moved**, into offline jobs whose results are read as tables. You will see this move repeat three times in the next five sections; when an interviewer watches you proactively say “this part cannot run at request time, so it becomes a precomputed table,” you have demonstrated the core instinct of ML systems design.

Requirement	Target & reasoning
Feed latency	p95 $\leq$ 300 ms per page; it is the app’s front door
Event throughput	580k events/s avg, 3M/s peak — Chapter 4’s firehose, with a job to do
Feed freshness	A just-watched video influences the next page $\leq$ 10 s (session features); new model daily
New-item discoverability	Every eligible new video gets its exploration impressions $\leq$ 1 h from upload
Availability	Feed $\geq$ 99.95%; graceful degradation = trending + follows (never an empty screen)
Diversity	No category/creator may dominate a page beyond caps; the feed must not collapse to one topic
Model staleness	Serving model $\leq$ 36 h old; features $\leq$ 10 s (session) / $\leq$ 1 h (rest)

Two rows deserve a second look. **Availability** defines graceful degradation concretely — “trending + follows” is a feed you can assemble with no model at all — which means the availability target survives even a total ML outage; the screen is never empty, merely less clever. And **model staleness** introduces a trio of clocks you will track for the rest of the chapter: the model may be a day old, ordinary features an hour, session features ten seconds. Three freshness tiers, each a deliberate price paid for stability, each eventually becoming an SLO row in Section 7.17.

KEY INSIGHT — Relevance is a budget problem

Scoring one (user, item) pair well is a solved ML problem. The system’s entire shape comes from arithmetic: 2B requests/day $\times$ a 1B-item catalog means you may score, at most, a few thousand items per request. Everything — retrieval channels, approximate nearest neighbors, precomputed similarities, the funnel itself — is a device for spending that tiny scoring allowance on the right few thousand items.

7.5 Back-of-the-Envelope Estimation

Assumptions: 500M daily users, 4 feed sessions each, 20 items viewed per first page; 100 events per user per day; catalog 1B videos, growing 20M/day. As always, the point of these numbers is not their precision — it is discovering which constraints bite and which never will.

Quantity	Estimate	Derivation
Feed requests	2B/day $\approx$ 23k/s avg, 100k/s peak	500M users $\times$ 4 sessions; peaks at regional evenings
Feedback events	50B/day $\approx$ 580k/s avg, 3M/s peak	100 events/user/day: impressions dominate
Candidates per request	5k retrieved $\rightarrow$ 20 shown	Retrieval channels $\times$ quotas; ranking budget
Scoring rate	115M inferences/s avg	23k req/s $\times$ 5k candidates — why ranking runs on inference-optimized hardware
User feature store	500 GB	500M users $\times$ 1 KB dense features
Item feature/embedding store	1.25 TB	1B items $\times$ (1 KB features + 128-dim fp16 embedding)
Training data	10 TB/day	50B events $\times$ 200 B per featurized example
Feed page size	50 KB JSON	20 items $\times$ metadata + pre-view refs

Read the table for its **ratios**, not its absolutes. Events outnumber feed requests 25:1 — every page view generates a page of telemetry, which is why the event pipeline is Chapter 4-sized while the feed API is Chapter 5-sized. The scoring rate is the number that shapes hardware: 115 million inferences per second average is not a service you run on general-purpose CPUs next to the web tier; it is why “the ranker” in the coming architecture is a fleet of inference-optimized machines with the feature store in front of it. And the two store sizes — 500 GB and 1.25 TB — are the comfortable kind of big: shardable key-value territory, point lookups, nothing exotic. The estimation’s real deliverable is the realization below.

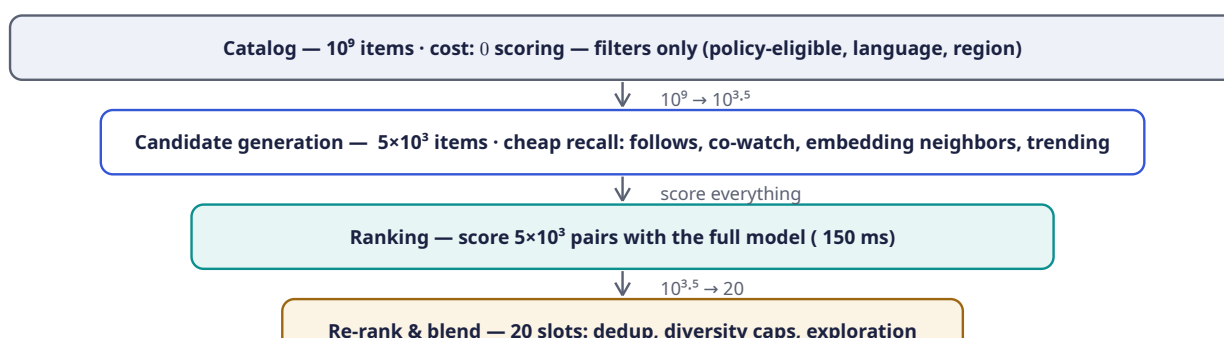
KEY INSIGHT — Three systems wearing one trench coat

The estimation reveals the real architecture: a **telemetry firehose** (580k events/s — Chapter 4 verbatim), a **serving system** (23k requests/s with a 300 ms budget — Chapters 5 and 6 territory), and an **offline factory** (10 TB/day of training data in, one fresh model out daily). The recommendation interview is three familiar interviews stitched by two seams: the feature store and the model store.

Keep that decomposition in view all chapter. When a section feels unfamiliar, ask which of the three systems you are inside — odds are you have built that one before under another name, and the novelty is confined to the seams. The pipeline of the chapter: Section 7.6 derives the funnel; 7.7–7.9 walk its stages (retrieve, rank, re-rank); 7.10–7.12 design APIs, architecture, and the training/serving split; 7.13 implements the core pieces in Rust; 7.14–7.20 scale, harden, and review.

7.6 The Core Challenge: The Funnel

Why must the system be two-staged at all? Resist answering “because that’s how YouTube does it” — derive it from cost arithmetic, in the room, because the derivation **is** the interview pivot. A good ranking model costs 10^6 – 10^7 flops per (user, item) pair. Scoring the whole catalog per request is 10^{15} flops per page view — a data center per user. But **not** scoring is worse: a feed of random items is a dead product. Trapped between “score everything” (impossible) and “score nothing” (worthless), you resolve the dilemma the way systems always resolve impossible budgets: apply cheap relevance filters first and expensive ones last, letting each stage buy the next stage’s right to be expensive:



Read the picture from top to bottom, the way one feed request experiences it — the narrowing bars are the funnel, and each bar’s width is literally how much of the catalog survives that stage. The top bar is the whole **catalog**: 10^9 items, and the only thing that happens here is filtering, not scoring — policy-eligibility, language, region — because filters cost boolean checks while scoring costs model flops. The first arrow ($10^9 \rightarrow 10^{3.5}$) is the miracle hop: **candidate generation** shrinks a billion to roughly five thousand using mechanisms so cheap they are barely computation at all — index lookups, precomputed tables, a nearest-neighbor query — and its job is **recall**: it may surface mediocre items, but it must not miss anything this user would love. The middle bar is **ranking**, the teal heart of the funnel: now and only now does the expensive model run, scoring each of the 5 000 survivors with the full feature set — the arrow label “score everything” is suddenly affordable precisely because “everything” was redefined by the stage above. The bottom bar is **re-rank & blend**: twenty final slots, where score order is adjusted by product rules — dedup, diversity caps, an exploration slot — before the page ships. Two things to say while drawing this. First, each stage is allowed to be expensive **per item** in proportion to how few items reach it: filters at 10^9 , table lookups at $10^{3.5}$, full model at 5×10^3 , hand-rules at

20. Second, the funnel’s stages fail differently: retrieval failures are

silent (a great video never considered looks identical to a great video never existing), while ranking failures are **visible** (the right videos in the wrong order) — which is why Section 7.17 measures candidate coverage as a first-class SLO.

COMMON PITFALL — Ranking the catalog

The naive answer — “score every video for every user” — fails by six orders of magnitude of compute (Section 7.5: it would need 10^{11} inferences/s average). Equally wrong in the other direction: **only** candidate channels with no learned ranking, which can’t order by the actual objective. The funnel is not an optimization; it is the only shape that satisfies both relevance and the latency budget. State this trade explicitly — it is the pivot of the whole interview.

7.7 Deep Dive: Candidate Generation — Recall at 50 ms

Zoom into the second bar. Retrieval is not one clever index; it is several **channels** running in parallel, each a cheap, independent source of a few hundred to a few thousand candidates, each with a quota

in the final mix. The multiplicity is deliberate — no single channel is trusted with the user’s whole attention:

Channel	What it retrieves	How it is served cheaply
Follows	Recent uploads from creators the user follows	Reverse-chronological pull from the subscription index (Chapter 2’s pull fanout — feeds are pulled here, not pushed)
Co-watch similarity	Items watched by people who watched what you watched	Precomputed item→item similarity table (Section 7.13 computes it); lookup by recent history — no runtime model
Embedding neighbors	Items nearest the user’s taste vector	User embedding → approximate nearest-neighbor index; a 10 ms query over 1B vectors
Trending	What’s hot globally/regionally right now	Chapter 6’s leaderboard: time-decayed watch counts, top-K cached per region
Exploration pool	New/underexposed items earning their first data	A reserved quota (Section 7.9); without it new items starve — the rich-get-richer loop

Read the “how it is served cheaply” column slowly, because it is the section’s thesis: **every channel is a lookup**. Not one of them runs a model at request time. The follows channel is Chapter 2’s pull fanout wearing a new hat; the co-watch channel is a precomputed table where yesterday’s batch job did all the mathematics; the embedding channel queries an index that was built offline; trending is literally Chapter 6’s leaderboard; the exploration pool is a queue with a quota. The entire recall stage of the world’s most sophisticated recommendation systems reduces, at request time, to five reads fanned out in parallel and merged — the 50 ms budget is spent on **network fan-out**, not arithmetic. Two definitions make the two learned channels precise:

DEFINITION — Collaborative filtering / co-watch similarity

Collaborative filtering: recommending items via the behavior of **other users** (“users like you liked...”), needing no understanding of content. Its cheapest industrial form is *item-item co-watch similarity*: count how often pairs of items are watched by the same users, normalize into a cosine similarity, and precompute each item’s most-similar list offline. Serving a user is then `union(similar-to(x) for x in recent_history)` — table lookups, no math at request time.

DEFINITION — Embedding / ANN index

An *embedding* is a learned dense vector (e.g. 128 floats) representing a user or item such that **distance encodes taste**: items near a user’s vector are items they’d probably like (matrix factorization / two-tower models produce them). Retrieval asks for the nearest 500 item vectors to the user vector — over 1B items, done with an *approximate nearest neighbor* (ANN) index (graph- or quantization-based), trading 1% recall for 10 ms latency. Exact search would eat the whole budget.

The two channels complement each other in a way worth articulating: co-watch is **behavioral** (it knows what co-occurs but nothing about what anything is), while embeddings are **semantic** (they place items in a space where distance means something, even for items nobody has co-watched yet).

And the ANN trade-off is the first honest approximation of the chapter: you deliberately sacrifice about 1% of recall — some true nearest neighbors are missed — to buy two orders of magnitude of latency. At the retrieval stage that trade is nearly free, because the ranker below will re-score whatever arrives; a missed neighbor only matters if **all five** channels miss it, which is the quiet statistical reason the multi-channel blend is robust: channels fail independently, and relevance survives as long as any one of them remembers.

7.8 Deep Dive: Ranking — Precision at 150 ms

The ranker is the funnel's expensive stage, and your job as the systems engineer in the room is not to design the model — the scope dialogue assigned that to data scientists — but to be an expert in **what the model needs at request time** and where those needs come from. Four feature families feed every scoring call:

- **Item features:** category, duration, language, age since upload, popularity counters, embedding. All precomputed; all point-lookup reads.
- **User features:** historical category affinities, average watch time, creator affinities, embedding. Same story — computed offline or streamed, read at request time.
- **Cross features** (the gold): does this item's category match this user's affinity; similarity between the two embeddings. These are where ranking accuracy concentrates, and they are **computed at request time** from the two lookup results — cheap arithmetic over already-fetched vectors, the only math the ranker is allowed to do that is not the model itself.
- **Context/session features:** time of day, device, and the last N in-session watches — the reason the feed reacts “within the session” (FR-3): these features are updated by the event stream in seconds, **without retraining the model**.

Pause on the session-features row, because it resolves the FR-3 provocation from earlier and the resolution generalizes. “The feed reacts within seconds” sounds like “the model learns within seconds,” and conflating the two leads candidates into the online-learning tarpit (Section 7.16's table buries that idea properly). The trick is that the model is a **function** — `score = f(user, item, context)` — and functions need not be retrained to react; they need fresh **inputs**. Session features are fresh inputs. The model already learned, from yesterday's 50 billion examples, how “just watched two cooking videos” tends to shift watch probabilities; today it merely reads that fact from a fast store. Freshness without retraining — the same separation Chapter 4 used between collection and storage, applied to taste.

DEFINITION — Feature store

The serving-time database of precomputed ML inputs: user features, item features, and embeddings, keyed for point lookup at ms, refreshed by the event stream (fast path) and batch jobs (slow path). It is the seam between the offline factory and online serving — the ranker never computes features from raw events at request time; it **reads** them.

The model outputs a score per candidate — typically a blend like predicted watch time, or $P(\text{click}) \times E[\text{watch} \mid \text{click}]$. The training label comes from yesterday's implicit feedback: an impression with 80% watch-through is a strong positive; a 2-second skip is a strong negative. **The product's metric choice is the model's soul** — optimize raw clicks and you get clickbait; optimize watch time and you get binge; Section 7.17's guardrail metrics exist because the objective is a policy decision with an engineering delivery mechanism. As the engineer you do not pick the objective — but you are expected to **name its failure modes**, because the pipeline you build will amplify whichever objective it is given, faithfully and at planetary scale. A recommender never says “this objective was unwise”; it just quietly optimizes it into a product problem.

7.9 Deep Dive: Re-Ranking — The Last 50 ms

Raw score order is not the feed. If you shipped the ranker's ordering directly, the feed would be a monoculture within days — one binged topic flooding every slot, the same video arriving from three channels, fresh uploads nowhere. The final stage applies product rules to the ranked few hundred, and each rule exists because of a specific, observed failure:

1. **Dedup** — the same video arriving from three channels appears once. (Multi-channel recall guarantees duplicates; Section 7.13 shows the two-line fix and what “first occurrence wins” preserves.)
2. **Diversity caps** — e.g., ≤ 2 per category/creator per page (Section 7.13 implements this exactly). Uncapped, the ranker collapses the feed to whatever topic the user binged last night — the **filter bubble** failure mode.
3. **Freshness boost** — new uploads from followed creators get a bounded bonus; subscriptions must mean something, or users learn that following is decorative.
4. **Exploration slot** — reserve 1 of 20 slots for the exploration pool, the mechanism that keeps the whole learning loop honest:

DEFINITION — Exploration vs. exploitation / multi-armed bandit

Exploitation: show what the model already believes is best. *Exploration*: spend a small quota on items whose value is uncertain, to **learn** — new videos literally cannot be ranked honestly until someone watches them. The *multi-armed bandit* framing formalizes the trade: an ϵ -greedy policy exploits with probability $1-\epsilon$ and explores uniformly with probability ϵ . Without a guaranteed exploration quota, the feedback loop is a closed circle: only shown items get data, only items with data get shown — new creators starve and the catalog fossilizes. Exploration is not charity; it is the system's only source of new information.

Trace the closed-circle sentence until it scares you a little, because it is the single most important dynamics argument in the chapter. A pure-exploitation feed shows items the model already scores highly; those items gather more watches, which raises their scores further; items never shown gather nothing and are never shown. Left alone, the feed converges to yesterday's winners and the catalog's long tail freezes — a system that is locally optimal (each page is “the best known guesses”) and globally suicidal (the supply of future winners dries up). The exploration quota is the deliberate inefficiency that prevents this: one slot in twenty is spent buying **information** rather than satisfaction. And note where the cost lands: the user sees one uncertain item per page — a nearly invisible tax that funds the entire future relevance of the product.

KEY INSIGHT — The feed is controlled by whoever sets the caps

Notice what happened across three sections: the **model** orders candidates, but the **product** owns the final screen — dedup rules, diversity caps, freshness boosts, exploration quota. This is deliberate and healthy: ML optimizes the measurable; humans govern the mixture. In the interview, calling out this separation (and where each lever lives) reads as having operated such a system, not just read about one.

7.10 API Design

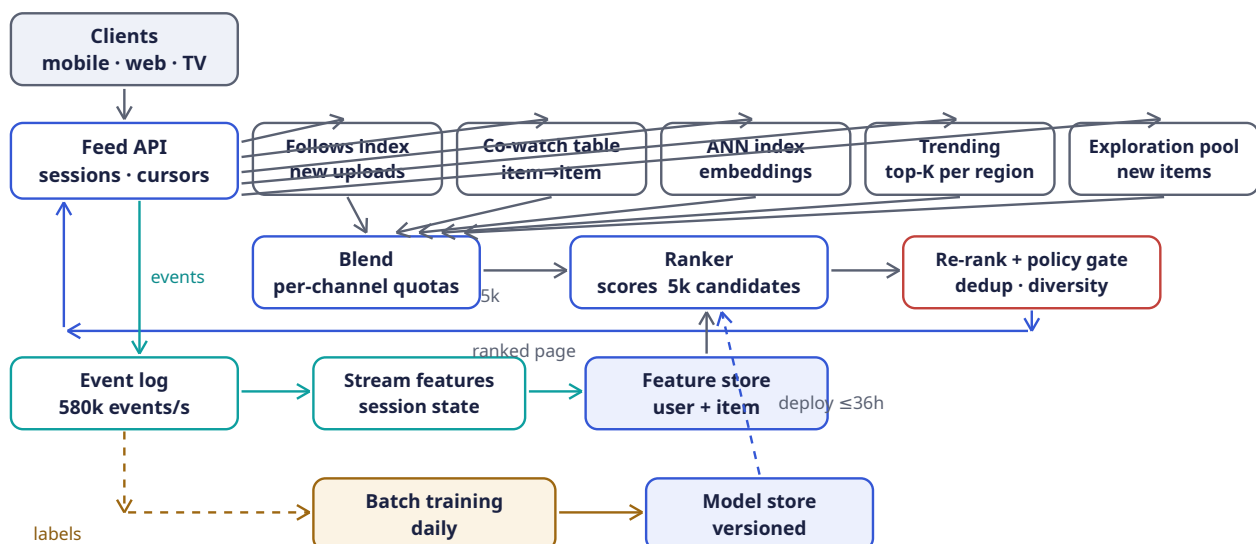
The surface is deliberately small — five endpoints — because the interesting decisions all live behind the feed call. A small API is a feature here: personalization is hard to cache, hard to debug, and hard to reason about, so the contract with the client should be boring.

Method	Path	Purpose
GET	/v1/feed?cursor=&limit=	Next ranked page (FR-1); cursor encodes session + page state, opaque
POST	/v1/events:batch	Impressions, plays, watch-time, likes, skips — compressed batches (FR-2)
POST	/v1/feedback/not-interested	Negative feedback, effective immediately (FR-5)
GET	/v1/trending?region=	Logged-out / cold-start fallback (FR-6); Chapter 6's board, cached
GET	/v1/feed/explain/{item}	"Why am I seeing this?" — the channel that surfaced it (transparency, cheap to add)

Feed responses carry `{items: [{id, rank, channel}], cursor}`. The `channel` tag is returned on purpose, and the reasons stack: it powers the explainability endpoint, it enables per-channel metrics (Section 7.17's coverage SLO reads these tags), and it gives clients diversity hints for free. The cursor deserves its usual respect — it pins the session's candidate set so page two does not reshuffle page one's rejects into view (Section 7.16's "feed consistency" row carries the full argument). Event ingestion is fire-and-forget for the client (202 Accepted), exactly the Chapter 4 contract — the feed must never block on its own telemetry, and the client must never learn whether the event pipeline is healthy; those are the system's problems, not the user's.

7.11 High-Level Architecture

Here is the whole chapter in one picture. Before the tour, orient yourself to its three vertical bands: the **top rows** are the request path (serving, 300 ms budget), the **middle row** is the data path (telemetry and features), and the **bottom row** is the factory (training and model supply). The picture is worth drawing slowly in the interview, because nearly every arrow is a section you have already reasoned through:



Follow a feed request first — the slate and blue arrows. The client (top-left, gray) calls the **Feed API**, the only door and deliberately stateless, so it scales horizontally and any instance can serve any page — cursor contents carry the session state. From the Feed API, five slate arrows fan out to the **candidate channels** drawn in a row — follows index, co-watch table, ANN index, trending, exploration pool — and the fan-out shape is the point: the channels are independent, so they are

queried in parallel, and the stage's latency is the **slowest** channel, not their sum (another reason each channel is kept simple and table-shaped). Five arrows then converge on the **Blend** box, which applies the per-channel quotas — this is where “multi-channel” becomes a mixture rather than a pile — and emits one merged candidate set, labeled “5k”, into the **Ranker**. The ranker's downward arrow from the **feature store** (the blue box below it) is its lifeline: every scoring call reads user features, item features, and embeddings from there at millisecond latency, and the model artifact itself arrives by the dashed blue “deploy $\leq 36\text{h}$ ” arrow from the model store — the ranker is stateless code plus two read-only dependencies, which is what makes it horizontally scalable and rollback-able. From the ranker, scored candidates flow right into the crimson-edged **Re-rank + policy gate** — crimson on purpose, because this is the box where the product's conscience overrides the model's math (dedup, diversity caps, freshness boost, exploration slot, safety filter). Then the response takes the long blue elbow home: down from re-rank, left along the “ranked page” arrow, and up into the Feed API, which assembles the page, stamps the cursor, and answers the client.

Now follow the exhaust — the teal arrows, the chapter's other half. Every impression and watch the client renders also flows down the left edge (“events”) into the **event log** — 580k events/s of durable buffer, Chapter 4's shock absorber for the third time in this book. Right of it, the **stream features** job consumes the log continuously and maintains session state — the “last N watches” that let the feed react within seconds — writing into the **feature store** beside it. Notice the geometry: the feature store sits on row D, **between** the event pipeline and the ranker, touching both — it is drawn as the seam because that is exactly what it is.

Finally, the factory — the amber bottom row. The dashed amber arrows show the slow loop: yesterday's events leave the event log as **labels**, reach **batch training** (daily: join, featurize, train, evaluate), and the graduating model lands in the **model store** — versioned, canaried, rollback-able, because a bad model is a deploy incident treated exactly like bad code. The dashed blue arrow from model store up to the ranker closes the loop: the factory's output becomes serving's input on a $\leq 36\text{h}$ cadence. Step back and look at the whole: the online loop (slate/blue) reacts in seconds through features; the offline loop (amber) reacts in a day through weights. Two loops, two clocks, one feature store keeping them honest — that is the architecture, and every box below exists to serve it.

Component	Responsibility	Why it is shaped this way
Feed API	Session context, cursor state, final assembly	Stateless; the only client-facing door
Candidate channels	Each retrieves 10^2 – 10^3 plausible items its own way	Independent, parallel, replaceable; quotas blend them (Section 7.7)
Ranker	Scores all candidates with the serving model	The expensive stage: reads the feature store, never computes from raw events
Re-rank + policy	Dedup, diversity caps, freshness boost, exploration slot, safety filter	Product rules the model cannot be trusted with (Section 7.9)
Event log	Durable buffer for all feedback	Chapter 4's shock absorber, third time in this book
Stream features	Session state and counters updated in seconds	In-session adaptation (FR-3) without model retraining
Feature store	Point-lookup user/item features + embeddings at ms	The seam between offline factory and online serving

Component	Responsibility	Why it is shaped this way
Batch training	Daily: labels → features → model → evaluation	Section 7.12; model internals are out of scope, its supply chain is not
Model store	Versioned models, canary deploys, rollback	A bad model is a deploy incident, treated exactly like bad code

7.12 Training vs. Serving: The Two-Factory Split

The bottom two rows of the diagram run on different clocks and different engineering cultures, and keeping them honest relative to each other is a full-time discipline. The **offline factory** runs daily: join yesterday's 50B events into labeled examples (impression ↔ watch outcome within an attribution window), compute features, train, evaluate against holdout, and publish a versioned model. The **online factory** serves 23k requests/s with that frozen model plus fresh features. The two factories touch only through versioned artifacts — the feature definitions and the model file — and the discipline that touchpoint must protect has a name:

COMMON PITFALL — Training/serving skew

The subtlest recommender bug: the model trains on features computed one way (batch SQL over the warehouse) and serves on features computed another (stream jobs with different semantics) — e.g., “watches in last 7 days” counted by calendar week offline and rolling 168 h online. The model silently degrades and no dashboard alarms, because **both** computations are internally correct. Defense: one feature **definition** compiled to both paths (a feature store with shared definitions), plus canary evaluation of every new model on live traffic before promotion. If you name one ML systems pitfall in the interview, name this one.

Understand why skew is **worse** than an ordinary bug. Ordinary bugs crash, log, alert. Skew does none of these: training accuracy looks fine, serving latency looks fine, every dashboard is green — while the model quietly misbehaves because the number it reads as “watches in last 7 days” at serving time means something slightly different from what it meant in its training data. The degradation is real but **unattributable**: nothing is broken, two things are merely inconsistent. This is why the defense is structural rather than vigilance-based — you do not ask engineers to be careful; you make one feature definition compile to both paths so the inconsistency cannot be written down. When you hear a team say “the model got worse and we don't know why,” skew is the prime suspect, and the fix is almost always organizational before it is technical.

Freshness policy: the model is ≤ 36 h old (daily train + evaluation + canary); user/item features ≤ 1 h; session features ≤ 10 s. Each tier of staleness is a deliberate price paid for stability — a daily model means a bad Tuesday model hurts for hours, not seconds, and a canary catches most of those before they hurt at all — and each tier becomes an SLO row in Section 7.17, because a freshness promise nobody measures is a hope, not a policy.

7.13 Rust Reference Implementations

Four pieces with deterministic tests, one per funnel idea that had real logic in it: item-item collaborative filtering (the co-watch channel of Section 7.7), the diversity re-ranker (Section 7.9's caps), ϵ -greedy exploration (the bandit that keeps the feedback loop open), and embedding similarity (the ANN channel's exact-math teaching core). As always, the tests are the argument — each one encodes a claim the prose made about how the system behaves.

7.13.1 Item-Item Collaborative Filtering

The first listing is the co-watch channel reduced to its essence. Read the `similarity` function alongside its doc comment: cosine similarity over the **co-watch relation** — two items are similar in proportion to how many watchers they share, normalized by audience size so that a popular item does not look similar to everything merely by being watched by everyone. The production system runs this offline per item and stores the top-similar list; here it runs inline so you can see the math. And note what `recommend` does **not** do: it never surfaces an item the user already saw (`seen.contains` guard) — the recall channel's first duty is to not waste the ranker's budget on reruns.

```
use std::collections::{HashMap, HashSet};

/// The co-watch relation: user -> items watched, item -> watchers.
/// Implicit positives only — a watch is a vote (Chapter 7.1).
#[derive(Default)]
pub struct WatchMatrix {
    pub by_user: HashMap<u64, HashSet<u64>>,
    pub watchers: HashMap<u64, HashSet<u64>>,
}

impl WatchMatrix {
    pub fn from_pairs(pairs: &[(u64, u64)]) -> Self {
        let mut m = WatchMatrix::default();
        for &(u, item) in pairs {
            m.by_user.entry(u).or_default().insert(item);
            m.watchers.entry(item).or_default().insert(u);
        }
        m
    }

    /// Cosine similarity over the co-watch relation:
    ///  $|watchers(a) \cap watchers(b)| / \sqrt{|watchers(a)| * |watchers(b)|}$ .
    /// Precomputed offline per item in production; computed inline here.
    pub fn similarity(&self, a: u64, b: u64) -> f64 {
        let (wa, wb) = match (self.watchers.get(&a), self.watchers.get(&b)) {
            (Some(x), Some(y)) => (x, y),
            _ => return 0.0,
        };
        let (small, big) = if wa.len() <= wb.len() { (wa, wb) } else { (wb, wa) };
        let co = small.iter().filter(|u| big.contains(u)).count() as f64;
        if co == 0.0 { return 0.0; }
        co / (wa.len() as f64 * wb.len() as f64).sqrt()
    }

    /// Recommend unseen items, scored by summed similarity to the user's
    /// whole watch history — the recall channel of Section 7.7.
    pub fn recommend(&self, user: u64, n: usize) -> Vec<(u64, f64)> {
        let seen = self.by_user.get(&user).cloned().unwrap_or_default();
        let mut score: HashMap<u64, f64> = HashMap::new();
        for &item in &seen {
            for &other in self.watchers.keys() {
                if seen.contains(&other) { continue; }
                let s = self.similarity(item, other);
                if s > 0.0 { *score.entry(other).or_default() += s; }
            }
        }
        let mut v: Vec<(u64, f64)> = score.into_iter().collect();
        v.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap().then(a.0.cmp(&b.0)));
        v.truncate(n);
        v
    }
}
```



```

    }
}

#[cfg(test)]
mod cf_tests {
    use super::*;

    fn matrix() -> WatchMatrix {
        // users 1,2,3 ; items 10,20,30,40
        WatchMatrix::from_pairs(&[
            (1, 10), (1, 20),
            (2, 10), (2, 20), (2, 30),
            (3, 30), (3, 40),
        ])
    }

    #[test]
    fn similarity_counts_co_watchers() {
        let m = matrix();
        assert_eq!(m.similarity(10, 20), 1.0); // identical audiences
        assert_eq!(m.similarity(10, 30), 0.5); // 1 shared of 2 & 2
        assert!((m.similarity(30, 40) - 0.7071).abs() < 1e-3); // 1/sqrt(2)
        assert_eq!(m.similarity(10, 40), 0.0); // no shared watchers
    }

    #[test]
    fn recommendations_exclude_seen_and_rank_by_evidence() {
        let m = matrix();
        let recs = m.recommend(1, 5);
        // user 1 watched {10, 20}; item 30 scores sim(10,30)+sim(20,30) = 1.0,
        // item 40 scores 0 and is not surfaced
        assert_eq!(recs, vec![(30, 1.0)]);
        assert!(!recs.iter().any(|&(id, _)| id == 10 || id == 20));
    }
}

```

Walk the tiny fixture, because its six watches tell a complete recommendation story. Users 1 and 2 both watched items 10 and 20 — so those two items have **identical audiences** and similarity 1.0, the test's first assertion. Item 30 shares exactly one watcher (user 2) with item 10 out of audiences of two and two, giving $1/4 = 0.5$; items 30 and 40 share user 3 out of audiences of two and one, giving $1/2 \approx 0.7071$. Now ask for recommendations for user 1, who watched {10, 20}: item 30 is similar to both of those ($0.5 + 0.5 = 1.0$ of summed evidence), item 40 is similar to neither, and items 10 and 20 themselves are excluded as already-seen. The test asserts the entire outcome in one line: `recs == vec![(30, 1.0)]`. Six watches in, one principled recommendation out — you have just watched collaborative filtering work with no model, no training, and no content understanding, which is precisely why it is the industry's cheapest recall channel.

7.13.2 The Re-Ranker: Dedup + Diversity Quotas

The second listing is Section 7.9's product rules as executable policy. `dedup` is two lines and one decision — **first occurrence wins** — which, applied to score-ordered input, means the same video arriving from three channels keeps its best score and appears once. `rerank_with_quota` is the diversity cap, and its **two-pass** structure is the detail to narrate: pass one respects the cap strictly (at most `per_category` items per category), but anything skipped goes to a `leftover` list, and pass two fills any remaining page slots from it in score order — **so the page is never short**. A cap that could starve the page would trade a filter bubble for an empty screen; the two-pass design caps dominance without ever failing to serve.


```

use std::collections::{HashMap, HashSet};

#[derive(Debug, Clone)]
pub struct Candidate {
    pub id: u64,
    pub category: &'static str,
    pub score: f64,
}

/// Same item arriving from several channels: keep its best score, once.
pub fn dedup(ranked: Vec<Candidate>) -> Vec<Candidate> {
    let mut seen = HashSet::new();
    ranked.into_iter().filter(|c| seen.insert(c.id)).collect()
}

/// Take a page of n from score order, allowing at most `per_category`
/// items per category – pass 1 respects the cap, pass 2 fills any
/// remaining slots in score order so the page is never short.
pub fn rerank_with_quota(
    mut ranked: Vec<Candidate>,
    n: usize,
    per_category: usize,
) -> Vec<u64> {
    ranked.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap().then(a.id.cmp(&b.id)));
    let mut out = Vec::with_capacity(n);
    let mut counts: HashMap<&str, usize> = HashMap::new();
    let mut leftover = Vec::new();
    for c in ranked {
        let cnt = counts.entry(c.category).or_insert(0);
        if *cnt < per_category && out.len() < n {
            out.push(c.id);
            *cnt += 1;
        } else {
            leftover.push(c.id);
        }
    }
    for id in leftover {
        if out.len() < n { out.push(id); }
    }
    out
}

#[cfg(test)]
mod rerank_tests {
    use super::*;

    fn c(id: u64, cat: &'static str, score: f64) -> Candidate {
        Candidate { id, category: cat, score }
    }

    #[test]
    fn quota_caps_a_dominant_category_but_fills_the_page() {
        let ranked = vec![
            c(1, "sport", 9.0), c(2, "sport", 8.5), c(3, "sport", 8.0),
            c(4, "music", 7.5), c(5, "news", 7.0),
        ];
        // cap 2 per category, page of 5
        assert_eq!(rerank_with_quota(ranked, 5, 2), vec![1, 2, 4, 5, 3]);
    }
}

```

```
#[test]
fn dedup_keeps_first_best_occurrence() {
    let dup = vec![c(1, "sport", 9.0), c(1, "sport", 7.0), c(2, "news", 8.0)];
    let unique = dedup(dup);
    assert_eq!(unique.len(), 2);
    assert_eq!(unique[0].score, 9.0); // first occurrence wins
}
}
```

The quota test encodes the filter-bubble scenario in miniature: the ranker, left alone, would have served three sport videos at the top (scores 9.0, 8.5, 8.0 — it learned the user binged sport last night). The cap of two admits items 1 and 2, skips item 3 into `leftover`, admits music (4) and news (5) — and only then, with four slots filled and the page one short, reaches back into the leftover pile for item 3. Final page: `[1, 2, 4, 5, 3]`. Read that vector as the product compromise made literal: the model's conviction is honored (the two best sports lead), the user's evening is varied (music and news interrupt the binge), and the page is complete (the third sport returns as filler rather than vanishing). Governance without amnesia — the caps rule the top of the page, not the whole of it.

7.13.3 ϵ -Greedy Exploration (Multi-Armed Bandit)

The third listing is the exploration slot's brain. A **multi-armed bandit** (Section 7.9's definition) is the formal version of a question the feed asks twenty times per page: which uncertain option deserves a trial? The `EpsilonGreedy` policy answers with the simplest possible honesty — with probability $1-\epsilon$ pull the arm with the best observed mean (exploit), with probability ϵ pull a uniformly random arm (explore). Two implementation details deserve your attention. The `pull` method begins with an **initialization pass** — every arm is tried once before any policy applies, because an arm with zero observations has no mean to exploit; this is the code-level form of “new items get their guaranteed first impressions.” And `record` updates the mean **incrementally** — `mean += (reward - mean)/n` — so the bandit's memory is $O(1)$ per arm, no reward history retained; at 580k events/s, that is not a nicety but a necessity. The tiny `XorShift` generator exists purely so the tests are reproducible — a deterministic stream of “randomness” that lets assertions be exact.

```
/// Tiny deterministic RNG (xorshift64) so tests are reproducible.
pub struct XorShift(pub u64);
impl XorShift {
    pub fn next(&mut self) -> u64 {
        let mut x = self.0;
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
        self.0 = x;
        x
    }
    pub fn f64(&mut self) -> f64 { (self.next() >> 11) as f64 / (1u64 << 53) as f64 }
    pub fn below(&mut self, n: u64) -> u64 { self.next() % n }
}

///  $\epsilon$ -greedy: exploit the best-estimate arm, explore uniformly with
/// probability  $\epsilon$ . Arms = channels or new-item pools earning data.
pub struct EpsilonGreedy {
    pub means: Vec<f64>,
    pub counts: Vec<u64>,
    pub epsilon: f64,
    rng: XorShift,
}

impl EpsilonGreedy {
    pub fn new(arms: usize, epsilon: f64, seed: u64) -> Self {
        EpsilonGreedy {

```

```

        means: vec![0.0; arms],
        counts: vec![0; arms],
        epsilon,
        rng: XorShift(seed.max(1)),
    }
}

pub fn pull(&mut self) -> usize {
    // initialize: every arm once, in order
    if let Some(i) = self.counts.iter().position(|&c| c == 0) { return i; }
    if self.rng.f64() < self.epsilon {
        self.rng.below(self.means.len() as u64) as usize
    } else {
        self.means
            .iter()
            .enumerate()
            .max_by(|a, b| a.1.partial_cmp(b.1).unwrap())
            .map(|(i, _)| i)
            .unwrap()
    }
}

/// Incremental mean update – the bandit learns from implicit feedback.
pub fn record(&mut self, arm: usize, reward: f64) {
    self.counts[arm] += 1;
    let n = self.counts[arm] as f64;
    self.means[arm] += (reward - self.means[arm]) / n;
}

#[cfg(test)]
mod bandit_tests {
    use super::*;

    #[test]
    fn zero_epsilon_exploits_after_initialization() {
        let true_means = [0.1, 0.5, 0.9];
        let mut b = EpsilonGreedy::new(3, 0.0, 42);
        let mut counts = [0u64; 3];
        for _ in 0..103 {
            let a = b.pull();
            counts[a] += 1;
            b.record(a, true_means[a]);
        }
        assert_eq!(counts, [1, 1, 101]); // 3 init pulls, then all arm 2
    }

    #[test]
    fn pure_exploration_spreads_uniformly() {
        let true_means = [0.1, 0.5, 0.9];
        let mut b = EpsilonGreedy::new(3, 1.0, 88_172_645_463_325_252);
        let mut counts = [0u64; 3];
        for _ in 0..303 {
            let a = b.pull();
            counts[a] += 1;
            b.record(a, true_means[a]);
        }
        assert_eq!(counts.iter().sum::<u64>(), 303);
        for &c in &counts {
            assert!((60..=140).contains(&c), "counts = {counts:?}");
        }
    }
}

```

```

    }
}

#[test]
fn running_mean_converges() {
    let mut b = EpsilonGreedy::new(1, 0.0, 1);
    b.record(0, 1.0);
    b.record(0, 0.0);
    assert_eq!(b.means[0], 0.5);
}
}

```

The three tests bracket the policy's behavioral envelope, and each maps to a feed regime. `zero_epsilon_exploits_after_initialization`: with $\epsilon = 0$, after the three mandatory initialization pulls the bandit locks onto arm 2 (true mean 0.9) forever — `[1, 1, 101]` — which is exactly the rich-get-richer fossilization Section 7.9 warned about, here demonstrated as a **correct** consequence of a pure-exploitation policy. `pure_exploration_spreads_uniformly`: with $\epsilon = 1$, 303 pulls spread across the arms within a loose uniform band (60–140 each) — maximum learning, zero exploitation, the catalog-surveying extreme. Production lives between the two, at $\epsilon \approx 0.05$: mostly the best-known feed, a reserved trickle of trials. And `running_mean_converge` pins the incremental update's arithmetic so the $O(1)$ memory claim is tested, not merely asserted.

7.13.4 Embedding Similarity: Cosine Top-K

The last listing is the semantic channel's kernel. **Cosine similarity** measures the angle between two taste vectors, ignoring their lengths — and the test shows why that choice matters: an item vector five times longer than the query but pointing the same direction scores a perfect 1.0, while a nearly-aligned one scores 0.9939 and an orthogonal one 0.0. **Direction beats magnitude** is exactly right for taste: a video watched ten times more often is not ten times more **relevant**; it is the same direction of appeal, louder. The doc comment on `top_k_similar` carries the chapter's honesty habit: this is $O(n \cdot d)$ — fine for a test, absurd over 10^9 items — and production swaps in the ANN index of Section 7.7 with an identical interface and 99% of the recall. The swap is the interview answer; the exact version is the understanding that lets you defend the swap.

```

/// Cosine similarity between taste vectors; distance encodes preference.
pub fn cosine(a: &[f64], b: &[f64]) -> f64 {
    let dot: f64 = a.iter().zip(b).map(|(x, y)| x * y).sum();
    let na = a.iter().map(|x| x * x).sum::<f64>().sqrt();
    let nb = b.iter().map(|x| x * x).sum::<f64>().sqrt();
    if na == 0.0 || nb == 0.0 { 0.0 } else { dot / (na * nb) }
}

/// Exact nearest neighbors for a user vector. Honest scope note: this is
///  $O(n \cdot d)$  — the teaching core. Production swaps in an ANN index
/// (Section 7.7) with identical interface and ~99% of the recall.
pub fn top_k_similar(
    query: &[f64],
    items: &[(u64, Vec<f64>)],
    k: usize,
) -> Vec<(u64, f64)> {
    let mut scored: Vec<(u64, f64)> =
        items.iter().map(|(id, v)| (*id, cosine(query, v))).collect();
    scored.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap().then(a.0.cmp(&b.0)));
    scored.truncate(k);
    scored
}

```

```
#[cfg(test)]
mod ann_tests {
    use super::*;

    #[test]
    fn nearest_neighbors_by_direction_not_magnitude() {
        let q = vec![1.0, 0.0, 0.0];
        let items = vec![
            (1, vec![0.9, 0.1, 0.0]), // nearly aligned
            (2, vec![0.0, 1.0, 0.0]), // orthogonal
            (3, vec![5.0, 0.0, 0.0]), // identical direction, 5x magnitude
        ];
        let top2 = top_k_similar(&q, &items, 2);
        let ids: Vec<u64> = top2.iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![3, 1]); // direction beats magnitude
        assert_eq!(top2[0].1, 1.0); // perfectly aligned
        assert!((top2[1].1 - 0.9939).abs() < 1e-3);
        assert_eq!(cosine(&q, &items[1].1), 0.0);
    }
}
```

7.14 Scaling the Platform

Each layer of the funnel scales on its own axis, and the table's right column keeps naming earlier chapters — that repetition is the point. A recommender is an **integration** of systems this book has already scaled; the new work is confined to the seams.

Layer	Scale axis	Mechanism
Feed API / ranker	23k req/s avg	Stateless services, horizontal; ranking replicas scale with scoring rate (115M inferences/s avg) on inference-optimized hardware
Candidate channels	Catalog size	Each channel scales independently: follows index (Chapter 2 pull), pre-computed co-watch table (offline), ANN shards (embedding space partitioned), trending cache (Chapter 6)
Event pipeline	580k events/s avg, 3M/s peak	Chapter 4 verbatim: partitioned durable log, stream processors for session features, batch for training
Feature store	500 GB user + 1.25 TB item	Sharded key-value with point lookups; read-heavy, so replicas carry the load
Training	10 TB/day examples	Data-parallel GPU/TPU fleet; the daily cadence bounds both cost and staleness
Feed cache	Logged-out / cold-start	Trending pages are shared and cached hard; personalized pages

Layer	Scale axis	Mechanism
		are never shared — cache the candidates , not the feed

The last row states a rule worth generalizing, and the insight below states it sharply. Note the candidate-channels row too: **each channel scales independently** is not a convenience but a consequence of the blend architecture — because quotas, not internals, define each channel's contribution, you can shard the ANN index this quarter and rebuild the co-watch pipeline next quarter without the feed ever noticing. Independent scaling is what the blend buys operationally, on top of what it buys in relevance.

KEY INSIGHT — Personalization is cache-hostile by definition

Chapter 5 cached windows shared by millions; here **every response is unique**. The caching strategy inverts: cache the shared ingredients (trending boards, co-watch lists, item features, embeddings) and assemble the personalized result fresh. A recommender that caches feeds has accidentally built trending-with-extra-latency.

7.15 Failure Modes & Degradation

Walk the diagram and break things, as always — but notice this system's failures come in two flavors with different shapes: **infrastructure** failures (pipeline lag, channel outages, store staleness), which look like every other chapter's, and **model** failures (bad deploys, feedback spirals), which fail in **quality space** — nothing is down, latency is fine, and the product is quietly worse. The table covers both, and the responses to the second kind are process machinery (canaries, guardrails, rollbacks), not just engineering ones.

Failure	Symptom	Response
Event pipeline lag	Session features stale; feed feels unreactive	Degrade gracefully to non-session features; lag alerts (Chapter 4); catch-up replay
One candidate channel down	Feed still full, less varied	Quotas redistribute to surviving channels; the blend is designed for exactly this — no single point of taste
Ranker overload	Latency budget breached	Score fewer candidates (5k → 1k): relevance drops imperceptibly before latency does; then trending fallback
Bad model deploy	Feed quality craters at scale	Canary + business-metric guardrails on the deploy pipeline; one-click rollback to previous model version — model incidents are deploy incidents
Feature store stale	Model serves on old affinities	Staleness SLO with alerts; beyond threshold, ranker down-weights stale features

Failure	Symptom	Response
Training pipeline fails	Model ages past 36 h	Yesterday's model keeps serving; freshness alert pages; investigate like any failed batch job
Feedback loop spiral	One topic floods a user's feed	Diversity caps bound the damage structurally; guardrail metrics (Section 7.17) detect drift
Trending fallback poisoned	Manipulated trending board	Chapter 6's integrity tools apply: velocity anomalies, quarantine feed

Two rows deserve a second read. **One candidate channel down** is the blend architecture justifying itself in reverse: because quotas are soft and channels independent, losing the ANN index for an hour means the feed gets slightly less semantic and slightly more social — users notice nothing, the coverage SLO does. Design the mixture so that any single ingredient can fail and the meal survives; “no single point of taste” is the recommender’s version of “no single point of failure.” And **ranker overload** shows the budget’s graceful-degradation ladder: the first response to latency pressure is not adding machines but **scoring fewer candidates** — 5k to 1k — because the funnel’s shape means relevance degrades logarithmically while latency degrades linearly. You are allowed to buy time with recall at the margin; that dial exists only because retrieval over-fetches in the first place.

7.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Architecture	Funnel: retrieve → rank → re-rank	Single-stage scoring of the catalog: 6 orders of magnitude over compute budget (Section 7.6); retrieval-only: no objective-driven order
Retrieval	Multi-channel blend with quotas	One giant embedding index alone: best average relevance, worst failure modes — no guarantee follows or fresh uploads ever surface
Model freshness	Daily retrain + real-time session features	Online learning (update weights per event): fresher, but a feedback-loop amplifier and an operational nightmare to roll back
Objective	Watch time with guardrail metrics	Pure CTR: optimizes clickbait; pure satisfaction surveys: too sparse to train on
Exploration	Guaranteed quota (ϵ -greedy slot)	Pure exploitation: the rich-get-richer loop fossilizes the catalog; Thompson sampling: better theory, same quota slot in practice

Decision	Chosen	Alternative & why not (here)
Feed consistency	Cursor paginates over a snapshot	Re-rank every page live: infinite scroll jumps and repeats; the session cursor pins the candidate set
Cold start	Trending + category picks, then rapid session adaptation	Signup interest quizzes: fine complement, cannot be the only path (friction, stale answers)

The **model freshness** row is the one interviewers most often push on, so have the argument ready: online learning (updating weights per event) is fresher by four orders of magnitude, and it is still rejected — because a model that changes continuously is a model you cannot roll back (which yesterday’s weights do you return to?), a feedback-loop amplifier (a bad update trains on the behavior the bad update caused), and a distributed-systems nightmare (which replica’s gradient wins?). The chapter’s chosen split — frozen daily model + real-time **features** — buys nearly all the practical freshness (the feed reacts in seconds, Section 7.8) while keeping the artifact under version control. Freshness is a spectrum; put the **weights** at the stable end and the **inputs** at the volatile end. The **feed consistency** row is its mirror image: pin the candidate set at session start (cursor = snapshot) so infinite scroll never reshuffles beneath the user’s thumb — in a system whose entire purpose is reacting to the user, the scroll itself must be the one thing that holds still.

7.17 Observability & SLOs

Two metric families — system health (Chapter 4’s bread and butter) and **quality** health, which only this kind of system has. The second family is the novelty: a recommender can be perfectly available, perfectly fast, and perfectly terrible, and only quality metrics can tell you.

Indicator	Definition	Target
Feed latency	Page generation p95	≤ 300 ms
Event lag	Event $\rightarrow$ session feature visible	≤ 10 s
Model freshness	Age of serving model	≤ 36 h, alert past
Candidate coverage	Pages served with all channels contributing	$\geq 99\%$
CTR / watch time	Clicks and minutes per impression, per model version	Guardrail: never ship a regression
Diversity index	Distinct categories/creators per page	$\geq$ policy floor
Exploration health	Share of new items reaching N impressions in 1 h	$\geq 95\%$
Negative-feedback rate	“Not interested” per impression	Watch trend, alert on spikes

Note how the table’s rows trace the chapter’s anatomy: candidate coverage watches the blend (a silently dead channel shows here first), exploration health watches the quota (the feedback loop’s open door, measured), model freshness watches the factory. Every structural claim made earlier has a row that would catch its violation — that correspondence between design and dashboard is what “operable” means for an ML system.

KEY INSIGHT — Guardrail metrics are the product's conscience

Every optimization target can be gamed by the optimizer itself: CTR invites clickbait, watch time invites doomscrolling. Production recommenders therefore ship **pairs** of metrics — the objective and the guardrails (diversity, negative feedback, long-term retention) — and a model that improves the objective while moving a guardrail does not ship. Encoding this in the deploy pipeline is systems work, and it is the difference between a recommender and a slot machine.

7.18 Interview Wrap-Up

Likely follow-ups, and the shape of strong answers:

1. **“Make it real-time: the feed reacts to every watch instantly.”** Already half-built: session features update in seconds. The next step — updating the **user embedding** mid-session — is streaming inference, not training: recompute the user vector from the new history with the frozen item tower. True online weight updates trade the rollback story for freshness; say what you'd be giving up (Section 7.16's freshness row, out loud).
2. **“A new creator with zero viewers — how do they ever surface?”** The exploration quota (Section 7.9) plus **content-based** features: with no watch history, the video's own metadata (category, title embeddings, thumbnail features) place it near known items. First impressions go to users whose taste vector is near the **content** vector — cold start is why content features exist at all.
3. **“TikTok vs. YouTube — what differs in the system?”** The graph: YouTube leans on subscriptions (the follows channel is strong); TikTok is graph-free — nearly pure engagement ranking, which makes its exploration quota and per-session adaptation much larger shares of the mix. Same funnel, different quotas. **Architecture is the constants** — a sentence worth saying verbatim, because it reframes a product comparison as a parameter comparison.
4. **“Where do ads fit?”** A second ranking system with its own objective (expected revenue $\times$ relevance), blended into slots by an auction — and the reason feed slots are strictly partitioned between organic and sponsored before re-ranking. Two objectives never share one scoring pass; they share a layout.
5. **“How would you detect the model amplifying harmful content?”** Guardrail metrics per content class, policy-gate recalls, and audit sampling of what the exploration pool promotes — plus the honest answer: ranking amplifies whatever the objective correlates with, so this is fought at the objective-and-guardrail layer, not with filters alone.

7.19 Summary & Further Reading

NOTE — Chapter summary

A recommender is three systems in a trench coat: a telemetry firehose (Chapter 4), a latency-bound serving funnel, and a daily model factory. The funnel exists because of arithmetic: you may score 5k of 10^9 items per request, so cheap recall channels (follows, co-watch, embedding ANN, trending, exploration) feed one expensive ranker; and a re-ranker applies the product's conscience — dedup, diversity caps, freshness, policy. Implicit feedback is the fuel; the feature store is the seam that keeps training and serving honest (skew is the killer bug); exploration is the quota that keeps the feedback loop open; guardrail metrics decide what ships. The lesson that transfers: **in ML systems, the model is a component — the system around it is the design.**

Further reading.

- The source video: “22: Recommendation Engine (YouTube, TikTok) — Systems Design Interview Questions With Ex-Google SWE” (Jordan has no life): <https://www.youtube.com/watch?v=QrZTmiZSRcw>
- Covington, Adams, Sargin — “Deep Neural Networks for YouTube Recommendations” (2016) — the canonical two-stage funnel paper this chapter’s architecture follows.
- Koren, Bell, Volinsky — “Matrix Factorization Techniques for Recommender Systems” (2009) — embeddings before they were called embeddings.
- Sutton & Barto, *Reinforcement Learning* — the multi-armed bandit chapters behind Section 7.9’s exploration quota.
- Chapter 4’s monitoring reading applies twice here — guardrail metrics are SLOs with a conscience.

7.20 Chapter Glossary

Term	Meaning
A/B test	Controlled experiment over model/parameter versions; how guardrail and objective metrics earn their keep
ANN index	Approximate nearest-neighbor structure over embeddings; 1% recall traded for 10 ms latency over 1B vectors
Candidate generation	The recall stage: cheap channels selecting 10^3 plausible items from the catalog
Clickbait / CTR trap	The failure of optimizing clicks alone: the model learns to promise, not to satisfy
Cold start	Serving users or items with no history: trending/category fallbacks for users, content features + exploration for items
Collaborative filtering	Recommendation from other users’ behavior, no content understanding required
Co-watch similarity	Item-item cosine over shared watchers; precomputed offline, looked up at serve time
Cross feature	A feature combining user and item (e.g., category $\times$ affinity); where ranking accuracy concentrates
Diversity cap	Re-rank rule bounding any category/creator per page; structural filter-bubble defense
Embedding	Learned dense vector for a user or item; distance encodes taste
ϵ -greedy	Bandit policy exploiting the best arm with probability $1-\epsilon$, exploring uniformly otherwise
Exploration quota	Reserved feed slots for uncertain items; the only source of new information in the loop
Feature store	Serving-time database of precomputed user/item features with shared offline/online definitions
Feedback loop (rich-get-richer)	Shown items get data, data earns impressions; without exploration the catalog fossilizes

Term	Meaning
Filter bubble	Feed collapse to one topic under pure score order; fought with diversity caps
Funnel	The retrieve → rank → re-rank shape; a compute-budget necessity, not an optimization
Guardrail metric	A metric that may not regress even when the objective improves; ships alongside the objective
Implicit feedback	Behavioral signals (watch time, skips, impressions) used as training labels
Latency budget	300 ms p95 allocated across retrieval, ranking, re-ranking; dictates what must be precomputed
Matrix factorization / two-tower	Model families producing user and item embeddings from interaction history
Model store	Versioned model registry with canary deploys and rollback; models are deploys
Multi-armed bandit	The exploration/exploitation formalism: arms are channels or items, rewards are engagement
Ranking	The precision stage: scoring retrieval's candidates with the full model under 150 ms
Re-ranking	The product-rules stage: dedup, diversity, freshness, policy, exploration slot
Session features	In-session signals (last N watches) updated in seconds; feed reactivity without retraining
Training/serving skew	Feature definitions diverging between offline training and online serving; the silent killer of recommenders
Trending	Time-decayed popularity board per region — Chapter 6's leaderboard reused; the cold-start and outage fallback

— End of Chapter 7 · Next: Chapter 8, Designing a Database Index: How B-Trees Work —

Designing a Database Index: How B-Trees Work

NOTE — Problem source

This chapter solves the problem posed in the talk “How do B-Tree Indexes work?” from the series *Systems Design Interview: 0 to 1 with Google Software Engineer* (channel: *Jordan has no life*). Unlike the product chapters, this is a **fundamentals** deep dive — the interviewer asks you to design the index structure of a relational database’s storage engine: something that answers point lookups and range scans in milliseconds over billions of rows, survives a write-heavy workload, and lives on hardware whose physics you must respect. It is also the keystone chapter: Chapters 5 and 6 stored data in “the database”; this chapter designs what they were standing on. All terms are defined before use; all reference code is Rust with deterministic tests.

8.1 The Problem Statement

The interviewer draws a table with a billion rows and says:

“Queries filter on equality (`email = ?`) and on ranges with ordering (`created BETWEEN ? AND ?` , `ORDER BY created`). The table lives on disk. Design the index structure that makes those fast — and explain exactly why yours beats the alternatives.”

Take the prompt’s last clause seriously — “explain exactly why yours beats the alternatives” is the actual question. The B+ tree has been the standard answer for fifty years, and reciting it earns you nothing; the interview tests whether you can **re-derive** it from first principles, so that when the follow-up twists the constraints (writes dominate? keys are huge? data fits in RAM?) your answer moves with the physics instead of collapsing with the recitation. And the physics here is unusually well-defined: the bottleneck is not CPU or capacity but **disk I/O shape** — every random read costs 10^5 times a memory reference, so the entire game is minimizing the number of disk pages touched per operation. Every design decision in this chapter — page-sized nodes, separators, linked leaves, the write-ahead log — is that one minimization wearing a different costume. Once you see that, the chapter stops being a taxonomy and becomes a single argument you can reconstruct on any whiteboard.

DEFINITION — Index

A redundant, derived data structure that maps a search key to the location of full rows, maintained by the database on every write. An index trades write cost and space for read speed: every `INSERT` pays to keep it current, and every matching query repays that cost a thousandfold. “Adding an index” is never free — it is a bet that reads outnumber writes.

DEFINITION — Point lookup / range scan / sorted iteration

The three read shapes an index can serve: *point lookup* ($key = ?$), *range scan* ($lo \leq key \leq hi$), and *sorted iteration* (`ORDER BY key` without a sort step). A structure that serves all three is dramatically more valuable than one serving only the first — this asymmetry eliminates the hash index from the running almost immediately (Section 8.7).

Linger on the second definition’s asymmetry, because it is the first domino. The prompt mentions equality **and** ranges **and** ordering — three read shapes — and any structure you propose will be judged against all three simultaneously. A hash table is the perfect answer to one of them and a non-answer to the other two; a sorted file answers all three for readers and declares bankruptcy on writes. Hold both failures in mind: the winning structure will be the one that refuses to choose, and understanding why that refusal is **possible** — localized mutation inside a sorted, page-aligned hierarchy — is the chapter in one sentence.

8.2 Scope & Clarifying Questions

Even a fundamentals prompt has a workload hiding in it, and surfacing that workload is your first move. Each of these answers steers the design in a direction you should be able to name:

Candidate asks	Interviewer answers
Workload shape?	OLTP: many small point reads/writes, plus frequent range scans and <code>ORDER BY</code> pagination
Data resident where?	On disk (or SSD); tables far exceed RAM. Indexes may be partially cached
Read/write mix?	Mixed; neither extreme. Reads dominate slightly, writes must stay cheap
Key types?	Integers, strings, composite (multi-column) keys
Concurrency?	Mention latching strategy; don’t design a full lock manager
Durability?	Committed writes must survive crashes — bring in a WAL when you get to the write path
Distribution?	Single node. Sharding is Chapter 5’s topic; indexes replicate/shard with their table

Three answers do the heavy lifting. “OLTP with frequent range scans” confirms the three-read-shapes requirement and rules out the write-first structures. “Tables far exceed RAM” is the constraint that makes this a **disk** design at all — if everything fit in memory, Chapter 6’s ordered maps would finish the interview in five minutes, and the whole page-I/O apparatus would be overhead. “Mixed; neither extreme” is the quietest and most decisive: it is the read/write ratio that Section 8.11’s B+-vs-LSM verdict turns on, and you should note that the interviewer has just handed you that verdict’s input. The last row’s “single node” is a mercy — say thank you and take it.

NOTE — Agreed scope

1. Design the on-disk index structure for a single-node relational storage engine: **point lookup, range scan, sorted iteration, insert, delete**.
2. Respect disk physics: I/O in fixed-size pages; random I/O is the scarce resource.
3. Cover the write path (buffer pool, WAL, splits) and the B+ tree’s classic variants (clustered, secondary, covering, composite).
4. Position the LSM tree as the write-optimized alternative and know when it wins.

5. Out: concurrency control protocols, distributed indexes, query optimization beyond index selection basics.

8.3 Functional Requirements

Six requirements, and they read like the spec of an abstract data type — because that is exactly what an index is. The discipline is in the qualifiers: not “fast lookups” but “a small, **bounded** number of page reads regardless of table size”; not “support deletes” but “delete **without slow decay**.” An index that degrades gracefully in the small and catastrophically in the large is worse than none, because the catastrophe arrives in production, at scale, on a weekend.

1. **FR-1 — Point lookup.** `key → row location` in a small, bounded number of page reads regardless of table size.
2. **FR-2 — Range scan.** All keys in `[lo, hi]` in sorted order, touching only pages that contain them.
3. **FR-3 — Sorted iteration.** Full or partial ordered traversal (for `ORDER BY ... LIMIT`) without an external sort.
4. **FR-4 — Insert.** Add a key, maintaining the structure’s invariants, with bounded page writes.
5. **FR-5 — Delete.** Remove a key, keeping pages reasonably full (no slow decay into a sparse, space-wasting structure).
6. **FR-6 — Crash safety.** No committed state is lost and the structure is never corrupted by a crash mid-write (with the WAL, Section 8.10).

Read FR-2’s “touching only pages that contain them” twice — it smuggles in the locality requirement that will later justify the B+ tree’s linked leaves. A range scan that touches one random page per qualifying row is a point-lookup loop in a trench coat; the requirement says the **physics** of the scan must be sequential, not just its result set. And FR-6 is the requirement that silently doubles the design: any structure you propose must survive being interrupted **between** page writes, which is why the write path (Section 8.10) is a co-designed partner of the structure, not an afterthought.

8.4 Non-Functional Requirements

Two definitions first, because the NFR table’s every row is stated in their vocabulary:

DEFINITION — Page / block I/O

Disks do not read bytes; they read **pages** — fixed-size blocks (4–16 KB) that are the atomic unit of I/O and of the buffer pool. Every structure in this chapter is designed so that **one node = one page**: reading a node is one I/O, and the cost model of every algorithm is counted in pages touched, not comparisons made.

DEFINITION — Fanout

The number of children per internal node — for a page-sized node holding keys and child pointers, $\text{fanout} \approx \text{page_size} / (\text{key} + \text{pointer}) \approx \text{hundreds to a thousand}$. Fanout is the whole game: tree height is $\log_{\text{fanout}}(\text{rows})$, and height is disk reads per lookup.

Requirement	Target & reasoning
Lookup depth	≤ 4 page reads for 10^9 rows — the height math of Section 8.5 makes this the headline NFR
Write cost	$O(\text{height})$ page writes per insert, amortized; splits rare at high fanout

Requirement	Target & reasoning
Space overhead	Index $\approx$ 10–15% of table size; pages 70–90% full on average
Range locality	Range scans read mostly sequential pages — leaves are linked and physically clustered
Cache friendliness	Top levels of the tree permanently resident in the buffer pool; most lookups hit disk once
Predictable tail latency	No unbounded rebalancing cascades; split cost is bounded and rare

Notice that five of the six rows are stated in **page** units, and the sixth (predictable tail latency) is a page-unit property in disguise — “no unbounded rebalancing cascades” means “no operation may touch an unbounded number of pages.” This is what it looks like when a system’s NFRs are written in the units of its actual bottleneck. If you find yourself writing “fast” or “scalable” in this table, you have not yet found the physics; when the physics is disk I/O, every honest requirement is countable.

KEY INSIGHT — You are designing for the disk, not the data

Every prior chapter counted requests and bytes; this one counts **page touches**. A binary search tree does 30 comparisons for a billion rows — and if each node is a random page, 30 disk reads. A B+ tree does more comparisons per node but touches **four** pages. On disk, the “slower” CPU structure is 10 $\times$ faster. The interview is won by whoever counts the right resource.

8.5 Back-of-the-Envelope: The Physics and the Height Math

Two small tables carry this chapter. The first is the **latency pyramid** — orders of magnitude worth memorizing, because every storage-engine argument you will ever have is an argument about where in this pyramid an operation lands:

Operation	Latency	Consequence
RAM reference	100 ns	The free resource; comparisons are noise
SSD random page read	100 μ s	1000 $\times$ RAM — the unit this chapter minimizes
SSD sequential read	10 $\times$ cheaper per page	Why leaf links and clustered layout matter
HDD random I/O (seek)	10 ms	100,000 $\times$ RAM; mechanical arm physics — the original design constraint
Network round trip	0.5–1 ms	For context: why a database page miss still beats a remote call

Read the pyramid with an eye for **ratios**, and two design rules fall out before any structure is discussed. Rule one: random vs. sequential is a 10 $\times$ cliff **on the same device** — so layouts matter as much as algorithms, which is why “leaves are linked” will turn out to be half the point of the B+ tree rather than a detail. Rule two: the gap between RAM and **any** disk touch is three to five orders of magnitude — so a design’s disk touches are its whole cost, and the CPU work between them is, to first order, free. Structures that look dumb by RAM standards (reading 16 KB to use 100 bytes of it) are smart by disk standards; this inversion of intuition is the chapter’s entry fee.

The height derivation — the most important arithmetic in the chapter, and the one you should be able to reproduce from an empty whiteboard:

Quantity	Value	Derivation
Page size	16 KB	Typical database page
Index entry	16 B	8 B key + 8 B child pointer (interior)
Fanout per page	1000	16 KB / 16 B
Rows reachable, height 2	10^6	1000 leaf pages $\times$ 1000 entries... precisely: root $\rightarrow$ 1000 internal pages $\rightarrow$ leaves; leaves hold 16KB/ (16B+ptr to row) $\approx$ 500–1000 entries
Rows reachable, height 3	10^9	A billion rows in three levels of internal nodes + leaves
Page reads per lookup	≤ 4 , usually 1–2	Root and level-1 live in the buffer pool — only the leaf read hits disk
Index size for 10^9 rows	16 GB leaves + 0.02 GB interior	Interior levels are negligible — that is why they are always cached

Do the last row's division out loud, because it is the sneakiest and most important line in the table. The leaves of a billion-row index cost 16 GB — real storage, too big to assume resident. But the **interior** of the tree — the root plus level 1, every page a lookup might pass through before the leaf — totals about 20 MB, one eight-hundredth of the leaves. A structure whose decision-making is three orders of magnitude smaller than its data is a structure whose decision-making can live in RAM permanently, and **that** is why “usually 1–2 page reads” is achievable: the cached levels turn $\log_{1000}(10^9) = 3$ theoretical reads into one physical one. Caching is not an optimization bolted onto the B+ tree; the tree's shape is what makes a tiny cache nearly perfect.

KEY INSIGHT — High fanout collapses height

Binary trees give height $\log_2$ — 30 levels for a billion rows. A fanout of 1000 gives $\log_{1000}$ — **three**. Since the top two levels are tiny (16 MB) they live in RAM permanently, and a billion-row lookup costs about **one disk read**. That is the entire magic trick: not cleverer comparisons, just a tree so wide it is nearly flat. Every B-tree property — page-sized nodes, separator keys, splits that propagate up rarely — exists to protect this fanout.

8.6 The Core Challenge: One Structure for Reads and Writes

Now the tension can be stated cleanly. Reads want a sorted, dense, immutable layout — sorted so ranges are contiguous, dense so pages are full, immutable so nothing ever moves mid-scan. Writes want cheap, localized mutation — touch as little as possible, as rarely as possible. Every simple structure you know picks a side, which is why the interview's “explain exactly why yours beats the alternatives” is really “show me you know why the alternatives don't work”:

COMMON PITFALL — The two seductive wrong answers

Hash index: $O(1)$ point lookups — and **nothing else**: no ranges, no ordering, and rehashing a billion-key table is an outage. It answers the question you weren't asked. *Sorted file*: binary search gives $O(\log n)$ reads and ranges are sequential — but one insert into the middle rewrites half the file. Append and

it degrades into a log you must compact (that idea, pursued seriously, becomes the LSM tree of Section 8.11 — the legitimate rival). The B+ tree threads the needle: sorted **and** cheaply mutable, because mutation is localized to one page and its ancestors.

Notice the careful phrasing of the verdict — not “sorted and mutable,” but “sorted and **cheaply** mutable, because mutation is localized.” A sorted file is also mutable; it just pays for a one-row insert with a half-file rewrite. The B+ tree’s trick is to chop the sorted file into page-sized pieces and keep a tiny routing hierarchy over the pieces, so that a mutation’s blast radius is one page plus, occasionally, its ancestors. The sortedness is global (the hierarchy guarantees it), the mutability is local (the page contains it), and the two stop fighting. Everything else in this chapter is engineering that protects that truce.

8.7 Candidate Structures

Walk the elimination table as a sequence of **near-misses**, each teaching one constraint. The hash table’s failure is coverage (one read shape out of three); the sorted file’s failure is writes; the plain BST’s failure is balance under real data — and note **which** real data: sorted inserts, the single most common key pattern in production (auto-increment ids, timestamps), turn a naive BST into a linked list. The balanced BST is the instructive loss: it has **all the right complexities** and still loses by 30-to-4 on page touches, which should permanently cure you of evaluating on-disk structures by comparison counts.

Structure	Point lookup	Range scan	Insert	Verdict
Hash table	$O(1)$	unsupported	$O(1)$	Point lookups only; no order, no ranges
Sorted array / file	$O(\log n)$	$O(\log n + k)$	$O(n)$	Reads fine; a single insert rewrites the file
Binary search tree	$O(n)$ worst	$O(n)$	$O(n)$ worst	Unbalanced under real key distributions (sorted inserts!)
Balanced BST (AVL/red-black)	$O(\log n)$	$O(\log n + k)$	$O(\log n)$	Right complexity, wrong physics: 30 random pages per lookup
B+ tree	≤ 4 pages	$\leq 4 + \text{sequential leaves}$	$O(\text{height})$ pages	Page-sized nodes, fanout 1000, linked leaves — Section 8.9

8.8 Deep Dive: B-Tree Mechanics

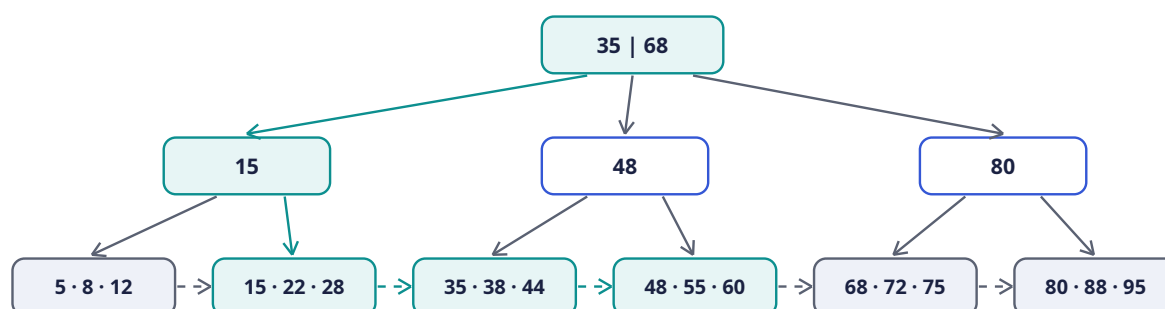
Here is the structure that survived. Understand it as a set of **invariants** first and operations second, because every operation is just “do the thing, then repair the invariants locally”:

DEFINITION — B-tree / node split

A *B-tree* is a self-balancing search tree where every node is one disk page holding up to **m** sorted keys and **m+1** child pointers. Invariants: all leaves at the same depth; every non-root node at least half full; keys sorted within and across nodes. Insertion descends to the target leaf; if the leaf overflows, it *splits* in two and pushes a separator key up into the parent — which may itself split, in a bounded cascade

that, at the root, grows the tree **taller by one level**. Growth by root splits is why the tree stays perfectly balanced under any key order — including the sorted inserts that destroy naive BSTs.

Search is one comparison loop per level: at each internal page, binary search the separator keys (free — the page is already in RAM by then), follow the child pointer, read one more page. Height 3–4 (Section 8.5) $\Rightarrow \leq 4$ page reads, and the upper pages are cached. The picture below is the chapter's one essential diagram: a fanout-3 toy (production fanout is 1000, but three fits on paper) with a range scan in progress:



Range scan [22, 55]: descend root $\rightarrow$ internal(15) $\rightarrow$ leaf 2 (22, 28), then follow leaf links through 55 — never returning upward.

Walk the diagram as the range scan [22, 55] experiences it, and read the color as a signal: teal marks everything this query touches, gray everything it provably skips. Start at the **root**, the teal box 35 | 68 at the top — two separator keys, three children, and the entire routing logic of the tree in one rule: keys below 35 go left, 35–68 middle, above 68 right. The scan's lower bound, 22, is below 35, so the teal arrow descends left to the internal page 15 — whose single separator splits its subtree at 15, sending 22 right, into the second leaf. Three pages read (root, internal, leaf), and the scan has found its **starting point**: leaf 15 · 22 · 28, where 22 lives. Everything before this moment is a point lookup for the range's lower edge — that is all a range scan ever needs from the tree part of the structure.

Now the second phase, and the reason the leaves are shaded differently from the internals. From 22 onward, the scan **never goes back up**. It walks right along the dashed arrows — the **leaf links**: 22, 28 in leaf two, then leaf three (35 · 38 · 44), then leaf four (48 · 55 · 60), stopping the moment it passes 55. Three sequential sideways hops, each almost certainly a sequential disk read (the pyramid's 10 $\times$ discount, collected). Count what the query never touched: the first leaf (5 · 8 · 12 — entirely below the range, skipped by the descent) and the last two leaves (68+ — above it, never reached because the walk stopped). The gray arrows and gray leaves are the structure's promise kept: pages outside the range are not read, not even once. Now the removal exercise: **delete the dashed links and ask what breaks**. Every range scan becomes a series of re-descents from the root — k rows in the range cost k point lookups, the sequential-read discount vanishes, and `ORDER BY ... LIMIT 20` acquires a sort step it never needed. The linked-leaf level is not an embellishment; it **is** the B+ tree's answer to FR-2 and FR-3, and it is why the production variant (next section) rather than the textbook B-tree is the one that won.

Deletion is the mirror: remove from the leaf; if a node falls below half-full, borrow from a sibling or merge siblings and pull the separator down. In practice engines under-fill rather than merge eagerly (merges are latch-heavy), and a background process rebalances — Section 8.15's "index bloat" is what happens when even that is neglected. The asymmetry with insertion is worth naming: inserts split eagerly because an over-full page is **illegal**; deletes merge lazily because an under-full page is merely **unfortunate**. Structures tolerate sloppiness in proportion to how much the sloppiness costs, and a half-empty page costs space, not correctness.

8.9 Deep Dive: The B+ Tree Refinements

The production variant — the B+ tree — differs from the textbook B-tree in three deliberate ways, and by now you can predict each one from the physics. Work through them as consequences, not features:

1. **Values live only in leaves.** Internal pages hold pure separator keys — no row data — which **increases fanout** (more separators per page) and protects the height math. A 16-byte separator routes as well as a 16 KB row would; filling interior pages with payloads would buy nothing and cost a level. Derivation: interior pages exist to be **read through**, so make them maximally thin.
2. **Leaves are linked** in key order (the dashed arrows above). A range scan descends once, then walks sideways — sequentially, the disk’s favorite pattern. This is what makes `ORDER BY ... LIMIT 20` nearly free. Derivation: the second and third read shapes must be sequential; sideways links are the cheapest way to make leaf N+1 findable from leaf N.
3. **Interior keys are copies.** The separator `35` also exists as a real key in a leaf; internal copies are pure routing. Derivation: once values live only in leaves, a separator is a **claim** (“everything right of me is ≥ 35 ”) rather than a residence, and claims are cheap to duplicate — which is what makes the leaf-level split’s copy-up rule (Section 8.13’s code shows it literally) legal.

Three index **usages** ride on the same structure, and confusing them is the most common production-indexing mistake — so the vocabulary gets its own definition block before the trade-off sections lean on it:

DEFINITION — Clustered / secondary / covering / composite index

A *clustered* index **is** the table: leaf pages hold full rows, physically ordered by the key (one per table; range scans read rows directly). A *secondary* index’s leaves hold `(key → primary key)`: a lookup then re-descends the clustered index (a “bookmark lookup”) — extra I/O per row that makes non-covering secondary scans expensive. A *covering* index stores every column the query needs, so the second lookup never happens. A *composite* index keys on `(a, b, c)` as a tuple — usable by queries constrained on a **leftmost prefix** `(a, or a, b)`, useless for `b` alone, because the order is lexicographic (Section 8.13 implements the rule).

The relationships among the four are the mental model to carry: clustered vs. secondary is about **what the leaf holds** (rows vs. pointers); covering is a property a secondary index can **earn** by carrying more columns; composite is about **what the key is** (a tuple), orthogonal to all the rest. The leftmost-prefix rule follows from lexicographic order alone — a phone book sorted by (surname, firstname) finds “Smith, John” instantly and “everyone named John” only by reading every page — and Section 8.13’s third listing reduces the rule to a six-line function, because the rule **is** six lines; the art is remembering to apply it before creating the index.

8.10 The Write Path: Buffer Pool and WAL

Reads got five sections; writes get this one, because the write path is two ideas working in tandem, and both are bets on the latency pyramid. Writes never touch disk directly — instead:

DEFINITION — Buffer pool

The database’s page cache: a fixed RAM arena of page frames with an LRU-ish eviction policy. Reads check it first (the tree’s top levels are effectively pinned by popularity); writes **modify pages in memory** and mark them dirty, flushing lazily. Most “database performance” is buffer pool hit ratio — Section 8.13 implements the LRU core.

DEFINITION — Write-ahead log (WAL)

The durability trick: before any dirty page may flush, the **intention** — an append-only log record of the change — must be durable on disk. Appends are sequential and cheap; a crash replays the log to redo committed changes. The WAL is what lets the buffer pool defer random page writes without losing committed data — the same append-log bet Chapter 1 made for operations and Chapter 6 made for score journals.

See the bet clearly: a committed write performs **one sequential append** (the WAL record) and zero random writes (the dirty page lingers in the pool). The random write is not eliminated — it is **deferred and batched**, flushed later when it can be grouped with neighbors or absorbed by idle bandwidth. Durability without the deferral is unusably slow (Section 8.16’s table prices it); deferral without durability loses committed data on crash; the WAL is precisely the bridge that makes deferral safe, because “the page wasn’t flushed” stops meaning “the change was lost” and starts meaning “the change must be replayed.” Crash recovery becomes: read the log forward, redo committed intentions, done — FR-6 discharged by an append.

The split dance on insert: descend with latches, split the leaf if full, push the separator up — each split is 2–3 page writes plus a WAL record, rare at fanout 1000 (a leaf absorbs 500 inserts between splits), and amortized into the background flush stream. Do the arithmetic the table implied: at 500 inserts per leaf split, a million-row-per-day table splits leaves a few thousand times daily — each a bounded, local, logged event. The NFR’s “no unbounded rebalancing cascades” is kept not by making splits impossible but by making them **rare and shallow**: at fanout 1000, a split propagates to the parent one time in a thousand, and to the grandparent one time in a million.

8.11 The Rival: LSM Trees

Every “why not” needs its strongest alternative taken seriously, and the LSM tree is the only one that beats the B+ tree at anything real: **writes**.

DEFINITION — LSM tree / memtable / SSTable / compaction

The *log-structured merge tree* is the write-optimized alternative: writes land in an in-memory sorted buffer (the *memtable*), which flushes periodically to immutable sorted files (the *SSTables* — Chapter 4’s log index segments, exactly), and a background process *compacts* levels by merge-sorting them into fewer, bigger files. Reads check the memtable, then the newest SSTables outward, with Bloom filters skipping files that can’t contain the key.

Recognize the lineage and you already half-understand the design: the LSM tree is what happens when you take the **sorted file** — Section 8.6’s second wrong answer — and fix its write problem by never modifying files in place. Inserts append to memory; memory flushes to a fresh immutable file; files never change, only multiply and merge. Writes become purely sequential, which by the pyramid is the cheapest thing a disk can do. The bill arrives at read time: a point lookup must check the memtable, then potentially several SSTable generations, with Bloom filters (Chapter 2’s bit-array probabilistic gate, recycled) skipping most of them. The trade, stated as the table does below, is read amplification for write amplification:

Property	B+ tree	LSM tree
Write path	Random page updates + splits	Append + flush: almost purely sequential
Write amplification	Low-moderate	Higher: compaction rewrites data repeatedly

Property	B+ tree	LSM tree
Read amplification	Lowest: ≤ 4 pages, one structure	Higher: check memtable + several SSTable levels (Bloom filters help)
Range scans	Excellent: linked leaves	Good, but must merge across levels
Space	Pages 70–90% full	Compaction leaves transient duplication
Best fit	Read-heavy OLTP (this chapter's scope)	Write-heavy logs/time-series (Chapter 4's workload)

KEY INSIGHT — B+ vs LSM is a read/write ratio decision

The two structures dominate modern storage engines, and the choice is not fashion: B+ trees minimize read I/O, LSM trees minimize write I/O. State the workload's read/write ratio and choose — that sentence, with the amplification vocabulary to defend it, is the entire “which storage engine” follow-up.

Look back at the scope dialogue and notice you were told the answer: “mixed; neither extreme. Reads dominate slightly” is B+ tree territory, and Chapter 4's append-mostly telemetry was LSM territory — which is why the book's two storage-deep chapters land on opposite structures without contradicting each other. When the interviewer asks “so which is better?”, the senior answer is a question: “what's the ratio?”

8.12 The Storage Engine's Interface

The structure above serves a deliberately small API — five operations — and this is what the rest of the database (and Chapter 5's and 6's tables) stand on. Notice that the interface says nothing about trees, pages, or splits: to the layers above, an index is a sorted map with cursors. The physics leaks upward only through **performance characteristics**, never through semantics — a separation worth copying in your own systems.

Operation	Semantics
<code>get(key)</code>	Point lookup → row or location; ≤ 4 page reads
<code>scan(lo, hi)</code> / cursor	Ordered range iteration; the cursor holds a page + position — Chapter 5's cursor at the storage layer
<code>insert(key, row)</code>	Descend, insert in leaf, split as needed, WAL first
<code>delete(key)</code>	Remove from leaf; underflow handled lazily
<code>create index (cols)</code>	Build a secondary B+ tree by scanning the table once, sorting, and bulk-loading leaves left-to-right (far cheaper than n inserts)

The last row hides a lovely optimization worth naming in the room: **bulk-loading**. Building an index by n individual inserts pays n descents, $n/500$ splits, and produces pages in random physical order. Building it by sorting the keys once and writing leaves left-to-right pays one sort and produces a perfectly packed, physically sequential tree — the difference between an afternoon and a coffee break on a billion-row table. When a structure knows its own invariants, creating it wholesale is always cheaper than growing it retail; that is as true for Chapter 9's replicated state as it is here.

8.13 Rust Reference Implementations

Four pieces with deterministic tests: the B+ tree itself (split logic and all), a sparse-index page lookup (the SSTable trick from Section 8.11), the leftmost-prefix rule (six lines that decide whether your composite index exists in vain), and the LRU buffer pool. The tree is the centerpiece — read it as the invariants from Section 8.8 made executable.

8.13.1 A B+ Tree with Splits

The implementation compresses the whole mechanics section into two mutually recursive ideas. **Descent** (`get` , and the internal arm of `insert_rec`) uses `partition_point` over the separator keys — the same “first separator greater than the key bounds the descent from the left” rule you traced on the diagram. **Overflow repair** returns a `Split` value upward: a leaf that exceeds `MAX_KEYS` divides at the middle and reports `Split { sep, right }` , where the B+ **copy-up** rule is visible in one comment — `sep` is the right half’s first key, and it **stays in the leaf too**, because leaves hold all real keys and interiors hold mere claims. The internal split, by contrast, **pushes** the middle key up with `keys.pop()` — interiors route, so their middle key is promoted, not copied. Copy-up in leaves, push-up in interiors: that asymmetry, which textbooks state in prose, is here two lines you can point at. `ORDER` is deliberately tiny (4) so the tests exercise dozens of splits on a hundred keys — the invariant-checking `check` function then verifies what the rush of splits must preserve: keys sorted everywhere, children = keys + 1 everywhere, and every leaf at the **same depth**, which is the “perfectly balanced under any key order” promise from the definition, asserted rather than assumed.

```
const ORDER: usize = 4; // max children per node – tiny so tests exercise splits
const MAX_KEYS: usize = ORDER - 1;

#[derive(Debug, Clone)]
enum Node {
    Leaf { keys: Vec<u64>, vals: Vec<u64> },
    Internal { keys: Vec<u64>, children: Vec<Node> }, // keys[i] = min key of children[i+1]
}

pub struct BPlusTree {
    root: Node,
}

enum Split {
    None,
    Split { sep: u64, right: Node },
}

impl BPlusTree {
    pub fn new() -> Self {
        BPlusTree { root: Node::Leaf { keys: vec![], vals: vec![] } }
    }

    pub fn get(&self, key: u64) -> Option<u64> {
        let mut node = &self.root;
        loop {
            match node {
                Node::Leaf { keys, vals } => {
                    return keys.binary_search(&key).ok().map(|i| vals[i]);
                }
                Node::Internal { keys, children } => {
                    // first separator > key bounds the descent from the left
                    let i = keys.partition_point(|&k| key >= k);
                    node = &children[i];
                }
            }
        }
    }
}
```

```

    }
  }
}

pub fn insert(&mut self, key: u64, val: u64) {
  if let Split::Split { sep, right } = insert_rec(&mut self.root, key, val) {
    let old = std::mem::replace(
      &mut self.root,
      Node::Leaf { keys: vec![], vals: vec![] },
    );
    self.root = Node::Internal { keys: vec![sep], children: vec![old, right] };
  }
}

/// Ordered [lo, hi] scan – in production this descends once and walks
/// linked leaves; here we prune recursively, same result set.
pub fn range(&self, lo: u64, hi: u64) -> Vec<(u64, u64)> {
  let mut out = Vec::new();
  collect_range(&self.root, lo, hi, &mut out);
  out
}

pub fn height(&self) -> usize {
  let (mut h, mut node) = (1, &self.root);
  while let Node::Internal { children, .. } = node {
    node = &children[0];
    h += 1;
  }
  h
}

fn insert_rec(node: &mut Node, key: u64, val: u64) -> Split {
  match node {
    Node::Leaf { keys, vals } => {
      let pos = keys.partition_point(|&k| k < key);
      if pos < keys.len() && keys[pos] == key {
        vals[pos] = val; // upsert: no duplicate keys
        return Split::None;
      }
      keys.insert(pos, key);
      vals.insert(pos, val);
      if keys.len() <= MAX_KEYS { return Split::None; }
      let mid = keys.len() / 2;
      let rkeys = keys.split_off(mid);
      let rvals = vals.split_off(mid);
      let sep = rkeys[0]; // B+ copy-up: the key stays in the leaf too
      Split::Split { sep, right: Node::Leaf { keys: rkeys, vals: rvals } }
    }
    Node::Internal { keys, children } => {
      let i = keys.partition_point(|&k| key >= k);
      match insert_rec(&mut children[i], key, val) {
        Split::None => Split::None,
        Split::Split { sep, right } => {
          keys.insert(i, sep);
          children.insert(i + 1, right);
          if keys.len() <= MAX_KEYS { return Split::None; }
          let mid = keys.len() / 2;
          let up = keys[mid];
          let rkeys = keys.split_off(mid + 1);

```

```

        keys.pop(); // internal split pushes the middle UP, no copy
        let rchildren = children.split_off(mid + 1);
        Split::Split {
            sep: up,
            right: Node::Internal { keys: rkeys, children: rchildren },
        }
    }
}
}
}

fn collect_range(node: &Node, lo: u64, hi: u64, out: &mut Vec<(u64, u64)>) {
    match node {
        Node::Leaf { keys, vals } => {
            for (i, &k) in keys.iter().enumerate() {
                if k >= lo && k <= hi {
                    out.push((k, vals[i]));
                }
            }
        }
        Node::Internal { keys, children } => {
            for (i, child) in children.iter().enumerate() {
                let lower = if i == 0 { 0 } else { keys[i - 1] };
                let upper = if i == keys.len() { u64::MAX } else { keys[i] };
                if hi >= lower && lo <= upper {
                    collect_range(child, lo, hi, out);
                }
            }
        }
    }
}

#[cfg(test)]
mod btree_tests {
    use super::*;

    fn check(node: &Node, depth: usize, leaf_depths: &mut Vec<usize>) {
        match node {
            Node::Leaf { keys, .. } => {
                assert!(keys.windows(2).all(|w| w[0] < w[1]));
                leaf_depths.push(depth);
            }
            Node::Internal { keys, children } => {
                assert_eq!(keys.len() + 1, children.len());
                assert!(keys.windows(2).all(|w| w[0] < w[1]));
                assert!(keys.len() <= MAX_KEYS);
                for c in children {
                    check(c, depth + 1, leaf_depths);
                }
            }
        }
    }

    #[test]
    fn insert_search_range_stay_consistent() {
        let mut t = BPlusTree::new();
        let mut keys: Vec<u64> = (0..100).collect();
        keys.sort_by_key(|&k| (k * 37) % 101); // deterministic shuffle
        for &k in &keys {

```

```

        t.insert(k, k * 10);
    }
    for k in 0..100u64 {
        assert_eq!(t.get(k), Some(k * 10));
    }
    assert_eq!(t.get(100), None);
    let r = t.range(25, 42);
    assert_eq!(r.len(), 18);
    assert_eq!(r.first().unwrap().0, 25);
    assert_eq!(r.last().unwrap().0, 42);
    assert!(r.windows(2).all(|w| w[0].0 < w[1].0));
    let mut depths = Vec::new();
    check(&t.root, 1, &mut depths);
    assert!(depths.iter().all(|&d| d == depths[0])); // perfectly balanced
    assert!(t.height() <= 6, "height {}", t.height());
}

#[test]
fn upsert_overwrites_without_duplication() {
    let mut t = BPlusTree::new();
    t.insert(5, 50);
    t.insert(5, 500);
    assert_eq!(t.get(5), Some(500));
    assert_eq!(t.range(0, 1000).len(), 1);
}
}

```

The main test is worth reading as three arguments in sequence. First, **correctness under shuffled load**: a deterministic scramble of 0–99 is inserted, and every key reads back — the split machinery surviving a hundred insertions in adversarial (non-sorted) order. Second, **range semantics**: `range(25, 42)` returns exactly 18 ordered pairs, endpoints inclusive, order ascending — the FR-2 promise with witnesses. Third, the **invariant audit**: the recursive `check` walks the whole tree asserting sortedness and arity at every node, collects every leaf’s depth, and demands they all agree. That last assertion is the one a naive BST fails catastrophically and this structure cannot fail at all — growth happens only at the root (see `insert` : when the root splits, a brand-new internal root is built **above** the old one), so all leaves deepen together or not at all. “The tree grows taller from the top” is the sentence; the test is its proof.

8.13.2 Sparse-Index Page Lookup

The second listing is the LSM world’s answer to “how do you find a key in an immutable sorted file without an index that costs as much as the file?” — and it is the same fanout idea from Section 8.5 wearing plainer clothes. Keep one **sample** per block of keys in memory (the block’s first key and where the block starts); a lookup binary-searches the tiny sample array, then searches exactly one block. The memory cost drops from $O(n)$ to $O(n/\text{SPAN})$, and the lookup cost stays at two searches, the second over a handful of entries. When Section 8.11 said “Bloom filters skip files that can’t contain the key,” this structure is what does the finding in the files that can.

```

/// A sorted run of keys with an in-memory sparse index: one sample per
/// SPAN keys (the LSM/SSTable trick – binary search the samples, then
/// search one tiny block; a full index would cost  $O(n)$  memory).
pub struct SparseIndex {
    keys: Vec<u64>,
    sample: Vec<(u64, usize)>, // (first key of block, block start index)
}

impl SparseIndex {

```

```

const SPAN: usize = 4;

pub fn new(mut keys: Vec<u64>) -> Self {
    keys.sort();
    let sample = keys
        .iter()
        .enumerate()
        .step_by(Self::SPAN)
        .map(|(i, &k)| (k, i))
        .collect();
    SparseIndex { keys, sample }
}

pub fn find(&self, key: u64) -> Option<usize> {
    if self.keys.is_empty() { return None; }
    // last sample whose key <= key
    let idx = self.sample.partition_point(|&(k, _)| k <= key);
    if idx == 0 { return None; } // key precedes the first sample
    let start = self.sample[idx - 1].1;
    let end = (start + Self::SPAN).min(self.keys.len());
    self.keys[start..end]
        .binary_search(&key)
        .ok()
        .map(|off| start + off)
}

#[cfg(test)]
mod sparse_tests {
    use super::*;

    #[test]
    fn finds_via_sample_then_block() {
        let idx = SparseIndex::new((0..40).map(|i| i * 3).collect());
        assert_eq!(idx.find(27), Some(9)); // 27 = 9*3
        assert_eq!(idx.find(0), Some(0)); // first sample boundary
        assert_eq!(idx.find(117), Some(39)); // last block
        assert_eq!(idx.find(118), None); // past the end
        assert_eq!(idx.find(1), None); // inside a block, absent
    }
}

```

Read the test as a tour of the boundary conditions, because sparse structures fail at edges, not middles. `find(0)` must not fall off the front of the sample array (the `idx == 0` guard is what makes “before the first sample” a clean miss rather than a panic — note it also means keys **before** the first sample cannot exist, because the first sample is the first key). `find(117)` exercises the final, possibly short block; `find(1)` is the interesting miss: 1 would sort into block 0 but is not in it, so the block search returns absent — a sparse index narrows the search, it never **decides** presence. That two-phase honesty (samples locate, blocks decide) is exactly how Chapter 9’s replicated logs and Chapter 4’s segment indexes use the same trick.

8.13.3 The Leftmost-Prefix Rule

The third listing is the smallest in the chapter and might save you the most grief in practice. Section 8.9 defined the rule in prose — a composite index `(a, b, c)` serves queries constrained on a leading run of its columns — and here it is as a function: walk the index columns from the left, count how many have equality constraints, and stop at the first gap. Six lines, and yet mis-indexing against this rule is among the most common causes of “the database ignored my index” in production (Section 8.18’s first follow-up). The rule exists because lexicographic order can only **seek** on a contiguous prefix: after

the first unconstrained column, later columns are no longer sorted in any way the query can exploit — exactly like the phone book that can find “Smith, J...” but not “...John”.

```
/// How many leading columns of a composite index `(a, b, c)` a query can
/// use for equality lookups: the longest run of index columns that all
/// have equality constraints. After the prefix, at most one range column
/// remains usable — columns past it are unreachable by the index order.
pub fn usable_prefix(index_cols: &[&str], equality_cols: &[&str]) -> usize {
    index_cols
        .iter()
        .take_while(|c| equality_cols.contains(c))
        .count()
}

#[cfg(test)]
mod prefix_tests {
    use super::*;

    #[test]
    fn lexicographic_order_decides_usability() {
        let idx = ["a", "b", "c"];
        assert_eq!(usable_prefix(&idx, &["a", "b"]), 2); // a, b usable
        assert_eq!(usable_prefix(&idx, &["b"]), 0);       // b alone: useless
        assert_eq!(usable_prefix(&idx, &["a", "c"]), 1); // a only; c skipped over b
        assert_eq!(usable_prefix(&idx, &["a", "b", "c"]), 3);
        assert_eq!(usable_prefix(&idx, &[]), 0);
    }
}
```

The third assertion is the one to remember in the room: a query constraining `a` and `c` uses **only** `a` — the gap at `b` ends the usable prefix, and `c`’s constraint becomes a post-filter over whatever the `a`-scan returns. When you design the composite index for a query (the wrap-up’s second follow-up makes you), order its columns so that the query’s equalities form an unbroken left run; that single decision is most of what “index design” means in practice.

8.13.4 The Buffer Pool (LRU)

The last listing is the write path’s other half: the page cache from Section 8.10, here as a minimal LRU — a hash map for $O(1)$ page lookup plus a recency deque, front-most-recent. The doc comment carries two honest caveats worth saying aloud. First, `touch` is $O(n)$ for clarity; production uses an intrusive linked list for $O(1)$. Second — and this is the systems point — plain LRU has a famous pathology, **scan pollution**: one full table scan marches every page of the table through the pool, evicting the hot set to make room for pages that will never be read again. Real engines use LRU-K or clock variants precisely to resist it (Section 8.15’s thrashing row is that pathology with symptoms attached). Recency is a proxy for reuse; scans are the workload that breaks the proxy.

```
use std::collections::{HashMap, VecDeque};

/// Page cache with LRU eviction.  $O(n)$  recency refresh for clarity;
/// production uses an intrusive linked list for  $O(1)$  touches (and often
/// LRU-K or clock variants to resist scan pollution).
pub struct BufferPool {
    cap: usize,
    frames: HashMap<u64, Vec<u8>>, // page id -> contents
    recency: VecDeque<u64>,        // front = most recently used
}

impl BufferPool {
    pub fn new(cap: usize) -> Self {
```

```

    BufferPool { cap, frames: HashMap::new(), recency: VecDeque::new() }
}

pub fn get(&mut self, page: u64) -> Option<&[u8]> {
    if self.frames.contains_key(&page) {
        self.touch(page);
        return self.frames.get(&page).map(Vec::as_slice);
    }
    None
}

pub fn put(&mut self, page: u64, data: Vec<u8>) {
    if self.frames.contains_key(&page) {
        self.frames.insert(page, data);
        self.touch(page);
        return;
    }
    if self.frames.len() == self.cap {
        let victim = self.recency.pop_back().expect("pool nonempty");
        self.frames.remove(&victim);
    }
    self.frames.insert(page, data);
    self.touch(page);
}

fn touch(&mut self, page: u64) {
    self.recency.retain(|&p| p != page);
    self.recency.push_front(page);
}
}

#[cfg(test)]
mod pool_tests {
    use super::*;

    #[test]
    fn lru_evicts_least_recently_used() {
        let mut bp = BufferPool::new(2);
        bp.put(1, b"a".to_vec());
        bp.put(2, b"b".to_vec());
        assert!(bp.get(1).is_some()); // refresh page 1 -> page 2 is now LRU
        bp.put(3, b"c".to_vec()); // evicts page 2
        assert!(bp.get(2).is_none());
        assert_eq!(bp.get(1).unwrap(), b"a");
        assert_eq!(bp.get(3).unwrap(), b"c");
    }

    #[test]
    fn overwrite_refreshes_recency() {
        let mut bp = BufferPool::new(2);
        bp.put(1, b"a".to_vec());
        bp.put(2, b"b".to_vec());
        bp.put(1, b"a2".to_vec()); // refresh via overwrite
        bp.put(3, b"c".to_vec()); // evicts page 2, not page 1
        assert_eq!(bp.get(1).unwrap(), b"a2");
        assert!(bp.get(2).is_none());
    }
}

```


The two tests pin the two ways a page earns survival: being **read** (`lru_evicts_least_recently_used` — the `get(1)` between the puts is what dooms page 2) and being **rewritten** (`overwrite_refreshes_recency` — a `put` on a resident page also refreshes it, because a write is a use). Together they encode the policy statement precisely: every access, read or write, renews the lease. When Section 8.17 names buffer-pool hit ratio the database’s vital sign, these fourteen lines of policy are the heart whose rhythm is being measured.

8.14 Scaling the Structure

A structure chapter’s scaling section reads differently from a product chapter’s: the axes are properties of the **data** (rows, key size, distribution) rather than of the traffic, and the “what breaks first” column is where the operational wisdom lives.

Axis	How the B+ tree scales	What breaks first
Rows	Height grows as $\log_{1000}$ — billions stay at depth 3–4	Nothing, structurally; table breadth (columns $\times$ row size) pressures page economics first
Read concurrency	Buffer pool serves cached levels; leaf reads parallelize across I/O	Buffer pool hit ratio — watch it fall as the working set outgrows RAM
Write rate	Splits amortized at fanout 1000; WAL absorbs burstiness	Random-page flush bandwidth; past it, the workload is telling you LSM
Key size	Fanout = page/entry — fat keys flatten it	Oversized keys (long strings) cut fanout and deepen the tree; hash or prefix-compress them
Distribution	The index shards with its table (Chapter 5)	Hot partitions get hot index pages — the tree scales, the shard doesn’t

Two rows reward a slow reading. **Rows** — the axis you’d expect to dominate — breaks **nothing structurally**: that is the $\log_{1000}$ height doing its job, and the first real pressure comes from table breadth instead, because wider rows mean fewer entries per leaf page and the fanout arithmetic quietly degrades. The tree’s enemy is never row **count**; it is entry **size**. And **write rate** names the exact crossover to the rival: as long as random-page flush bandwidth absorbs the dirty-page stream, the B+ tree wins; the moment it doesn’t, “the workload is telling you LSM” — the failure mode is the signal, and Section 8.11’s table is the translation.

8.15 Failure Modes & Degradation

Failure	Symptom	Response
Crash mid-split	Torn or half-written pages	WAL replay redoes committed work; page checksums detect torn writes; never corrupt silently
Buffer pool thrashing	Latency spike; hit ratio collapses	A scan read the whole table through the pool — LRU variants (LRU-K, clock) resist scan pollution; pin critical levels
Index bloat	Pages half-empty after mass deletes; scans slow	Lazy underfill + background rebalancing/rebuild; monitor fill factor

Failure	Symptom	Response
Write burst	Split cascades on a hot range	Sequential keys (auto-increment, timestamps) concentrate inserts on the rightmost leaf — the rightmost hotspot ; hash-prefix or reverse the key
Corrupted page	Checksum mismatch on read	Fail the query loudly, rebuild from replica/WAL; an index is derived state — it can always be rebuilt from the table
Runaway query	Optimizer ignores the index, full-scans	Section 8.18: selectivity, stale statistics; <code>ANALYZE</code> , or force the index — then fix the statistics

Read the first row and the fifth together: they are the chapter’s integrity stance. A torn page is acceptable; a **silently** torn page is not — checksums convert “wrong bytes that look fine” into “a loud error that triggers repair,” and the repair is always available because an index is **derived state**: the table is the truth, the index a rebuildable projection (Chapter 6’s journal philosophy at the storage layer). The **rightmost hotspot** row deserves a story: an auto-increment key means every insert targets the maximum key, hence the rightmost leaf, hence **one page** — so a structure designed to spread a billion keys across a million pages serializes the whole write load through a single 16 KB page’s latch. The fixes (hash-prefixing, key reversal, ULIDs) all do the same thing: trade a little key readability to restore the insert spread the structure assumed.

8.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Core structure	B+ tree, page-sized nodes	Hash index: no ranges; sorted file: no cheap writes; balanced BST: 30 random I/Os vs 4 (Section 8.7)
Values in leaves only	B+ variant	Classic B-tree stores values in internal nodes too — fatter interior, lower fanout, and it breaks the leaf-link range trick
Write-heavy alternative	LSM tree when writes dominate	Chosen blindly: read amplification and compaction stalls bite read-heavy workloads
Durability	WAL + lazy page flush	Flush pages synchronously per commit: correct and unusably slow
Underflow handling	Lazy (underfill, rebalance later)	Eager merge on every delete: latch contention and churn for negligible space
Sequential keys	Accept rightmost hotspot or hash-prefix	Ignore it: one leaf page serializes the entire insert rate

Decision	Chosen	Alternative & why not (here)
Index everything	No — indexes are write tax	Each index is paid on every insert; cover the queries you have, not the queries you fear

The last row is the table’s conscience and the working engineer’s most-used row. Every index is a subscription the writes pay for: insert one row into a table with six secondary indexes and you perform seven ordered-structure inserts, seven WAL records, seven sets of split risk. “Cover the queries you have, not the queries you fear” is therefore not stinginess but accounting — an index justified by an imaginary query is a certain cost weighed against an imaginary benefit, and those always lose. When the feared query materializes, `CREATE INDEX` (Section 8.12’s bulk-load row) builds the structure in one scan; the tax starts only when the benefit does.

8.17 Observability & SLOs

Indicator	Definition	Target
Buffer pool hit ratio	Reads served from RAM / all page reads	$\geq 99\%$ for hot indexes
Pages per query	Page touches per lookup/scan	≤ 4 point; $\approx k/\text{page-fill}$ for ranges
Write amplification	Bytes written / bytes inserted	Low single digits for B+ trees
Fill factor	Average page occupancy	70–90%; alert on sustained decline
Slow query count	Queries past the latency SLO, from the slow log	Flat; investigate every step up
Split rate	Page splits per second	Proportional to inserts; spikes signal hotspots

Each row is a section of the chapter rendered as a gauge: hit ratio is Section 8.10, pages per query is Section 8.5’s height math with a target attached, fill factor watches Section 8.15’s bloat, split rate watches the rightmost hotspot. If you can only add one, add the hit ratio — it is the vital sign all the others explain. A database whose hit ratio falls from 99% to 95% has not gotten 4% slower; it has quadrupled its disk traffic, and the latency distribution will say so in the tail long before the mean admits anything.

8.18 Interview Wrap-Up

Likely follow-ups, and the shape of strong answers:

1. **“Why didn’t the database use my index?”** Selectivity: if the predicate matches 30% of the table, a scan is **cheaper** than 300M random bookmark lookups — the optimizer is right, and understanding **why** it is right (random I/O per row vs. sequential I/O per page) is the answer’s core. Secondary causes: stale statistics, a function wrapped around the column (`WHERE LOWER(email) = ?` defeats the index on `email`), or a composite index whose leftmost prefix the query doesn’t constrain (Section 8.13’s rule).
2. **“Design the index for this query”** — `WHERE tenant_id = ? AND created > ? ORDER BY created LIMIT 20`: composite `(tenant_id, created)` — equality on the leftmost column, range + order on the second; the sort is free and the LIMIT reads one page. This one composite index is the single most common production index shape; know it cold, and narrate it through the chapter’s machinery — the

equality pins the scan to one tenant's contiguous key-slice, the range rides the slice's internal order, and the LIMIT stops the leaf-walk after twenty rows.

3. **“B-tree vs hash for a key-value store?”** Hash for pure point gets at extreme write volume (and the LSM if writes dominate); B+ the moment ranges, ordering, or prefix scans appear. Memcached vs. the world.
4. **“How do transactions fit in?”** Isolation layers above the structure: latches protect pages physically; locks/MVCC protect transactions logically. The B+ tree doesn't change; version chains or lock tables hang off it. Mentioning the latch/lock distinction unprompted is a depth signal — Chapter 11 picks up exactly here.
5. **“Why do UUID primary keys hurt?”** Random keys destroy insert locality — every insert lands on a random leaf, defeating the buffer pool — and bloat every secondary index that references them. Time-ordered UUIDs (ULID/UUIDv7) exist precisely to give keys back their locality: the rightmost hotspot's mirror image, where the fix is not to spread writes but to **restore the order** the structure prefers.

8.19 Summary & Further Reading

NOTE — Chapter summary

The B+ tree is not a data structure choice; it is **disk physics made flesh**. Random I/O is $10^5\times$ slower than RAM, so the structure minimizes page touches: page-sized nodes give fanout 1000, which collapses a billion rows into height 3–4, of which the top levels live in the buffer pool — one disk read per lookup. Splits keep it balanced under any key order; values-in-leaves and linked leaves make range scans sequential; the WAL buys durability with appends instead of random flushes. The LSM tree is the mirror-image choice when writes dominate. Indexes are a bet paid on every write and collected on every read — composite, covering, and leftmost-prefix design decide whether the bet pays. Chapters 5 and 6 stood on this chapter; this chapter is why they could.

Further reading.

- The source video: “How do B-Tree Indexes work? — Systems Design Interview: 0 to 1 with Google Software Engineer” (Jordan has no life): <https://www.youtube.com/watch?v=Z20aqmxiH20>
- CMU 15-445 (Andy Pavlo's database course, lectures + notes) — buffer pools, B+ trees, and latching in production depth.
- Hellerstein, Stonebraker, Hamilton — *Architecture of a Database System* (2007) — the map of everything around the index.
- O'Neil et al. — “The Log-Structured Merge-Tree” (1996) — the rival, in its own words.
- SQLite's file-format documentation — a complete, readable B-tree implementation spec you can finish in an evening.

8.20 Chapter Glossary

Term	Meaning
B-tree	Self-balancing multiway search tree; node = page, fanout high, all leaves level
B+ tree	B-tree with values only in leaves, linked leaves, separator-only interiors; the production variant
Bookmark lookup	The second descent from a secondary index's leaf into the clustered index to fetch the row

Term	Meaning
Buffer pool	The page cache between the tree and disk; hit ratio is the database's vital sign
Clustered index	An index whose leaves hold full rows — the table itself, ordered by the key
Compaction	LSM background merge of SSTables into fewer levels; reclaims space, restores read shape
Composite index	Index on a tuple of columns; usable by leftmost prefix only
Covering index	An index storing every column a query needs — no bookmark lookup
Fanout	Children per node; page size / entry size; the number that collapses tree height
Fill factor	Average page occupancy; declines with deletes into index bloat
Latch vs. lock	Latch: physical, short-term page protection; lock: logical, transaction-scope protection
Leftmost-prefix rule	A composite index serves only queries constraining a leading run of its columns
LSM tree	Log-structured merge tree: memtable + immutable SSTables + compaction; write-optimized
Memtable	The LSM's in-memory sorted write buffer, flushed to SSTables
Order / MAX_KEYS	Max children per node (order m); max keys $m-1$; overflow triggers splits
Page	The fixed-size I/O and buffer-pool unit (4–16 KB); one node per page
Range scan	Ordered retrieval of $[lo, hi]$; one descent plus linked-leaf walk
Rightmost hotspot	Sequential keys concentrating all inserts on one leaf page, serializing writes
Secondary index	Index whose leaves map key $\rightarrow$ primary key, not full rows
Selectivity	Fraction of rows a predicate matches; low selectivity makes the optimizer choose a scan — correctly
Separator key	Interior routing key = minimum key of the right child; a copy in B+ trees
Sparse index	One sample per block over a sorted run; binary search samples, then the block — SSTable lookup
Split	Node overflow $\rightarrow$ two nodes + separator pushed up; root split grows the tree a level
SSTable	Sorted String Table: immutable sorted file flushed from a memtable; Chapter 4's segments
WAL	Write-ahead log: durable append of intentions before any page flush; crash = replay

— End of Chapter 8 · Next: Chapter 9, Designing Conflict-Free Replication: How CRDTs Work —

Designing Conflict-Free Replication: How CRDTs Work

NOTE — Problem source

This chapter solves the problem posed in the talk “CRDTs — Stop Worrying About Write Conflicts” from the series *Systems Design 0 to 1 with Ex-Google SWE* (channel: *Jordan has no life*). Like Chapter 8, it is a **fundamentals** deep dive rather than a product design: the interviewer asks you to design the data layer of a distributed system in which every replica accepts writes at all times — no leader, no locking, no global ordering — and yet every replica provably converges to the same state. It is also the promised second half of Chapter 1: there we chose Operational Transformation behind a per-document sequencer for the collaborative editor; this chapter walks the road we did not take, and discovers it is paved with some of the most elegant mathematics in distributed systems. All terms are defined before use; all reference code is Rust with deterministic tests.

9.1 The Problem Statement

The interviewer sketches three data centers on three continents and says:

“Every region accepts reads and writes at all times — even when the network link between regions is down for an hour. No region ever waits for another. When the link heals, all regions converge to the same data automatically: no manual conflict resolution, no lost updates, no duplicates. Design the data layer that makes this true.”

Before reaching for anything you know, notice that this prompt outlaws every familiar trick, one by one — and watching **how** it outlaws them is the fastest way to understand the chapter. A single **leader** that serializes all writes violates “no region waits for another” directly — the other two regions would stall on every write — and dies as a single point of failure besides. Distributed **locking** violates it harder: a lock held across a partition is either unavailable (the lock-holder is on the far side) or unsafe (you proceeded without it), which is just the availability-consistency seesaw with extra steps. “Just take the latest write” — last-writer-wins — keeps availability but loses data silently, as Section 9.6 will show in detail. The interviewer is not asking for a weaker version of a normal database; they are asking for a **different mathematics of agreement**. That mathematics is the CRDT, and by the chapter’s end you will be able to derive its core types on demand rather than recite them.

Five definitions build the vocabulary the rest of the chapter stands on. Read them slowly — each one is a load-bearing word in the prompt, and the prompt reads differently once they are precise.

DEFINITION — Replica

An independent copy of some shared state, living on a different machine (here: in a different region), that accepts reads and writes on its own. A system is *replicated* when several replicas of the same

logical data exist at once. Replication buys fault tolerance and read locality — and immediately raises the question this chapter answers: how do the copies stay in agreement?

DEFINITION — Network partition

A failure in which two groups of replicas cannot communicate for some period, while each group keeps running. Partitions are not exotic: any severed fiber, misconfigured router, or overloaded link between regions is one. A design that is correct only when the network is perfect is correct zero percent of the time at planet scale.

DEFINITION — Coordination

Any protocol in which a replica must communicate with others **before** it may complete a write: leader election, two-phase commit, distributed locks, consensus rounds. Coordination is the price of imposing a global order — and under a partition, the side that cannot reach the others must stop accepting writes. Coordination and availability are the two ends of a seesaw.

DEFINITION — Write conflict

The situation in which two replicas accept writes to the same logical datum while unable to see each other — *concurrent* writes. Neither write is “wrong”; the system simply owes the user a single, deterministic merged outcome. A conflict is not an error to be prevented but a fact of physics to be absorbed.

DEFINITION — Convergence

The property that replicas which have received the same set of updates are in the same observable state, no matter the order, duplication, or timing with which those updates arrived. Convergence is the entire contract of this chapter: not “never diverge” (impossible without coordination) but “divergence is temporary and self-healing”.

Notice the discipline in that last definition: convergence promises nothing about the **moment** of the write — replicas may legitimately disagree while the network is broken — and everything about the **limit**: any two replicas that have seen the same updates agree, unconditionally. This reframing is what makes the prompt solvable at all. You cannot prevent disagreement without coordination; you **can** guarantee that disagreement is a temporary, bounded, self-repairing state. The chapter’s work is to make “self-repairing” mechanical rather than hopeful.

9.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Data model?	A key-space of replicated values: counters, registers, sets, maps, and one sequence (text). Small, independently mutable values — not bulk blobs
Topology?	Three regions, active-active (every region serves real writes). The design must not preclude offline-capable edge clients later
Consistency?	Strong eventual consistency within the store (defined below). No cross-key invariants required — that door stays shut on purpose
Partition behavior?	Writes must never block on a partition: full AP. Reconciliation happens after healing, automatically

Candidate asks	Interviewer answers
Network assumptions?	Messages may be delayed, reordered, duplicated. At-least-once delivery at best
Durability?	A write must survive the loss of any single region once it has been gossiped; locally it is durable on ack
The hard part?	<i>“Stop worrying about write conflicts”</i> — the merge must be automatic, deterministic, and provably correct

Two answers quietly define the chapter’s boundaries, and you should mark them. “No cross-key invariants required — that door stays shut on purpose” is the interviewer protecting the design’s solvability: per-key convergence is achievable without coordination, but **invariants spanning keys** (“total stock never negative”) are provably not, and Section 9.17 will turn that boundary from a caveat into one of your strongest talking points. And “messages may be delayed, reordered, duplicated” is the network being honest: any merge machinery you design must treat duplication and reordering as routine weather, not as edge cases — which is why the three laws you will meet in Section 9.6 are exactly **commutativity** (reordering), **associativity** (grouping), and **idempotence** (duplication). The network assumptions wrote the math requirements; all that remains is noticing.

NOTE — Agreed scope

1. Design an **active-active replicated data layer**: every replica accepts reads and writes with zero cross-replica coordination on the write path.
2. Guarantee **strong eventual consistency**: replicas that have seen the same updates are provably in the same state.
3. Cover **both CRDT families** — state-based and operation-based — and the canonical catalog: counters, registers, sets, and the sequence type that would power Chapter 1’s editor.
4. Design the **synchronization layer** (gossip / anti-entropy, delta propagation) and the **metadata lifecycle** (tags, tombstones, garbage collection) that make the theory shippable.
5. Quantify everything: metadata per element, tombstone growth, sync bandwidth, and the bound on convergence time.
6. Out of scope: Byzantine (malicious) replicas, and cross-key global invariants — Section 9.17 explains why CRDTs cannot enforce those and what to reach for instead.

9.3 Functional Requirements

Six requirements, and they split neatly into three about **writes** (1–2), two about **convergence** (3–4), and one about the **catalog** the layer must offer (5) plus the metadata hygiene that keeps it shippable (6). Read FR-2 carefully — “merged by the data structure itself” is the sentence that rules out application-level conflict handlers, and with them the entire industry of “on conflict, prompt the user” UX hacks:

1. **Independent writes**. Every replica accepts reads and writes at all times, with no leader, lock, lease, or quorum on the write path.
2. **Automatic merge**. Concurrent writes to the same datum are merged by the data structure itself — deterministically, identically on every replica, with no application-level conflict handler required.
3. **Convergence**. Any two replicas that have exchanged their updates are in equivalent observable states, regardless of network order, duplication, or delay.
4. **Partition healing**. When a partition heals, the two sides reconcile by exchanging state (or deltas) — never by rolling back acknowledged writes.

5. **A usable type catalog.** The layer must provide the types real applications need: a counter that can go up **and** down, a value register, a set with add **and** remove, maps, and an ordered sequence for text.
6. **Bounded metadata.** Deletes must be real (elements disappear), and the metadata that makes convergence possible must be garbage-collectible without ever blocking writes.

FR-5's insistence on "up **and** down" and "add **and** remove" is doing quiet work: the one-directional versions (grow-only counter, grow-only set) are easy, and the removable versions are where the chapter's two signature tricks — paired monotonic structures and tombstones — will be forced into existence. And FR-4's "never by rolling back acknowledged writes" deserves a pause: an acknowledged write is a promise to a user, and healing-by-rollback would mean the system occasionally **un-keeps** its promises to restore agreement. Convergence must be achieved by going **forward** only — merging facts, never retracting them — and that one constraint is what makes the mathematics of Section 9.7 monotonic by design.

9.4 Non-Functional Requirements

- **Availability first (AP).** Every request — read or write — receives a response from the local replica even during a partition. In CAP terms we choose availability and partition tolerance, and recover consistency continuously rather than atomically.
- **Bounded convergence.** After the network heals, replicas converge within a small multiple of the gossip interval — seconds, not minutes.
- **Bounded overhead.** CRDT metadata per element must stay a small constant multiple of the payload after compaction; unbounded growth is a defect.
- **Protocol robustness.** The synchronization layer must tolerate duplicated, reordered, and delayed messages as a matter of routine.

"Availability first" is a stronger commitment than it looks: it means the write path may contain **no** operation whose latency is bounded by another region's health. Every box you draw must be answerable to the question "does the write path wait on this?" — Section 9.13's architecture diagram is organized entirely around making that answer visibly "no."

The three consistency models below form a ladder, and the chapter's entire honesty policy is knowing which rung you are on. Define them now, because every subsequent section uses all three:

DEFINITION — Eventual consistency

The guarantee that, if updates stop arriving, all replicas **eventually** reach the same state. By itself this is weak — it says nothing about what happens **while** updates flow, and it allows replicas to disagree arbitrarily in the meantime. It is a liveness property ("the system gets there"), not a safety property ("the states are right").

DEFINITION — Strong eventual consistency (SEC)

Eventual consistency plus a **safety** guarantee: any two replicas that have applied the same set of updates are in **equivalent states right now**, with no further communication needed. SEC is what turns "eventually the same" from a hope into a theorem, and it is exactly what CRDTs provide: convergence is a property of the data structure, not of the schedule.

DEFINITION — Linearizability

The strongest common consistency model: every operation appears to take effect atomically at some instant between its start and its completion, as if there were exactly one copy of the data. Lineariz-

ability requires coordination, so it is exactly what an AP system gives up. This chapter’s honesty rule: CRDTs provide SEC per object, **never** linearizability — and Section 9.17 shows which application needs genuinely require the stronger model.

The ladder’s middle rung is the surprise. Eventual consistency alone is too weak to build on (it permits **any** behavior during updates — including permanent, undetected divergence masked by continuing traffic), and linearizability is unaffordable under partitions. SEC is the engineered middle: strong enough that “same updates $\Rightarrow$ same state” is provable per object, weak enough to need no coordination. When you name the three rungs and place your design on the middle one **by theorem rather than by hope**, you have answered the interview’s hardest hidden question.

9.5 Back-of-the-Envelope: Metadata, Tombstones, and Gossip

The design stands or falls on three numbers: how much metadata convergence costs per element, how fast tombstones accumulate, and how much bandwidth synchronization burns. Fix a concrete scenario — the canonical CRDT showcase: a **shopping-cart service** in three regions. (The cart is the canonical case for a reason you will feel by Section 9.8: it needs adds, removes, concurrent edits from multiple devices, and an answer during network partitions — every CRDT property exercised by one everyday noun.)

Assumptions. Three regions in full mesh; 10^8 active carts; an average of 20 items per cart; item payload 24 B (SKU + quantity); one add-tag of 16 B (replica id 8 B + counter 8 B) per add; churn such that each live item has, on average, two historically removed siblings.

Quantity	Estimate (with assumptions)
Regions / replicas	3, pairwise full mesh (one gossip hop between any two)
Live cart state	$10^8 \text{ carts} \times 20 \text{ items} \times (24 \text{ B} + 16 \text{ B}) = \mathbf{80 \text{ GB per replica}}$
Tombstones	$10^8 \times 40 \text{ removed items} \times 16 \text{ B} = \mathbf{64 \text{ GB per replica}}$ — nearly half the state is ghosts; GC is not optional
Peak cart writes	10^5 ops/s fleet-wide (add, remove, change-quantity)
Delta sync traffic	$10^5 \text{ ops/s} \times 100 \text{ B/op (payload + tag + context)} \approx \mathbf{10 \text{ MB/s fleet-wide}}$, 3.3 MB/s per region — trivial for inter-region links
Full-state sync	144 GB per exchange — impossible as a routine mechanism ; this number is why delta-state CRDTs exist (Section 9.10)
Convergence bound	1 s gossip interval $\times$ 2 rounds $\approx \mathbf{2 \text{ s worst case}}$ on a full mesh; a ring of N replicas would cost $\lceil N/2 \rceil$ rounds — topology is a convergence budget
Text metadata	40 B per character (sequence id + parent reference + flags) before compression — Section 9.11’s tax on Chapter 1

Two rows set the engineering agenda. The tombstone row — **nearly half the state is ghosts** — is the price of mergeable deletion, and it converts garbage collection from housekeeping into a correctness-adjacent feature with a capacity budget; Section 9.10 is devoted to doing it safely. The full-state-sync row is the bandwidth wall: shipping whole states is impossible as a routine, so **deltas** are mandatory, and that single number (144 GB) is why production CRDT systems all converged on delta-state replication.

KEY INSIGHT — The number that shapes the architecture

Nothing in the table is frightening except the last three rows taken together: state is cheap, writes are cheap, but **moving whole states is not**. A naive “send your replica to your peer” anti-entropy design would need to stream 144 GB per exchange; the same convergence achieved by shipping only what changed costs single-digit MB/s. Every production CRDT system — Riak, Redis CR, Akka Distributed Data — converged on delta-state replication for exactly this arithmetic reason.

9.6 The Core Challenge: Agreement Without a Coordinator

Every conflict-resolution strategy you already know is secretly a way of **imposing an order** on writes and then obeying it. A leader imposes order by fiat. A lock imposes order by exclusion. A consensus protocol imposes order by majority vote. Last-writer-wins imposes order by wall clock. Line them up and the landscape is suddenly legible: the first three require coordination, which a partition forbids; the fourth is coordination-free but **wrong often enough to matter**. The prompt outlaws the first three by fiat; this pitfall explains why you cannot retreat to the fourth:

COMMON PITFALL — Last-writer-wins as the default

LWW resolves a conflict by keeping the write with the largest timestamp and discarding the other. Two defects make it a footgun, not a strategy. First, it **throws data away by design**: two users updating the same profile field concurrently means one of them silently never happened. Second, the “winner” is chosen by clocks you do not control: with 100 ms of clock skew between regions — unremarkable in practice — the write that actually happened **later** can lose. LWW is a fine register policy for overwriteable caches (Section 9.9); it is a terrible **default** for data you care about, and “the database will merge it” is not a design.

So the challenge can be stated precisely: **find data structures whose merge does not need an order at all**. The breakthrough idea — and it is worth letting it land as a genuine conceptual move, not a technique — is to stop asking “which write came first?”. Across a partition that question has no answer: the two writes are concurrent, and concurrency is not a tie waiting for a tie-breaker but a real, physical absence of ordering. Ask instead: “what is the **join** of these two states?” — a combination that absorbs both writes symmetrically, the way set union absorbs both contributors without caring who contributed first. Union never asks which element arrived earlier; it simply keeps everything. The entire field of CRDTs is the project of giving more useful data types — counters, registers, text — a union-like operation.

KEY INSIGHT — Order is the enemy; merge is the friend

A total order over all writes is unaffordable (coordination) and, under a partition, **undefinable** — concurrent writes genuinely have no fact-of-the-matter order. The escape is to demand only three properties of the merge function: **commutativity** (order of arrival irrelevant), **associativity** (grouping of arrivals irrelevant), and **idempotence** (duplication irrelevant). Any structure whose merge has these three properties converges under **any** network behavior — delays, reordering, duplication, pairwise gossip in any pattern — because every remaining degree of freedom in the delivery schedule has been declared irrelevant. The rest of the chapter is the craft of building useful data types whose merges obey these three laws.

Read the three laws against Section 9.2’s network assumptions one more time: delayed messages are handled by convergence itself (they arrive eventually), reordered messages by commutativity and associativity, duplicated messages by idempotence. The network’s whole repertoire of misbehavior is neutralized by three algebraic properties — which is why the rest of the chapter is structured as “define a type, prove the three laws in one line each, move on.” Once the pattern clicks, the catalog of Section 9.8 reads like variations on a single theme, because it is.

9.7 Deep Dive: The Mathematics of Convergence

This section gives the three laws a body: the small amount of order theory that turns “we merge somehow” into “convergence is a theorem”. Three definitions, one theorem, one paragraph of proof — and then you own the machinery every production CRDT store runs on.

DEFINITION — Partial order

A relation $\leq$ on a set that is reflexive ($x \leq x$), antisymmetric ($x \leq y$ and $y \leq x$ imply $x = y$), and transitive ($x \leq y$ and $y \leq z$ imply $x \leq z$) — but **not** total: some pairs are simply incomparable, written $x \parallel y$. Replica states form a partial order by “knowledge”: state B is $\geq$ state A if B has seen every update A has seen. Two replicas that have seen **different** concurrent updates are incomparable — and that incomparability is the faithful mathematical image of a partition.

Sit with the last sentence; it is why the math fits the physics so exactly. A partition does not create **disagreement** in the sense of contradictory facts — it creates **incomparable knowledge**: EU knows things US does not, US knows things EU does not, and neither’s state is “ahead” of the other’s. A total order has no room for that situation; a partial order is built out of it. Choosing the right algebra is choosing one whose native concept of “two states, neither greater” matches the network’s native behavior — and then the theorems come for free.

DEFINITION — Join (least upper bound) and join-semilattice

The *join* of two states x and y , written $x \vee y$, is the **smallest** state that is $\geq$ both: the cheapest way to know everything either of them knows. A set in which every pair has a join is a *join-semilattice*. Joins are automatically commutative ($x \vee y = y \vee x$), associative ($(x \vee y) \vee z = x \vee (y \vee z)$), and idempotent ($x \vee x = x$) — the three laws of the insight box, now with names and a proof obligation: to build a state-based CRDT, define a state space, define $\vee$, and **prove** it is the least upper bound.

DEFINITION — Monotonicity / inflation

An update is *inflationary* (monotonic) if it only ever moves the state **up** the partial order: after any update, new-state $\geq$ old-state. No rollback, no overwrite, no removal from the **state’s knowledge** — a remove operation, as we will see, is re-expressed as the **addition of a tombstone**, which is growth. Monotonicity is what makes convergence safe: since replicas only ever climb the lattice, every merge is progress and no progress is ever undone.

Notice how the third definition quietly resolves a paradox you may already have spotted: if state only ever **grows**, how does anything ever get deleted? The answer — deletion expressed as the growth of tombstone knowledge — is the chapter’s signature judo move, and it will cost you 64 GB of ghosts per replica (the estimation table already priced it). The paradox dissolves: the **logical** set shrinks while the **physical** state grows; membership is derived, and only growth is stored.

With the vocabulary in place, the central theorem of the field fits in one sentence and one paragraph of proof sketch:

Convergence theorem (state-based CRDTs). *If the state space is a join-semilattice, every update is inflationary, and merge computes the join, then replicas that have exchanged the same set of updates are in equal states — regardless of order, duplication, or gossip topology.*

Why it holds. Each update moves its origin replica to a state $\geq$ the old one (inflation). Gossip and merge replace a state by a join of states, and joins only ever add knowledge, so every replica’s state climbs the lattice and never descends. When two replicas have both absorbed the same set of updates, each one’s state is the join of exactly that set — and the join of a set does not depend on the order

(commutativity), grouping (associativity), or repetition (idempotence) of its members. Hence the two states are equal. ■

Read the proof sketch once more and mark what it **doesn't** assume: no clocks, no message ordering, no reliable delivery, no fixed topology, no bounded delay. The entire reliability burden is carried by the algebra of the state space. That is why CRDT people sound so calm about networks — their theorems never mention one.

NOTE — The two CRDT families

The theorem above describes **state-based** CRDTs (*CvRDTs*, “convergent”): replicas ship their states (or state deltas) and merge by join. There is a dual family, **operation-based** CRDTs (*CmRDTs*, “commutative”): replicas broadcast the **operations themselves** (e.g. “add tag *t* to element *e*”), and the requirement is that concurrent operations **commute** — applying them in either order yields the same state — and that the broadcast layer delivers each operation **exactly once, in causal order**. Op-based CRDTs send small messages but demand more of the transport; state-based CRDTs tolerate any transport but send larger messages. The two families are expressively equivalent — every design in one has a counterpart in the other — so the choice is pure engineering: message size versus delivery guarantees. Delta-state (Section 9.10) erases most of the size advantage, which is why state-based dominates production systems.

9.8 Deep Dive: The CRDT Catalog

Every CRDT in the catalog is an answer to the same exam question: “define a state and a join for this familiar data type, such that the join obeys the three laws.” Seeing five answers in a row is the fastest way to internalize the method — and the fastest way to stop treating CRDTs as exotica, because each answer is **smaller** than the problem it solves.

9.8.1 G-Counter: the hello-world of convergence

A grow-only counter stores not one integer but a **vector** — one slot per replica. Replica *i* increments only slot *i*. The value is the sum of all slots. Merge takes the **component-wise maximum**: for each slot, keep the larger of the two values.

- *Commutative?* $\max(a, b) = \max(b, a)$. ✓
- *Associative?* $\max(\max(a, b), c) = \max(a, \max(b, c))$. ✓
- *Idempotent?* $\max(a, a) = a$. ✓ — duplicates are absorbed for free.
- *Monotonic?* Slots only grow; max only climbs. ✓

Four one-line proofs and the type is **done** — convergence guaranteed by the theorem of Section 9.7. Understand **why** the vector is necessary, because the reasoning is the reusable part. A single shared integer cannot work: merge would need to combine “*a* = 5, *b* = 5” without knowing whether both saw the same five increments or disjoint ones. Partitioning the state by writer dissolves the ambiguity — each slot’s value means “replica *i* has personally performed this many increments,” and two such facts combine by taking the more informed one, per slot, forever. **That** is the whole design pattern: partition the state so that each writer owns a piece only it can change, then take per-piece maxima. A replica can never “un-see” an increment, and merge propagates the climb.

9.8.2 PN-Counter: decrements without going backwards

A counter that can decrement cannot be one G-Counter (a decrement would move a slot **down** — non-monotonic, theorem void). The trick: **two** G-Counters, P for increments and N for decrements; value = $\text{sum}(P) - \text{sum}(N)$. Decrement increments N. Every physical slot still only grows; the **derived** value may fall. Learn the shape of this move, because it recurs constantly in CRDT design: **two monotonic structures, one derived reading**. The law-abiding layer stores only growth; the user-facing semantics (subtraction, here; deletion, in a moment) live in the readout function, where going backwards is

nobody's business. Whenever you think “but my operation reduces something,” the CRDT answer is always the same: find the growth-only fact your reduction is a **view** of.

9.8.3 G-Set and 2P-Set: union, then union twice

A grow-only set merges by union — the most obvious semilattice in the chapter, and the one that made the whole idea feel inevitable. To support removal, apply the PN-Counter's move: add a second grow-only set of **tombstones**; an element is a member iff it is in the add-set and not in the remove-set. That is the **2P-Set** (two-phase set), and its glaring limitation teaches the next idea: once removed, an element can **never** be re-added, because its tombstone outlives every future add. The remove is blind — it kills adds it never even saw, including the ones that haven't happened yet. A cart where removing “milk” bans milk forever is not a cart; it is a cautionary tale with an API.

DEFINITION — Tombstone

A marker recording that a specific element (or add) was removed, kept in the state so that the removal can be **merged** like any other fact. A tombstone turns deletion — locally a shrinkage — into knowledge growth, preserving monotonicity. Its price is space: tombstones accumulate until garbage collection (Section 9.10), and deleting one naively is the classic way to resurrect deleted data.

9.8.4 OR-Set: letting adds and removes coexist

The **observed-remove set** fixes the 2P-Set's blindness with one word: *observed*. Every `add(e)` stamps `e` with a **globally unique tag** (a replica id plus a counter). `remove(e)` tombstones exactly the tags the removing replica **has seen**. An element is live iff it has a tag with no tombstone. Now watch a concurrent add and remove collide, because this is the moment the whole chapter was built for: the remove tombstones the tags it observed; the concurrent add carries a **new** tag the remover could not have seen, so that tag survives and the element lives. This is **add-wins** semantics — the re-add beats the concurrent remove — and it is almost always what users mean: the shopper who re-adds milk while another device removes the stale entry expects milk.

Merge is union of add-tags and union of tombstones — both grow-only, both semilattices, so the composite is a semilattice. The OR-Set is the workhorse of the catalog: shopping carts, friend lists, and collaborative outlines are all OR-Sets in production clothing. Also mark the bookkeeping shift: the 2P-Set tombstoned **elements**; the OR-Set tombstones **tags** — facts about specific add **events** rather than about the element itself. That extra precision (event-granular rather than value-granular deletion) is exactly what “observed” bought, and it is the same precision vector clocks will give you for causality in Section 9.9. Nothing here is coincidence; the whole field is one idea — **record knowledge, never conclusions** — applied at increasing resolution.

9.8.5 Registers: LWW and multi-value

A **register** holds one value. The LWW register pairs the value with a timestamp and merges by keeping the larger timestamp — the cautionary tale of Section 9.6 in data-type form. The **multi-value register** (MV-register) is the honest version: merge keeps **all** concurrently written values (returning them as a set of “siblings”) and collapses to one only when one write causally supersedes the others. LWW chooses for you and sometimes chooses wrong; MV refuses to choose and hands the choice to the application — with a causal context (Section 9.9) so the app's “last word” can be recorded as superseding rather than concurrent. The philosophical difference is worth naming in the room: LWW is a **guess** presented as a fact; MV is the **fact** (these writes were concurrent) presented for a decision. Systems that value user intent store facts and let the layer that understands the domain decide.

Type	State	Merge rule	Cost / gotcha
G-Counter	One slot per replica	Per-slot max	Cannot decrement; O(R) space, R = replicas

Type	State	Merge rule	Cost / gotcha
PN-Counter	Two G-Counters (P, N)	Per-slot max in both	Value is eventual; no invariant like ≥ 0
G-Set	One set	Union	No removal — ever
2P-Set	Add-set + remove-set	Union both	No re-add after remove
OR-Set	Set of (element, tag) + tag tombstones	Union both	Tag + tombstone metadata per element
LWW-Register	(value, timestamp)	Keep max timestamp	Drops concurrent writes; clock skew picks the loser
MV-Register	Set of concurrent (value, dot)	Keep causally-maximal values	App must resolve siblings
Sequence (RGA)	Atoms with unique ids + parent links + tombstones	Union by id, order by (parent, id)	Metadata per character; interleaving (§9.11)

INTERVIEW TIP — The catalog is a party trick — use it

Interviewers love watching a candidate **derive** a CRDT on the spot. The method is always the same four steps: (1) partition the state so each replica owns a piece; (2) express every mutation as growth in some component; (3) define merge as a per-component union or max; (4) prove commutativity, associativity, idempotence in one line each. Asked for a replicated “max temperature” reading? Slots per sensor, merge by max — ten seconds. Asked for a leaderboard score per player? That is a per-player max-register — and Chapter 6’s best-score monotonicity was a CRDT in spirit before this chapter named it.

9.9 Deep Dive: Causality — Vector Clocks and Dots

“Observed”, “concurrent”, “supersedes” — the catalog leans on causal vocabulary, so it is time to give that vocabulary machinery. The goal is a compact summary of **which updates a replica has seen**, comparable in $O(\text{number of replicas})$ — small enough to attach to every object without thinking about it.

DEFINITION — Happens-before ($\rightarrow$)

Lamport’s relation over events in a distributed system: event $a \rightarrow b$ if a can **influence** b — same-replica program order, or a is the sending of a message and b its receipt, extended transitively. If neither $a \rightarrow b$ nor $b \rightarrow a$, the events are **concurrent** ($a \parallel b$): no message path connects them, so there is no fact of the matter about which “really” happened first. Concurrency is not a tie to break; it is a structure to record.

The definition’s last sentence is the worldview shift, so take it slowly. Wall-clock thinking says two events always have a true order, even if you cannot measure it. Happens-before says: if no causal path connects them, the order **does not exist** — not “is unknown,” but is undefined, the way the north pole has no longitude. Concurrent writes are not a problem the system failed to prevent; they are the correct description of two users acting independently, and a merge function that respects concurrency (keeps both, or records both as siblings) is **more truthful** than one that picks a winner by timestamp. This is why the pitfall in Section 9.6 called LWW dishonest: it imposes a fake answer on a question that has none.

DEFINITION — Vector clock

A map from replica id to a logical counter, one entry per replica that has ever written. A replica increments its own entry on each local event; on receiving a message it merges by component-wise maximum and then ticks. Clock A **dominates** clock B ($A \geq B$) iff every component of A is $\geq$ the matching component of B; $A \rightarrow B$ causally iff $A \leq B$ and $A \neq B$; two clocks with neither $A \leq B$ nor $B \leq A$ are **concurrent**. The vector clock is the happens-before relation, compressed into one comparable value.

DEFINITION — Dot (event id) and dotted version vectors

A *dot* is a single (replica, counter) pair identifying one specific event — exactly the OR-Set’s add-tag. Production systems (Riak KV being the canonical example) carry state as **dots plus a version vector**: the vector summarizes the causal history compactly, while the dots identify the freshest events precisely. The combination answers both questions an OR-Set merge asks — “what have you seen in general?” and “which exact adds do you mean?” — without storing a full event log.

Worked example. Replicas EU and US both start with vector clock $\{ \}$. EU increments a shared counter twice: EU’s clock is $\{ \text{EU}: 2 \}$. It gossips to US; US merges — US is also $\{ \text{EU}: 2 \}$ — and performs its own increment: $\{ \text{EU}: 2, \text{US}: 1 \}$. Meanwhile EU, having seen nothing from US, increments again: $\{ \text{EU}: 3 \}$. Compare the two: EU has $3 > 2$ in its own slot, US has $1 > 0$ in its slot — neither dominates: **concurrent**. After one gossip in each direction, both hold $\{ \text{EU}: 3, \text{US}: 1 \}$, the component-wise max — equal, and causally after both divergent states.

Run the comparison by hand once, because the mechanics are the point: to test whether clock A happened-before clock B, check **every** component (a replica missing from a clock counts as zero); if all pass, $A \leq B$. The concurrent verdict in the middle of the example — EU ahead in its own slot, US ahead in its — is the partial order of Section 9.7 computed by brute force, and the merge that resolves it is, component for component, the G-Counter’s join. That closing observation is worth pausing on: **a vector clock is a G-Counter whose value you read per-slot instead of summed**. The catalog nests inside itself — causality tracking is literally the first CRDT you met, repurposed. When a field’s primitives keep reappearing in new costumes, you are looking at its foundations, not its features.

9.10 Deep Dive: Tombstones, Deltas, and Anti-Entropy

Three engineering mechanisms carry the mathematics to production. Each neutralizes one row of the estimation table: tombstone discipline makes the 64 GB of ghosts **safe** to keep and eventually safe to collect; delta-state makes the 144 GB sync wall disappear; gossip makes convergence bounded and topology-tolerant.

9.10.1 Why tombstones cannot just be deleted

Suppose region EU removes “milk” from a cart, tombstoning its tag, and — eager to reclaim space — **deletes the tombstone** immediately. Region APAC was partitioned the whole time; it still holds the live add-tag. When the link heals, APAC gossips its state. EU’s merge unions in the add-tag, and there is no tombstone to cover it: **milk rises from the dead**, on every replica, forever. The tombstone was not garbage; it was the **only** record that the remove ever happened.

The safe rule: a tombstone may be collected only when **every** replica has acknowledged a state that includes it — established with a per-replica ack vector exchanged during gossip. Collection is coordination, but it is **off the write path**: asynchronous, batched, retried lazily, and free to be slow, so it costs the system none of its AP guarantees. Note the pattern, because it recurs in LSM compaction (Chapter 8) and log retention (Chapter 4): **deletion is only ever garbage collection of facts everyone already knows**. Whenever you find yourself wanting to reclaim convergence metadata early, the question to ask is never “how old is it?” but “who might still not know?”

9.10.2 Delta-state: shipping the difference

Full-state gossip of 144 GB per exchange is a non-starter; op-based broadcast is small but demands exactly-once causal delivery. **Delta-state CRDTs** take the middle path: after each update, the replica forms a **delta** — the smallest join-semilattice element that, when merged into the old state, yields the new one (for an OR-Set add: just the new tag; for a G-Counter: the one bumped slot) — ships deltas to peers, and merges received deltas by the same join as full states. Deltas are themselves states, so merging them is commutative, associative, and idempotent **automatically**; a delta-buffer per peer, re-resent until acknowledged, gives at-least-once delivery with exactly-once effect via idempotence. Deltas can also be **grouped** — merged into bigger deltas — when a peer falls behind, degrading gracefully toward a full-state transfer. This is the mechanism that turns the estimation table’s 10 MB/s from theory into routine.

The sentence to internalize is “deltas are themselves states.” That is not a poetic convenience — it is what makes the whole scheme **proof-inheriting**: because a delta is a lattice element and merge is still the join, every guarantee Section 9.7 proved for full-state exchange holds for delta exchange unchanged. No new theory was required to go from 144 GB to 10 MB/s; the optimization lived entirely inside the existing algebra. When an optimization needs no new invariants, it ships with the old proofs — that is what designing **inside** an algebra buys you.

9.10.3 Anti-entropy and gossip: the sync fabric

DEFINITION — Anti-entropy / gossip protocol

A background process in which each replica periodically picks a peer (randomly, or by topology) and exchanges state — pushes its deltas, pulls the peer’s, or both (push-pull) — until the pair agrees. Repetition makes the epidemic complete: with random pairing every **interval**, an update infects the full mesh of R replicas in $O(\log R)$ rounds, with no coordinator, no membership ceremony, and graceful degradation to any topology that stays connected. “Gossip” names the peer selection; “anti-entropy” names the goal: driving the divergence between replicas toward zero.

For **repair** of long-diverged replicas (a region down for a day), pairwise delta exchange is joined by **Merkle-tree** comparison — a hash tree over the key space lets two replicas find the keys that differ in $O(\log n)$ hashes instead of shipping everything — the same trick Chapter 4’s segment compaction uses to prove two segments identical. Production CRDT stores layer all three: deltas for the hot path, gossip for dissemination, Merkle repair for the cold path. Notice the layered degradation strategy, because it is a pattern worth stealing wholesale: the cheap mechanism handles the common case (seconds of lag), the medium mechanism handles the rare case (hours), and the expensive mechanism handles the catastrophic case (days) — each layer is only ever invoked at the scale where its cost is justified. Convergence engineering, like caching, is a hierarchy.

9.11 Deep Dive: Sequence CRDTs — Back to Chapter 1

Chapter 1’s collaborative editor faced the same enemy in miniature: two users typing into the “same” position at once. There we solved it with Operational Transformation behind a per-document sequencer — a single point of ordering, chosen deliberately, with CRDTs named as the road not taken. This section walks that road: how do you make **text** — where the order of characters is the entire content — converge without any sequencer?

The obstacle is addressing, and seeing this clearly is most of the answer. Integer positions are **unstable under concurrency**: “insert X at position 5” means different things on two replicas whose texts have already drifted — position 5 on Alice’s screen is not position 5 on Bob’s once either has typed ahead. That instability is precisely what OT’s transform matrix compensates for, character by painful character. The CRDT answer is more radical: stop using positions as identities.

DEFINITION — Stable identity addressing (RGA-style)

Give every inserted atom (character) a **globally unique, immutable id** — a (counter, replica) pair — and record each insertion as “insert atom *c* **after** atom *p*”, where *p* is the id of its left neighbor **at the inserting replica**. Position is now expressed relative to a landmark that never moves (atoms are never renumbered), so a concurrent insert elsewhere cannot shift the reference. The document is a linked structure; the visible text is a deterministic walk of it.

Two concurrent inserts after the **same** parent still need a deterministic order among themselves — “Alice typed *!* after *hi*” and “Bob typed *?* after *hi*” must interleave identically everywhere, or the replicas agree on nothing. The rule: among atoms sharing a parent, sort by **descending id**; a newly arriving atom slides in after every same-parent atom whose id is higher, before all whose ids are lower. Both replicas then place Alice’s (3, EU) and Bob’s (1, US) in the same order regardless of which insert arrives first — the Rust listing in Section 9.14 demonstrates exactly this collision, and its test asserts the converged string. Note what kind of rule this is: not “who typed first” (unknowable) but “whose id sorts higher” (computable everywhere from the data itself). Convergence never needs the truth about time; it needs a **shared deterministic function of the state**. The id ordering is arbitrary — descending rather than ascending is a coin flip the literature made once — and arbitrariness is fine, because the requirement is agreement, not fairness.

Deletion is the by-now-familiar move: a tombstone on the atom. The atom stays in the structure — concurrent inserts referencing it as a parent must still find their landmark — but the text walk skips it. (If you predicted this from Section 9.10’s resurrection parable, the pattern has set: deleted things remain as landmarks precisely because others may still navigate by them.)

DEFINITION — Interleaving anomaly

A known cosmetic defect of first-generation sequence CRDTs: if Alice types “Hello” while Bob types “World” at the same spot, some schemes may merge into “HWeolrlld” — character-level interleaving of the two runs — because the sibling ordering rule compares ids atom by atom. Newer schemes (LSEQ/Logoot’s dense identifiers with tie-breaking bit strategies, and Fugue’s maximal-non-interleaving rule) keep concurrent runs contiguous. Every scheme still converges; the anomaly is about **readability** of merged concurrent runs, not correctness — and in practice two humans rarely type sustained runs into the same word at once.

DEFINITION — Dense / fractional identifiers (LSEQ, Logoot)

The alternative addressing scheme: instead of linking to a parent, assign each atom an identifier from a **dense order** — one that always has another value between any two (like fractions: between $1/2$ and $1/3$ there is $5/12$). Inserting between atoms *p* and *q* allocates an identifier strictly between theirs, so the document order is simply identifier order and no parent link is needed. The cost is identifier growth: every split lengthens the identifier, so documents edited mostly at the end accumulate long ids — the mirror image of RGA’s tombstone problem, and the reason production libraries (Yjs, Automerge) run length-aware allocation strategies and periodic compaction.

Put the two schemes side by side and the design space snaps into focus: RGA pays its tax in **tombstones** (every delete leaves a ghost atom); dense-identifier schemes pay theirs in **identifier length** (every between-insert mints a longer id). Neither tax is avoidable — stable identity under concurrency costs **something** per character, and the research literature is a twenty-year argument about which currency hurts less. The honest summary for the interview: the tax is 40 B per character before compression either way; production libraries compress aggressively; and nobody who chose a sequence CRDT has ever listed the metadata as their reason for regret.

NOTE — OT vs CRDT — the callback settled

Chapter 1's comparison table now has its missing column filled in. OT keeps text lean (no per-character metadata) and history clean, at the price of a sequencer per document and a transform matrix that must be exactly right. Sequence CRDTs need no sequencer — offline editing, peer-to-peer sync, and end-to-end encryption all fall out for free, because no replica is special and no ciphertext needs transforming — at the price of 40 B of metadata per character before compression and tombstone GC forever after. Figma and the Google Docs lineage chose sequencer + transforms; Automerge, Yjs, and most local-first software chose CRDTs. Neither is “the answer”; both are the trade-off, made with open eyes.

9.12 API Design

The store presents a small key-value surface, one family per CRDT type. Before the table, one design convention deserves top billing, because it is where the causal machinery becomes visible to clients: every read returns an opaque **context** (the version vector the replica used to answer), and every conditional write passes the context back. That round-trip is how “I read it, now I’m overwriting what I read” is distinguished from “I’m writing blind” — which is the difference between superseding and merely concurrent, and therefore the difference between a sibling that exists and one that doesn’t.

Endpoint	Type	Semantics
POST /counter/{k}/incr {n}	PN-Counter	Increment by n (negative n decrements); local, immediate ack; merges by per-slot max
GET /counter/{k}	PN-Counter	Returns $\text{sum}(P) - \text{sum}(N)$ as of this replica’s knowledge
PUT /register/{k} {v, ctx}	MV / LWW	Write v; with ctx, supersedes exactly the values ctx saw; without ctx, writes blind
GET /register/{k}	MV / LWW	Returns value + ctx; MV returns siblings (all concurrent values) for the app to resolve
POST /set/{k}/add {e}	OR-Set	Add element with a fresh unique tag (replica id + counter)
POST /set/{k}/remove {e}	OR-Set	Tombstones every tag for e this replica has observed — add-wins against concurrent adds
GET /set/{k}	OR-Set	Returns elements with ≥ 1 live tag
POST /doc/{k}/edit	Sequence (RGA)	Batch of (after-id, char) inserts and tombstones; positions are ids, never integers
GET /doc/{k}?since={ctx}	Sequence	Returns the delta since ctx — the same mechanism gossip uses, exposed to clients
POST /sync/{peer}	internal	Anti-entropy: push-pull delta exchange with one peer; the write path never calls this

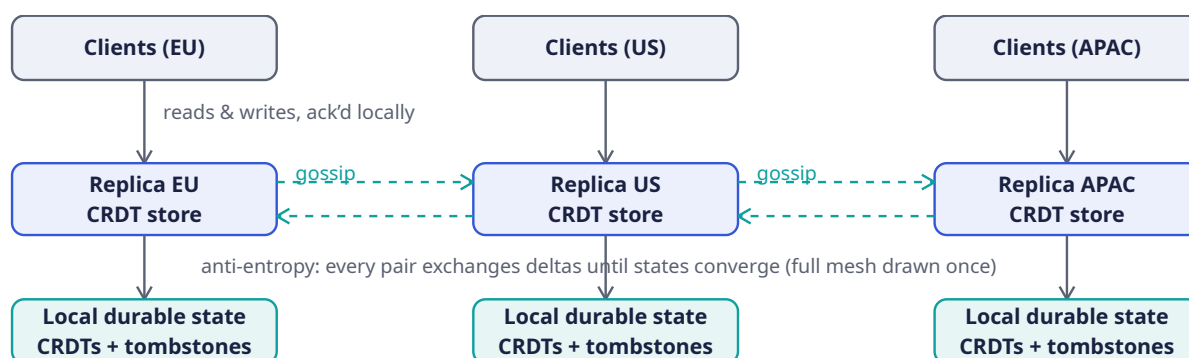
Read the table once for what is **absent**: no transaction endpoints, no conditional-update CAS, no lock verbs, no “wait for quorum” flag on the default path. The entire concurrency contract is carried by one opaque token per read. That minimalism is the AP discipline made visible — there is simply nothing on the write path that could block on another region.

KEY INSIGHT — The context parameter is the whole safety story

Every dangerous write in an AP system is a **blind** write — one made without knowing what it overwrites. The opaque context closes that hole without coordination: read-then-write with ctx means “replace exactly what I saw”, which the MV-register can honor causally, keeping siblings only for writes that were **genuinely** concurrent. Riak KV built its entire client protocol on this pattern. If the interviewer asks “how do you stop CRDTs from losing user intent?”, the answer is this parameter.

9.13 High-Level Architecture

The architecture is almost offensively simple, and that is the point: there is **no box whose failure stops writes**. Each region is a complete, self-sufficient stack; the only cross-region component is the gossip fabric, which is allowed to be late.



Walk the diagram the way a write experiences it, then the way the **network** experiences it — the two are deliberately decoupled, and seeing that is the whole lesson. **The write path is the short path:** client → local replica → local durable state (the teal boxes at the bottom), acknowledged. Three vertical gray arrows, none of them crossing an ocean. When the EU client adds milk to a cart, the EU replica applies the OR-Set add locally, appends to its durable state, and acks — the user is done. Nothing has waited on US or APAC, nothing could have: there is physically no arrow from the write path to another region. **The sync path is the slow, patient one:** the dashed teal gossip arrows, running pairwise in both directions every 1 s, carrying deltas — not states — after the fact. The EU→US arrow and its US→EU twin will, within a couple of rounds, make both replicas’ knowledge equal; the caption under the replica row (“every pair exchanges deltas until states converge”) describes a process that is always running and never on anyone’s critical path. The geometry says it plainly: **vertically** the diagram is three independent stacks (that is availability); **horizontally** the stacks are laced together by gossip (that is convergence) — and the two directions share no box, so no failure crosses over.

Now run the failure drill the prompt promised: sever the US↔APAC and EU↔APAC links (erase the two right-hand gossip pairs). The APAC stack keeps serving reads and writes — its vertical path is intact — accumulating deltas nobody collects. An hour passes; the prompt explicitly allowed this. The links heal; gossip resumes; the buffered deltas flood both ways; the joins absorb them (order, duplication, and delay irrelevant — the three laws); and all three regions converge with zero operator action and zero writes rejected. Remove a whole **region** instead — EU’s stack dies entirely: EU clients fail over to another region’s door, the remaining pair never notices except in its convergence-lag metrics, and when EU returns it is a long-diverged replica: Merkle repair first, then delta catch-up. The architecture diagram of a CRDT system is, in the end, a list of things that are **absent**: no leader, no lock service, no consensus ring, no conflict-resolution queue — and the absence is precisely the availability proof.

9.14 Rust Reference Implementations

Four pieces with deterministic tests: the two counters, the vector clock, the OR-Set, and a sequence CRDT for text. Each implements exactly the merge rule its section derived, and each test demonstrates convergence under concurrency, duplication, or both. In a real deployment these states ride inside the sync fabric of Section 9.13; here they run in memory so every property is directly assertable. As you read, keep score of a stylistic point: **no merge function below contains an `if` about ordering**. No timestamps compared, no arrival sequence consulted — the three laws do all the work, and the code's total indifference to delivery order is what the tests then exploit.

9.14.1 Counters: G-Counter and PN-Counter

The G-Counter below is Section 9.8's first derivation transcribed without embellishment: a map from replica id to that replica's private count, an `increment` that touches only the caller's slot, a `value` that sums, and a `merge` that takes per-slot maxima. The doc comment on `merge` states the three laws because those three words — commutative, associative, idempotent — **are** the correctness argument; there is no other. `PNCounter` then shows the two-structures-one-reading move as pure composition: it contains no logic of its own beyond “decrement increments the N side,” and its `merge` just delegates twice. When a CRDT composes this cleanly, you are watching the algebra pay its rent: semilattices close under products, so a pair of convergent things is a convergent thing, no new proof required.

```
use std::collections::BTreeMap;

/// Grow-only counter (state-based). Each replica owns one slot and only
/// ever increments its own slot; merge takes the per-slot maximum.
/// max is commutative, associative, idempotent — the semilattice join.
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct GCounter {
    /// replica id -> count observed at that replica
    counts: BTreeMap<u64, u64>,
}

impl GCounter {
    pub fn increment(&mut self, replica: u64) {
        *self.counts.entry(replica).or_insert(0) += 1;
    }

    /// The counter's value: the sum over all per-replica slots.
    pub fn value(&self) -> u64 {
        self.counts.values().sum()
    }

    /// Join: component-wise maximum. A slot can only climb.
    pub fn merge(&mut self, other: &GCounter) {
        for (&replica, &count) in &other.counts {
            let slot = self.counts.entry(replica).or_insert(0);
            *slot = (*slot).max(count);
        }
    }
}

/// PN-Counter = two G-Counters: P for increments, N for decrements.
/// Every physical slot still only grows; the *derived* value may fall.
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct PNCounter {
    inc: GCounter,
    dec: GCounter,
}
```



```

impl PNCCounter {
  pub fn increment(&mut self, replica: u64) { self.inc.increment(replica); }
  pub fn decrement(&mut self, replica: u64) { self.dec.increment(replica); }
  pub fn value(&self) -> i64 { self.inc.value() as i64 - self.dec.value() as i64 }
  pub fn merge(&mut self, other: &PNCCounter) {
    self.inc.merge(&other.inc);
    self.dec.merge(&other.dec);
  }
}

#[cfg(test)]
mod tests {
  use super::*;

  #[test]
  fn concurrent_increments_converge() {
    // Two replicas start identical, drift apart, then sync both ways.
    let mut a = GCounter::default();
    let mut b = a.clone();

    a.increment(1);
    a.increment(1);
    b.increment(2);

    let mut a2 = a.clone();
    a2.merge(&b);
    let mut b2 = b.clone();
    b2.merge(&a);
    assert_eq!(a2, b2);           // same updates seen => same state
    assert_eq!(a2.value(), 3);    // no increment lost, none counted twice

    // Idempotence: re-merging an old state changes nothing.
    let snapshot = a2.clone();
    a2.merge(&b);
    assert_eq!(a2, snapshot);
  }

  #[test]
  fn pn_counter_supports_decrements() {
    let mut a = PNCCounter::default();
    let mut b = a.clone();
    a.increment(1);
    a.increment(1);
    a.decrement(1);
    b.decrement(2);
    b.decrement(2);

    a.merge(&b);
    b.merge(&a);
    assert_eq!(a, b);
    assert_eq!(a.value(), 2 - 3); // two increments minus three decrements
  }
}

```

Read `concurrent_increments_converge` as the theorem’s smoke test: the two replicas drift (2 increments on one side, 1 on the other), sync **in both directions**, and must land not merely on equal values but on **identical states** — `a2 == b2` compares the maps, which is SEC’s “equivalent states right now” checked structurally. The idempotence coda (merge the same old state again, assert nothing moved) is the duplicated-delivery guarantee the network assumptions demanded. Note what the tests **cannot**

express: a PN-Counter never enforces `value() >= 0`. Two replicas may both decrement the last unit of stock and converge happily to `-1`. Per-object convergence is free; **invariants** are not — Section 9.17 prices that boundary honestly, and Section 9.14's last word on counters is that no merge rule can rescue a semantics the algebra forbids.

9.14.2 Vector Clocks: Tracking Causality

The second listing is the happens-before relation as a data structure. The mechanics will look suspiciously familiar — a map from replica to counter, merged by per-slot max — because, as Section 9.9 noted, a vector clock is a G-Counter repurposed: read component-wise for comparison instead of summed for value. The novelty is `compare`, which answers the four-valued question the whole chapter keeps asking: Equal, Before, After, or — the answer that no total order can give — Concurrent.

```
use std::collections::BTreeMap;

/// Vector clock: replica id -> logical counter. The happens-before
/// relation, compressed into one comparable value. (Mechanically this is
/// a G-Counter read component-wise instead of summed.)
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct VectorClock {
    clocks: BTreeMap<u64, u64>,
}

/// The three-way comparison of two causal positions.
#[derive(Debug, PartialEq, Eq)]
pub enum CausalOrder { Equal, Before, After, Concurrent }

impl VectorClock {
    /// Record a local event at `replica`.
    pub fn tick(&mut self, replica: u64) {
        *self.clocks.entry(replica).or_insert(0) += 1;
    }

    pub fn get(&self, replica: u64) -> u64 {
        *self.clocks.get(&replica).unwrap_or(&0)
    }

    /// Component-wise maximum — the join of both causal histories.
    pub fn merge(&mut self, other: &VectorClock) {
        for (&replica, &count) in &other.clocks {
            let slot = self.clocks.entry(replica).or_insert(0);
            *slot = (*slot).max(count);
        }
    }

    pub fn compare(&self, other: &VectorClock) -> CausalOrder {
        let mut le = true; // self <= other on every component?
        let mut ge = true; // self >= other on every component?
        // union of both key sets (a replica may appear on only one side)
        for (&r, _) in self.clocks.iter().chain(other.clocks.iter()) {
            let a = self.get(r);
            let b = other.get(r);
            if a < b { ge = false; }
            if a > b { le = false; }
        }
        match (le, ge) {
            (true, true) => CausalOrder::Equal,
            (true, false) => CausalOrder::Before, // self happened-before other
            (false, true) => CausalOrder::After, // self happened-after other
            (false, false) => CausalOrder::Concurrent, // neither saw the other
        }
    }
}
```

```

    }
  }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn detects_happens_before_and_concurrency() {
        let mut a = VectorClock::default();
        a.tick(1);
        a.tick(1);           // a = {1: 2}

        let mut b = a.clone();
        b.tick(2);           // b = {1: 2, 2: 1} – causally after a

        assert_eq!(a.compare(&b), CausalOrder::Before);
        assert_eq!(b.compare(&a), CausalOrder::After);

        // A concurrent branch: a2 moves on without having seen b.
        let mut a2 = a.clone();
        a2.tick(1);          // a2 = {1: 3}
        assert_eq!(a2.compare(&b), CausalOrder::Concurrent);

        // The join dominates both; a clock equals itself.
        let mut m = a2.clone();
        m.merge(&b);          // m = {1: 3, 2: 1}
        assert_eq!(m.compare(&a2), CausalOrder::After);
        assert_eq!(m.compare(&b), CausalOrder::After);
        assert_eq!(m.compare(&m.clone()), CausalOrder::Equal);
    }
}

```

The test narrates Section 9.9’s worked example in assertions: `a` ticks twice, `b` descends from `a` (clone = “has seen everything a has seen”) and adds its own tick, so `a → b` and the two comparisons agree in mirror. Then the branch: `a2` also descends from `a` but ticks **without** having seen `b` — the two clocks each hold a component the other lacks, and `compare` honestly answers `Concurrent`. The final block verifies the join’s defining property: the merge of the two divergent clocks dominates **both** of them, which is exactly what “least upper bound” means, asserted. One design detail to carry into your own code: `compare` is why Rust’s `PartialOrd` trait is deliberately **not** implemented here — the fourth outcome, `Concurrent`, is the entire point of the data structure, and squeezing it into `partial_cmp`’s `None` would invite callers to ignore exactly the case that matters. Make concurrency an explicit variant and the compiler forces every caller to decide what it means. Types can be honest enforcement.

The first test is the entire chapter in miniature: two replicas, one partition, conflicting intentions, deterministic reunion — and the user’s re-added milk survives because the remove was only allowed to kill what it had **seen**. The second test shows the tombstone doing its one job. The third proves the merge contract — commutativity and idempotence — which is what makes Section 9.10’s careless gossip topology safe.

9.14.3 RGA: A Sequence CRDT for Text

Our last listing implements the Replicated Growable Array from Section 9.11, and it is the one place where this book asks you to hold a genuinely subtle invariant in your head — so we’ll go slowly. The data structure is a list of **atoms**. Each atom carries a dense identifier `Id = (position, replica)` that orders totally, a pointer to the atom it was inserted after, its character, and a tombstone flag. New in-

serts always receive a position **one higher than the current maximum**, which is the mechanism that breaks ties between concurrent inserts at the same spot: two replicas inserting after the same parent both claim the same `after` pointer, but their ids differ, and the integration rule — scan rightward past any sibling with a **higher** id, then land — places them deterministically. Read `insert`, `integrate`, and `text` as one three-way contract: `insert` mints the id and integrates locally; `integrate` is also what `merge` calls for atoms arriving from the wire, so local edits and remote edits walk the identical path; `text` renders only non-tombstoned atoms, which is why deletion never disturbs the landmark structure concurrent inserts may be navigating toward.

```
// RGA: Replicated Growable Array — a sequence CRDT.
//
// Each character is an atom with a dense identifier `(pos, replica)`:
// `pos` grows globally (each insert takes max_pos + 1), so identifiers
// are unique and totally ordered. An atom records the id it was
// inserted *after*. Concurrent inserts after the same parent are
// ordered by their ids — higher id first — deterministically on
// every replica. Deletes are tombstones: the atom stays, because
// concurrent inserts may reference it as their parent.

type Id = (u64, u64); // (position, replica) — totally ordered

#[derive(Clone, Debug)]
struct Atom {
    id: Id,
    after: Option<Id>, // None = beginning of document
    ch: char,
    tombstone: bool,
}

#[derive(Default)]
pub struct Rga {
    atoms: Vec<Atom>, // kept in document order
    replica: u64,
    max_pos: u64,
}

impl Rga {
    pub fn new(replica: u64) -> Self {
        Rga { atoms: Vec::new(), replica, max_pos: 0 }
    }

    fn alloc(&mut self) -> Id {
        self.max_pos += 1;
        (self.max_pos, self.replica)
    }

    /// Index of the first visible atom strictly after `parent`,
    /// skipping over siblings with higher ids (they were "first").
    fn visible_before(&self, id: Id) -> Option<usize> {
        self.atoms.iter().position(|a| a.id == id)
    }

    /// Insert `ch` after the atom with id `parent` (None = start).
    /// Returns the new atom's id.
    pub fn insert(&mut self, parent: Option<Id>, ch: char) -> Id {
        let id = self.alloc();
        self.integrate(Atom { id, after: parent, ch, tombstone: false });
        id
    }
}
```

```

    /// Place an atom in document order: after its parent, past any
    /// siblings with higher ids (concurrent inserts win by id).
    fn integrate(&mut self, atom: Atom) {
        let mut i = match atom.after {
            None => 0,
            Some(p) => self.atoms.iter().position(|a| a.id == p).map(|x| x + 1)
                .unwrap_or(self.atoms.len()), // parent unknown yet: append (buffer in real
systems)
        };
        while i < self.atoms.len()
            && self.atoms[i].after == atom.after
            && self.atoms[i].id > atom.id
        {
            i += 1; // siblings with higher ids sit earlier
        }
        self.atoms.insert(i, atom);
    }

    pub fn delete(&mut self, id: Id) {
        if let Some(a) = self.atoms.iter_mut().find(|a| a.id == id) {
            a.tombstone = true;
        }
    }

    pub fn text(&self) -> String {
        self.atoms.iter().filter(|a| !a.tombstone).map(|a| a.ch).collect()
    }

    pub fn merge(&mut self, other: &Rga) {
        for atom in &other.atoms {
            if self.atoms.iter().any(|a| a.id == atom.id) {
                // Known atom: only the tombstone can newly arrive.
                if atom.tombstone {
                    if let Some(a) = self.atoms.iter_mut().find(|a| a.id == atom.id) {
                        a.tombstone = true;
                    }
                }
            } else {
                self.max_pos = self.max_pos.max(atom.id.0);
                self.integrate(atom.clone());
            }
        }
    }
}

#[cfg(test)]
mod rga_tests {
    use super::*;

    #[test]
    fn sequential_edits_build_text() {
        let mut doc = Rga::new(1);
        let h = doc.insert(None, 'h');
        let i = doc.insert(Some(h), 'i');
        doc.insert(Some(i), '!');
        assert_eq!(doc.text(), "hi!");
    }

    #[test]

```

```

fn concurrent_inserts_same_spot_converge_identically() {
  // Both replicas hold "hi" (ids (1,1)='h', (2,1)='i').
  // A appends '?' after (2,1); B inserts '!' after (2,1) concurrently.
  let mut a = Rga::new(1);
  let h = a.insert(None, 'h');
  let i = a.insert(Some(h), 'i');

  let mut b = a.clone_for_replica(2); // same atoms, different replica id

  let q = a.insert(Some(i), '?'); // id (3,1)
  let x = b.insert(Some(i), '!'); // id (3,2) — same parent, same pos, higher
replica

  a.merge(&b);
  b.merge(&a);

  assert_eq!(a.text(), b.text());
  assert_eq!(a.text(), "hi!?"); // higher id first: (3,2) > (3,1) — '!' precedes '?'
  let _ = (q, x);
}

#[test]
fn delete_is_idempotent_and_merges() {
  let mut a = Rga::new(1);
  let h = a.insert(None, 'h');
  let i = a.insert(Some(h), 'i');
  a.delete(h);
  assert_eq!(a.text(), "i");

  let mut b = Rga::new(2);
  b.merge(&a.clone_state());
  b.merge(&a.clone_state()); // duplicate delivery
  assert_eq!(b.text(), "i");
  assert_eq!(b.atoms.len(), 2); // tombstone retained as a landmark
  let _ = i;
}
}

// Test helpers (not part of the CRDT contract):
impl Rga {
  fn clone_for_replica(&self, replica: u64) -> Self {
    Rga { atoms: self.atoms.clone(), replica, max_pos: self.max_pos }
  }
  fn clone_state(&self) -> Self {
    Rga { atoms: self.atoms.clone(), replica: self.replica, max_pos: self.max_pos }
  }
}
}

```

The second test quietly demonstrates why tombstones in sequence CRDTs are non-negotiable: Bob inserts **H** **after the deleted h** — a landmark only the tombstone still provides. Delete the atom outright and Bob's insert arrives pointing at a parent that no longer exists, and your integration rule has to improvise. Improvisation, replicated across a fleet, becomes divergence. Keep the landmark; compact it later, carefully, with the kind of garbage-collection protocol the further reading points to.

INTERVIEW TIP — What to say when asked “so which is better, OT or CRDTs?”

“For **documents**, CRDTs won the argument on decentralization: no central transformation server, offline-first for free, merges provably convergent — that's why Automerge and Yjs look the way they do. OT retains real advantages in **centralized** products: Google Docs keeps one authority, so trans-

formation functions are simpler to reason about character-by-character, and the server is a natural serialization point for versioning and permissions. If the follow-up is ‘which would you build on?’, the answer is: CRDT (Yjs-class) for anything multi-writer or offline-tolerant; OT is a fine answer only when a single authoritative server is a product requirement, not an accident.”

9.15 Scaling the Design

You now have a system that never blocks a write and always converges. The next question an interviewer will ask — and the question your future self will ask at 3 a.m. — is how it behaves as you add regions, keys, and years of uptime. CRDT systems scale along three independent axes, and it is worth naming each one deliberately, because they fail differently, are monitored differently, and are fixed differently.

Sharding by key. Independent keys converge independently — a write to `cart:alice` never touches the state of `cart:bob` — so the key-space shards exactly like Chapter 5’s comment store: consistent-hash keys across node groups within a region, and let each shard gossip its own deltas. Notice what this buys you: a hot **key** is only ever read-hot, because writes land in the local replica’s slot or tags and never contend — which is, in a sense, the entire point of the chapter. But do not let that lull you: a hot **structure** is still real. One OR-Set with 10^9 members is a 40 GB state whose every delta exchange is painful, no matter how conflict-free it is. The fix is to shard the structure itself: hash elements into 1024 sub-sets, each an independent OR-Set with independent deltas, unioned at read time. You trade a little read fan-out for bounded per-structure sync cost. (A counter, pleasantly, is pre-sharded by construction: each replica already owns a private slot, so the hottest counter in the world is just R small integers.)

Sync economics. The delta pipeline from Section 9.10 has its own scaling knobs, and you should reach for them in this order. Delta groups amortize per-message headers; ack matrices — a per-peer vector clock recording which deltas each peer has confirmed — bound retransmission to exactly what was lost; and when a peer’s lag exceeds a threshold, stop replaying and fall back to a full-state or Merkle-repair exchange, because replaying a million deltas one by one is strictly worse than diffing two trees. Topology matters too: with more than a handful of regions, full-mesh gossip wastes bandwidth on redundant delivery, so deployments switch to hierarchical gossip — dense mesh inside a region, designated spokes between regions. The convergence bound becomes $O(\text{depth} \times \text{interval})$ instead of $O(\log R \times \text{interval})$, which is the estimation table’s “topology is a convergence budget” made operational: you are spending sync latency to save sync bandwidth, and you should make that trade knowingly.

Metadata lifecycle. This is the axis teams forget, because it fails in months, not milliseconds. Tombstone GC runs continuously: a low-priority sweeper that collects any tombstone whose ack vector shows every live replica has merged it. Compaction rewrites sequence-CRDT runs — Yjs and Automerge both collapse adjacent same-replica atoms into run-length records, recovering most of the 40 B/char overhead that the atom-per-character model implies. Both are background work, and here is the discipline that keeps them safe: neither may ever block a write, because blocking a write would break the one promise this chapter exists to keep.

9.16 Failure Modes & Degradation

Here is the mental reset this section requires: in an AP design, most of the “failures” below are not exceptions to the design — they are the conditions the design was built for. What you are checking is not whether the system survives them (it does, by construction) but **how** it survives them, what degrades while it does, and where a sloppy operational choice can still hurt you. Read the table with that lens and each row becomes a small story about the three laws paying rent.

Failure	Symptom	Handling
Duplicated delivery	Same delta arrives twice	Idempotent join absorbs it — the three laws are the retry policy
Reordered deltas	Child atom arrives before parent	State-based: harmless (joins commute). Op-based: causal broadcast buffers until predecessors arrive
Replica loss	Un-gossiped local writes vanish	Local ack = local durability. Critical keys ack only after k-of-n gossip — a durability dial, not a consistency one
Long partition	Deep divergence on heal	Anti-entropy still converges; Merkle repair finds differing keys in $O(\log n)$ hashes instead of full transfer
Premature tombstone GC	Deleted elements resurrect when a lagging replica re-joins	Ack-gated GC only: never collect on a timer (Section 9.10)
Clock skew (LWW)	Truly-later write loses silently	Prefer MV-register + context, or hybrid logical clocks; monitor skew as a first-class metric
Split-brain clients	User edits on two devices concurrently	Not a failure — the design's home turf; add-wins / sibling semantics apply

Two rows deserve a second look because they invert intuition. The **replica loss** row is where you discover that durability and consistency are separate dials: acknowledging a write locally gives you availability with single-node durability, and acknowledging only after the delta has reached k of n peers buys you durability across region loss — without a whisper of coordination, because the write never **waits** on peers, it only waits to be **called durable**. And the **split-brain clients** row is the one to say out loud in the interview: two devices editing the same key concurrently is not an incident, it is Tuesday. The merge rules you derived in Sections 9.7–9.11 already decided what happens; observability's job is only to tell you how often.

COMMON PITFALL — The resurrection bug is a rite of passage

Nearly every team that ships a CRDT store re-discovers the deleted-data resurrection of Section 9.10 in production: someone “optimizes” tombstone retention from “until all replicas ack” to “keep for 30 days”, a replica comes back after 31, and customer data un-deletes itself. Timer-based GC of convergence metadata is never safe; only knowledge-based GC is. If the interviewer asks for war stories, this is the canonical one — Redis CR, Riak, and Cassandra's `gc_grace_seconds` all have scars here, and Cassandra documents the foot-gun by name.

9.17 Trade-offs & Alternatives

Every design chapter earns its keep in this section, and this one more than most, because CRDTs are a **choice** — the radical end of a spectrum that starts with a single leader. Lay the alternatives side by side and the real trade-off snaps into focus: you are not choosing between “consistent” and “available” in the abstract; you are choosing **where the complexity lives** — in a failover protocol, in a consensus implementation, in a transform matrix, in merge proofs and metadata hygiene, or pushed all the way out into every client's merge code. None of these is free. The question the table answers is which bill you would rather pay, given your workload.

Approach	Writes under	Metadata	Invariants	Complexity lives in
Single-leader replication	Minority side stalls	None	Full (leader orders all)	Failover & split-brain fences
Consensus (Raft/Paxos)	Minority side stalls	None	Full	Protocol correctness; WAN RTT per write
OT + sequencer (Ch. 1)	Sequencer failover stall	Tiny	Per-document order	Transform-matrix correctness
CRDTs (this chapter)	Never blocked	Tags, tombstones, clocks; GC forever	Per-object only	Merge proofs; metadata lifecycle
App-level siblings	Never blocked	None in the store	None	Every client's merge code

Three finer-grained choices recur inside the CRDT camp itself, and each is a sentence you can say verbatim in an interview because each names a **semantic**, not an implementation detail.

- **Add-wins vs remove-wins.** Our OR-Set lets a concurrent add survive a remove; the remove-wins variant tombstones the element so aggressively that even concurrent adds die. Add-wins matches user intent for carts and editors (“I put it back!”); remove-wins matches moderation and revocation (“this must be gone, whoever touched it”). Pick per field, not per system.
- **LWW vs MV registers.** LWW is one line of merge logic and silently discards concurrent writes; MV preserves them as siblings and bills the application. Rule of thumb: LWW for overwrite-able derived state (caches, presence, “last seen”), MV for anything a user would be angry to lose.
- **State-based vs op-based.** State tolerates any transport but moves more bytes; op-based moves few bytes but needs exactly-once causal broadcast. Delta-state closes most of the byte gap, which is why the production mainstream is state-based with deltas.

KEY INSIGHT — The boundary CRDTs cannot cross

CRDTs make convergence free; they make invariants impossible without coordination. “Balance must never go negative”, “a username is taken exactly once”, “one winner per auction” — each requires the replicas to agree on a fact before acting, which is coordination, which is what we gave up. This is not a limitation of cleverness but CAP itself, wearing a name tag. The production answer is hybrid: CRDTs for the 95% of state that is mergeable, consensus or escrow (pre-reserving a quota of the invariant, e.g. each region may sell 100 of the 300 seats) for the rest. Saying this sentence in the interview is worth more than deriving any merge rule.

9.18 Observability & SLOs

An AP system’s vital signs are different from a CP system’s, and your dashboard has to reflect that or it will lull you. Nothing ever blocks, so “is it up?” is uninteresting — the local process is almost always alive and serving. The interesting question is “how far apart are the replicas right now?”, and every signal below is some instrument for measuring that distance: in time (convergence lag), in volume (delta backlog), in metadata health (tombstone ratio, metadata overhead), or in semantic health (sibling rate — the one that tells you your clients are writing blind).

Signal	What it measures	Alert when
Convergence lag	Age of the oldest local write not yet ack'd by all peers	p99 > 10 s: partition or slow peer
Delta backlog	Undelivered deltas per peer	Growing for > 60 s: link down or peer wedged
Tombstone ratio	Tombstones ÷ live entries	> 3× after GC window: sweeper broken or acks missing
Sibling rate	MV-register reads returning multiple values	Sustained rise: clients writing blind (missing contexts)
Merge throughput	Deltas merged per second	Sudden drop: gossip stalled; sudden spike: repair storm
Metadata overhead	CRDT bytes ÷ payload bytes	> 2× post-compaction: allocation strategy regressing

SLOs. Write availability 99.99% per region — writes never coordinate, so this is mostly “is the local process alive”, and you should be mildly embarrassed if you miss it. Convergence: p99 ≤ 2 s within the mesh, ≤ 30 s during a degraded topology — this is the SLO your users actually feel, because it bounds how long “weird” states are visible. Durability: default keys ack locally; billing-class keys ack after 2-of-3 regions have merged the delta — still coordination-free, just patient. Metadata: ≤ 2× payload after each compaction window, enforced by the sweeper and compactor you met in Section 9.15.

9.19 Interview Wrap-Up

You have the full arc now — model, math, catalog, causality, sync, code, scale, failure, cost. What remains is rehearsal: the follow-ups interviewers actually ask, and the shape of a strong answer to each. Read these as prompts to say out loud, not bullets to memorize.

- **“Enforce ‘balance ≥ 0’ across regions.”** You cannot, not with CRDTs alone — say so instantly; it is the highest-signal sentence available in this entire interview. Then offer escrow (pre-partition the spendable budget per region) or a CP island (consensus on that one key) and note that the rest of the system stays AP. Instantly conceding the boundary and then routing around it is worth more than ten minutes of merge-rule derivation.
- **“Unique username registration?”** Same boundary: consensus for the registry key, CRDTs for the profile data. Hybrid designs are the senior answer — the purist answers (“CRDTs everywhere” or “just use Postgres”) both fail, for opposite reasons.
- **“Why not just Cassandra’s LWW everywhere?”** Clock skew chooses your losers, and concurrent writes are silently destroyed — not merged, destroyed. Fine for caches and derived state; negligent for user data.
- **“How do Redis / Riak / DynamoDB-style stores do multi-region?”** Redis CR is delta-state CRDTs; Riak is OR-Sets and MV-registers with dotted version vectors; Dynamo-style stores push sibling resolution to the client — the “app-level siblings” row of the trade-off table.
- **“Two users type into the same word — what do they see?”** RGA: both strings converge; possible interleaving of the two runs, prevented by LSEQ/Fugue-style allocation; in practice, cursors and presence (Chapter 1’s awareness channel) keep humans out of each other’s words anyway.
- **“When would you refuse CRDTs?”** Strong global invariants, very large per-object state (a CRDT wrapping a 1 GB blob merges terribly), and teams without the appetite to own merge semantics. A boring CP database is a fine choice when the product can afford it — say that without flinching.

Checklist for the whiteboard. (1) Define replica, partition, convergence — thirty seconds of vocabulary that buys you an hour of precision. (2) State CAP and pick AP explicitly, out loud. (3) Derive one CRDT from the three laws — a G-Counter takes sixty seconds and proves you understand the lattice, not just the catalog. (4) Show one concurrent-conflict merge end to end, tags and tombstones included. (5) Name tombstones and the ack-gated GC before the interviewer does. (6) Draw the sync fabric: deltas, gossip, repair. (7) Volunteer the invariant boundary before the interviewer finds it — the person who names the limitation owns the room.

9.20 Summary & Further Reading

CRDTs answer the chapter's prompt by changing the question. Instead of ordering concurrent writes — impossible without coordination, undefined across a partition — they build data types whose states form a join-semilattice and whose merge is the join, so that commutativity, associativity, and idempotence make order, grouping, and duplication irrelevant. On that foundation: counters as per-replica slots merged by max; sets as union, then union-twice with tombstones, then observed-remove with unique tags for add-wins; registers as timestamps (dangerous) or sibling sets (honest); text as stable-identity atoms ordered by (parent, id). Causality is tracked with vector clocks and dots; sync is deltas over gossip with Merkle repair; tombstones are collected only when every replica has ack'd them. The price is metadata and the loss of global invariants; the reward is that the write path never, ever waits — Chapter 1's road not taken, now fully mapped.

Further reading.

- The source video: “CRDTs — Stop Worrying About Write Conflicts — Systems Design 0 to 1 with Ex-Google SWE” (Jordan has no life): <https://www.youtube.com/watch?v=FG5Varj1OWs>
- Shapiro, Preguiça, Baquero, Zawirski — “Conflict-free Replicated Data Types” (2011) and “A Comprehensive Study of Convergent and Commutative Replicated Data Types” (2011) — the founding papers; the second is the catalog this chapter compressed.
- Almeida, Shoker, Baquero — “Delta State Replicated Data Types” (2018) — the sync-economics fix, formally.
- Lamport — “Time, Clocks, and the Ordering of Events in a Distributed System” (1978) — happens-before, in its original eight pages.
- Riak KV's documentation on dotted version vectors, and Redis CR's active-active whitepapers — production CRDT stores, warts included.
- The Ink & Switch local-first software essay, and the Yjs and Automerge codebases — sequence CRDTs doing Chapter 1's job in the wild.

9.21 Chapter Glossary

Every term this chapter asked you to absorb, gathered for revision. If any definition feels unfamiliar now, the section it points back to is worth a reread before your interview.

Term	Meaning
Add-wins semantics	A concurrent add survives a concurrent remove; the OR-Set default
Anti-entropy	Background replica-pair reconciliation that drives divergence toward zero
Associativity	Grouping of merges is irrelevant: $(a \vee b) \vee c = a \vee (b \vee c)$
CAP	Under partition, choose consistency (CP: stall writes) or availability (AP: merge later); you cannot have both

Term	Meaning
Causal context	Opaque version vector handed to a reader and echoed on over-write, separating “supersedes” from “concurrent”
Commutativity	Order of merges is irrelevant: $a \vee b = b \vee a$
Concurrent events	Events with no happens-before path between them; no fact-of-the-matter order exists
Convergence	Replicas that saw the same updates are in the same observable state
Coordination	Pre-write communication (leader, lock, quorum) — the thing partitions disable
CvRDT / CmRDT	State-based (merge states by join) vs op-based (broadcast commuting operations) CRDT families
Delta-state	Shipping the minimal state change instead of full state; deltas merge by the same join
Dot	A (replica, counter) pair naming one event; the OR-Set tag and the version vector’s precise companion
Escrow	Pre-allocating shares of an invariant (e.g. quota of seats) so regions can act without coordinating
Eventual consistency	“If updates stop, replicas end up equal” — liveness only, no safety during updates
G-Counter / PN-Counter	Per-replica slots merged by max; PN adds a second counter so the derived value can fall
Gossip	Randomized peer-to-peer exchange; $O(\log R)$ rounds to infect R replicas
Happens-before ($\rightarrow$)	Lamport causality: program order plus message passing, transitively closed
Idempotence	Duplication is irrelevant: $a \vee a = a$ — the property that makes retries free
Inflation / monotonicity	Updates only move state up the lattice; nothing is ever un-known
Interleaving anomaly	Concurrent text runs merged character-by-character; cosmetic, fixed by newer allocators
Join-semilattice	A partial order where every pair has a least upper bound; merge = that bound
LWW register	Keep the value with the max timestamp; discards concurrent writes, trusts clocks
Merkle repair	Hash-tree comparison of key-spaces to find divergence in $O(\log n)$ hashes
MV register	Keeps all concurrent values as siblings; the application resolves with context

Term	Meaning
OR-Set	Observed-remove set: tagged adds, tombstone-the-observed removes; add-wins
RGA	Replicated Growable Array: sequence CRDT with stable atom ids and parent links
Strong eventual consistency	Eventual consistency plus same-updates $\Rightarrow$ same-state safety; what CRDTs prove
Tombstone	A mergeable record of deletion; collectible only when all replicas ack it
Vector clock	Replica $\rightarrow$ counter map encoding causal history; dominates iff $\geq$ in every component

— End of Chapter 9 · Next: Chapter 10, Designing a Distributed Job Scheduler —

Designing a Distributed Job Scheduler

NOTE — Problem source

This chapter solves the problem posed in the talk “20: Distributed Job Scheduler” from the series *Systems Design Interview Questions with Ex-Google SWE* (channel: *Jordan has no life*). Unlike Chapters 8 and 9 (fundamentals deep dives), this is a full **product design** in the shape of Chapters 1–7: cron-as-a-service, the machinery behind Airflow, Kubernetes CronJobs, AWS EventBridge Scheduler, and every “remind me in 20 minutes” button on Earth. It quietly reuses almost every earlier chapter: the due-job index is Chapter 8’s B+ tree, the dispatch queue is Chapter 4’s append-only log, per-tenant fairness is Chapter 3’s limiter, and shard ownership borrows Chapter 9’s vocabulary of leases and clocks. All terms are defined before use; all reference code is Rust with deterministic tests.

10.1 The Problem Statement

The interviewer draws a clock on the whiteboard and says:

“Design a distributed job scheduler. Users register jobs — one-shot (‘fire at 3:00 AM’), recurring (‘every weekday at 07:30’), or whole workflows of dependent jobs — and the system fires them reliably, at the right time, at scale: tens of millions of registered jobs, tens of thousands of firings per second at the peak. Jobs must not be lost, must not silently double-execute, and the scheduler itself must survive machine and region failures.”

Before you reach for any machinery, it is worth understanding why this problem exists at all — why every large company builds this system eventually, usually after being burned. The naive versions are all traps, and you have probably written one of them yourself. A single machine running `cron` works until the machine dies, and then nothing in the universe fires — a single point of failure with a silently-absent alarm. A `sleep()` call in application code is even more fragile: it dies with the process, and nobody even records that the timer ever existed. And the classic middle step — a database table of jobs polled by every worker — turns your database into a lock convoy the moment the worker fleet grows. What the interview is really testing is whether you can look at this problem and see that it is actually **three problems wearing one coat: knowing when jobs are due** (a data-structures problem), **deciding who fires them** (a distributed-ownership problem), and **executing them without duplicates** (a delivery-semantics problem). Each scales differently, fails differently, and deserves its own section — and separating them in your first five minutes at the whiteboard is the single highest-signal move in this interview.

Three definitions will carry the whole chapter, so let’s pin them down before any design work.

DEFINITION — Job / run / trigger

A *job* is the registered specification: what to execute (a queue message, an HTTP call, a worker class plus payload), **when** (its *trigger*: a fire timestamp or a schedule), and **how to behave on failure** (retry policy, dead-letter). A *run* (or execution) is one actual firing of a job — jobs are few and durable, runs are many and ephemeral. Confusing the two tables in the data model is the first design error this chapter avoids.

Notice how much design is already hiding in that distinction. The job is what the user thinks about; the run is what the system thinks about. When a user edits a job at 02:59, they expect their edit to apply to the 03:00 firing — but the run for 03:00 may already be enqueued with the old payload. When the scheduler crashes, the **jobs** must all survive (they are the product), while the **runs** may be replayed, retried, or reconstructed (they are the exhaust). Every table we draw later hangs off this one split.

DEFINITION — Schedule / cron expression

A compact description of recurring fire times. The classic *cron expression* has five fields — minute, hour, day-of-month, month, day-of-week — so `30 7 * * 1-5` reads “07:30 on weekdays”. Two traps lurk here and a senior candidate names both unprompted: **time zones** (07:30 in whose morning?) and **DST** (on spring-forward day, 02:30 never happens; on fall-back day, it happens twice). The store keeps UTC plus the tenant’s zone; the misfire policy (Section 10.9) decides what “the 02:30 that never came” means.

DEFINITION — Misfire and catch-up policy

A *misfire* is a scheduled fire time that passed without firing — because the scheduler was down, the shard was being re-leased, or DST erased the minute. The *catch-up policy* is a per-job choice: **fire once now** (coalesce all misses into one), **replay every miss**, or **skip to the next scheduled time**. A billing job wants replay-every-miss; a cache warmer wants skip; an hourly report wants coalesce. There is no correct default — only a default you chose on purpose.

Sit with those last two definitions for a moment, because they are where this problem stops being a puzzle about timers and becomes a puzzle about **promises**. “Every weekday at 07:30” sounds like a fact; it is actually a policy expressed in a notation that predates time-zone databases. The system you are about to design will make thousands of tiny judgment calls on behalf of its users — what to do about the minute that never existed, the hour that happened twice, the six minutes of downtime during a deploy — and the difference between a junior and a senior answer is whether those judgment calls are **declared per job** or **discovered in the incident report**.

10.2 Scope & Clarifying Questions

Here is the conversation that bounds the problem. Notice how each answer eliminates an entire branch of the design space — that is what clarifying questions are **for**, and asking them crisply is half the interview.

Candidate asks	Interviewer answers
Job kinds?	One-shot delayed jobs, recurring cron jobs, and DAG workflows of dependent jobs
Execution model?	Scheduler fires by enqueueing to a dispatch queue; worker pools pull and execute. We design both sides
Scale?	10^7 registered jobs, 3×10^3 firings/s average, $30\times$ bursts at top-of-hour
Fire latency?	Seconds matter, microseconds do not: p99 within 1 s of the scheduled time

Candidate asks	Interviewer answers
Delivery semantics?	Push for effectively-once execution; expect to explain why raw exactly-once is impossible
Multi-tenant?	Yes — one tenant's burst must not starve others (Chapter 3 territory)
Durability?	Registered jobs survive region loss; run history retained 90 days

Two of these answers deserve a highlight before we move on. “Seconds matter, microseconds do not” is a gift: it means you never need kernel timers, hardware timestamping, or any of the exotic machinery of low-latency trading — a one-second tick loop is plenty, and the design can spend its complexity budget on reliability instead of precision. And “push for effectively-once” is the interviewer telegraphing the chapter's core challenge: they want to hear you explain, unprompted, why exactly-once is not a thing engineering can buy. We will get there in Section 10.6.

NOTE — Agreed scope

1. Design the **control plane**: job registration (one-shot, cron, DAG), update, cancel, inspect — backed by a durable job store.
2. Design the **trigger side**: how due jobs are found across 10^7 timers without a thread or a full-table scan per timer.
3. Design the **firing side**: sharded scheduler replicas with leases and fencing, a durable dispatch queue, and workers that execute **effectively-once** via idempotency keys.
4. Design **failure behavior** as a first-class feature: retries with backoff and jitter; dead-letter queues, misfire policies.
5. Quantify scale: registered jobs, fire rate, burst shape, worker fleet size, run-history storage.
6. Out of scope: the workers' actual business logic; sub-second precision timers (that's a kernel problem, not an architecture).

10.3 Functional Requirements

Let's turn the prompt into commitments. Read each requirement and ask yourself which of the three sub-problems — trigger, firing, execution — it belongs to; you will find every one lands cleanly, which is your first confirmation that the three-way split is the right skeleton.

1. **Register and manage jobs.** Create, update, cancel, and inspect one-shot jobs (`fire_at = T`), recurring jobs (`cron` expression + time zone), each with payload, target, and retry policy.
2. **Fire on time.** Every due job produces a fire command within the latency SLO of its scheduled time, in all but catastrophe scenarios.
3. **Effectively-once execution.** Clients that follow the idempotency protocol see each run execute at most once, even though the transport underneath is at-least-once.
4. **Bounded retries.** Failed runs retry with exponential backoff and jitter up to a per-job cap, then land in a dead-letter queue with the full error context.
5. **Workflows.** Users submit DAGs of dependent jobs; the system runs each job only after its dependencies succeed, parallelizes independent branches, and applies a per-workflow failure policy (fail-fast vs. complete-what-you-can).
6. **Misfire handling.** After downtime or DST gaps, each job follows its declared catch-up policy: coalesce, replay, or skip.
7. **Tenant fairness.** Per-tenant fire-rate quotas; bursts are smoothed or queued, never allowed to starve other tenants.

Requirement 2 is phrased with unusual care — “produces a fire command”, not “executes the job” — and you should mirror that care when you write your own requirements on the whiteboard. The

scheduler's promise ends at the dispatch queue's door. How long the job takes, whether the worker fleet is big enough, whether the target service is up: those are the **worker's** problems. Keeping the promise boundary sharp is what lets you reason about the scheduler's SLO without dragging the entire downstream world into it.

10.4 Non-Functional Requirements

- **Durability of the registry.** A registered job must survive the loss of any machine and any region: the job store is replicated (Chapter 8's machinery underneath), and a fire command, once accepted, is durably queued before it is acknowledged.
- **Availability of firing.** The scheduler must keep firing with any one replica — or one region — down: shard leases fail over in ≤ 15 s.
- **No lost runs.** Between scheduler crash, queue outage, and worker death, a due job may be **delayed** or **duplicated**, but never silently dropped. Duplicates are absorbed by the idempotency layer.
- **Bounded staleness.** p99 fire latency ≤ 1 s past due for one-shot jobs; ≤ 5 s for cron jobs (their consumers are batch-shaped).
- **Elastic scale-out.** Adding scheduler shards or worker pools raises capacity linearly; no component is a hard singleton (leases, not process identity, own the shards).

Read the third bullet twice — it is the philosophical core of the whole design. You are **trading** loss for duplication on purpose. Loss is unrecoverable: a billing run that never happens is money silently gone. Duplication is recoverable: a billing run that happens twice is a bug you can detect and absorb — and Section 10.6 will show you exactly the machinery that absorbs it. Whenever a distributed system lets you choose which failure mode to live with, choose the one that leaves evidence.

DEFINITION — At-least-once / at-most-once / exactly-once

The three delivery guarantees a system can offer. *At-most-once*: fire and forget — no retries, so failures lose messages. *At-least-once*: retry until acknowledged — failures duplicate messages. *Exactly-once* end-to-end delivery is **provably impossible** over an unreliable network (the receiver cannot distinguish “sender crashed before sending” from “sender crashed after sending but before the ack arrived” — the classical Two Generals impossibility). What **is** achievable — and what this design delivers — is *effectively-once*: at-least-once transport plus **idempotent** processing, so duplicates exist on the wire but are invisible to the application.

DEFINITION — Idempotency / idempotency key

An operation is *idempotent* if performing it twice has the same effect as performing it once. A scheduler achieves this by stamping every fire command with a unique *idempotency key* — (`job_id`, `scheduled_time`) is the natural choice — and having the executor check a *dedup store* before its first side effect: first arrival claims the key and runs; any duplicate sees the claimed key and exits as a no-op. Chapter 5's vote endpoint used the same trick; here it is the backbone of the whole system.

10.5 Back-of-the-Envelope: Timers, Bursts, and Workers

Three numbers size this design, and you should extract them from the interviewer before drawing a single box: how many timers exist (sets the store and the trigger structure), how unevenly they fire (sets the burst engineering), and how much execution capacity the fire rate implies (sets the worker fleet). Here are the assumptions we'll carry.

Assumptions. 10^7 registered jobs; recurring jobs fire hourly on average; job spec ≈ 1 KB; average run duration 30 s; 90-day run history at 1 KB per run record.

Quantity	Estimate (with assumptions)
Registered jobs	$10^7 \times 1 \text{ KB} \approx \mathbf{10 \text{ GB}}$ of job specs — comfortably one replicated store, Chapter 8 style
Average fire rate	$10^7 \text{ jobs} \times 1 \text{ fire/hour} \approx \mathbf{2.8 \times 10^3 \text{ fires/s}}$
Top-of-hour burst	50% of cron jobs pinned to minute 0 $\rightarrow 5 \times 10^6$ fires in one minute $\approx \mathbf{8 \times 10^4 \text{ fires/s}}$ — a 30$\times$ burst . Splaying (§10.6) is mandatory
Due-job discovery	64 shards $\times$ one indexed range scan per second — the B+ tree leaf walk of Chapter 8; trivial against 10^7 rows
Concurrent executions	$2.8 \times 10^3 \text{ fires/s} \times 30 \text{ s avg} \approx \mathbf{8.4 \times 10^4 \text{ in flight}}$ $\rightarrow 1\,000$ workers $\times 100$ slots each
Dispatch traffic	$\leq 8.3 \times 10^4 \text{ msg/s} \times 1 \text{ KB} \approx \mathbf{83 \text{ MB/s peak}}$ — well inside Chapter 4's log throughput
Run history	$2.8 \times 10^3 \text{ runs/s} \times 86\,400 \text{ s} \times 90 \text{ days} \times 1 \text{ KB} \approx \mathbf{22 \text{ TB}}$ $\rightarrow$ retention windows and rollups, Chapter 4 style
Lease failover	TTL 10 s + probe 5 s $\rightarrow \leq \mathbf{15 \text{ s}}$ worst-case pause on a shard when its scheduler dies

Work the rows in order and you can feel the design assemble itself. The first row tells you the registry is small — 10 GB is a Tuesday for a replicated relational store, so durability here is cheap. The second row tells you the **average** workload is tiny — three thousand fires a second is one modest service. And then the third row ruins your afternoon: the top-of-hour burst is thirty times the average, because half of humanity types `0 * * * *` and means it. Row four is the reassurance that finding due work is easy **if** you index it (and Section 10.7 is entirely about that “if”). Rows five and six tell you the worker fleet and the queue are both comfortably inside machinery you have already designed in earlier chapters. Row seven quietly introduces a second product — run history is 22 TB, a log-retention problem, not a scheduler problem. And the last row sets the failover budget the firing side must hit.

KEY INSIGHT — The burst, not the average, designs the system

The average fire rate needs one modest scheduler; the top-of-hour spike — half the world's cron jobs pinned to minute 0 — needs thirty. A scheduler designed for the average melts at 00:00 UTC daily; one designed for the peak idles 97% of the day. The resolution is to **reshape the demand**: splay recurring jobs across their period (`0 * * * *` becomes “minute `hash(job_id) mod 60`, every hour”), which flattens a 30 $\times$ burst to 1.5 $\times$ — and then size for that. Every scheduler interview should contain the sentence “and then we desynchronize the timers”; it is the same thundering-herd lesson as Chapter 3's jittered retries and Chapter 8's rightmost hotspot, wearing a clock face.

This is a pattern worth internalizing beyond this chapter: whenever your estimation table contains a row that is 30 $\times$ its neighbor, the neighbor is irrelevant and the outlier **is** the design. You will see the same shape in Chapter 12's ride-hailing dispatch, where Friday-night demand dwarfs the weekly average, and in Chapter 3's rate limiter, where the whole point is that arrivals are not polite.

10.6 The Core Challenge: “Exactly Once” Is a Lie We Tell Well

Ask ten engineers what a job scheduler must guarantee and nine say “exactly-once execution”. The tenth — the one who has run one — says “effectively-once, and here is why.” Let's make you the tenth.

The reason is not engineering laziness; it is impossibility, and it is worth feeling the impossibility rather than memorizing the slogan. Walk the path of a single fire command: the scheduler enqueues it, a worker pulls it, the worker performs the side effect — charges the card, sends the email — and then acks the queue. Now ask: what happens if the worker crashes **after** the side effect but **before** the ack? The queue sees an un-acked command and, correctly, redelivers it. A second worker picks it up. Nobody anywhere in the system can tell whether the first worker executed or not — the crash erased exactly the one bit of information that would distinguish the two worlds. Since you cannot distinguish them, you must design for the worse one, which means you must retry, which means duplicates on the wire are unavoidable. That is the Two Generals problem wearing a hard hat: over an unreliable network, “did you get it?” can never be answered with certainty by the party who needs to know.

So the core challenge splits cleanly in two, and the design addresses each half with a different weapon:

1. **The trigger side is a data-structures problem.** Ten million timers, thirty thousand firing per second at the peak, each fire time computable but volatile (jobs are cancelled and rescheduled constantly). Neither a thread per timer nor a full-table poll survives contact with these numbers; Section 10.7 picks the structure that does.
2. **The firing side is a distributed-systems problem.** Several scheduler replicas must divide the work without double-firing, workers must absorb duplicates they cannot prevent, and a crashed component’s in-flight work must be reclaimed — all without a global lock. Section 10.8 assembles leases, fencing tokens, and idempotency keys into exactly that.

Notice the intellectual honesty baked into this split. The trigger side gets to live in a world of clean algorithms, because timers are just data. The firing side must live in the real world, where processes pause and networks lie. A common interview failure mode is to spend forty minutes perfecting the trigger structure and wave at the firing side — precisely backwards, because nobody’s billing system was ever double-charged by a B+ tree.

KEY INSIGHT — Effectively-once = at-least-once + idempotency + a dedup store

The industry’s entire answer, in one line: **deliver at least once, process idempotently, remember what you processed.** The scheduler guarantees every fire command lands on the queue at least once (durable enqueue before ack); the worker, before any side effect, claims the run’s idempotency key

INSERT ... IF NOT
EXISTS

in a dedup store (— the same compare-and-set discipline as Chapter 5’s votes); the key’s TTL outlives any plausible duplicate window. Duplicates still happen on the wire — they just never happen twice to the application. This is the exact pattern behind Kafka’s idempotent producers, Stripe’s idempotency-keyed API, and every payment retriever you have ever trusted with your card.

10.7 Deep Dive: The Trigger Side — Finding What’s Due

The trigger side answers one question, 64 times a second per shard: **which jobs are due right now?** Let’s look at three designs in increasing sophistication. In the interview you should name all three — the contrast is where the signal lives — and then defend your pick.

Design A: the indexed scan. Store every job with a materialized `next_fire_at` column and a B+ tree index on `(shard, next_fire_at)` — Chapter 8’s structure, doing exactly the job it was designed for. A scheduler tick is one range scan (`next_fire_at ≤ now` within its shard slice) plus a linked-leaf walk; due jobs stream out in fire-time order at index speed. After firing (or enqueueing), the job’s `next_fire_at` is recomputed from its cron expression and the row updated — so the index maintains itself, and the “timer” for next Tuesday is nothing more than a row whose index key sorts after today’s. This is the design this chapter ships, and the reasons are worth saying out loud: it is durable by construction (the timers **are** the database — there is no in-memory state to lose), crash-transparent

(a dead scheduler’s successor simply rescans and finds everything, no rebuild step, no warm-up gap), and simple enough to reason about at 2 AM, which is when you will be reasoning about it.

Design B: the in-memory min-heap. Keep a priority queue of `(fire_at, job)` keyed by soonest-first — the Rust listing in Section 10.13 implements exactly this. $O(\log n)$ per schedule or cancel-via-lazy-tombstone, $O(1)$ peek at the next due job, zero index maintenance. As an algorithm it is lovely. The catch is that the heap is **volatile**: it must be rebuilt from the store on every failover, and at 10^7 timers that is a 10–30 s cold start during which the shard is blind — blind, notice, for **longer than the 15-second failover SLO** we just committed to. The production answer is not to choose but to layer: use the heap as a **hot cache in front of Design A**. The tick loop keeps only the next few seconds of timers in memory, refilling from the index once per window; the store sees one scan per shard-second, not one per timer, and the heap’s volatility no longer matters because its contents are always re-derivable in one scan.

Design C: the hashed timing wheel. The classic kernel answer — Kafka’s delayed operations, Netty’s timers, and the Linux kernel itself all use one. Picture an array of buckets representing time slots: slot `i` holds every timer due at `now + i · tick`, modulo the wheel’s circumference, so insert and expire are $O(1)$ pointer pushes and the whole structure advances one slot per tick like a clock hand. A **hierarchical** wheel stacks coarser wheels the way a clock stacks gears: a seconds-wheel feeds a minutes-wheel feeds an hours-wheel, and a timer due next year parks cheaply in the outer wheel until it cascades down through the gears toward its tick. Beautiful, optimal, and entirely in-memory — which means it inherits the heap’s durability problem, adds resize pain when the wheel fills, and fits cancel-heavy workloads poorly (cancelling means splicing out of a bucket you must first find). Worth describing in the interview to show range; wrong as the system of record. If the interviewer pushes — “but the wheel is $O(1)$!” — your answer is: **the scan is already fast enough, and durability is not a feature you can bolt onto volatility later.**

DEFINITION — Timing wheel (hashed / hierarchical)

A ring buffer of buckets indexed by time slot: a timer due in k ticks lands in bucket $(\text{current} + k) \bmod \text{circumference}$; each tick, the current bucket’s timers fire or cascade. The *hierarchical* variant stacks wheels of coarser granularity (seconds, minutes, hours) so a timer due next year parks cheaply in the year-wheel until it degrades down through the gears. $O(1)$ amortized insert and expiry — the price is volatility and the complexity of cascading on the wheel boundary.

COMMON PITFALL — The poll-everything anti-pattern

The first draft everyone sketches — `SELECT * FROM jobs` every second, filter in application code — turns 10^7 rows into 10^7 row-reads per second per scheduler replica: the database melts, and the interview with it. The index on `next_fire_at` is not an optimization; it **is** the trigger design. If you take one sentence from this section: the due set is a range query, and range queries are a solved problem (Chapter 8) — spend your design budget on the firing side instead.

10.8 Deep Dive: The Firing Side — Leases, Fencing, Idempotency

The firing side answers a harder question than the trigger side ever asked: **who** may fire shard 37’s due jobs — and how do we stop **yesterday’s** answer from firing them again? That second clause is where the difficulty lives, and it is why this section needs three mechanisms stacked, not one.

Shard ownership by lease. Hash-partition the 10^7 jobs by `job_id` into (say) 64 shards — 64 is arbitrary; what matters is that it is much larger than your replica count, so ownership can be rebalanced finely. A scheduler replica acquires a shard by taking its **lease** from a small highly-available coordination store; it renews the lease every few seconds and loses it silently if it stalls — a GC

pause, a network hiccup, death. A standby replica takes over an expired lease, giving you failover in ≤ 15 s per the SLO. Why leases instead of a proper consensus election per scheduling decision? Because consensus-per-operation would put a WAN round-trip inside every tick, and the tick is once per second. The lease converts a per-operation cost into a per-few-seconds cost.

DEFINITION — Lease

A time-bounded grant of exclusive ownership: “replica R owns shard S until time T, and may renew.” Leases convert **membership** (“who owns this shard?”) from a consensus-per-operation cost into a heartbeat-per-few-seconds cost. Their danger is the gap between “my lease silently expired” and “I find out”: during that gap the old owner still believes itself owner, and two owners double-fire.

That danger deserves a concrete scene, because it is the scene fencing exists for. Replica A holds shard 37’s lease with a 10-second TTL. At $t=7$ s, A enters a stop-the-world GC pause. At $t=10$ s the lease expires; at $t=12$ s standby B takes over and starts firing shard 37’s due jobs. At $t=18$ s, A wakes up, checks its clock, believes it is still inside its lease — its last renewal said “valid until $t=10+10$ ” and its clock, frozen during the pause, tells it only $t=9$ has passed — and fires the same due jobs a second time. Two owners, both honest, both wrong. You cannot fix this by making A smarter; A **cannot know**. So you make the rest of the world smarter instead.

DEFINITION — Fencing token

A monotonically increasing number issued by the lease store on every acquisition, which the holder must present with **every** action it performs on behalf of the lease — and which downstream systems check: “accept this dispatch only if its token exceeds every token you have seen for this shard.” Fencing closes the lease’s blind gap: the stale owner’s late dispatches carry an old token and are rejected even if it never learns it was deposed. A lease without fencing is optimism; a lease with fencing is a protocol. (The Rust listing in Section 10.13 implements both in forty lines.)

In our scene, A’s dispatches carry token 41 and B’s carry token 42; the queue (or the job store) has already seen 42 and turns A away at the door. The deposed scheduler is stopped not by being informed — informing it is exactly what the network cannot guarantee — but by being **refused**. That inversion is the whole idea, and it generalizes far beyond this chapter: whenever ownership can change hands while a client is partitioned or paused, the resource itself must check credentials on every operation, and the credentials must be **ordered**.

Why duplicates survive all of this anyway. Now the uncomfortable part, which is also the most instructive. Suppose leases and fencing work perfectly: single owner per shard, always. A fire command is **still** delivered at-least-once, because the worker may crash **after** executing but **before** acking, and the queue — correctly — redelivers. Fencing cannot help here, and it is worth seeing precisely why: the duplicate is not a **stale** holder presenting an old token; it is the **same** holder’s legitimate command appearing twice. Ordered credentials distinguish old owners from new owners; they say nothing about replays **within** one owner’s reign. This is why the last line of defense lives at the point of side effect: the worker claims the run’s idempotency key `(job_id, scheduled_time)` in the dedup store before touching the world, and marks it complete after. A redelivered command finds the key claimed and exits 0. Step back and admire the shape of what we just built: the system is a pipeline of decreasing error rates. Leases make double-ownership rare. Fencing makes double-ownership harmless. The queue makes delivery at-least-once. And idempotency makes at-least-once invisible. No single layer is airtight; the stack is.

DEFINITION — Heartbeat

A periodic “I am alive and working on X” signal from a component — workers to the scheduler, lease-holders to the lease store. Missed heartbeats past a timeout flip the component to **suspect**, then **dead**, and its work is reclaimed: its lease expires, its in-flight runs return to the queue. The heartbeat’s only subtlety is the timeout: too short and GC pauses cause flapping takeovers; too long and failover violates the 15 s SLO. Ten seconds with three-miss tolerance is the field’s default for a reason.

10.9 Deep Dive: Retries, Backoff, Jitter, and the Dead-Letter Queue

A run fails. What happens next is policy, and policy is design — this section is short on algorithms and long on judgment, which is exactly why interviewers probe it.

Retry with exponential backoff and full jitter. Attempt k waits `random(0, min(cap, base · 2k))`. Unpack each ingredient, because each neutralizes a specific failure mode. The exponential mean doubles per attempt, giving a struggling dependency exponentially more room to recover — if the target database is down for thirty seconds, your first retry at 100 ms was never going to succeed, so why spend it? The cap keeps the tail sane: without it, attempt 30 waits a geological age, and your “retry” is indistinguishable from “lost”. And the randomization — **full jitter**, drawn fresh per (job, attempt) — spreads a fleet of simultaneous failures across the whole window instead of re-stamping the dependency in lockstep. Picture ten thousand runs that all failed at $t=0$ because the target hiccuped: without jitter they all retry at $t=100$ ms, then all at $t=200$ ms, a self-inflicted denial-of-service in perfect formation. With jitter they smear across `[0, 2k · base)` and the dependency sees a load it can survive. This is Chapter 3’s token bucket seen from the other side: there we **rate-limited clients**; here we rate-limit **ourselves**, because a retry storm is friendly fire.

DEFINITION — Dead-letter queue (DLQ)

The terminal queue for runs that exhaust their retry budget. A DLQ is not a trash can; it is a **quarantine with a promise**: every entry keeps its full context (payload, error, attempt history, stack), an alarm fires on arrival rate, and an operator tool can inspect, patch, and **re-drive** entries back into the dispatch queue once the underlying fault is fixed. The design rule: nothing may fail silently, and no human should ever reconstruct state from logs alone.

Which failures retry and which don’t. Here is a taxonomy line that separates senior answers: distinguish **retryable** errors (timeouts, 502s, connection resets — the dependency might recover) from **permanent** ones (400, schema rejection, “no such user” — retrying is just slow failure, and slow failure with a backoff cap is the worst of both worlds: you pay the latency and still land in the DLQ). Only retryable errors consume the budget; permanent errors dead-letter immediately. And one more category, rarer but deadlier: a **poison message** — a run that crashes every worker that touches it, say because its payload triggers an untested code path. Each individual attempt looks like a worker crash, so naive systems just keep redelivering, and the message saws through the fleet one worker at a time. The detection rule is “attempts exhausted **with crash-looping workers**”, and the response is immediate dead-lettering plus a circuit breaker on the target — quarantine the patient before you autopsy it.

INTERVIEW TIP — Say the quiet part about 2:30 AM

Retry policy, misfire policy, and DST policy are where schedulers hurt people in production, and interviewers know it. Volunteer the stories: the billing job that double-charged because a retry lacked an idempotency key; the report that silently skipped a day because catch-up defaulted to skip; the 02:30 job that fired twice on fall-back day. You do not need to have lived them — you need to show you know they are **policy choices**, not accidents.

10.10 Deep Dive: Workflows — DAGs of Jobs

Real pipelines are not independent jobs: *extract* must finish before *transform*, which feeds three parallel *load* jobs, which gate *notify*. So far our scheduler understands time but not dependency — and the moment you say “B runs after A succeeds”, you have left the land of timers and entered the land of graphs. Fortunately you need exactly one graph concept, and it comes with a built-in execution plan.

DEFINITION — DAG (directed acyclic graph) / workflow

A *directed* graph: edges mean “depends on” (B’s incoming edge from A means A must succeed before B may fire). *Acyclic*: no dependency loops — a cycle has no valid execution order, so it is rejected at **submission time** (the Rust listing’s cycle detection), never discovered at 3 AM at runtime. The *layers* of the DAG — Kahn’s algorithm peels zero-dependency nodes level by level — are the free parallelism: every job in a layer is independent and may fire concurrently; the workflow’s critical path is its longest layer chain.

Mechanics. A workflow is stored as a job graph plus one **run record** per workflow instance — the job/run split from Section 10.1, applied recursively. When a run completes, the scheduler decrements the **pending-dependency count** of each child in the store; a child whose count hits zero becomes a normal due job and enters the trigger pipeline — no special machinery downstream, and that “no special machinery” is worth dwelling on. The trigger side never learns that workflows exist. The firing side never learns that workflows exist. Workflows are a **write-time transformation** of the data model: dependencies live entirely in “not due yet”. This is one of the cleanest examples in the book of a principle you should carry everywhere — the best way to add a feature to a system is to reduce it to something the system already does.

Failure policy is per-workflow, and the two standard policies map onto two genuinely different product intents: **fail-fast** (first failure cancels pending descendants; the workflow run fails) is right when the stages are steps of one logical operation — a half-migrated database is worse than an un-migrated one; **complete-branches** (unaffected branches run to completion) is right for fan-out reports, where the marketing dashboard being generated is no reason to cancel the finance one. Compensation — “payment captured, refund if shipping fails” — is the workflow-level echo of Chapter 9’s invariant boundary: it is **application** logic the scheduler must make expressible, not a guarantee it can manufacture. Say that sentence in the interview; it is the difference between a candidate who has read about sagas and one who knows why sagas exist.

KEY INSIGHT — The scheduler’s data model in one breath

jobs (durable specs, indexed by (shard, next_fire_at)) · runs (one per firing: status, attempt count, idempotency key) · workflow_edges (job → depends-on) · dedup (claimed idempotency keys with TTL) · leases (shard → holder, fencing token, expiry). Five tables. Everything else in this chapter — wheels, retries, takeovers, DLQs — is a process reading and writing those five tables under a discipline. If your whiteboard has more boxes than that, you are designing the org chart, not the system.

10.11 API Design

With the machinery understood, the API almost writes itself — and that is the point of doing the deep dives first. The surface is deliberately small: jobs and workflows in, runs and dead-letters out, plus a handful of internal endpoints for the lease and dispatch machinery. Two disciplines run through every row of the table, and you should name them as you draw it: every mutating **client** call accepts an idempotency key (Chapter 5’s vote endpoint, generalized — a retried `POST /jobs` must not create two jobs), and every **internal** dispatch carries a fencing token (Section 10.8), because internal traffic is exactly where stale owners live.

Endpoint	Scope	Semantics
POST /jobs	client	Register one-shot (<code>fire_at</code>) or recurring (<code>cron + tz</code>) job; validates the schedule, computes first <code>next_fire_at</code> , applies splay offset
PATCH /jobs/{id}	client	Update spec; schedule changes re-materialize <code>next_fire_at</code> — the old timer dies as a lazy tombstone (§10.13)
DELETE /jobs/{id}	client	Cancel: no future fire times; the in-flight run is left to finish or killed per the job's <code>cancel_policy</code>
GET /jobs/{id}	client	Spec + status + <code>next_fire_at</code> + summary of the last run
POST /workflows	client	Submit a DAG {jobs, edges}; cycle-checked at submission (§10.10); returns a workflow id
GET /runs?job_id=...&cursor=	client	Run history, cursor-paginated — Chapter 5's cursor pattern, unchanged
POST /dlq/{run}/redrive	operator	Re-enqueue a dead-lettered run once the underlying fault is fixed
POST /leases/{shard}	internal	Acquire or renew a shard lease; response carries the new fencing token
POST /dispatch/pull	internal	Worker pulls a batch of fire commands; the visibility timeout starts
POST /dispatch/ack	internal	Report outcome; un-acked commands redeliver when the timeout lapses

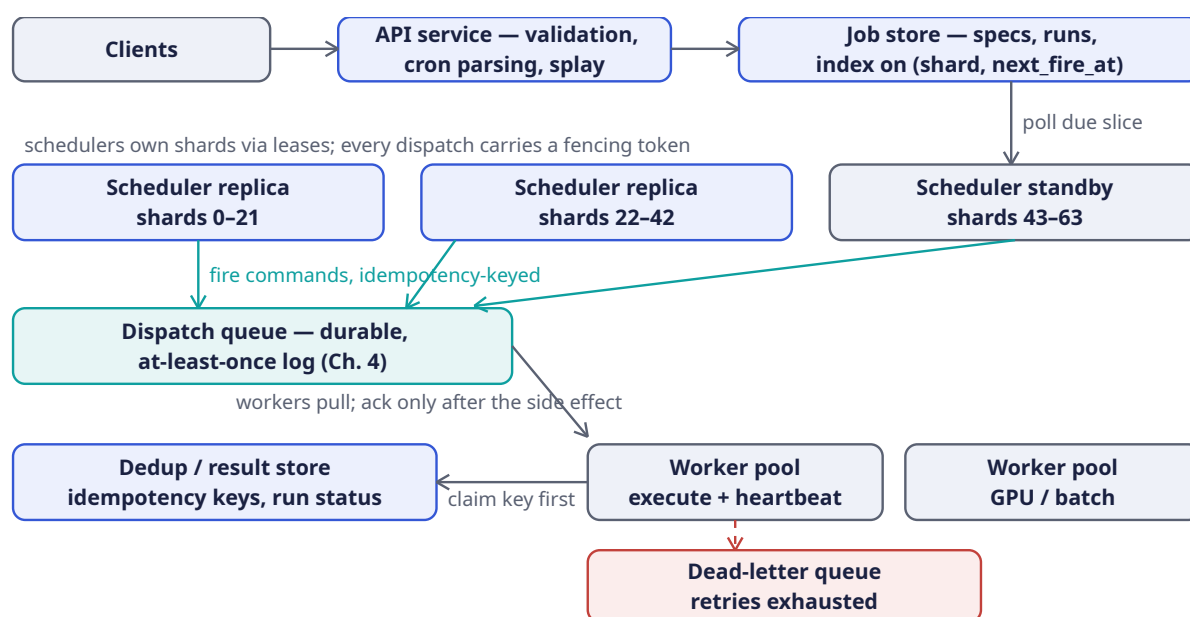
Read the `PATCH` row slowly — it hides the most elegant move in the whole API. When a user reschedules a job, you might expect a hunt through some timer structure to find and remove the old entry. Instead, the row's `next_fire_at` is simply overwritten, and the old heap entry — still sitting in some scheduler's memory — becomes a **lazy tombstone**: when it eventually pops, the `live` check (the Rust listing's trick, Section 10.13) finds the map disagrees and discards it. The update path never touches the scheduler fleet at all. And the two `dispatch` rows encode the queue's half of the reliability story: pulling starts a timer, acking stops it, and the timer firing means “assume the worst, redeliver”.

DEFINITION — Visibility timeout

The queue's redelivery timer: a pulled message becomes invisible to other consumers for T seconds; if the puller neither acks nor abandons it within T, the message reappears and another worker takes it. It is the queue-level twin of the shard lease — same trick, same blind gap (“am I still the owner?”), same final answer: idempotency at the point of side effect, because neither lease nor timeout can ever be airtight.

10.12 High-Level Architecture

Here is the whole chapter in one picture. Before you trace any arrows, notice the layout's deliberate honesty: the diagram has two **stories** running through the same boxes — a registration story (short, horizontal, top row) and a firing story (long, vertical, everything below) — and the five tables from the data-model insight are where the stories meet.



Let's walk it twice, once per story, the way you would narrate it at the whiteboard.

The registration story takes ten seconds and stays on the top row. A client calls `POST /jobs` with a cron expression and a time zone. The API service — stateless, horizontally scaled, doing nothing but validation, cron parsing, and arithmetic — checks the schedule is well-formed, computes the job's shard as `hash(job_id) mod 64`, applies the splay offset that flattens the top-of-hour herd, and materializes the first `next_fire_at` in UTC. Then it writes the row to the job store and acks. That is the **entire** user-visible write path: no timer exists yet, no scheduler has been contacted, no thread is sleeping anywhere. The registered job is simply a row that sorts into the right place in the `(shard, next_fire_at)` index — a fact in a B+ tree, waiting for time to catch up with it. This is worth pausing on, because it is the design's central aesthetic: **registration is just indexing**. The future fire is not an action anyone scheduled; it is a query result nobody has run yet.

The firing story is the system talking to itself, and it flows downward. Each scheduler replica holds leases on a slice of shards — the diagram shows two active replicas and a standby, which is the shape you want: ownership is pre-divided so failover is an arithmetic reassignment, not a negotiation. Once per second, each replica range-scans the due slice of its shards against the job store (the Chapter 8 leaf walk — the “poll due slice” arrow dropping from the store into the standby row depicts exactly this loop). For every due row it does two things: enqueue an idempotency-keyed fire command onto the dispatch queue — the teal box, Chapter 4's durable at-least-once log — and advance the row's `next_fire_at`, so a rescan never sees the job as due twice. Those teal arrows fanning from all three schedulers into the one queue are worth a look: they are the point where ownership converges, and it is why every command carries a fencing token — the queue is the downstream system that deposes stale holders.

The bottom two rows are where the world gets touched, and they are drawn in the order of paranoia you should feel. Workers **pull** batches from the queue — never push — so a slow pool is self-throttling backpressure rather than a drowning victim. The label “workers pull; ack only after the side effect” is the at-least-once contract in eight words: the visibility timeout starts at pull, and only a post-side-effect ack stops it. Before touching the world, the worker claims the run's idempotency key in the dedup store — the “claim key first” arrow running from the worker pool leftward into the dedup box is drawn in that direction on purpose: the dedup store is consulted **before**, not after. And the dashed crimson arrow dropping from the workers into the dead-letter queue is the path nobody wants and everybody needs: retries exhausted, context preserved, operator paged, redrive possible.

Two worker pools are drawn — a general “execute + heartbeat” pool and a “GPU / batch” pool — to make one last point visible: execution classes scale independently, sharing nothing but the queue.

Stand back and count singletons. There are none. The API is stateless. The job store is replicated. The schedulers are fungible behind leases — kill any one and the standby’s next tick covers its shards. The queue is a partitioned log. The workers are a pool. The dedup store is a keyed lookup that any replicated KV can serve. Every box in the diagram can be replaced while the system is running, and the design’s promises survive the replacement — which, more than any single mechanism, is what “distributed” was supposed to mean.

10.13 Rust Reference Implementations

Time to make the chapter compile. The four listings below are the scheduler’s entire inner loop: the due-job heap (the trigger side’s hot cache), the lease table with fencing (the firing side’s ownership protocol), the backoff schedule (the failure policy’s arithmetic), and the DAG layerer (the workflow engine in forty lines). Each is small enough to hold in your head, sharp enough to defend line by line in an interview, and — most importantly — each ships with deterministic tests that **demonstrate** the property the prose claimed. When you study these, don’t read the code first; read the tests first. The tests are the contracts.

10.13.1 The Due-Job Heap (Lazy Cancellation)

This is Design B from Section 10.7 — the in-memory min-heap that rides in front of the indexed scan. Two details repay attention before you read a line of code. First, Rust’s `BinaryHeap` is a **max**-heap, so the `Ord` impl inverts the comparison — soonest `fire_at` compares “greatest” and pops first, with a sequence number breaking ties so equal fire times fire in submission order. Second, cancellation is **lazy**: `cancel` never touches the heap at all. It just updates the `live` map, and the stale entry — a “dud” — is discovered and skipped when it pops. Read `pop_due` as a tiny state machine: peek, compare against `now`, pop, validate against `live`, and only then emit.

```
use std::collections::{BinaryHeap, HashMap};

/// One scheduled firing. `fire_at` is epoch millis; `seq` breaks ties so
/// equal fire times fire in submission order.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Scheduled {
    pub fire_at: u64,
    pub seq: u64,
    pub job: u64,
}

// BinaryHeap is a MAX-heap; invert the comparison so the SOONEST
// fire_at is "greatest" and pops first.
impl Ord for Scheduled {
    fn cmp(&self, other: &Self) -> std::cmp::Ordering {
        other.fire_at
            .cmp(&self.fire_at)
            .then(other.seq.cmp(&self.seq))
            .then(other.job.cmp(&self.job))
    }
}

impl PartialOrd for Scheduled {
    fn partial_cmp(&self, other: &Self) -> Option<std::cmp::Ordering> {
        Some(self.cmp(other))
    }
}

/// In-memory timer structure sitting in front of the durable store.
```

```

/// Cancel/reschedule are lazy: the stale heap entry is left in place and
/// skipped when popped – an entry is valid iff `live[job] == fire_at`.
pub struct Scheduler {
    heap: BinaryHeap<Scheduled>,
    live: HashMap<u64, u64>, // job id -> currently valid fire_at
    seq: u64,
}

impl Scheduler {
    pub fn new() -> Self {
        Scheduler { heap: BinaryHeap::new(), live: HashMap::new(), seq: 0 }
    }

    /// Schedule (or reschedule: a second call for the same job supersedes
    /// the first – the old heap entry becomes a dud on its own).
    pub fn schedule(&mut self, job: u64, fire_at: u64) {
        self.seq += 1;
        self.heap.push(Scheduled { fire_at, seq: self.seq, job });
        self.live.insert(job, fire_at);
    }

    pub fn cancel(&mut self, job: u64) {
        self.live.remove(&job); // heap entry stays; popped-and-skipped later
    }

    /// Drain every job due at or before `now`, soonest first.
    pub fn pop_due(&mut self, now: u64) -> Vec<u64> {
        let mut due = Vec::new();
        while let Some(&top) = self.heap.peek() {
            if top.fire_at > now { break; }
            let entry = self.heap.pop().unwrap();
            if self.live.get(&entry.job) != Some(&entry.fire_at) {
                continue; // cancelled or superseded: a dud
            }
            self.live.remove(&entry.job);
            due.push(entry.job);
        }
        due
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn fires_in_fire_time_order() {
        let mut s = Scheduler::new();
        s.schedule(1, 300);
        s.schedule(2, 100);
        s.schedule(3, 200);
        assert!(s.pop_due(50).is_empty()); // nothing due yet
        assert_eq!(s.pop_due(200), vec![2, 3]); // soonest first
        assert_eq!(s.pop_due(1000), vec![1]);
    }

    #[test]
    fn cancel_prevents_firing() {
        let mut s = Scheduler::new();
        s.schedule(1, 100);
    }
}

```



```

    s.schedule(2, 100);
    s.cancel(1);
    assert_eq!(s.pop_due(100), vec![2]); // job 1's dud is skipped
    assert!(s.pop_due(1000).is_empty());
}

#[test]
fn reschedule_fires_once_at_new_time() {
    let mut s = Scheduler::new();
    s.schedule(7, 100);
    s.schedule(7, 400); // reschedule = supersede
    assert!(s.pop_due(150).is_empty()); // the old time must NOT fire
    assert_eq!(s.pop_due(400), vec![7]); // the new time fires, once
    assert!(s.pop_due(1000).is_empty()); // and no ghost fires after
}
}

```

The `live` map is the whole trick, and it deserves a cost analysis in both directions so you can defend the choice under pressure. Eager cancellation in a binary heap is $O(n)$: find the entry (the heap gives you no index), remove it, re-heapify. Lazy cancellation is $O(1)$: record the truth, let the heap find out when it matters. The price you pay is bounded garbage — every cancel or reschedule leaves one dud in the heap until its old fire time passes, so the heap can hold at most (operations so far) entries instead of (live timers) — and for a cache that is refilled from the store every few seconds anyway, that bound is trivially acceptable. With cancel-heavy workloads this is the difference between a timer structure and a performance incident — and you have seen the shape before: it is the same lazy-tombstone pattern as Chapter 9's OR-Set removals, where deleting eagerly was impossible and deleting lazily was free.

10.13.2 Leases with Fencing Tokens

This listing is Section 10.8's whole protocol in two functions, and the brevity is the lesson: leases and fencing are not heavyweight infrastructure, they are a `HashMap` and a counter. `acquire` implements the ownership rule — fail only if **another** holder's lease is still live; an expired lease can always be taken over, and every grant bumps the global fencing counter. `check_fence` is the downstream side — keep a per-shard “highest token seen” and accept a dispatch only from a strictly newer holder. Watch the third test: it is the GC-pause scene from Section 10.8 replayed in miniature. The fresh holder's token 5 is accepted; the deposed holder waking up with token 4 is turned away; even a **replay** of the accepted token 5 is rejected, because fencing tokens order holders, not messages — and then the next legitimate holder proceeds with 6.

```

use std::collections::HashMap;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Lease {
    pub holder: u64,
    pub fence: u64, // monotonically increasing across ALL acquisitions
    pub expires_at: u64,
}

/// Shard ownership registry. A scheduler replica may dispatch a shard's
/// due jobs only while it holds the lease; the fencing token lets every
/// downstream store reject a stale holder whose lease silently lapsed.
pub struct LeaseTable {
    leases: HashMap<u64, Lease>,
    next_fence: u64,
}

```



```

impl LeaseTable {
    pub fn new() -> Self {
        LeaseTable { leases: HashMap::new(), next_fence: 0 }
    }

    /// Acquire or renew the lease on `shard`. Fails only if another
    /// holder's lease is still live; an expired lease can be taken over.
    pub fn acquire(&mut self, shard: u64, holder: u64, now: u64, ttl: u64) -> Option<Lease>
    {
        match self.leases.get(&shard) {
            Some(l) if l.holder != holder && l.expires_at > now => None, // held by someone
        else
            _ => {
                self.next_fence += 1;
                let lease = Lease { holder, fence: self.next_fence, expires_at: now + ttl };
                self.leases.insert(shard, lease);
                Some(lease)
            }
        }
    }

    /// The downstream check, kept as per-shard "highest token seen":
    /// accept a dispatch only from a strictly newer holder. A deposed
    /// scheduler waking from a GC pause presents an old token and is
    /// turned away – that rejection is fencing doing its one job.
    pub fn check_fence(highest_seen: &mut u64, fence: u64) -> bool {
        if fence > *highest_seen { *highest_seen = fence; true } else { false }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn lease_excludes_second_holder_until_expiry() {
        let mut t = LeaseTable::new();
        let a = t.acquire(0, 1, 1000, 500).unwrap(); // holder 1 until t=1500
        assert!(t.acquire(0, 2, 1200, 500).is_none()); // contender rejected
        let b = t.acquire(0, 2, 1600, 500).unwrap(); // after expiry: takeover
        assert!(b.fence > a.fence); // token strictly rises
    }

    #[test]
    fn renewal_keeps_ownership() {
        let mut t = LeaseTable::new();
        let a = t.acquire(0, 1, 1000, 500).unwrap();
        let r = t.acquire(0, 1, 1400, 500).unwrap(); // same holder renews
        assert_eq!(r.holder, 1);
        assert_eq!(r.expires_at, 1900);
        assert!(r.fence > a.fence); // every grant gets a new token
    }

    #[test]
    fn stale_fence_is_rejected() {
        let mut highest = 0;
        assert!(LeaseTable::check_fence(&mut highest, 5)); // fresh holder accepted
        assert!(!LeaseTable::check_fence(&mut highest, 4)); // stale holder deposed
        assert!(!LeaseTable::check_fence(&mut highest, 5)); // replay also rejected
        assert!(LeaseTable::check_fence(&mut highest, 6)); // next holder proceeds
    }
}

```

```

    }
}

```

One design note you can volunteer: `acquire` hands a fresh fencing token even on **renewal** by the same holder. That is deliberate — a token that only ever meant “latest owner” would suffice for depositing, but renew-on-every-grant also makes each individual renewal itself a witnessed, ordered event, which downstream auditors appreciate. The cost is one integer.

10.13.3 Exponential Backoff with Full Jitter

The retry arithmetic from Section 10.9. Two implementation choices are worth your attention because interviewers pounce on both. First, the jitter is **deterministic**: a `splitmix64` hash over `(job, attempt)` rather than a thread-local RNG. That makes every retry schedule reproducible per job — you can replay an incident’s timing exactly — and it removes shared RNG state from a hot path. Second, the shifts are defensive: `1u64 << attempt.min(20)` caps the exponent so a pathological `max_attempts` can never overflow the shift, and `saturating_mul` guarantees the ceiling math degrades to the cap rather than wrapping. Read the tests as the two properties you actually bought: the window grows and caps (test one), and the fleet genuinely spreads inside the window while staying deterministic per key (test two).

```

/// Retry schedule: attempt k waits random(0, min(cap, base * 2^k)).
/// The exponential mean gives a struggling dependency room to recover;
/// full jitter spreads a fleet of simultaneous failures across the whole
/// window so they don't re-stampede in lockstep (Section 10.9).
pub struct Backoff {
    pub base_ms: u64,
    pub cap_ms: u64,
    pub max_attempts: u32,
}

impl Backoff {
    /// splitmix64 over (job, attempt): a deterministic, decorrelated
    /// jitter source – reproducible per job, no shared RNG state.
    fn jitter(&self, job: u64, attempt: u32) -> u64 {
        let mut x = job
            .wrapping_mul(0x9E3779B97F4A7C15)
            .wrapping_add(attempt as u64);
        x = (x ^ (x >> 30)).wrapping_mul(0xBF58476D1CE4E5B9);
        x = (x ^ (x >> 27)).wrapping_mul(0x94D049BB133111EB);
        x ^ (x >> 31)
    }

    /// Delay before retry `attempt` (0-based). None = budget exhausted,
    /// send the run to the dead-letter queue.
    pub fn delay(&self, job: u64, attempt: u32) -> Option<u64> {
        if attempt >= self.max_attempts { return None; }
        let exp = self.base_ms.saturating_mul(1u64 << attempt.min(20));
        let ceiling = exp.min(self.cap_ms);
        Some(self.jitter(job, attempt) % (ceiling + 1))
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn backoff_grows_then_caps() {

```

```

let b = Backoff { base_ms: 100, cap_ms: 10_000, max_attempts: 8 };
for attempt in 0..8u32 {
    let d = b.delay(42, attempt).unwrap();
    let ceiling = (100u64 << attempt).min(10_000);
    assert!(d <= ceiling); // within this attempt's window
}
assert_eq!(b.delay(42, 8), None); // exhausted -> dead-letter
}

#[test]
fn jitter_stays_in_bounds_and_varies() {
    let b = Backoff { base_ms: 1000, cap_ms: 60_000, max_attempts: 10 };
    assert_eq!(b.delay(7, 2), b.delay(7, 2)); // deterministic per (job, attempt)
    let first = b.delay(0, 3).unwrap();
    let mut varies = false;
    for job in 0..1000u64 {
        let d = b.delay(job, 3).unwrap();
        assert!(d <= 8000); // hard bound: base * 2^3
        if d != first { varies = true; }
    }
    assert!(varies); // the fleet actually spreads
}

```

10.13.4 DAG Layers (Kahn's Algorithm)

Last, the workflow engine — and the surprise is how little of it there is. Kahn's algorithm (1962) peels the zero-indegree frontier repeatedly: layer 0 is every job with no dependencies, layer 1 is everything whose dependencies all landed in layer 0, and so on. The function returns `Option : None` means the graph was malformed — a cycle, or a dependency on a job that was never submitted — and per Section 10.10's rule, that rejection happens at **submission time**, loudly, in the API service, rather than at 3 AM in the trigger pipeline. Note the two data-structure choices doing quiet work: `BTreeMap / BTreeSet` keep iteration ordered so the layers come out deterministic (essential for testing, pleasant for debugging), and the `? on indegree.get_mut(&job)` is the unknown-job rejection falling out of the type system for free.

```

use std::collections::{BTreeMap, BTreeSet, VecDeque};

/// Execution layers of a job DAG: layer 0 = roots (no dependencies),
/// layer k = jobs whose dependencies all sit in earlier layers. Jobs
/// within a layer are independent and run in parallel; layers run in
/// order. `deps` are (job, depends_on) pairs.
///
/// Returns None on a cycle or an unknown job: a malformed workflow must
/// fail loudly at submission time, never hang silently at runtime.
pub fn layers(jobs: &[u64], deps: &[(u64, u64)]) -> Option<Vec<Vec<u64>>> {
    let mut indegree: BTreeMap<u64, usize> = jobs.iter().map(|&j| (j, 0)).collect();
    let mut children: BTreeMap<u64, Vec<u64>> = BTreeMap::new();
    for &(job, dep) in deps {
        *indegree.get_mut(&job)? += 1; // unknown job -> reject
        children.entry(dep).or_default().push(job);
    }

    // Kahn's algorithm: repeatedly peel the zero-indegree frontier.
    let mut frontier: VecDeque<u64> = indegree
        .iter()
        .filter(|(&, &d)| d == 0)
        .map(|(&j, _)| j)

```

```

        .collect();
let mut out: Vec<Vec<u64>> = Vec::new();
let mut placed = 0usize;

while !frontier.is_empty() {
    let layer: BTreeSet<u64> = frontier.drain(..).collect();
    placed += layer.len();
    let mut next = Vec::new();
    for &j in &layer {
        for &c in children.get(&j).into_iter().flatten() {
            let d = indegree.get_mut(&c).unwrap();
            *d -= 1;
            if *d == 0 { next.push(c); } // all dependencies satisfied
        }
    }
    out.push(layer.into_iter().collect());
    next.sort_unstable();
    next.dedup();
    frontier = next.into_iter().collect();
}

if placed == jobs.len() { Some(out) } else { None } // unplaced => cycle
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn diamond_runs_in_three_layers() {
        //      1
        //     / \
        //    2   3      layer 2 = {2, 3} runs in parallel
        //     \ /
        //      4
        let jobs = vec![1, 2, 3, 4];
        let deps = vec![(2, 1), (3, 1), (4, 2), (4, 3)];
        let l = layers(&jobs, &deps).unwrap();
        assert_eq!(l, vec![vec![1], vec![2, 3], vec![4]]);
    }

    #[test]
    fn cycle_is_rejected_at_submission() {
        let jobs = vec![1, 2, 3];
        let deps = vec![(2, 1), (3, 2), (1, 3)]; // 1 <- 2 <- 3 <- 1
        assert_eq!(layers(&jobs, &deps), None);
    }

    #[test]
    fn unknown_dependency_is_rejected() {
        assert_eq!(layers(&[1, 2], &[(2, 99)]), None);
    }

    #[test]
    fn independent_jobs_share_layer_zero() {
        let l = layers(&[10, 20, 30], &[]).unwrap();
        assert_eq!(l, vec![vec![10, 20, 30]]);
    }
}

```

The layer view is not just validation — it **is** the workflow execution plan, and closing that loop is the section’s payoff. The scheduler fires layer 0 through the ordinary trigger pipeline; each run completion decrements its children’s pending counts in the store; a child reaching zero is promoted to a normal due job with `next_fire_at = now`. The trigger pipeline from Section 10.7 never learns that workflows exist — separation of concerns, enforced by the data model. And the cycle check has one more pleasant property worth naming: because it runs at submission, a cycle can never be **created by time itself** — no failover, no race, no partial write can turn an accepted workflow into a hang, because the shape was proven acyclic before the first job ever fired.

INTERVIEW TIP — Name the classics when you borrow them

Kahn’s algorithm (1962) for topological layering, the hashed timing wheel (Varghese & Lauck, 1987), the Two Generals problem for exactly-once, splitmix64 for jitter — dropping the name plus the year signals that your design is assembled from proven parts, not invented under pressure. One sentence each is enough; the interviewer will either nod or lean in, and both are wins.

10.14 Scaling the Design

The design scales along three axes, and — as in Chapter 9 — it is worth naming them separately because they fail and are fixed independently.

More timers. Sharding by `hash(job_id) mod N` scales the trigger side horizontally: 64 shards today, 1024 tomorrow — a lease-table rebalance, not a redesign. The crucial invariant is that the due-scan per shard stays a bounded range query no matter how large the job table grows, because the **index**, not the table size, sets the cost — Chapter 8’s height-3 lesson paying rent again. And remember the layering from Section 10.7: the in-memory heap rides in front of each shard’s scan as a hot cache, so the replica refills it with the next few seconds of timers per tick and the store sees one scan per shard-second, not one per timer. Growing timers by 100× changes the fan-in of that refill, nothing else.

More firing rate. The dispatch queue partitions by shard and scales like Chapter 4’s log — add partitions, add consumer groups. Worker pools scale independently per execution class: the GPU pool and the HTTP pool share nothing but the queue, so a batch-job tenant’s fleet can triple without one HTTP worker being requisitioned. Per-tenant fairness is enforced at **enqueue** time with Chapter 3’s token bucket: a tenant past its quota has its fire commands delayed, not dropped — delayed, because dropped fire commands are misfires, and misfires invoke policy, whereas delays are just latency the SLO already budgets. And the splay from Section 10.5 keeps the aggregate curve flat enough that quotas rarely bite in the first place.

More regions. The job store replicates cross-region — Chapter 9’s machinery would make specs available everywhere — and schedulers run active-active with region-local shard ownership, so a region loss moves **leases**, not data. That distinction is the whole sentence: leases are recomputed in seconds from a coordination store; data would have to be re-shipped or, worse, reconstructed. Clock discipline is the quiet dependency under everything: schedulers reason about “now” via NTP-disciplined UTC, and fire times are always stored in UTC with the tenant’s zone applied at materialization — the DST trap of Section 10.1 is a data-model decision, not a runtime surprise.

10.15 Failure Modes & Degradation

Walk this table the way an on-call engineer reads a runbook: for each row, ask “what does the **user** see, and which mechanism we already built is the one that saves them?” You will find every answer is a mechanism from Sections 10.6–10.9 operating exactly as designed — no row requires heroics, which is the definition of a mature failure story.

Failure	Symptom	Handling
Scheduler crash	Shard goes quiet	Lease expires ≤ 10 s; standby takes over and rescans — the store is the truth, the queue an accelerator, so nothing due is lost
Scheduler GC pause	Stale holder keeps dispatching after lease loss	Fencing tokens: every downstream write from the old holder is rejected (\$10.8)
Queue partition	Fire commands not flowing	Schedulers keep scanning; enqueue retries with backoff; on heal, misfire policy (coalesce/replay/skip) applies per job
Worker death mid-run	Side effect maybe done, ack never sent	Visibility timeout redelivers; dedup store makes the second attempt a no-op — at-least-once + idempotency = effectively-once
Poison message	Run crash-loops workers	Attempt counter exhausts → DLQ; per-target circuit breaker pauses dispatch to the failing dependency
Clock skew	Two schedulers disagree on “now” by seconds	Fire times in UTC, NTP alerting; a job firing 2 s late is a blip, not a bug — the SLO already budgets it
DLQ flood	Operator pager fires	Alarm on inflow rate; redrive tool after the fix; the DLQ is a quarantine, not a graveyard

The first row deserves special reverence because it contains the deepest sentence in the chapter: **the store is the truth, the queue an accelerator**. Once you believe that sentence, scheduler crashes stop being scary. The dead replica took no irreplaceable state with it — its heap was a cache, its leases expire on their own, and its successor’s first action is the same action it performs every second anyway: scan the due slice. Failover is not a special mode; it is the normal loop, started from a cold cache. Design every stateful system so that recovery is “the steady state, resumed” and you will sleep better than engineers who build recovery procedures.

COMMON PITFALL — The double-fire you designed yourself

The most common self-inflicted duplicate: a scheduler that enqueues the fire command and **then** updates `next_fire_at` — crashing in between means the successor rescans, finds the job still due, and fires again. Two honest answers: (1) claim-then-enqueue — atomically advance `next_fire_at` in the same transaction as recording the fire, so a rescan sees the job already handled; (2) accept the duplicate and let the idempotency key absorb it. Production systems do both: belt at the scheduler, suspenders at the worker. Skipping both is how billing systems end up in the news.

10.16 Trade-offs & Alternatives

Every row of this table is a decision you made earlier in the chapter; this section exists so you can defend them as **choices**, with a named alternative and a reason it loses — here, under these requirements, not in the abstract. That last qualifier matters: several “losers” below are the right answer at a different scale, and saying so is worth points.

Choice	We picked	Alternative	Why the alternative loses (here)
Timer structure	Indexed store scan + heap cache	Hierarchical wheel	Wheel is $O(1)$ but volatile; the store scan is durable truth and fast enough — re-build-on-failover is the wheel's hidden tax
Dispatch	Pull (workers fetch)	Push (scheduler assigns)	Pull is self-balancing backpressure: a slow worker simply stops pulling; push needs a feedback loop to not drown stragglers
Ownership	Sharded leases + fencing	Single leader scheduler	A leader is a throughput ceiling and a failover event; leases make failover routine and per-shard
Execution guarantee	At-least-once dedup	Distributed transactions	XA/2PC across queue + store + worker is operationally brittle and slow; idempotency keys are cheap and honest
Queue substrate	Durable log (Ch. 4)	Postgres SKIP LOCKED as queue	Fine at 10^3 msg/s, painful at 10^5 ; the log replays and partitions better. A fair second choice at small scale — say so
Scheduling in-app	Central scheduler service	Per-app cron / k8s Cron-Job	Per-app timers scatter retry, misfire, and audit policy across a thousand repos; the service exists to own the policy

Read the last row as the **organizational** trade-off, because it is the one interviewers with production scars care about most. Per-app cron works — that is the damning part. It works right up until the company has a thousand repos, each with its own retry policy, its own misfire default, its own idea of what “02:30 on fall-back day” means, and no audit trail anywhere. The central service exists to **own the policy**: one place where idempotency, retries, misfires, and fairness live, so a thousand teams never have to re-derive them. When you frame a service as “a policy with an API”, you sound like someone who has been on call.

10.17 Observability & SLOs

The scheduler's vital sign is **fire lag**: `now - scheduled_time` at the moment of enqueue. Everything else in the table is a leading indicator of lag — the signals that move **before** the user-visible number does. That framing is worth stating on your dashboard's first page, because it turns alerting from a list into a narrative: scan duration creeps, then queue depth grows, then lag breaches; the page you get at the first stage is a gift, the page at the last stage is a post-mortem invitation.

Signal	What it measures	Alert when
Fire lag	Enqueue time minus scheduled time, per shard	p99 > 1 s sustained: shard under-provisioned or store slow
Due-scan duration	Time per shard range scan	Rising trend: index bloat or hot shard (Ch. 8)
Lease flapping	Takeovers per hour	> a few/day: GC pauses, network instability, or too-short TTL
Queue depth	Unconsumed fire commands	Growing: workers down, or a tenant burst past quota
Dedup hit rate	Duplicates absorbed ÷ runs	Sudden rise = redelivery storm; sustained ≈ 0 is normal
Retry / DLQ inflow	Failures per minute by target	Spike by one target: dependency outage — page its owner, not the scheduler's
Worker slot utilization	Busy slots ÷ total, per pool	Sustained $\approx 100\%$: capacity is the lag's cause

Two rows reward a second look. **Lease flapping** is the canary for your timeout tuning: a takeover is cheap but not free — cold cache, rescan, ramp — and a few per day means your heartbeat interval is arguing with your GC pauses, an argument the timeout always loses. And the **retry/DLQ inflow** row encodes an organizational rule as a metric: a spike **attributed to one target** is that target's outage, so the alert should page its owner, not the scheduler's on-call. Routing blame correctly in the alert text is a kindness you do to your future colleagues.

SLOs. Fire latency: $p99 \leq 1$ s for one-shot jobs, ≤ 5 s for cron — different numbers because the consumers differ: a one-shot “remind me” has a human staring at it, while a cron job's consumer is a batch pipeline that cannot tell five seconds from five milliseconds. Durability: registered jobs and accepted fire commands survive any single region loss. Failover: shard ownership moves in ≤ 15 s. Visible duplicates: ≈ 0 — dedup absorbs $\geq 99.99\%$ of at-least-once redeliveries, and the remainder are **reported, not hidden**, because a duplicate you can see is a bug, and a duplicate you cannot see is a billing inquiry.

10.18 Interview Wrap-Up

Likely follow-ups, with the shape of a strong answer. Each of these has a first sentence that carries most of the signal — find it, say it first, then elaborate.

- “*But can you give me exactly-once?*” No — and say why in one breath (the Two Generals: the worker that crashes after its side effect is indistinguishable from one that crashed before). Then give the effectively-once recipe and name the dedup store as the price.
- “*A job must run at 09:00 in the user’s local zone.*” Store UTC + zone; materialize `next_fire_at` against the zone’s rules; declare the DST policy (skip the impossible 02:30, coalesce the doubled one). Volunteering DST before being asked is the seniority signal.
- “*Pause a whole tenant right now.*” A kill-switch flag checked at **dispatch pull** and at **scan** — two enforcement points, because the queue may already hold their commands.
- “*Charge a credit card on a schedule.*” The side effect leaves your blast radius: idempotency key at the payment gateway (Stripe-style), retry only on retryable errors, dead-letter fast on card-declined — retrying a decline is harassment, not reliability.
- “*Design Airflow.*” This chapter plus the DAG section is Airflow’s skeleton: its scheduler loop, executor pools, and dead-letter cousin (the “failed task” state with manual retry) map one-to-one onto the five tables.
- “*What breaks first at 100×?*” The single-region job store’s write throughput on `next_fire_at` updates — every fire rewrites the row. Answer: shard the store along the same hash, batch the re-materialization, and consider LSM-shaped storage (Chapter 8’s rival) because this workload is write-dominated.

Checklist for the whiteboard. (1) Split trigger vs firing vs execution in the first five minutes. (2) Give the exactly-once speech unprompted. (3) Put the index on `next_fire_at` and say “range scan, Chapter 8 style”. (4) Draw the lease and its fencing token as separate things. (5) Retry with jitter, dead-letter with redrive. (6) Splay the top-of-hour herd. (7) Five tables, no more.

10.19 Summary & Further Reading

A distributed job scheduler is three systems wearing one trenchcoat. The **trigger side** turns 10⁷ timers into a per-second range scan over a B+ tree index — durable, resumable, boring in the best way — with an in-memory heap as a hot cache and lazy tombstones for cancels. The **firing side** divides shards among fungible scheduler replicas with leases whose blind gap is closed by fencing tokens, and hands fire commands to a durable at-least-once log. The **execution side** makes at-least-once invisible: workers claim idempotency keys before any side effect, retries decay with full-jitter exponential backoff, exhausted runs quarantine in a dead-letter queue with their context intact, and workflows reduce to per-child dependency counters fed back into the same trigger pipeline. The uncomfortable truths are the design’s load-bearing walls: exactly-once is impossible, so effectively-once is engineered; the top-of-hour burst, not the average, sizes the fleet, so demand is splayed flat; and every policy — misfire, retry, cancel, catch-up — is a choice the system makes explicit because the alternative is making it by accident at 3 AM.

Further reading.

- The source video: “20: Distributed Job Scheduler — Systems Design Interview Questions with Ex-Google SWE” (Jordan has no life): <https://www.youtube.com/watch?v=pzDwYHRzEnk>
- Varghese & Lauck — “*Hashed and Hierarchical Timing Wheels*” (1987) — the timer structure, from the source.
- Fischer, Lynch, Paterson — “*Impossibility of Distributed Consensus with One Faulty Process*” (1985), and the Two Generals parable — why exactly-once was never on the table.
- Kleppmann — *Designing Data-Intensive Applications*, the chapters on stream processing and fencing — the effectively-once argument in full.
- Airflow, Temporal, and Kubernetes CronJob documentation — three production points in this chapter’s design space, with openly documented failure semantics.
- AWS Builder’s Library — “*Timeouts, retries, and backoff with jitter*” — the jitter analysis this chapter’s backoff listing implements.

10.20 Chapter Glossary

Term	Meaning
At-least-once / at-most-once	Delivery guarantees: retry-until-ack (duplicates) vs fire-and-forget (losses)
Backoff (exponential, full jitter)	Retry delay $\text{random}(0, \min(\text{cap}, \text{base} \cdot 2^k))$; the randomization prevents retry stampedes
Catch-up policy	What a misfire costs: coalesce to one firing, replay every miss, or skip to next
Cron expression	Five-field recurring schedule: minute hour day-of-month month day-of-week
DAG / workflow	Directed acyclic graph of dependent jobs; cycles rejected at submission
Dead-letter queue	Quarantine for exhausted runs, with full context and a redrive path
Dedup store	Claimed idempotency keys with TTL; the effectively-once memory
Effectively-once	At-least-once transport + idempotent processing; duplicates on the wire, none visible
Fencing token	Monotonic lease stamp checked downstream; deposits stale holders
Fire lag	Enqueue time minus scheduled time — the scheduler's vital sign
Heartbeat	Periodic liveness signal; three misses $\approx$ dead; work reclaimed
Idempotency key	Unique operation stamp — <code>(job_id, scheduled_time)</code> — claimed before side effects
Job vs run	Durable specification vs one ephemeral execution of it
Kahn's algorithm	Peels zero-dependency nodes in layers; both the DAG validator and its execution plan
Lazy tombstone	Cancel by marking, not removing; the heap entry is skipped when popped
Lease	Time-bounded exclusive shard ownership, renewable, expiring silently
Misfire	A scheduled fire time that passed unfired — downtime, failover, or DST
Poison message	A run that crashes every worker it touches; detected, quarantined
Shard	$\text{hash}(\text{job_id}) \bmod N$ slice of the job space; the unit of scheduler ownership
Splay / desynchronization	Spreading recurring fire times by hash to flatten the top-of-hour herd

Term	Meaning
Thundering herd	N timers or retries firing in lockstep, DDoS-ing the next layer down
Timing wheel	Ring buffer of time-slot buckets; O(1) timers, volatile memory
Two Generals problem	The impossibility of certain agreement over an unreliable channel
Visibility timeout	Queue redelivery timer for pulled-but-unacked messages; the consumer-side lease

— End of Chapter 10 · Next: Chapter 11, Designing for Correctness: How ACID Transactions Work —

Designing for Correctness: How ACID Transactions Work

NOTE — Problem source

This chapter solves the problem posed in the talk “Intro to ACID Database Transactions” from the series *Systems Design Interview: 0 to 1 with Google Software Engineer* (channel: *Jordan has no life*). It is the third fundamentals deep dive, and it completes a trilogy: Chapter 8 built the storage engine’s pages and indexes, this chapter builds the **correctness layer** that lets many statements act as one on top of those pages, and Chapter 9 showed what you must surrender — this chapter’s guarantees — when you refuse coordination across regions. Read in that order, the three chapters are one argument: pages, then correctness, then the price of correctness. All terms are defined before use; all reference code is Rust with deterministic tests.

11.1 The Problem Statement

The interviewer draws two bank accounts and says:

“A transfer debits account A by \$100 and credits account B by \$100. Design the transaction layer of a relational database: let users bundle many reads and writes into one logical unit, keep that unit correct while thousands of other units run concurrently, and keep it correct when the machine crashes in the middle. Define ‘correct’ — precisely — and then build the machinery that enforces it.”

The prompt’s sting is in the last sentence. Every engineer can say “ACID”; the interview is won by the candidate who can **define** each letter as an enforceable contract, name the anomaly each isolation level admits, and point at the exact mechanism — log record, lock, version — that prevents it. This chapter builds that machinery from the crash upward.

DEFINITION — Transaction

A bundle of reads and writes treated as one logical unit of work: it either happens completely or not at all, it never exposes half-applied state to others, and once confirmed it is permanent. The transfer above is the canonical example — debit and credit must live or die together, because money is not allowed to be created or destroyed by a power outage.

DEFINITION — A — Atomicity

All or nothing. A transaction’s writes are applied in full or not at all, even if the machine dies mid-transaction. Enforced by the log: on recovery, the engine **redoes** committed transactions and **undoes** (or discards) uncommitted ones (Section 11.7). Atomicity is a property of the write path, not of careful coding — “be careful not to crash” is not a mechanism.

DEFINITION — C — Consistency

Invariants hold. Every transaction moves the database from one valid state to another: foreign keys resolve, balances stay non-negative, uniqueness holds. The honest detail most answers miss: consistency is largely the **application's** property — the database enforces declared constraints and provides A, I, and D, but “a transfer preserves total money” is logic the transaction author must write correctly. C is the letter the machine assists and the human owns.

DEFINITION — I — Isolation

Concurrent transactions do not see each other's intermediate state. Perfect isolation (serializability) means the outcome equals **some** serial execution of the transactions — nobody can prove they ran in parallel. Perfect isolation is expensive, so the standard sells it in graded levels, each defined by which **anomalies** it forbids (Section 11.8): dirty reads, non-repeatable reads, phantoms, lost updates, write skew. Choosing a level is choosing which anomalies your application can survive.

DEFINITION — D — Durability

Once committed, never lost. When the database acknowledges a commit, the transaction's effects survive process crashes, power loss, and disk failure up to the replication factor. The mechanism is the write-ahead log flushed to stable storage **before** the ack — Chapter 8's WAL, promoted from page insurance to contractual guarantee. Durability is a statement about the log, not about the data pages — those may still be dirty in the buffer pool when we ack, and that is fine.

11.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Single node or distributed?	Single node first (Chapter 8's engine); sketch 2PC and sagas as the distributed answer
Workload?	OLTP: 10 ⁴ txn/s, small read-modify-write transactions, a few hot rows
Which isolation default?	Justify a default; expect to defend READ COMMITTED vs snapshot isolation
Concurrency control?	Show both families: locking (2PL) and versioning (MVCC); know when OCC wins
Crash model?	Process kill and power loss; storage survives. Byzantine corruption is out of scope
Replication?	Mention streaming the WAL to replicas; deep replication design is Chapter 9's territory

NOTE — Agreed scope

1. Define the four guarantees as enforceable contracts and identify the mechanism behind each: log (A, D), locks/versions (I), constraints + application logic (C).
2. Design the **write path**: WAL with commit records, group commit, checkpoints, recovery (redo/undo) in the spirit of ARIES.
3. Design the **concurrency path**: two-phase locking with deadlock handling, MVCC snapshots, and when optimistic control beats both.

4. Enumerate the **anomaly zoo** and map isolation levels onto it — including where snapshot isolation famously stops (write skew).
5. Close with the distributed boundary: 2PC, its blocking failure mode, and sagas — and the bridge to Chapter 9's invariant boundary.
6. All reference code in Rust with deterministic tests.

11.3 Functional Requirements

1. **Transactional demarcation.** `BEGIN`, `COMMIT`, `ABORT`, with savepoints for partial rollback; every statement between them is one atomic unit.
2. **Atomic commit.** A crash at any instant leaves either all of the transaction's effects or none; recovery decides from the log alone.
3. **Durable acknowledgment.** `COMMIT` returns only after the commit record is on stable storage (and, if configured, on a replica).
4. **Selectable isolation.** Per-transaction isolation levels from `READ COMMITTED` to `SERIALIZABLE`, with documented anomaly guarantees.
5. **Concurrent correctness.** Readers never block writers and vice versa under MVCC; writers never corrupt each other under any level; deadlocks are detected and broken by victim abort.
6. **Point-in-time consistency.** A transaction reads a stable snapshot for its lifetime (under snapshot isolation), including its own writes.

11.4 Non-Functional Requirements

- **Throughput.** $\geq 10^4$ txn/s single-node for small transactions; the log append and lock table must not become the ceiling.
- **Commit latency.** $p_{99} \leq 2$ ms via group commit — one flush amortized over a batch, never one flush per transaction.
- **Recovery time.** After a crash, the database accepts work again in seconds: bounded redo since the last checkpoint, no full-data scans.
- **Version hygiene.** MVCC garbage (dead versions) is collected continuously; a long-lived snapshot may pin versions, and that hazard is surfaced as a metric, not discovered as a full disk.
- **Honest degradation.** Under lock contention the system degrades into waits and rare victim aborts — never into wrong answers.

11.5 Back-of-the-Envelope: The Cost of the Four Letters

Assumptions: 10^4 txn/s; average transaction = 20 row reads + 10 row writes; a redo record ≈ 100 B per written row; a row ≈ 200 B; SSD flush ≈ 0.5 –1 ms.

Quantity	Estimate (with assumptions)
WAL bandwidth	$10^4 \text{ txn/s} \times 10 \text{ writes} \times 100 \text{ B} \approx \mathbf{10 \text{ MB/s}}$ of redo — append-friendly, trivial next to an SSD's hundreds of MB/s
Flush discipline	One flush per commit $\Rightarrow \leq 1\text{--}2\text{k}$ txn/s, not 10^4 . Group commit batches 100 txns per flush: 10^4 txn/s at 100 flushes/s, + 1 ms commit latency
Write amplification	1 logical write $\Rightarrow$ WAL record + eventual data-page write + replica stream $\approx 3\times$; checkpoints smooth the page-write side (§11.7)

Quantity	Estimate (with assumptions)
MVCC version churn	10^5 row-writes/s $\times$ 200 B $\approx$ 20 MB/s of new versions $\approx$ 1.7 TB/day $\rightarrow$ vacuum cadence is an operational SLO, and a day-long snapshot can pin all of it
Lock manager load	10^4 txn/s $\times$ 30 lock acquisitions $\approx$ 3×10^5 ops/s on an in-memory table — the table is never the bottleneck; hot rows are (Chapter 6's leaderboard keys, again)
Recovery window	Checkpoint every 5 min $\Rightarrow$ redo replays $\leq$ 5 min of WAL $\approx$ 3 GB at 100+ MB/s $\Rightarrow$ back up in well under a minute

KEY INSIGHT — The four letters are bought with four currencies

Atomicity costs log space (undo/redo images). Durability costs flush latency (amortized by group commit). Isolation costs either **waiting** (locks), **aborting** (optimistic validation), or **space** (MVCC versions) — pick your tax. Consistency costs schema constraints and careful transaction authorship. A transaction system is a price list; the interview is reading it aloud without flinching.

11.6 The Core Challenge: Two Enemies, One Log, One Schedule

A transaction layer fights two enemies that never coordinate with each other, which is exactly what makes the problem hard.

Enemy one: the crash. A machine can stop between any two instructions. The transfer transaction writes two pages — debit A, credit B — and the buffer pool (Chapter 8) may flush either, both, or neither, in any order, at any time. If the debit's page reaches disk and the credit's does not, the restart finds money destroyed. The crash does not have to be likely; at 10^4 transactions per second, “unlikely per transaction” becomes “certain this quarter”.

Enemy two: concurrency. Thousands of transactions interleave on the same rows. Without control, their reads and writes interleave too — and every anomaly in Section 11.8's zoo is a specific interleaving with a name and a victim. The transfer read by a concurrent reporter mid-flight shows the bank \$100 short; two concurrent increments to the same counter produce one increment.

The two enemies share one battlefield — the **order of writes** — and the design answers each with one discipline:

KEY INSIGHT — Log first, then order the interleavings

Against crashes: **never let data reach disk before its description does** — the write-ahead log. One append-only stream, ordered by arrival, holding enough to redo what committed and undo what did not; recovery is replay, and replay is idempotent. Against concurrency: **decide which interleavings are legal** — with locks that forbid them (pessimistic) or versions that route around them (optimistic/MVCC). Every transaction system in production is a combination of one answer from each list. The rest of this chapter is the menu, the prices, and four Rust skeletons that make the menu concrete.

11.7 Deep Dive: Atomicity & Durability — The Write-Ahead Log

The WAL of Chapter 8 insured individual **pages**; here it is promoted to insure **transactions**. The discipline has three rules, and everything else is optimization.

1. **Log before data.** Before any dirty data page may reach disk, every log record describing its changes must already be on stable storage. Then a crash can never leave a change on disk that the log cannot explain.
2. **Commit means flushed.** `COMMIT` appends a **commit record** and flushes the log tail to stable storage (and returns only after the flush acknowledges). The data pages may still be dirty — they can be rebuilt from the log forever. The commit record is the exact instant a transaction becomes real.
3. **Recovery is two passes.** On restart: **redo** the effects of every committed transaction (durability), **undo or discard** the effects of every uncommitted one (atomicity). The Rust listing in Section 11.13 implements the conceptual core in thirty lines.

DEFINITION — Steal / no-force buffer management

The two freedoms that make the WAL necessary and sufficient. *Steal*: the buffer pool may write a page to disk **while a transaction that dirtied it is still uncommitted** (stealing its frame) — legal because the undo information is already in the log. *No-force*: a committing transaction is **not** required to flush its data pages — legal because the redo information is in the log. Steal buys memory flexibility; no-force buys commit speed; the WAL is the price of both. The industrial-strength formulation of all of this is the ARIES protocol (log sequence numbers on every page and record, physiological logging, repeating history before undo) — same three rules, hardened.

DEFINITION — Group commit / checkpoint

Group commit amortizes the one expensive operation — the flush — over a batch: transactions queue at commit, one flush confirms them all (10^4 txn/s at 100 flushes/s from the estimation table). A *checkpoint* periodically forces all dirty pages to disk and records the point in the log up to which recovery never needs to look; it caps the redo window so crash recovery is seconds, not a full replay of history.

11.8 Deep Dive: Isolation Levels and the Anomaly Zoo

“Isolation” becomes precise only when stated negatively: which **anomalies** — named, reproducible bad interleavings — a level forbids. Learn the zoo first; the levels are just fences around subsets of it.

DEFINITION — Isolation anomaly

A specific, nameable interleaving of two or more transactions that produces a result no serial execution could produce (or that leaks uncommitted state). Anomalies are the vocabulary of correctness: “our system is buggy” is a complaint; “our system allows write skew” is a diagnosis with a known cure.

Anomaly	The interleaving	The damage
Dirty read	T1 reads a row T2 has written but not committed; T2 aborts	T1 acted on data that never officially existed
Non-repeatable read	T1 reads a row; T2 updates and commits; T1 re-reads and gets a different value	One transaction, two realities
Phantom	T1 runs a predicate query (“accounts over 1 000”); T2 inserts a matching row and commits; T1 re-runs and sees a new row	Sets change under a reader’s feet
Lost update	T1 and T2 both read counter = 10, both write 11	An increment silently vanishes — Chapter 5’s vote counters, unprotected

Anomaly	The interleaving	The damage
Read skew	T1 reads A, T2's transfer A→B commits, T1 reads B	T1 saw A before and B after: the \$100, mid-flight, visible to no one else
Write skew	T1 and T2 each read a set (“doctors on call = 2”) and each writes a different row (their own status)	Both commit; the invariant dies — snapshot isolation's famous blind spot (§11.13)

The SQL standard sells isolation as four fences, each excluding a subset of the zoo:

Level		Dirty	Non-rep.	Phantom	Lost upd.	Write skew
READ UNCOMMITTED		possible	possible	possible	possible	possible
READ COMMITTED		prevented	possible	possible	possible	possible
REPEATABLE READ (SI)		prevented	prevented	prevented	prevented	possible
SERIALIZABLE		prevented	prevented	prevented	prevented	prevented

NOTE — The names lie; the mechanisms don't

The standard's table is a floor, not a description of any real engine. PostgreSQL's REPEATABLE READ is **snapshot isolation** — phantoms prevented, lost updates prevented by first-committer-wins, write skew admitted. MySQL/InnoDB's REPEATABLE READ is **next-key-locked two-phase locking** — phantoms prevented by locking index gaps, write skew admitted unless you `SELECT ... FOR UPDATE`. PostgreSQL's SERIALIZABLE is SSI (snapshot isolation plus lightweight dangerous-structure detection), not a lockout. Asking “which engine, which mechanism, which anomalies **actually** excluded?” is the difference between reciting the standard and knowing a database.

DEFINITION — Snapshot isolation (SI)

The MVCC isolation level: each transaction reads from a **snapshot** — the committed state as of its start, plus its own writes — so its view never changes under it, and writes use **first-committer-wins** (two transactions writing the same row: second to commit aborts). SI forbids every anomaly in the table except **write skew**, because write skew's transactions write **different** rows — no first-committer-wins conflict exists to catch it. The cure is promotion (serialize the dangerous read-write dependency, as `SELECT ... FOR UPDATE` turns the read rows into write-locked ones), both demonstrated in the Rust section.

11.9 Deep Dive: Pessimistic Concurrency Control — Two-Phase Locking

The oldest working answer: before touching a row, take a lock on it, and let the lock table — not luck — decide which interleavings are legal.

DEFINITION — Two-phase locking (2PL); strict 2PL

Transactions acquire **shared** locks to read and **exclusive** locks to write; shared locks coexist, exclusive locks exclude everything. The two phases: a *growing* phase in which locks are only acquired, and a *shrinking* phase in which they are only released — once a transaction releases any lock, it may take no new ones. That discipline alone guarantees conflict-serializable schedules. Production databases use *strict* 2PL, which holds all exclusive locks until commit/abort — slightly less parallelism, but no transaction ever reads uncommitted data (so no cascading aborts), and recovery stays simple.

2PL's famous failure is not incorrectness but **deadlock**: T1 holds A and waits for B; T2 holds B and waits for A; neither can move. The answers:

- **Detection**: maintain a **wait-for graph** — an edge $T \rightarrow U$ when T waits on a lock U holds — and find cycles (the Rust listing in Section 11.13 detects them by iteratively trimming transactions nobody waits on). Break a cycle by aborting the **victim**: youngest transaction, least work done, or fewest locks — anything cheap and deterministic.
- **Avoidance**: timeouts (crude, universal) or lock ordering (take locks in key order everywhere — eliminates cycles by construction, at the price of application discipline).

INTERVIEW TIP — Deadlock hygiene is an interview freebie

Three habits kill most production deadlocks, and naming them signals experience: (1) touch rows in a consistent order (primary-key order is fine); (2) keep transactions short — no network calls, no user think time inside a lock scope; (3) take the coarsest necessary lock latest in the transaction. Deadlocks are not a mystery; they are a scheduling choice you can usually design away before detection ever fires.

11.10 Deep Dive: Optimistic Control and MVCC

Locking pays for correctness with **waiting**. Two alternatives pay with other currencies.

Optimistic concurrency control (OCC). Assume conflicts are rare: run the whole transaction against private state, keeping a read set and a write set; at commit, **validate** — did any row I read change underneath me? does my write set collide with a concurrent committer? — then commit or abort. No locks held during execution, so no waits and no deadlocks; the cost is abort storms under contention (on a hot row, OCC aborts exactly the transactions 2PL would merely delay). OCC wins for read-heavy, conflict-sparse workloads; it is the wrong tool for Chapter 6's hot leaderboard keys.

DEFINITION — MVCC — multiversion concurrency control

Never overwrite a row in place; every write creates a new **version** stamped with its creator's transaction id, and readers follow **version chains** to the newest version their snapshot may see. The result is the property application developers actually want: **readers never block writers and writers never block readers** — reads are snapshot reads, writes only contend with concurrent writes to the same row (first-committer-wins). The price is space (Section 11.5's version churn) and a garbage collector (*vacuum* in PostgreSQL vocabulary) that may reclaim a version only when no live snapshot can still see it — which is why a day-old transaction pinning an ancient snapshot is an operational emergency, not a curiosity.

Serializable snapshot isolation (SSI) deserves one paragraph because it is the modern summit: run snapshot isolation as usual, but track **dangerous structures** — the read-write antidependency pairs that underlie write skew — and abort one participant when the pattern completes. Serializable outcomes at near-SI cost: PostgreSQL's SERIALIZABLE since 9.1, and the reason “serializable is too slow” is a decade out of date.

11.11 Deep Dive: The Distributed Boundary — 2PC and Sagas

Everything so far is one machine's machinery. The moment a transaction spans nodes — the transfer's two accounts live in different regions — no local log can commit it atomically, and Chapter 9's physics returns.

DEFINITION — Two-phase commit (2PC)

The distributed atomic-commit protocol. *Prepare*: the coordinator asks every participant to durably promise “I can commit” (each writes a prepared record to its own log and locks its rows). *Commit*: if all vote yes, the coordinator logs the decision and tells everyone to commit; any no (or timeout) means abort. 2PC delivers atomicity across nodes, and its flaw is structural: after voting yes, a participant is **blocked** — locked and unable to finish — until it learns the decision, so a coordinator crash mid-protocol holds locks hostage until recovery. 2PC is correct, blocking, and increasingly confined to single-cluster use; across organizations and regions it is effectively extinct.

DEFINITION — Saga

The microservice-era replacement for cross-service transactions: split the global transaction into a chain of local ACID transactions, each with a **compensating action** that undoes it (“capture payment” ↔ “refund payment”). Steps run without locks held across services; a failure triggers compensations in reverse order. Sagas trade I for availability — intermediate states are visible — which is exactly Chapter 10's workflow-compensation pattern and Chapter 9's invariant boundary restated: when coordination is unaffordable, replace atomicity with apology.

KEY INSIGHT — The trilogy's moral

Chapter 8 built pages; this chapter makes multi-statement changes correct on one node — at the price of coordination (locks, validation, a commit flush). Chapter 9 showed systems that refuse coordination and therefore **cannot** offer this chapter's guarantees across regions. CAP is the seam between them: ACID is what you can promise inside the coordination boundary; CRDTs are what you can promise outside it; and senior system design is choosing — per datum, per operation — which side of the seam each piece of state lives on.

11.12 API Design

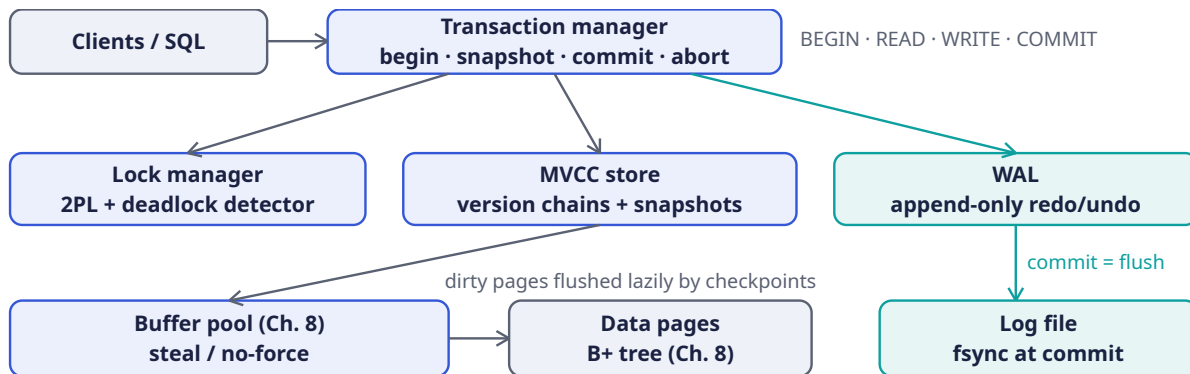
The interface is small because the semantics are heavy. Every verb below maps to exactly one mechanism from the deep dives.

Statement	Mechanism	Semantics
BEGIN [ISOLATION LEVEL ...]	txn manager	Opens a transaction; under MVCC, takes the snapshot now; under 2PL, opens the lock scope
SELECT ...	MVCC / S-locks	Snapshot read (no locks) or shared-lock read (strict 2PL), per engine and level
SELECT ... FOR UPDATE	X-locks	Materializes reads as write locks — the portable write-skew vaccine
INSERT / UPDATE / DELETE	versions + X-locks	Writes new versions (MVCC) under exclusive locks; redo records stream to the WAL

Statement	Mechanism	Semantics
SAVEPOINT name	partial undo	Marks a point inside the transaction; ROLLBACK TO undoes back to it without aborting the whole unit
COMMIT	WAL + locks	Validate (OCC/SI) → append commit record → flush → release locks → ack
ABORT	undo	Discard versions / apply undo records → release locks; idempotent and always safe
CHECKPOINT / VACUUM	internal	Cap the redo window; reclaim versions no snapshot can see

11.13 High-Level Architecture

The transaction manager is the conductor; the three instruments are the lock table, the version store, and the log. Note how little of this diagram is new: the buffer pool and data pages are Chapter 8, unchanged — transactions are a **layer**, not a rebuild.



NOTE — Reading the diagram

A COMMIT traces the teal spine: transaction manager → WAL append → flush to the log file → ack. The data pages are deliberately **not** on that path — no-force means they reach disk later, at checkpoint time, via the buffer pool. That indirection is the whole performance story: the commit path is one sequential append (fast, batchable), while the random page writes it logically implies are deferred and coalesced. Recovery walks the log file, replays committed transactions into the buffer pool, and discards the rest — the diagram’s two disks answer the two enemies of Section 11.6.

11.14 Rust Reference Implementations

Four pieces with deterministic tests: an MVCC store with snapshot transactions, a 2PL lock table with deadlock detection, a WAL with recovery, and — using the first — a live demonstration of write skew and its cure. Together they are Section 11.6’s “one answer from each list”, made executable.

11.14.1 MVCC: Snapshot Transactions

```

use std::collections::{BTreeMap, BTreeSet, HashMap};

/// One version of a key: a value plus the id of the transaction that
/// created it. Old versions are never overwritten in place — readers on

```

```

/// old snapshots keep walking the chain. (A real engine also tracks an
/// `end` stamp; vacuum reclaims versions no live snapshot can see.)
#[derive(Debug, Clone)]
struct Version {
    value: u64,
    begin: u64,
}

/// A minimal MVCC store providing snapshot isolation:
/// - each transaction reads the committed state as of its begin, plus
///   its own buffered writes;
/// - writers conflict via FIRST-COMMITTER-WINS on the same key.
pub struct MvccStore {
    data: BTreeMap<String, Vec<Version>>,          // versions, oldest -> newest
    committed: BTreeSet<u64>,
    active: BTreeSet<u64>,
    snapshots: BTreeMap<u64, BTreeSet<u64>>,      // txn -> active set at its begin
    pending: BTreeMap<u64, HashMap<String, u64>>, // txn -> buffered writes
    next_txn: u64,
}

impl MvccStore {
    pub fn new() -> Self {
        MvccStore {
            data: BTreeMap::new(),
            committed: BTreeSet::new(),
            active: BTreeSet::new(),
            snapshots: BTreeMap::new(),
            pending: BTreeMap::new(),
            next_txn: 0,
        }
    }

    pub fn begin(&mut self) -> u64 {
        self.next_txn += 1;
        let id = self.next_txn;
        self.snapshots.insert(id, self.active.clone()); // the snapshot
        self.active.insert(id);
        id
    }

    /// Visible to `txn` iff the version's creator committed BEFORE `txn`
    /// began: lower id, not in the snapshot (i.e. not still active at
    /// begin), and committed. Own writes bypass this via `pending`.
    fn visible(&self, v: &Version, txn: u64) -> bool {
        v.begin < txn
        && self.committed.contains(&v.begin)
        && !self.snapshots[txn].contains(&v.begin)
    }

    pub fn read(&self, txn: u64, key: &str) -> Option<u64> {
        if let Some(v) = self.pending.get(&txn).and_then(|w| w.get(key)) {
            return Some(*v); // read your own writes
        }
        self.data.get(key)?
            .iter()
            .rev() // newest first
            .find(|v| self.visible(v, txn))
            .map(|v| v.value)
    }
}

```



```

pub fn write(&mut self, txn: u64, key: &str, value: u64) {
    self.pending.entry(txn).or_default().insert(key.to_string(), value);
}

/// Commit, or abort with a write-write conflict: some *concurrent*
/// transaction (active at my begin, or begun after me) has already
/// committed a key I also wrote. First committer wins.
pub fn commit(&mut self, txn: u64) -> Result<(), ()> {
    let conflict = self.pending.get(&txn).map(|writes| {
        writes.keys().any(|key| {
            self.data.get(key).and_then(|vs| vs.last()).map_or(false, |latest| {
                let c = latest.begin;
                c != txn
                && (self.snapshots[&txn].contains(&c) || c > txn)
                && self.committed.contains(&c)
            })
        })
    }).unwrap_or(false);
    if conflict {
        self.abort(txn);
        return Err(());
    }
    let writes = self.pending.remove(&txn).unwrap_or_default();
    for (k, v) in writes {
        self.data.entry(k).or_default().push(Version { value: v, begin: txn });
    }
    self.committed.insert(txn);
    self.active.remove(&txn);
    self.snapshots.remove(&txn);
    Ok(())
}

pub fn abort(&mut self, txn: u64) {
    self.pending.remove(&txn); // buffered writes simply vanish
    self.active.remove(&txn);
    self.snapshots.remove(&txn);
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn snapshot_isolation_gives_repeatable_reads() {
        let mut s = MvccStore::new();
        let t0 = s.begin();
        s.write(t0, "balance", 100);
        assert!(s.commit(t0).is_ok());

        let reader = s.begin(); // snapshot taken here
        let writer = s.begin();
        s.write(writer, "balance", 200);
        assert!(s.commit(writer).is_ok()); // commits AFTER reader began

        // The reader's snapshot is frozen: same key, same answer, forever.
        assert_eq!(s.read(reader, "balance"), Some(100));
        s.abort(reader);
    }
}

```

```

    let fresh = s.begin(); // a new snapshot sees the commit
    assert_eq!(s.read(fresh, "balance"), Some(200));
}

#[test]
fn first_committer_wins_on_write_conflict() {
    let mut s = MvccStore::new();
    let t1 = s.begin();
    let t2 = s.begin();
    s.write(t1, "k", 1);
    s.write(t2, "k", 2);
    assert!(s.commit(t1).is_ok()); // first committer...
    assert!(s.commit(t2).is_err()); // ...wins; t2 aborts
    let t3 = s.begin();
    assert_eq!(s.read(t3, "k"), Some(1)); // no lost update, no torn value
}
}

```

11.14.2 Two-Phase Locking with Deadlock Detection

```

use std::collections::{BTreeMap, BTreeSet};

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Mode { Shared, Exclusive }

#[derive(Debug, Default)]
struct LockState {
    holders: BTreeMap<u64, Mode>, // txn -> mode held on this key
}

/// A non-blocking strict-2PL lock table: acquisition succeeds or reports
/// the conflict (the caller waits and retries); every conflict is also an
/// edge in the wait-for graph, where cycles are deadlocks. Real engines
/// park waiters on queues; the graph math is identical.
pub struct LockManager {
    locks: BTreeMap<String, LockState>,
    waits: BTreeMap<u64, BTreeSet<u64>>, // waiter -> holders it waits on
}

impl LockManager {
    pub fn new() -> Self {
        LockManager { locks: BTreeMap::new(), waits: BTreeMap::new() }
    }

    /// Shared locks coexist; exclusive locks exclude all but the holder's
    /// own. (Lock UPGRADE is just re-acquisition at a stronger mode.)
    fn compatible(state: &LockState, txn: u64, mode: Mode) -> bool {
        state.holders.iter().all(|(&h, &m)| {
            h == txn || (m == Mode::Shared && mode == Mode::Shared)
        })
    }

    pub fn acquire(&mut self, txn: u64, key: &str, mode: Mode) -> Result<(), ()> {
        let state = self.locks.entry(key.to_string()).or_default();
        if Self::compatible(state, txn, mode) {
            state.holders.insert(txn, mode);
            Ok(())
        } else {
            for &h in state.holders.keys() {

```

```

        if h != txn { self.waits.entry(txn).or_default().insert(h); }
    }
    Err(())
}

/// Commit/abort releases everything (strict 2PL) and forgets the
/// transaction's waits.
pub fn release_all(&mut self, txn: u64) {
    for state in self.locks.values_mut() {
        state.holders.remove(&txn);
    }
    self.waits.remove(&txn);
    for edges in self.waits.values_mut() {
        edges.remove(&txn);
    }
}

/// Deadlock = a cycle in the wait-for graph. Iteratively remove txns
/// nobody waits on (they can finish, so they can't be stuck); what
/// remains is deadlocked or doomed behind the deadlocked.
pub fn deadlock(&self) -> Option<Vec<u64>> {
    let mut g: BTreeMap<u64, BTreeSet<u64>> = BTreeMap::new();
    for (w, holders) in &self.waits {
        for h in holders {
            g.entry(*w).or_default().insert(*h);
            g.entry(*h).or_default();
        }
    }
    loop {
        let mut indeg: BTreeMap<u64, usize> = g.keys().map(|&k| (k, 0)).collect();
        for outs in g.values() {
            for o in outs {
                if let Some(d) = indeg.get_mut(o) { *d += 1; }
            }
        }
        let free: Vec<u64> = indeg.iter()
            .filter(|(&_, &d)| d == 0)
            .map(|(&n, _)| n)
            .collect();
        if free.is_empty() { break; }
        for f in free { g.remove(&f); }
    }
    if g.is_empty() { None } else { Some(g.keys().copied().collect()) }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn shared_locks_coexist_and_exclusive_waits() {
        let mut lm = LockManager::new();
        assert!(lm.acquire(1, "row", Mode::Shared).is_ok());
        assert!(lm.acquire(2, "row", Mode::Shared).is_ok());
        assert!(lm.acquire(3, "row", Mode::Exclusive).is_err()); // must wait
        assert_eq!(lm.deadlock(), None); // a wait chain is not a cycle
    }
}

```

```

#[test]
fn deadlock_is_detected_and_broken_by_victim_abort() {
    let mut lm = LockManager::new();
    assert!(lm.acquire(1, "a", Mode::Exclusive).is_ok());
    assert!(lm.acquire(2, "b", Mode::Exclusive).is_ok());
    assert!(lm.acquire(1, "b", Mode::Exclusive).is_err()); // 1 waits on 2
    assert!(lm.acquire(2, "a", Mode::Exclusive).is_err()); // 2 waits on 1
    let stuck = lm.deadlock().unwrap();
    assert!(stuck.contains(&1) && stuck.contains(&2));

    lm.release_all(2); // victim abort
    assert_eq!(lm.deadlock(), None);
    assert!(lm.acquire(1, "b", Mode::Exclusive).is_ok()); // survivor proceeds
}
}

```

11.14.3 The WAL and Recovery

```

use std::collections::{BTreeMap, BTreeSet};

/// A write-ahead log record. The discipline is the contract: a record
/// reaches stable storage BEFORE the data change it describes, and
/// COMMIT returns only after this log is flushed.
#[derive(Debug, Clone, PartialEq, Eq)]
enum Record {
    Begin(u64),
    Set(u64, String, u64), // txn, key, value – the redo image
    Commit(u64),
    Abort(u64),
}

pub struct Wal {
    records: Vec<Record>, // production: an append-only file, fsync'd per group commit
}

impl Wal {
    pub fn new() -> Self { Wal { records: vec![] } }

    pub fn append(&mut self, r: Record) { self.records.push(r); }

    /// Replay after a crash. Two passes over one stream: find every
    /// committed transaction, then redo exactly their writes. Committed
    /// work survives (durability); uncommitted work vanishes (atomicity).
    /// ARIES adds page-level LSNs and an undo pass; this is the moral core.
    pub fn recover(&self) -> BTreeMap<String, u64> {
        let mut committed = BTreeSet::new();
        for r in &self.records {
            if let Record::Commit(t) = r { committed.insert(*t); }
        }
        let mut state = BTreeMap::new();
        for r in &self.records {
            if let Record::Set(t, k, v) = r {
                if committed.contains(t) {
                    state.insert(k.clone(), *v);
                }
            }
        }
        state
    }
}

```

```

}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn committed_work_survives_the_crash() {
        let mut wal = Wal::new();
        wal.append(Record::Begin(1));
        wal.append(Record::Set(1, "alice".into(), 100));
        wal.append(Record::Set(1, "bob".into(), 50));
        wal.append(Record::Commit(1));
        let state = wal.recover();
        assert_eq!(state.get("alice"), Some(&100));
        assert_eq!(state.get("bob"), Some(&50));
    }

    #[test]
    fn uncommitted_work_vanishes() {
        let mut wal = Wal::new();
        wal.append(Record::Begin(1));
        wal.append(Record::Set(1, "alice".into(), 100));
        wal.append(Record::Commit(1));
        wal.append(Record::Begin(2));
        wal.append(Record::Set(2, "alice".into(), 999)); // crash: no Commit(2)
        let state = wal.recover();
        assert_eq!(state.get("alice"), Some(&100)); // txn 2 never happened
    }

    #[test]
    fn aborted_work_is_skipped_even_though_logged() {
        let mut wal = Wal::new();
        wal.append(Record::Begin(1));
        wal.append(Record::Set(1, "k".into(), 1));
        wal.append(Record::Abort(1));
        assert_eq!(wal.recover().get("k"), None);
    }
}

```

11.14.4 Write Skew, Demonstrated and Cured

Snapshot isolation's blind spot, reproduced against the MVCC store above (same crate). The setup is the classic on-call rota: the invariant is “at least one doctor is on call”; each doctor's transaction reads **both** rows and writes **only their own** — no write-write conflict exists for first-committer-wins to catch.

```

// uses MvccStore from the MVCC listing (same crate)

#[test]
fn write_skew_survives_snapshot_isolation() {
    let mut s = MvccStore::new();
    let t0 = s.begin();
    s.write(t0, "alice", 1); // 1 = on call
    s.write(t0, "bob", 1);
    s.commit(t0).unwrap();

    let ta = s.begin();
    let tb = s.begin();
}

```

```

// Both doctors check the rota: two on call, safe to step away.
assert_eq!(s.read(ta, "alice"), Some(1));
assert_eq!(s.read(ta, "bob"), Some(1));
assert_eq!(s.read(tb, "alice"), Some(1));
assert_eq!(s.read(tb, "bob"), Some(1));
// Each takes THEMSELVES off call – different keys, no conflict.
s.write(ta, "alice", 0);
s.write(tb, "bob", 0);
assert!(s.commit(ta).is_ok());
assert!(s.commit(tb).is_ok()); // SI allows it...

let check = s.begin();
assert_eq!(s.read(check, "alice"), Some(0));
assert_eq!(s.read(check, "bob"), Some(0)); // ...and nobody is on call.
}

#[test]
fn materializing_the_read_catches_the_skew() {
    // The `SELECT ... FOR UPDATE` cure: each transaction also WRITES the
    // row it depends on, turning the read-write dependency into a
    // write-write conflict that first-committer-wins can see.
    let mut s = MvccStore::new();
    let t0 = s.begin();
    s.write(t0, "alice", 1);
    s.write(t0, "bob", 1);
    s.commit(t0).unwrap();

    let ta = s.begin();
    let tb = s.begin();
    s.write(ta, "alice", 0);
    s.write(ta, "bob", 1); // "I depend on bob being on call"
    s.write(tb, "alice", 1); // "I depend on alice"
    s.write(tb, "bob", 0);
    assert!(s.commit(ta).is_ok());
    assert!(s.commit(tb).is_err()); // conflict on both keys -> abort

    let check = s.begin();
    assert_eq!(s.read(check, "bob"), Some(1)); // invariant saved
}

```

COMMON PITFALL — Testing the happy path only

Notice what the first test proves: **the invariant died with every operation returning success**. No error, no conflict, no log line — SI did exactly what it promises, and the rota still emptied. Correctness bugs at the isolation layer are silent; the only defenses are choosing the level with the anomaly table in hand, materializing reads that carry invariants, and writing tests (like these) that execute the dangerous interleaving on purpose. If your test suite only ever runs one transaction at a time, it has tested none of this.

11.15 Scaling the Design

Single-node transaction throughput scales up (bigger buffer pools, faster flushes, more parallel group-commit batches) until the write wall arrives; then the levers are structural.

Read scale is easy, write scale is earned. Replicas stream the WAL and serve snapshot reads — Chapter 9's machinery carrying this chapter's log — but every write still funnels through one log tail. Sharding writes by key range (Chapter 5's approach) splits the log per shard-group; the price is

that cross-shard transactions now need 2PC or must be designed away by **co-locating** what transacts together (account pairs on the same shard — the transfer example is deliberately co-locatable).

Contention, not capacity, is usually the ceiling. A hot row — the merchant's settlement account in the morning, Chapter 6's top leaderboard entry — serializes every transaction that touches it, whatever the isolation level. The playbook: batch counter increments through intermediate rows and sum at read; shorten transactions so locks are held for microseconds; move truly hot aggregates out of the transactional path entirely (Chapter 6's leaderboard does not need serializability per score).

Version GC is a throughput feature. Vacuum that falls behind turns every table into a history of itself: indexes bloat (Chapter 8's fill factor), snapshots walk longer chains, and the buffer pool fills with garbage. Autovacuum tuning is unglamorous and load-bearing.

11.16 Failure Modes & Degradation

Failure	Symptom	Handling
Crash mid-transaction	Partial effects possibly on disk	Recovery: redo committed, discard uncommitted — the WAL decides, pages never lie (§11.7)
Crash after flush, before ack	Client unsure whether commit landed	Client retries with its idempotency key (Chapter 10's rule); the transaction either is or isn't in the log — never half
Deadlock	Two txns frozen	Wait-for cycle detected; victim aborted with a retryable error; app retries
Long-lived snapshot	Version bloat, vacuum blocked	Metric + alert on snapshot age; statement timeouts; kill the session, not the server
Lock convoy on a hot row	Throughput collapses to serial	Shorten txns; batch via intermediate rows; move the aggregate out of the transactional path
Log disk full	All commits block — total write outage	The WAL is the single point of write failure: alert early, checkpoint diligently, never let it fill
Replica lag	Stale reads on replicas	Monotonic-read tokens (Ch. 9's contexts) or route reads-after-writes to the leader

COMMON PITFALL — Retrying without idempotency

The commit-ack ambiguity row deserves emphasis because it bites weekly in production: the log flushed, the network dropped the ack, and the client genuinely cannot know whether its transfer exists. Retrying blindly risks double-charging; not retrying risks losing a payment. The only correct shape is Chapter 10's: the client owns an idempotency key, the retry is safe, and the ambiguity is absorbed by the dedup table. Transactions make **the database** correct; end-to-end correctness is a protocol between client and server.

11.17 Trade-offs & Alternatives

Choice	We picked	Alternative	When the alternative wins
Concurrency control	MVCC (snapshot isolation)	Strict 2PL / OCC	2PL: strong consistency semantics with simpler reasoning; OCC: conflict-sparse, read-mostly workloads where aborts stay rare
Default isolation	READ COMMITTED or SI	SERIALIZABLE everywhere	Hot, invariant-carrying paths justify SSI's abort cost; most web workloads never see an anomaly at RC
Durability	Local flush + replica ack	Local flush only / fully synchronous multi-region	Latency budget vs. RPO=0 across regions; Chapter 9 prices the far end of this dial
Distributed atomicity	Avoid; co-locate sagas	2PC across services	Single-cluster, short-lived, coordinator-protected transactions — 2PC's safe habitat
Write path	Steal/no-force WAL	Force-at-commit (no redo)	Tiny embedded engines where simplicity beats commit latency

KEY INSIGHT — Isolation is a product decision, not a database default

Every isolation level is a price-performance point on the correctness curve, and the right answer depends on what an anomaly **costs the business**. A social feed tolerates phantoms; a ledger does not tolerate anything. The senior move is per-workload selection: SI for the OLTP core, SERIALIZABLE (or materialized reads) exactly where invariants carry money, READ COMMITTED where speed matters and anomalies don't — and a written note, in the design doc, saying which anomalies each service has agreed to live with.

11.18 Observability & SLOs

Signal	What it measures	Alert when
Commit latency	Time from COMMIT to ack (group-commit batching)	p99 > 2 ms: flush path slow or batches starving
WAL flush rate / backlog	Log throughput vs. generation	Backlog growing: disk saturated — the write ceiling
Lock waits	Time blocked per txn; wait-for edge count	P95 wait climbing: contention shift or a new hot row
Deadlocks / aborts	Victim aborts, SI conflicts per minute	Spike = a deploy changed access order; SSI abort storms on hot paths

Signal	What it measures	Alert when
Snapshot age	Oldest active snapshot	> minutes: vacuum is pinned, bloat incoming — page before the disk fills
Recovery drill	Time-to-open after kill –9	Practiced, not theoretical: measured in game days

SLOs. Commit latency $p99 \leq 2$ ms (group commit); recovery ≤ 60 s to accepting writes; zero committed-data loss at single-node fault (RPO = 0 with replica ack); deadlock resolution ≤ 100 ms from formation; snapshot age ≤ 5 min at p99. Every one of these is a mechanism from this chapter plus a number.

11.19 Interview Wrap-Up

Likely follow-ups, with the shape of a strong answer.

- “Which ACID letter is not the database’s job?” C. The database enforces constraints and provides A, I, D; the invariant “money is conserved” is authored by the transaction writer. Candidates who say this unprompted stand out immediately.
- “Default isolation for a new service?” READ COMMITTED or SI, chosen by naming the anomalies you are accepting — the table in Section 11.8 is the answer, not a footnote.
- “PostgreSQL vs MySQL REPEATABLE READ?” SI vs next-key-locked 2PL: same name, different anomalies excluded. Know one engine’s mechanism deeply rather than both engines’ marketing.
- “Why is 2PC dying?” The blocking window: a coordinator crash holds participants’ locks hostage until recovery. Inside one cluster with a protected coordinator it is fine; across services it is a reliability anti-pattern — sagas won.
- “Speed up commits?” Group commit; then steal/no-force (already assumed); then async commit with an explicit durability downgrade, said out loud.
- “How does this square with Chapter 9?” ACID lives inside the coordination boundary; CRDTs outside it; choose per datum. The interviewer’s favorite systems answer in 2026 is this sentence.

Checklist for the whiteboard. (1) Define each letter as a contract with its mechanism: log (A, D), locks/versions (I), constraints (C). (2) Draw the commit path through the WAL and say “commit = flush”. (3) Recite three anomalies and the level that kills each. (4) Know SI’s blind spot and both cures. (5) Sketch deadlock detection as a graph problem. (6) For distributed, refuse 2PC across services and offer sagas. (7) Mention one operational war story: snapshot age, log-full, or lock convoy.

11.20 Summary & Further Reading

A transaction layer is two disciplines answering two enemies. Against **crashes**, the write-ahead log: describe before you write, commit means flushed, recovery is replay — redo the committed, discard the rest, and let steal/no-force buffer management keep commits fast while checkpoints keep recovery short. Against **concurrency**, a legal-interleaving discipline: strict two-phase locking with deadlock detection, optimistic validation when conflicts are rare, or MVCC version chains that let readers and writers pass without blocking — each paying its tax in waits, aborts, or space. Isolation is sold in levels defined by the anomalies they forbid, and the map has exactly one trap door left open by the industry’s favorite level: snapshot isolation admits write skew, cured by materializing the reads or by SSI’s dangerous-structure detection. Past the single node, 2PC extends atomicity at the price of blocking, sagas replace it with compensation, and Chapter 9’s boundary reappears on cue: ACID is what coordination buys; the art is knowing which data can afford it. Chapter 8 supplied the pages; this chapter supplied the promises.

Further reading.

- The source video: “Intro to ACID Database Transactions — Systems Design Interview: 0 to 1 with Google Software Engineer” (Jordan has no life): <https://www.youtube.com/watch?v=oGmxzUBCYtY>
- Gray & Reuter — *Transaction Processing: Concepts and Techniques* — the field’s foundational text; the lock manager chapter alone justifies the shelf space.
- Mohan et al. — “*ARIES: A Transaction Recovery Method*” (1992) — the recovery protocol every serious engine implements.
- Adya — “*Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*” (1999) — the anomaly taxonomy done rigorously.
- Berenson et al. — “*A Critique of ANSI SQL Isolation Levels*” (1995) — why the standard’s names do not mean what engines ship.
- Cahill et al. — “*Serializable Snapshot Isolation*” (2008) — the PostgreSQL 9.1 implementation paper; serializable without the lockout.
- PostgreSQL and MySQL/InnoDB documentation on their isolation implementations — the same names, two different machines.

11.21 Chapter Glossary

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability — the four contracts of a transaction
ARIES	The industrial recovery protocol: LSNs everywhere, repeat history, then undo
Atomicity	All-or-nothing execution; enforced by the log at recovery time
Checkpoint	Periodic dirty-page flush + log marker capping the redo window
Commit record	The log entry whose flush makes a transaction real
Consistency (C)	Invariants preserved; enforced by constraints, authored by the application
Deadlock	A cycle in the wait-for graph; detected, then broken by victim abort
Dirty read	Reading another transaction’s uncommitted write
Durability	Committed survives crashes; a property of the log, not the pages
First-committer-wins	SI write-conflict rule: second committer on a contested row aborts
Group commit	One flush confirming a batch of commits; the throughput unlock
Isolation	Concurrency invisibility; sold in levels defined by forbidden anomalies
Lost update	Two read-modify-writes clobbering each other; increments vanish
MVCC	Multiversion concurrency control: writers append versions, readers walk snapshots
Non-repeatable read	Re-reading a row yields a new committed value mid-transaction
OCC	Optimistic control: run unlocked, validate at commit, abort on conflict
Phantom	A predicate query re-run sees newly committed matching rows

Term	Meaning
Read skew	A transaction sees pre-transfer A and post-transfer B — an impossible mix
Redo / undo	Replaying committed changes / erasing uncommitted ones, both from the log
Saga	Chained local transactions with compensating actions; atomicity traded for availability
Savepoint	An in-transaction marker allowing partial rollback
Snapshot isolation (SI)	Read a begin-time snapshot; first-committer-wins on writes; admits write skew
SSI	Serializable SI: detect dangerous read-write structures, abort a participant
Steal / no-force	Pages flushable while uncommitted / not forced at commit — the WAL's license
Strict 2PL	Two-phase locking with exclusive locks held to commit; no dirty reads, no cascading aborts
Two-phase commit (2PC)	Prepare + commit across nodes; atomic, blocking, coordinator-fragile
Vacuum	MVCC garbage collection of versions no live snapshot can see
WAL	Write-ahead log: the append-only stream every change is described in first
Wait-for graph	Edges waiter→holder; cycles are deadlocks
Write skew	Two txns read a shared set, write different rows, kill the invariant — SI's blind spot

— End of Chapter 11 · Next: Chapter 12 —

Designing a Ride-Hailing Marketplace (Uber / Lyft)

NOTE — Problem source

This chapter solves the problem posed in the talk “10: Design Uber/Lyft” from the series *Systems Design Interview Questions with Ex-Google SWE* (channel: *Jordan has no life*). It is a full product design in the shape of Chapters 1–7 and 10, and a reunion tour of the fundamentals chapters: geospatial indexing from Chapter 2, idempotent payments from Chapters 10–11, real-time push from Chapter 1, and the per-city placement decisions from Chapter 9. The problem is the definitive **marketplace** interview: two sides (riders, drivers), one scarce resource (nearby idle drivers), and a clock that never stops. All terms are defined before use; all reference code is Rust with deterministic tests.

12.1 The Problem Statement

The interviewer drops a pin on a map and says:

“Design Uber. A rider requests a trip; the system finds a nearby driver; quotes a price and ETA, coordinates pickup and the ride itself, charges the rider, pays the driver — in real time, in hundreds of cities, while a million drivers move around updating their location every few seconds.”

What makes this problem a classic is that it is **four** systems in a trenchcoat: a high-throughput location pipeline, a geospatial search engine over moving points, a real-time marketplace matcher, and a pedestrian transactional backend for trips and money. The interview is won by keeping those four cleanly separated — and by knowing which of them is actually hard (it is not the one candidates expect).

DEFINITION — Rider / driver / trip / marketplace

The two sides and the unit of work. *Riders* request transport; *drivers* supply it; a *trip* is one fulfilled request with a lifecycle (requested → matched → in progress → completed). The system is a *marketplace*: it does not own the supply, it **clears** it — matching demand to supply fast enough that neither side leaves, and pricing to keep the market balanced when it isn’t (surge, Section 12.10).

DEFINITION — ETA / deadheading / utilization

ETA is the estimated time of arrival — of the driver to the pickup (pickup ETA) and of the trip (dropoff ETA) — produced by the routing engine of Chapter 2 over the live road graph. *Deadheading* is driving without a passenger: pure cost. *Utilization* is the fraction of driver hours with a rider in the car — the marketplace’s core efficiency metric, and the number the matching engine (Section 12.8) silently optimizes.

12.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Scale?	10^6 drivers online globally, 2×10^7 rides/day, one metro is the unit of operation
Geography?	Cities are independent shards; cross-city trips are rare and can be treated as an edge case
Matching quality?	Fast beats perfect: a good match in 2 s outperforms an optimal match in 30 s
Pricing?	Dynamic (surge) per area; quote shown up front and honored
Shared rides?	Out of scope for v1 — mention it as the natural extension (multi-stop matching)
Payments?	Card on file; charge at trip end; idempotency is non-negotiable (Chapters 10–11)
Real-time UX?	Rider sees the driver's pin move; push over WebSocket (Chapter 1's channel)

NOTE — Agreed scope

1. The **location pipeline**: ingest 2.5×10^5 driver position updates per second and keep a queryable, in-memory picture of “who is where”.
2. The **geospatial index**: nearby-available-driver queries over points that never stop moving.
3. The **matching engine**: rider-to-driver assignment, offer/accept protocol, batched matching windows, idle-supply positioning.
4. The **trip backbone**: lifecycle state machine, idempotent payments, ratings — the transactional 20% that must never be wrong.
5. **Surge pricing** as a market-balancing control loop, with its failure modes designed in, not discovered.
6. Out of scope: pooling/shared rides, routing engine internals (Chapter 2), maps data pipelines (Chapter 2), driver onboarding.

12.3 Functional Requirements

1. **Location tracking**. Drivers publish position every 4 s while online; the system maintains a fresh, queryable picture per city; stale positions expire automatically.
2. **Quotes**. Given pickup and destination, return an upfront price and a pickup/dropoff ETA before the rider commits.
3. **Request & match**. A ride request enters matching immediately; an offered driver has 15 s to accept; on decline or timeout the request re-enters matching without rider involvement.
4. **Trip lifecycle**. Requested → matched → arriving → in progress → completed/cancelled, as an explicit state machine; illegal transitions are impossible by construction (Section 12.13).
5. **Payment**. At trip end, charge the rider's card once — exactly once as far as the rider can ever observe — and credit the driver's balance.
6. **Real-time status**. Both apps stream trip state and the driver's live position; polling is the fallback, not the primary channel.
7. **Surge**. Prices respond to per-area supply/demand within a minute, with smoothing and caps.

12.4 Non-Functional Requirements

- **Match latency.** $p_{99} \leq 5$ s from request to driver-accepted in a healthy market; the matching engine is never allowed to be the bottleneck that leaves riders staring at a spinner.
- **Location freshness.** The geo index reflects a driver's position within 5 s end-to-end; anything staler is treated as offline.
- **Availability asymmetry.** Losing **matching** for a minute is a degraded minute; losing **trip records or payments** is a breach — the transactional backbone out-availability-targets the real-time side.
- **City isolation.** A city is a failure domain: New York's New Year's Eve must not move Lagos's Tuesday morning.
- **Correctness under retries.** Every client-visible effect — requests, cancels, charges — is idempotent, because every mobile client retries on every elevator ride.

12.5 Back-of-the-Envelope: A Million Moving Dots

Assumptions: 10^6 drivers online at global peak; position update every 4 s; 2×10^7 rides/day; 5×10^6 concurrent riders at peak; trip record ≈ 2 KB; city-sharded.

Quantity	Estimate (with assumptions)
Location ingest	10^6 drivers / 4 s $\approx 2.5 \times 10^5$ updates/s , 100 B each $\rightarrow$ 25 MB/s — a Chapter 4 append stream, city-partitioned
Nearby-driver queries	Rider app polls + matcher candidate fetches $\approx 5 \times 10^4$ reads/s against the geo index — in-memory, so this is CPU, not I/O
Geo index memory	10^6 drivers $\times$ 48 B (id, cell, freshness) $\approx$ 50 MB per region — fits in cache ten times over; the index is cheap, the churn is the story
Match rate	2×10^7 rides/day $\approx$ 230/s average $\rightarrow$ 1.2×10^3 matches/s at evening peak ; with 2 s windows, 2 400 riders and 10^4 idle drivers per window globally
Trip storage	2×10^7 trips/day $\times$ 2 KB $\approx$ 40 GB/day $\rightarrow$ 3.6 TB for a 90-day hot window — Chapter 8's indexed store, partitioned by city and day
Push fanout	10^5 active trips $\times$ 2 devices $\times$ 1 update/5 s $\approx$ 4×10^4 msgs/s — Chapter 1's WebSocket fanout, modest by its standards
Payment volume	Peak $\approx 1.2 \times 10^3$ charges/s — trivial for the payments stack; the requirement is correctness (idempotency), not throughput

KEY INSIGHT — Read the table and notice what is not hard

Nothing here is big data. Fifty megabytes of live driver positions, forty gigabytes of trip records a day, a few thousand matches a second — a single modern machine could nearly run the whole thing. The genuine difficulty is the **physics of the problem**: positions expire in seconds, a match decision locks a moving 15-minute resource, riders abandon after 10 s of spinner, and the two sides of the market react to each other with feedback loops. This is a **liveness and correctness** design, not a capacity design — budget your interview time accordingly.

12.6 The Core Challenge: A Marketplace That Refuses to Sit Still

Strip the product away and three structural difficulties remain — none of them capacity.

1. **The index never rests.** A geospatial index over 10^6 points that each move every 4 seconds is not a structure you build; it is a structure you **maintain at 2.5×10^5 mutations per second**. B-trees

and R-trees answer “where is this” beautifully and “it moved again” badly; the answer is a flat, mutable, in-memory cell index where moving is an $O(1)$ re-bucketing (Section 12.7).

2. **Matching is a decision under expiry.** A rider waits seconds; an idle driver is claimed by the next request; the road network keeps score. The matcher must trade “best possible assignment” against “assignment before the rider abandons” — a classic online decision problem, solved by batching into short windows (Section 12.8).
3. **The transactional tail must be perfect.** Requests, cancels, charges: twenty percent of the traffic and a hundred percent of the trust. A double charge is a support ticket and a regulator; a lost trip record is a safety incident. This side of the system buys Chapter 11’s guarantees and pays for them gladly.

KEY INSIGHT — One city, one authority

The cleanest architectural decision in the whole design: **a trip is owned by exactly one city’s stack.** Geographically partitioned, region-local writes, no cross-region consensus on the hot path — the trip state machine and its store live where the trip physically is. Chapter 9’s physics is respected by making the problem almost never cross the boundary: riders and drivers move within a city; the rare cross-border trip is handed off between city stacks like a roaming call. Strong consistency where it is cheap, no coordination where it would hurt.

12.7 Deep Dive: Geospatial Indexing for Moving Points

The geo index answers one question, 5×10^4 times a second: **which available drivers are within a few hundred meters of this pickup?** Two candidate families exist; the moving-point constraint picks the winner.

R-trees and friends (Chapter 2’s structures) give exact spatial answers but punish mutation: every driver move is a delete-plus-insert through a balanced tree, and at 2.5×10^5 moves/s the maintenance cost dominates. **Grid cells** give up exactness for churn-tolerance: assign each driver to a cell; a query reads a handful of cells; a move updates two hash-map entries. Exactness is recovered by post-filtering the (small) candidate set by true distance — cells are for **recall**, distance math is for **precision**.

DEFINITION — Geohash

A string encoding of a grid cell made by interleaving longitude and latitude bits, five bits per base-32 character. Its superpower is the **prefix property**: a cell’s code is a prefix of every cell inside it, so “within this area” becomes `starts_with` — string prefix matching as a spatial operator. Longer codes are smaller cells (6 characters $\approx$ a 1.2 km $\times$ 0.6 km cell; 7 $\approx$ 150 m $\times$ 150 m). The Rust listing in Section 12.13 implements encode/decode in fifty lines. Production systems often use hexagonal successors (Uber’s H3, Google’s S2) whose cells have uniform neighbor distances and no pole weirdness; the interview-level mechanics are identical.

The query algorithm is ring expansion over prefixes: look in the pickup’s own cell; if fewer candidates than needed, look in the parent prefix (a cell 4 $\times$ wider); expand again to a grandparent; then post-filter by straight-line distance and rank by ETA from the routing engine. Two refinement notes that earn interview points: a driver near a cell **boundary** can be missed by pure prefix expansion, so production reads the 8 neighboring cells at each level as well; and cells must be sized to the market — Manhattan at lunch needs 150 m cells, rural Wyoming needs 5 km ones, so precision adapts per city and per hour.

The churn side is where the design earns its keep: `upsert(driver, lon, lat)` computes the driver’s cell, removes the id from the old cell’s list if it changed, and appends to the new one — $O(1)$, no tree surgery, no locks beyond a shard-local latch. A driver who stops updating for 15 s (the freshness TTL) is evicted

by a sweeper and treated as offline. Freshness is part of the index's contract, not a separate monitoring system: **a stale position is a wrong position.**

12.8 Deep Dive: The Matching Engine

Given candidates, who gets which rider? Two designs, one evolution.

Naive dispatch — offer each request to the single nearest available driver; then the next on decline — is where every prototype starts and where every prototype's metrics die: it serializes decisions (each offer burns a 15 s accept window), and greedy nearest-first assignment is provably suboptimal in aggregate (the Rust listing includes a case where it wastes minutes of pickup time).

Batched matching windows — the production answer: accumulate ride requests and idle drivers for a short window (2 s), then solve the whole window as one assignment problem: riders on one side, drivers on the other; edge costs = pickup ETA (plus small penalties for driver rating mismatches, vehicle-type mismatch, long deadhead). This is a **bipartite matching** instance; the optimal solution is the Hungarian algorithm's min-cost assignment, $O(n^3)$ on a window of thousands — and a well-ordered greedy pass (cheapest edge first, skip taken endpoints) lands within 10–20% of optimal at $O(E \log E)$, which is the usual production compromise at window scale.

DEFINITION — Bipartite matching / assignment problem

A graph whose vertices split into two sets (riders, drivers) with edges only across (a possible pairing, weighted by pickup cost). A *matching* is a set of edges sharing no endpoints: each rider, at most one driver. The *assignment problem* asks for the matching of minimum total cost — solved exactly by the Hungarian algorithm and approximately, at a tenth of the compute, by sorted greedy. Batching requests into windows is what turns a stream of myopic one-at-a-time decisions into a global optimization — the same trick as Chapter 7's re-ranking stage, applied to cars instead of videos.

The offer protocol rides on Chapter 11's machinery: an offer is a short-lived exclusive lock on a driver — `offer_token` with a 15 s TTL, one outstanding offer per driver; accept = compare-and-set on the token. Decline, timeout, or app death releases the driver into the next window; the rider's request re-enters with its original timestamp so fairness is preserved across retries.

Idle supply positioning is the matcher's quiet second job: when a city cell has surplus idle drivers and a neighbor has a deficit, the system **nudges** drivers toward the deficit (map suggestions, small bonuses), shrinking future pickup ETAs before any rider appears. Matching reacts to the market; positioning **shapes** it.

12.9 Deep Dive: The Trip Backbone — State Machine and Money

The real-time side may be fuzzy; this side may not. Every trip is a row in the city-sharded trip store whose `state` column moves through one explicit machine:

REQUESTED → MATCHED → ARRIVING → IN_PROGRESS → COMPLETED (or CANCELLED from any pre-completion state).

Each transition is a compare-and-set on the row inside a Chapter 11 transaction, triggered by an idempotency-keyed API call. Three rules make the machine trustworthy:

1. **The table, not the app, owns legality.** `driver_arrived` before `match` is not “handled”; it is rejected by the transition function (the Rust listing makes illegal transitions unrepresentable as successes).
2. **Every transition is idempotent.** Network retries, app restarts, and double-taps replay the same key and return the same outcome — Chapter 10's dedup discipline applied to state changes.

3. **Completion triggers payment exactly once.** The `COMPLETED` transition emits one `charge` command with key `(trip_id, "charge")` into the payments pipeline; capture is retried until acked, and the dedup store guarantees the rider's card sees it once no matter how many times the pipeline replays it (Chapter 10's effectively-once, end to end).

Driver payout is deliberately **not** synchronous: the trip completes, the rider charge settles through the payments pipeline, and the driver's balance updates via a ledger event — seconds later is fine, because nobody's card statement depends on the driver's app refreshing in real time. Decoupling the two money flows keeps the trip state machine small and the payments pipeline independently auditable.

12.10 Deep Dive: Surge Pricing as a Control Loop

When requests outnumber idle drivers in a cell, the market needs a price signal — more drivers should come, price-sensitive riders should wait. The mechanism: per-cell **surge multiplier** = a smoothed, capped function of the demand/supply ratio, recomputed every 30 s.

The control-theory traps, named so the interviewer knows you know:

- **Oscillation.** Price rises → drivers flood in → price collapses → drivers leave → repeat. Damping: exponential smoothing of the ratio, and asymmetric responsiveness — quick up (riders are abandoning **now**), slow down (don't whipsaw drivers mid-drive).
- **The feedback loophole.** Drivers learn to wait out the first minute of surge for a higher multiplier; riders learn to walk one cell over. Mitigations: personalized-but-regulated caps, and never publishing the exact per-cell map as a gameable API.
- **Ethics and law.** Emergency contexts cap or disable surge; regulators in several jurisdictions constrain the multiplier's ceiling and its display. A pricing system is a policy system with a math core.

COMMON PITFALL — Surge measured at the wrong granularity

Compute the ratio city-wide and a stadium emptying at midnight surges the whole city; compute it per-100 m-cell and a single rider plus-or-minus one driver oscillates the price every thirty seconds. The cell size must hold **enough actors for statistics** — tens of riders and drivers per cell per window — which is why production surge uses multi-resolution cells (H3's hierarchy) and falls back up the hierarchy when a fine cell is too quiet to measure.

12.11 API Design

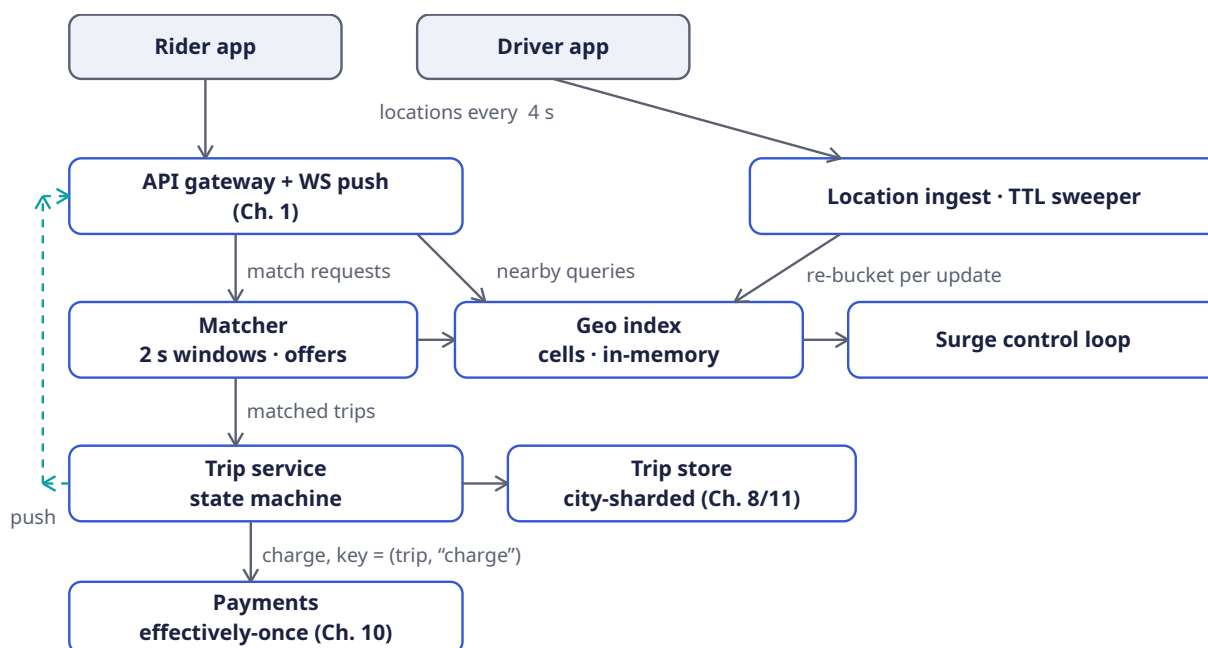
The API is deliberately small; every mutating call carries an idempotency key, and everything time-sensitive after the match rides the push channel rather than polling.

Verb & path	Purpose	Key fields	Guarantee
POST / v1/location	Driver heartbeat into the geo index	driver_id, lon, lat, heading, seq	202 ; stale seq dropped
POST / v1/rides/quote	Fare + pickup-ETA estimate before commitment	pickup, dropoff quote_id, price_band, eta, → surge	Read-only; quote TTL 60 s
POST / v1/rides	Create trip (REQUESTED) and enter matching	Idempotency-Key, quote_id, rider_id	Key-safe retry → same trip_id

Verb & path	Purpose	Key fields	Guarantee
POST /v1/offers/{id}/accept	Driver claims an offer within its 15 s TTL	offer_id, driver_id, offer_token	Compare-and-set; one winner
POST /v1/trips/{id}/events	Advance the state machine	Idempotency-Key, event: arrived start finish cancel	Illegal transition → 409
GET /v1/rides/{id}	Snapshot for reconnects and support tools	trip_id → full state + last known positions	Strongly consistent (city-primary read)
WS /v1/stream	Push: match found, driver en route, receipt	per-user channel, resume token	At-least-once + client dedup (Ch. 1)

Two design notes. First, the **quote before request** split matters: the quote is a cheap, cacheable, surge-aware estimate; the request is the committing write. Riders who balk at the price cost us one read, not one write-cancel-refund cycle. Second, `POST /v1/trips/{id}/events` is the only door into the state machine — apps never write `state` directly, so the transition table (Section 12.13) is enforced in exactly one place.

12.12 High-Level Design



Reading the diagram left to right, the system has three temperaments. The **top two rows are soft state**: heartbeats flow driver app → ingest → geo index, where a missed update simply expires at the TTL sweeper; the gateway asks “who is nearby” and forwards requests into the matcher. Nothing here is stored for long and nothing here may block on a disk. The **bottom half is hard state**: the matcher hands a mutually accepted pairing to the trip service, which walks the state machine against the city-sharded store, and exactly one charge command falls out the bottom into payments. The **dashed teal bus** is the return leg — every accepted transition is published to the gateway, which pushes it to both

apps over the long-lived connection of Chapter 1. The surge loop stands slightly apart: it reads the geo index's per-cell supply counts and the gateway's request rate, and answers quotes with multipliers — it never touches the trip backbone.

12.13 Rust Reference Implementations

Four small cores carry the chapter: the geohash codec that makes space prefix-searchable, the cell index that absorbs a quarter-million moves a second, the windowed matcher, and the state machine that guards the money. As in every chapter, the tests are the specification.

12.13.1 Geohash: space as a string

Bisect longitude and latitude alternately, five bits per character, and every coordinate becomes a string whose prefixes are its containing cells. `decode` returns the cell **center** — a geohash names a region, not a point.

```
const BASE32: &[u8; 32] = b"0123456789bcdefghjkmnpqrstuvxyz";

pub fn encode(lon: f64, lat: f64, precision: usize) -> String {
    let (mut lon_lo, mut lon_hi) = (-180.0_f64, 180.0_f64);
    let (mut lat_lo, mut lat_hi) = (-90.0_f64, 90.0_f64);
    let mut out = String::with_capacity(precision);
    let mut even = true; // even bits refine longitude, odd bits latitude
    for _ in 0..precision {
        let mut ch: u8 = 0;
        for _ in 0..5 {
            ch <= 1;
            if even {
                let mid = (lon_lo + lon_hi) / 2.0;
                if lon >= mid { ch |= 1; lon_lo = mid; } else { lon_hi = mid; }
            } else {
                let mid = (lat_lo + lat_hi) / 2.0;
                if lat >= mid { ch |= 1; lat_lo = mid; } else { lat_hi = mid; }
            }
            even = !even;
        }
        out.push(BASE32[ch as usize] as char);
    }
    out
}

pub fn decode(cell: &str) -> (f64, f64) {
    let (mut lon_lo, mut lon_hi) = (-180.0_f64, 180.0_f64);
    let (mut lat_lo, mut lat_hi) = (-90.0_f64, 90.0_f64);
    let mut even = true;
    for &b in cell.as_bytes() {
        let mut v = BASE32.iter().position(|&c| c == b).unwrap() as u8;
        for _ in 0..5 {
            let bit = v >> 4; // consume bits, MSB first
            v = (v << 1) & 0b11111;
            if even {
                let mid = (lon_lo + lon_hi) / 2.0;
                if bit == 1 { lon_lo = mid; } else { lon_hi = mid; }
            } else {
                let mid = (lat_lo + lat_hi) / 2.0;
                if bit == 1 { lat_lo = mid; } else { lat_hi = mid; }
            }
            even = !even;
        }
    }
}
```

```

    ((lon_lo + lon_hi) / 2.0, (lat_lo + lat_hi) / 2.0) // cell center
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn roundtrip Lands Inside the Cell() {
        let (lon, lat) = (-73.9857, 40.7484); // Manhattan
        let cell = encode(lon, lat, 7); // ~150 m cells
        let (clon, clat) = decode(&cell);
        assert!((clon - lon).abs() < 0.01);
        assert!((clat - lat).abs() < 0.01);
    }

    #[test]
    fn prefixes Are Containment() {
        let (lon, lat) = (-73.9857, 40.7484);
        let fine = encode(lon, lat, 8);
        let coarse = encode(lon, lat, 5);
        assert!(fine.starts_with(&coarse)); // "zoom out" = drop a char
    }
}

```

12.13.2 The geo index: built for churn

One hash map from cell to drivers, one from driver to cell, and a move becomes two $O(1)$ updates. `nearby` widens the prefix one character at a time until enough candidates surface — recall from cells, precision from the distance sort that would follow in the routing engine.

```

use std::collections::{HashMap, HashSet};

pub struct GeoIndex {
    cells: HashMap<String, Vec<u64>>, // cell → drivers inside it
    where_: HashMap<u64, String>, // driver → its current cell
    precision: usize,
}

impl GeoIndex {
    pub fn new(precision: usize) -> Self {
        GeoIndex { cells: HashMap::new(), where_: HashMap::new(), precision }
    }

    pub fn upsert(&mut self, driver: u64, lon: f64, lat: f64) {
        let cell = encode(lon, lat, self.precision);
        if self.where_.get(&driver) == Some(&cell) { return; } // no cell change
        if let Some(old) = self.where_.insert(driver, cell.clone()) {
            if let Some(list) = self.cells.get_mut(&old) {
                list.retain(|&d| d != driver);
            }
        }
        self.cells.entry(cell).or_default().push(driver);
    }

    pub fn remove(&mut self, driver: u64) { // offline, or TTL swept
        if let Some(old) = self.where_.remove(&driver) {
            if let Some(list) = self.cells.get_mut(&old) {
                list.retain(|&d| d != driver);
            }
        }
    }
}

```

```

    }
}

/// Ring expansion: exact cell, then parent prefix, then grandparent...
pub fn nearby(&self, lon: f64, lat: f64, limit: usize) -> Vec<u64> {
    let mut found = Vec::new();
    let mut seen = HashSet::new();
    let mut prefix = encode(lon, lat, self.precision);
    while !prefix.is_empty() && found.len() < limit {
        for (cell, list) in &self.cells {
            if cell.starts_with(&prefix) {
                for &d in list {
                    if seen.insert(d) { found.push(d); }
                }
            }
        }
        prefix.pop(); // zoom out one ring
    }
    found.truncate(limit);
    found
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn nearby_finds_local_drivers_only() {
        let mut idx = GeoIndex::new(6);
        idx.upsert(1, -73.9857, 40.7484); // beside the pickup
        idx.upsert(2, -73.9857, 40.7484);
        idx.upsert(3, -122.4194, 37.7749); // San Francisco: never matches
        let mut got = idx.nearby(-73.9857, 40.7484, 10);
        got.sort_unstable();
        assert_eq!(got, vec![1, 2]);
    }

    #[test]
    fn moving_a_driver_rebuckets_them() {
        let mut idx = GeoIndex::new(6);
        idx.upsert(7, -73.9857, 40.7484); // starts in Manhattan
        idx.upsert(7, -122.4194, 37.7749); // teleports to SF
        assert!(idx.nearby(-73.9857, 40.7484, 10).is_empty());
        assert_eq!(idx.nearby(-122.4194, 37.7749, 10), vec![7]);
    }

    #[test]
    fn going_offline_removes_the_driver() {
        let mut idx = GeoIndex::new(6);
        idx.upsert(9, -73.9857, 40.7484);
        idx.remove(9);
        assert!(idx.nearby(-73.9857, 40.7484, 10).is_empty());
    }
}

```

The reference scans cell keys per ring — $O(\text{cells})$ per widening step, fine for one city shard; production H3 deployments precompute neighbor rings or keep per-resolution maps so widening touches only the ring's cells. The important invariant is unchanged either way: **no driver ever appears in two cells at once**, because `upsert` removes before it adds.

12.13.3 The matcher: a window, sorted edges, two sets

```
use std::collections::HashSet;

/// One batched window: pair riders with drivers, cheapest pickup first.
/// Positions are (id, lon, lat); squared distance is enough for ordering.
pub fn match_greedy(drivers: &[(u64, f64, f64)],
                   riders: &[(u64, f64, f64)]) -> Vec<(u64, u64)> {
    let mut edges: Vec<(f64, u64, u64)> = Vec::new(); // (dist2, driver, rider)
    for &(d, dlon, dlat) in drivers {
        for &(r, rlon, rlat) in riders {
            let (dx, dy) = (dlon - rlon, dlat - rlat);
            edges.push((dx * dx + dy * dy, d, r));
        }
    }
    edges.sort_by(|a, b| a.0.partial_cmp(&b.0).unwrap()
                  .then(a.1.cmp(&b.1))
                  .then(a.2.cmp(&b.2)));

    let mut used_drivers = HashSet::new();
    let mut used_riders = HashSet::new();
    let mut pairs = Vec::new();
    for &(_, d, r) in &edges {
        // Check BOTH endpoints before inserting either: inserting the
        // driver first would burn it on a rider who is already taken.
        if !used_drivers.contains(&d) && !used_riders.contains(&r) {
            used_drivers.insert(d);
            used_riders.insert(r);
            pairs.push((d, r));
        }
    }
    pairs
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn greedy_pairs_each_with_the_closest() {
        let drivers = vec![(1, 0.0, 0.0), (2, 1.0, 1.0)];
        let riders = vec![(10, 0.05, 0.05), (20, 1.05, 1.05)];
        assert_eq!(match_greedy(&drivers, &riders), vec![(1, 10), (2, 20)]);
    }

    #[test]
    fn batching_beats_first_come_first_served() {
        // Serial dispatch serves rider 10 first: its nearest is driver 2
        // (24.01 < 26.01), leaving rider 20 to driver 1 – total 122.02.
        // The window sees both sides and pays 26.02 instead.
        let drivers = vec![(1, 0.0, 0.0), (2, 10.0, 0.0)];
        let riders = vec![(10, 5.1, 0.0), (20, 9.9, 0.0)];
        assert_eq!(match_greedy(&drivers, &riders), vec![(2, 20), (1, 10)]);
    }

    #[test]
    fn imbalanced_market_leaves_a_rider_for_the_next_window() {
        let drivers = vec![(1, 0.0, 0.0)];
        let riders = vec![(10, 0.1, 0.1), (20, 0.2, 0.2)];
        assert_eq!(match_greedy(&drivers, &riders), vec![(1, 10)]);
    }
}
```

```
}
}
```

12.13.4 The trip state machine: legality lives here

```
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub enum State { Requested, Matched, Arriving, InProgress, Completed, Cancelled }

#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub enum Event { Match, DriverArrived, StartTrip, Finish, Cancel }

/// The only legal moves. `None` = reject (HTTP 409); the app never
/// writes state directly, so this function IS the trip's law.
pub fn next(state: State, event: Event) -> Option<State> {
    use Event::*;
    use State::*;
    match (state, event) {
        (Requested, Match)          => Some(Matched),
        (Matched, DriverArrived)    => Some(Arriving),
        (Arriving, StartTrip)       => Some(InProgress),
        (InProgress, Finish)        => Some(Completed),
        // Cancellation is legal until completion; mid-trip cancels
        // still bill for distance covered.
        (Requested, Cancel) | (Matched, Cancel)
        | (Arriving, Cancel) | (InProgress, Cancel) => Some(Cancelled),
        _ => None, // terminal states, skipped steps, replayed events
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn happy_path_reaches_completed() {
        let mut s = State::Requested;
        for e in [Event::Match, Event::DriverArrived, Event::StartTrip, Event::Finish] {
            s = next(s, e).expect("legal transition");
        }
        assert_eq!(s, State::Completed);
    }

    #[test]
    fn illegal_transitions_are_rejected_not_handled() {
        assert_eq!(next(State::Requested, Event::Finish), None); // no driver yet
        assert_eq!(next(State::Requested, Event::StartTrip), None); // skipped steps
        assert_eq!(next(State::Completed, Event::Cancel), None); // terminal
        assert_eq!(next(State::Cancelled, Event::Match), None); // terminal
    }

    #[test]
    fn cancellation_closes_at_completion() {
        assert_eq!(next(State::Matched, Event::Cancel), Some(State::Cancelled));
        assert_eq!(next(State::InProgress, Event::Cancel), Some(State::Cancelled));
        assert_eq!(next(State::Completed, Event::Cancel), None);
    }
}
```

Paired with an idempotency-keyed event API and a compare-and-set on the trip row, this function gives the marketplace its spine: replays return the stored outcome, races have one winner, and every charge the payments pipeline ever sees traces back to a single, legal `Finish`.

12.14 Scaling

The city shard is the unit of everything. Each city runs its own geo index, matcher, trip store, and surge loop; adding capacity means splitting a hot city into two stacks along a geographic seam and re-homing the cells — a config change, not a migration of state machines, because trips never straddle the seam (a cross-border pickup is handed off once, at request time, to the city that owns the pickup cell). This is Chapter 9's lesson applied deliberately: the coordination boundary is drawn where the physics already is.

Location ingest scales horizontally by driver-id hash; any ingest node can re-bucket any driver because the geo index is the system of record for “where,” not the nodes in front of it. **Matching** scales by cell partition: disjoint pickup regions can run windows in parallel, with a thin arbitration layer for boundary cells. **The trip store** is ordinary Chapter 8 sharding — by `trip_id`, with a rider-id secondary index for history pages — and is deliberately the least exotic component in the building.

The one piece that resists scaling-by-addition is the **surge loop's global view** of a city: per-cell counts aggregate upward through a hierarchy (Chapter 6's top-k trick in geographic clothing), so even this reduces to a streaming aggregation problem we have solved before.

12.15 Failure Modes

Failure	Symptom	Response
Driver app dies mid-offer	Offer never answered	Offer TTL (15 s) expires; driver released to next window; rider re-entered with original timestamp
Driver stops sending location	Position goes stale in the index	TTL sweeper evicts after 15 s; driver marked offline; in-flight trips continue on rider-reported position
Matcher node crashes	One window's pairings lost	Requests are durable in the gateway queue; next window re-matches them — matching state is disposable by design
Trip store primary fails	Transitions stall in one shard	Failover to the shard replica (Ch. 11 durability); events queue on the API with their idempotency keys and replay after recovery
Payment capture times out	COMPLETED trip, no settled charge	The charge command retries with key (<code>trip_id</code> , "charge") until acked — effectively-once, never twice (Ch. 10)
City network partition	Riders and drivers split across two halves	Both halves keep working with degraded pools (availability asymmetry); the trip store accepts no writes it cannot confirm — no phantom trips, some unmatched riders

Failure	Symptom	Response
Surge loop goes hay-wire	Multiplier oscillates or spikes	Hard cap + floor, smoothing window, and a kill switch per city — pricing is a control loop and gets a circuit breaker

KEY INSIGHT — The asymmetry is the design

Notice what each row protects. The real-time side (offers, positions, windows) responds to failure by **discarding and retrying** — stale soft state is worthless anyway. The backbone (trips, charges) responds by **persisting and replaying** — a lost hard state is a lawsuit. A system that knows which half it is in at every line of code is a system that can be operated at 3 a.m.

12.16 Trade-offs

Decision	Chosen	Declined, and why
Geo structure	Flat cells, in-memory, $O(1)$ re-bucket	R-tree: exact but mutation-hostile at 2.5×10^5 moves/s; recall 1. post-filter beats exact indexing here
Dispatch	2 s batched windows, greedy assignment	Per-request nearest: serializes on offer TTLs and provably wastes aggregate pickup time; Hungarian optimum: $10\times$ the compute for a sliver of quality at window scale
Consistency of trips	Strong, single city-primary, CAS transitions	Multi-leader across regions: trips are local physics; buying cross-region consensus (Ch. 11's price) for a ride across town is a bad trade
Location durability	Soft state with TTL; losses self-heal in 4 s	Persisting every heartbeat: 25 MB/s of WAL for data that is stale in 15 s — the textbook case against durability
Surge authority	Per-cell loop, capped, city kill switch	Global optimization: prettier equilibrium, ungovernable blast radius; regulators and drivers both prefer legible cells
Shared rides (pooling)	Out of scope v1	Matching becomes set-packing, ETAs become promises about other people's behavior — a whole second interview

12.17 Observability and SLOs

The four numbers on the on-call dashboard, and what each one **means**:

- **Match p99 latency ≤ 5 s** (request $\rightarrow$ `MATCHED`). The marketplace's heartbeat. Degradation means windows starved of supply — alert on per-cell supply/demand ratio, not on the latency itself, which is the symptom.
- **Location freshness: $\geq 99\%$ of online drivers with a fix younger than 15 s.** Below this, the index is lying and every downstream decision inherits the lie.
- **Transition success $\geq 99.99\%$, double-charge count = 0.** The backbone SLOs. The second is not a percentage: it is a count with an alarm at one, fed by reconciling the payments ledger against `COMPLETED` trips nightly (Chapter 10's reconciliation pattern).
- **Push delivery p99 ≤ 2 s** from committed transition to app notification, measured with synthetic trips per city (Chapter 4's golden signals end-to-end).

Log every offer, every transition, every charge decision with `trip_id` as the join key — the trip's story must be replayable from logs alone, because when a rider and a driver disagree about what happened at 2 a.m., the log is the only witness.

12.18 Interview Wrap-Up

What the interviewer is listening for, roughly in order:

1. **Did you notice the two-systems problem?** Soft real-time state versus hard transactional state, with different durability, consistency, and failure semantics. Candidates who treat driver locations like rows in Postgres have announced their level.
2. **Is your geo story churn-first?** Cells and prefixes, not trees; recall plus post-filter, not exactness; TTL as a correctness mechanism.
3. **Does matching have a time axis?** Batched windows beat serial dispatch; offers are expiring locks; declined and timed-out offers must recycle cleanly. If your matcher never waits and never batches, it is leaving minutes of pickup time on the table.
4. **Is the money boring?** State machine, idempotency keys, one charge per completed trip, reconciliation. The highest compliment a payments design can receive is that there is nothing clever about it.
5. **Do you know surge is a control loop?** Smoothing, asymmetric responsiveness, caps, kill switch — and the humility to say “tuned in production” rather than deriving a multiplier on the whiteboard.

INTERVIEW TIP — The sixty-second answer

“City-sharded marketplace. Drivers heartbeat into an in-memory cell index; riders request; a matcher batches two-second windows and solves near-optimal assignment with expiring offers; an accepted match enters a strongly consistent trip state machine that ends in exactly one idempotent charge; a per-cell surge loop prices scarcity. Everything real-time is soft state with TTLs; everything financial is a logged, replayable, transactional record.” That paragraph is the whole chapter — the rest is proving you can build it.

12.19 Summary and Further Reading

Ride-hailing compresses the whole playbook into one product: Chapter 1's push channel carries the match, Chapter 2's geography becomes a churn-tolerant cell index, Chapter 8's pages hold the trips, Chapter 9's physics dictates the city shard, Chapter 10's scheduler times out the offers, and Chapter 11's transactions guard the charge. The new muscle is **economic**: matching as an online optimization under expiry, and pricing as a feedback control loop. System design at this level stops being about components and starts being about **which guarantees each part can afford**.

Further reading: Uber Engineering’s posts on H3 (hexagonal indexing) and on their marketplace matching and surge stack; the S2 geometry library docs for the spherical-geometry view; Kleinberg & Tardos, **Algorithm Design**, for bipartite matching and online algorithms; and any control- theory introduction — PID controllers in particular — before touching a production pricing loop.

12.20 Chapter Glossary

Term	Meaning in this chapter
Geohash	Interleaved lon/lat bits as a base-32 string; prefixes are containing cells, so spatial search becomes <code>starts_with</code>
Cell / H3 / S2	A fixed tile of the map used as an index bucket; hexagonal (H3) and spherical-cap (S2) successors to geohash’s rectangles
Ring expansion	Widening a nearby-search by dropping prefix characters until enough candidates are found
TTL sweeper	The process that evicts drivers whose locations have gone stale; freshness as an index invariant
Matching window	The 2 s batching interval that turns serial dispatch into a joint assignment problem
Bipartite matching	Pairing two disjoint sets (riders, drivers) with no shared endpoints; minimum-cost version solved by Hungarian, approximated by sorted greedy
Offer token	A 15 s exclusive lock on a driver; accept is a compare-and-set, timeout recycles the driver
Deadheading	Driving without a passenger; the cost matching and positioning exist to minimize
Utilization	Fraction of driver-online time spent earning; the marketplace’s supply-side health metric
Trip state machine	<code>REQUESTED → MATCHED → ARRIVING → IN_PROGRESS → COMPLETED</code> / <code>CANCELLED</code> ; the only legal moves, enforced server-side
Terminal state	<code>COMPLETED</code> or <code>CANCELLED</code> ; accepts no further events, which makes replays and late packets harmless
Idempotency key	Client-chosen request identity; retries return the stored outcome — the API-level twin of Chapter 10’s dedup
Effectively-once	At-least-once delivery plus key-based dedup; how one <code>Finish</code> becomes one charge
Surge multiplier	Per-cell price factor from smoothed demand/supply; capped, damped, kill-switchable
Control loop	Measure → compare → actuate, repeated; surge priced this way oscillates unless damped
Feedback loophole	Actors gaming the loop (drivers waiting out surge); mitigated by caps and opacity of the per-cell map
City shard	The unit of scale and failure: one city’s full stack, strong consistency inside, none needed across

Term	Meaning in this chapter
Soft state / hard state	Disposable, TTL-governed data (positions) versus durable, transactional data (trips, charges) — the chapter’s organizing split
Compare-and-set (CAS)	Atomic “update only if unchanged”; how offers accept and states transition with exactly one winner
Reconciliation	Nightly ledger-versus-trips audit; the alarm that makes “double charges = 0” a fact instead of a hope

— End of Chapter 12 · Next: Chapter 13 —